目　录

第 1 章　绪　论

1.1　复习笔记

（1）数据结构的基本结构

根据数据元素之间关系的不同特性，通常有下列四类基本结构：

②线性结构。数据元素之间存在一个对一个的关系。

③树形结构。数据元素之间存在一个对多个的关系。

④图状结构或网状结构。数据元素之间存在多个对多个的关系。

如图 1-1 所示为上述四类基本结构的关系图。

（2）数据结构的形式定义

（3）数据结构在计算机中的表示

数据结构在计算机中的表示（又称映象）称为数据的物理结构，又称存储结构。它包括数据元素的表示和关系的表示。

①元素的表示。计算机数据元素用一个由若干位组合起来形成的一个位串表示。

②关系的表示。计算机中数据元素之间的关系有两种不同的表示方法：顺序映象和非顺序映象。并由这两种不同的表示方法得到两种不同的存储结构：顺序存储结构和链式存储结构。

a．顺序映象的特点是借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系。

抽象数据类型可通过固有数据类型来表示和实现，即利用处理器中已存在的数据类型来说明新的结构，用已经实现的操作来组合新的操作。

1．抽象数据类型的 C 语言描述

（1）预定义常量和类型

```
#define TRUE 1
#define FALSE 0
#define OK    1
#define ERROR 0
#defineINFEASIBLE -1
#define OVERFLOW-2
```

（2）数据结构的表示和数据元素类型

```
typedef    //数据结构的表示（存储结构）用类型定义（typedef）描述
ElemType    //数据元素类型约定为 ElemType
```

（3）算法的函数描述

```
函数类型    函数名（函数参数表）{
语句序列
}
```

（4）赋值语句

```
简单赋值    变量名=表达式；
串联赋值    变量名 1=变量名 2=…=变量名 k=表达式；
成组赋值    （变量名 1，变量名 2，…，变量名 k）=（表达式 1，表达式 2，…，表达式 k）；
结构名=结构名
结构名=（值 1，…，值 k）；
变量名[]=表达式；
变量名[起始下标..终止下标]=变量名[起始下标..终止下标]；
```

交换赋值　变量名变量名
条件赋值　变量名=条件表达式 ？ 表达式 T：表达式 F；
（5）选择语句
条件语句 1 if（表达式）　语句；
条件语句 2 if（表达式）　语句；
else　语句；
开关语句 1 switch（表达式）{
case 值 1：语句序列 1；break；
…
case 值 n：语句序列 n；break；
default：语句序列 n+1；
}
开关语句 2　switch{
case 条件 1：　语句序列 1；break；
…
case 条件 n：语句序列 n；break；
default：　语句序列 n+1；
}
（6）循环语句
for 语句　for（赋初值表达式序列；条件；修改表达式序列） 语句；
while 语句　while（条件）　语句；
do-while 语句　do{
语句序列；
}while（语句）；
（7）结束语句
函数结束语句　return　表达式；
return；
case 结束语句　break；
异常结束语句　exit（异常代码）；
（8）输入和输出语句
输入语句　scanf（[格式串]，变量 1，…，变量 n）；
输出语句　printf（[格式串]，表达式 1，…，表达式 n）；
（9）基本函数
求最大值　max（表达式 1，…，表达式 n）
求最小值　min（表达式 1，…，表达式 n）
求绝对值　abs（表达式）
求不足整数值　floor（表达式）
求进位整数值　ceil（表达式）
判定文件结束　eof（文件变量） 或 eof
判定行结束　eoln（文件变量） 或 eoln
（10）逻辑运算
与运算&&：对于 A&&B，当 A 的值为 0 时，不在对 B 求值。
或运算||：对于 A||B，当 A 的值为非 0 时，不在对 B 求值。

正确性是指设计或选择的算法应当能正确地反映某种需求，否则，算法正确与否的衡量准则就不存在了。“正确”一词的含义大体可分为以下四个层次：
①程序不含语法错误。
②程序对于几组输入数据能够得出满足规格说明要求的结果。
③程序对于精心选择的、典型、苛刻而带有刁难性的几组输入数据能够得出满足规格说明要求的结果。
④程序对于一切合法的输入数据都能产生满足规格说明要求的结果。
可读性算法首先是方便人的阅读与交流，其次才是机器执行。可读性好有助于人对算法的理解，因为晦涩难懂的程序易于隐藏较多错误从而难以调试和修改。
健壮性是指当输入数据非法时，算法也能适当地作出反应或进行处理，而不会产生莫明其妙的输出结果。
（4）效率与低存储量需求
效率指的是算法执行时间。对于同一个问题如果有多个算法可以解决，执行时间短的算法效率高；存储量需求指算法执行过程中所需要的最大存储空间，这两者都与问题的规模有关。
算法执行时间需通过依据该算法编制的程序在计算机上运行时所消耗的时间来度量，度量一个程序的执行时间通常有两种方法：
（1）事后统计
很多计算机内部都有计时功能，有的甚至可精确到毫秒级，因此采用不同算法的程序可通过一组或若干组相同的统计数据以分辨优劣。
但这种方法有两个缺陷：一是必须先运行依据算法编制的程序；二是所得时间的统计量依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优劣。
（2）事前分析估算
①消耗时间的因素
一个用高级程序语言编写的程序在计算机上运行时所消耗的时间取决于下列因素：
a．程序依据的算法选用的策略。
b．问题的规模。
c．书写程序的语言。
d．编译程序所产生的机器代码的质量。
e．机器执行指令的速度。
显然，同一个算法用不同的语言实现，或者用不同的编译程序进行编译，或者在不同的计算机上运行时，效率均不相同，这表明使用绝对的时间单位衡量算法的效率是不合适的。
②时间复杂度
4．算法的存储空间需求
1.2　强化习题详解
1.3　考研真题与典型题详解
①事后统计。利用计算机内的计时功能，采用不同算法的程序可以用一组或多组相同的统计数据区分。其缺点，一是必须先运行依据算法编制的程序；二是所得时间的统计量依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优劣。
第 2 章　线性表
2.1　复习笔记
2.2　强化习题详解
2.3　考研真题与典型题详解
第 3 章　栈和队列

3.1　复习笔记

3.2　强化习题详解

9．试将下列递归过程改写为非递归过程。

```
void test（int &sum）
{
intx;
scanf（x）;
if（x==0） sum=0;
else
{
test（sum）;
sum+=x;
}
printf（sum）;
}
```

10．简述队列和栈这两种数据类型的相同点和差异处。

不同点：

①栈是一种后进先出的线性表，且只允许在栈顶进行插入和删除操作。

②队列是一种先进先出的线性表，且只允许在队头进行删除操作，在队尾进行插入操作。

11．写出以下程序段的输出结果（队列中的元素类型 QElemType 为 char）。

```
voidmain（）
{
QueueQ;
InitQueue（Q）;
charx = "e"
EnQueue（Q
EnQueue（Q
EnQueue（Q
DeQueue（Q
EnQueue（Q
DeQueue（Q
EnQueue（Q
While（!QueueEmpty（Q））
{
DeQueue（Q
printf（y）;
}
printf（x）;
}
```

12．简述以下算法的功能（栈和队列的元素类型均为 int）。

```
voidalgo3（Queue &Q）
{
StackS;
intd;
```

InitStack（S）;
while（!QueueEmpty（Q））
{
DeQueue（Q
Push（S
}
while（!StackEmpty（S））
{
Pop（S
EnQueue（Q
}
}
13．若以 1234 作为双端队列的输入序列，试分别求出满足以下条件的输出序列：
（1）能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的输出序列。
（2）能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的输出序列。
（3）既不能由输入受限的双端队列得到，也不能由输出受限的双端队列得到的输出序列。
答：（1）4132
（2）4213
（3）4231
14．假设以顺序存储结构实现一个双向栈，即在一维数组的存储空间中存在着两个栈，它们的栈底分别设在数组的两个端点。试编写实现这个双向栈 tws 的三个操作：初始化 inistack（tws）、入栈 push（tws
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
答：
设 head 指示循环队列中队头元素元素的位置，则有
head=（（q.rear+MAXLEN）-q.length+1）%MAXLEN
其中 MAXLEN 为队列可用的最大空间。
答：
答：
答：

3.3　考研真题与典型题详解
第 4 章　串
4.1　复习笔记
4.2　强化习题详解
4.3　考研真题与典型题详解
第 5 章　数组和广义表
5.1　复习笔记
5.2　强化习题详解
5.3　考研真题与典型题详解
第 6 章　树和二叉树
6.1　复习笔记
6.2　强化习题详解
6.3　考研真题与典型题详解
第 7 章　图
7.1　复习笔记
7.2　强化习题详解
7.3　考研真题与典型题详解
第 8 章　动态存储管理
8.1　复习笔记
8.2　强化习题详解
8.3　考研真题与典型题详解
第 9 章　查　找
9.1　复习笔记
9.2　强化习题详解
9.3　考研真题与典型题详解
第 10 章　内部排序
10.1　复习笔记
10.2　强化习题详解
10.3　考研真题与典型题详解
第 11 章　外部排序
11.1　复习笔记
11.2　强化习题详解
11.3　考研真题与典型题详解
第 12 章　文　件
12.1　复习笔记
12.2　强化习题详解
12.3　考研真题与典型题详解

第 1 章　绪　论

1.1　复习笔记

一、数据结构的定义

数据结构是一门研究非数值计算的程序设计问题中计算机的操作对象以及它们之间的关系和操作等的学科。

二、基本概念和术语

1．数据

数据（data）是对客观事物的符号表示，在计算机科学中是指所有能输入到计算机中并被计算机程序处理的符号的总称，它是计算机程序加工的“原料”。

2．数据元素

数据元素（data element）是数据的基本单位，在计算机程序中通常作为一个整体进行考虑和处理。

3．数据对象

数据对象（data object）是性质相同的数据元素的集合，是数据的一个子集。

4．数据结构

数据结构（data structure）是相互之间存在一种或多种特定关系的数据元素的集合。

（1）数据结构的基本结构

根据数据元素之间关系的不同特性，通常有下列四类基本结构：

①集合。数据元素之间除了“同属于一个集合”的关系外，别无其它关系。

②线性结构。数据元素之间存在一个对一个的关系。

③树形结构。数据元素之间存在一个对多个的关系。

④图状结构或网状结构。数据元素之间存在多个对多个的关系。

如图 1-1 所示为上述四类基本结构的关系图。

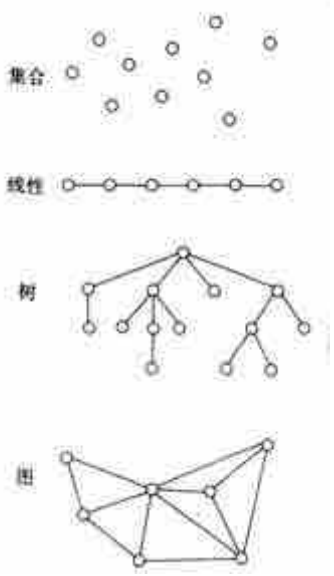


图 1-1 四类基本结构的关系图

（2）数据结构的形式定义

数据结构的形式定义为：数据结构是一个二元组 Data_Structure==（D，S）

其中：D 表示数据元素的有限集，S 表示 D 上关系的有限集。

（3）数据结构在计算机中的表示

数据结构在计算机中的表示（又称映象）称为数据的物理结构，又称存储结构。它包括数据元素的表示和关系的表示。

①元素的表示。计算机数据元素用一个由若干位组合起来形成的一个位串表示。

②关系的表示。计算机中数据元素之间的关系有两种不同的表示方法：顺序映象和非顺序映象。并由这两种不同的表示方法得到两种不同的存储结构：顺序存储结构和链式存储结构。

a．顺序映象的特点是借助元素在存储器中的相对位置来表示数据元素之间的逻辑关系。

b．非顺序映象的特点是借助指示元素存储地址的指针（pointer）表示数据元素之间的逻辑关系。

5．数据类型

数据类型（Data type）是一个值的集合和定义在这个值集上的一组操作的总称。按“值”的不同特性，高级程序语言中的数据类型可分为两类：一类是非结构的原子类型，值是不可分解的；另一类是结构类型，值是由若干成分按某种结构组成的，可以分解，并且它的成分可以是非结构的，也可以是结构的。

6．抽象数据类型

抽象数据类型（Abstract Data Type 简称 ADT）由一个值域和定义在该值域上的一组操作组成。抽象数据类型的定义仅取决于它的一组逻辑特性，而与其在计算机内部如何表示和实现无关。“抽象”的意义在于数据类型的数学抽象特性。若按其值的不同特性，抽象数据类型可细分为下列三种类型：原子类型，固定聚合类型和可变聚合类型。

7．多形数据类型

多形数据类型（polymorphic data type）是指其值的成分不确定的数据类型。

三、抽象数据类型的表示和实现

抽象数据类型可通过固有数据类型来表示和实现，即利用处理器中已存在的数据类型来说明新的结构，用已经实现的操作来组合新的操作。

1．抽象数据类型的 C 语言描述

（1）预定义常量和类型

```
#define T
#define FA
#define
#define ER
#defineINFEASIBLE -1
#define OVERFLOW-2
```

（2）数据结构的表示和数据元素类型

```
typedef    //数据结构的表示（存储结构）用类型定义（typedef）描述
ElemType    //数据元素类型约定为 ElemType
```

（3）算法的函数描述

```
函数类型    函数名（函数参数表）{
语句序列
}
```

（4）赋值语句

```
简单赋值    变量名=表达式；
串联赋值    变量名 1=变量名 2=…=变量名 k=表达式；
成组赋值    （变量名 1，变量名 2，…，变量名 k）=（表达式 1，表达式 2，…，表达式 k）；
结构名=结构名
结构名=（值 1，…，值 k）；
变量名[]=表达式；
变量名[起始下标..终止下标]=变量名[起始下标..终止下标]；
交换赋值    变量名变量名
条件赋值    变量名=条件表达式 ？ 表达式 T：表达式 F；
```

（5）选择语句

条件语句 1 if（表达式）　　语句；
条件语句 2 if（表达式）　　语句；
else　　语句；
开关语句 1 switch（表达式）{
case 值 1：语句序列 1；break；
…
case 值 n：语句序列 n；break；
default：语句序列 n+1；
}
开关语句 2　switch{
case 条件 1：　　语句序列 1；break；
…
case 条件 n：语句序列 n；break；
default：　语句序列 n+1；
}
（6）循环语句
for 语句　for（赋初值表达式序列；条件；修改表达式序列）　语句；
while 语句　while（条件）　　语句；
do-while 语句　do{
语句序列；
}while（语句）；
（7）结束语句
函数结束语句　return　　表达式；
return；
case 结束语句　break；
异常结束语句　exit（异常代码）；
（8）输入和输出语句
输入语句　scanf（[格式串]，变量 1，…，变量 n）；
输出语句　printf（[格式串]，表达式 1，…，表达式 n）；
（9）基本函数
求最大值　max（表达式 1，…，表达式 n）
求最小值　min（表达式 1，…，表达式 n）
求绝对值　abs（表达式）
求不足整数值　floor（表达式）
求进位整数值　ceil（表达式）
判定文件结束　eof（文件变量）　或 eof
判定行结束　eoln（文件变量）　或 eoln
（10）逻辑运算
与运算&&：对于 A&&B，当 A 的值为 0 时，不在对 B 求值。
或运算||：对于 A||B，当 A 的值为非 0 时，不在对 B 求值。

四、算法和算法分析

1．算法

（1）算法的定义

算法是对特定问题求解步骤的一种描述，它是指令的有限序列，其中每一条指令表示一个或多个操作。

（2）算法的 5 个重要特性

①有穷性；

②确定性；

③可行性；

④输入；

⑤输出。

2．算法设计的要求

（1）正确性（correctness）

正确性是指设计或选择的算法应当能正确地反映某种需求，否则，算法正确与否的衡量准则就不存在了。“正确”一词的含义大体可分为以下四个层次：

①程序不含语法错误。

②程序对于几组输入数据能够得出满足规格说明要求的结果。

③程序对于精心选择的、典型、苛刻而带有刁难性的几组输入数据能够得出满足规格说明要求的结果。

④程序对于一切合法的输入数据都能产生满足规格说明要求的结果。

（2）可读性（readability）

可读性算法首先是方便人的阅读与交流，其次才是机器执行。可读性好有助于人对算法的理解，因为晦涩难懂的程序易于隐藏较多错误从而难以调试和修改。

（3）健壮性（robustness）

健壮性是指当输入数据非法时，算法也能适当地作出反应或进行处理，而不会产生莫明其妙的输出结果。

（4）效率与低存储量需求

效率指的是算法执行时间。对于同一个问题如果有多个算法可以解决，执行时间短的算法效率高；存储量需求指算法执行过程中所需要的最大存储空间，这两者都与问题的规模有关。

3．算法效率的度量

算法执行时间需通过依据该算法编制的程序在计算机上运行时所消耗的时间来度量，度量一个程序的执行时间通常有两种方法：

（1）事后统计

很多计算机内部都有计时功能，有的甚至可精确到毫秒级，因此采用不同算法的程序可通过一组或若干组相同的统计数据以分辨优劣。

但这种方法有两个缺陷：一是必须先运行依据算法编制的程序；二是所得时间的统计量依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优劣。

（2）事前分析估算

①消耗时间的因素

一个用高级程序语言编写的程序在计算机上运行时所消耗的时间取决于下列因素：

a．程序依据的算法选用的策略。

b．问题的规模。

c．书写程序的语言。

d．编译程序所产生的机器代码的质量。

e．机器执行指令的速度。

显然，同一个算法用不同的语言实现，或者用不同的编译程序进行编译，或者在不同的计算机上运行时，效率均不相同，这表明使用绝对的时间单位衡量算法的效率是不合适的。

②时间复杂度

为了便于比较同一问题的不同算法的执行时间，通常的做法是从算法中选取一种对于所研究的问题（或算法类型）来说是基本运算的原操作，以该基本操作重复执行的次数作为算法的时间量度。一般情况下，算法中基本操作重复执行的次数是问题规模 n 的某个函数 f（n），算法的时间量度记作

T（n）—O（f（n））

它表示随问题规模 n 的增大算法执行时间的增长率和 f（n）的增长率相同，称作算法的渐近时间复杂度（asymptotic time complexity），简称时间复杂度。由于算法的时间复杂度考虑的只是对于问题规模 n 的增长率，因此在难以精确计算基本操作执行次数（或语句频度）的情况下，只需求出它关于 n 的增长率或阶即可。

4．算法的存储空间需求

类似于算法的时间复杂度，以空间复杂度（space complexity）作为算法所需存储空间的量度，记作

S（n）=O（f（n））

其中 n 为问题的规模。

1.2 强化习题详解

【基础知识题】

1．简述下列术语：数据，数据元素、数据对象、数据结构、存储结构、数据类型和抽象数据类型。

答：（1）数据是对客观事物的符号表示，在计算机科学中是指所有能输入到计算机中并能被计算机程序处理的符号的总称。

（2）数据元素是数据的基本单位，在计算机程序中通常作为一个整体进行考虑和处理。

（3）数据对象是性质相同的数据元素的集合，是数据的一个子集。

（4）数据结构是相互之间存在一种或多种特定关系的数据元素的集合。

（5）存储结构是数据结构在计算机中的表示（又称映象或数据的物理结构）。

（6）数据类型是一个值的集合和定义在这个值集上的一组操作的总称。

（7）抽象数据类型是指一个数学模型以及定义在该模型上的一组操作。

2．试描述数据结构和抽象数据类型的概念与程序设计语言中数据类型概念的区别。

答：数据结构是指相互之间存在一种或多种特定关系的数据元素的集合；抽象数据类型是指一个数学模型以及在该模型上的一组操作。

（1）抽象数据类型的定义仅取决于它的一组逻辑特性，而与其在计算机内部如何表示和实现无关。“抽象”的意义在于数据类型的数学抽象特性。

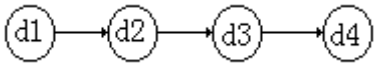
（2）抽象数据类型包含一般数据类型的概念，但含义比一般数据类型的范畴更广，它不再局限于各处理器中已定义并实现的数据类型，还包括用户在设计软件时自己定义的数据类型。

（3）抽象数据类型可通过固有数据类型来表示和实现，即利用处理器中已存在的数据类型来说明新的结构，利用已经实现的操作来进行新的操作。

3．设有数据结构（D，R），其中 $D=\{d1,d2,d3,d4\}$，$R=\{r\}$，

$r=\{(d1,d2),(d2,d3),(d3,d4)\}$，试按图论中图的画法惯例画出其逻辑结构图。

答：逻辑结构图如下：



4．试仿照三元组的抽象数据类型分别写出抽象数据类型复数和有理数的定义（有理数是其分子、分母均为自然数且分母不为零的分数）。

答：ADT Complex{

数据对象：D={r,i|r,i 为实数}

数据关系：R={<r,i>}

基本操作：

InitComplex（&C,re,im）

操作结果：构造一个复数 C，其实部和虚部分别为 re 和 im

DestroyCmoplex（&C）

操作结果：销毁复数 C

Get（C,k,&e）

操作结果：用 e 返回复数 C 的第 k 元的值

Put（&C,k,e）

操作结果：改变复数 C 的第 k 元的值为 e

IsAscending（C）

操作结果：如果复数 C 的两个元素按升序排列，则返回 1，否则返回 0

IsDescending（C）

操作结果：如果复数 C 的两个元素按降序排列，则返回 1，否则返回 0

Max（C,&e）

操作结果：用 e 返回复数 C 的两个元素中值较大的一个

Min（C,&e）

操作结果：用 e 返回复数 C 的两个元素中值较小的一个

}ADT Complex

ADT RationalNumber{

数据对象：D={s,m|s,m 为自然数，且 m 不为 0}

数据关系：R={<s,m>}

基本操作：

InitRationalNumber（&R,s,m）

操作结果：构造一个有理数 R，其分子和分母分别为 s 和 m

DestroyRationalNumber（&R）

操作结果：销毁有理数 R

Get（R,k,&e）

操作结果：用 e 返回有理数 R 的第 k 元的值

Put（&R,k,e）

操作结果：改变有理数 R 的第 k 元的值为 e

IsAscending（R）

操作结果：若有理数 R 的两个元素按升序排列，则返回 1，否则返回 0

IsDescending（R）

操作结果：若有理数 R 的两个元素按降序排列，则返回 1，否则返回 0

Max（R,&e）

操作结果：用 e 返回有理数 R 的两个元素中值较大的一个

Min（R,&e）

操作结果：用 e 返回有理数 R 的两个元素中值较小的一个
}ADT RationalNumber
5．在程序设计中，常用下列三种不同的出错处理方式：
（1）用 exit 语句终止执行并报告错误；
（2）以函数的返回值区别正确返回或错误返回；
（3）设置一个整型变量的函数参数以区别正确返回或某种错误返回。
试讨论这三种方法各自的优缺点。
答：（1）exit 常用于异常错误处理，它可以强行中断程序的执行，返回操作系统。
用 exit 语句终止执行并报告错误，没有回收占用的资源，但一般不会出现问题，操作系统会自动回收。另外，退出地点太多不利于调试。如果封装一个 exit 调试函数就另当别论。
（2）以函数的返回值区别正确返回或错误返回，常用于子程序的测试，比较容易发现错误的地方，便于实现程序的局部控制，不会直接终止程序的运行。但是判断太多，必须人工维护一份错误值列表，尤其是在写了越来越多的函数，错误值也越来越多的时候。
（3）用整型函数进行错误处理的优点是可以给出错误类型，便于迅速确定错误。
6．在程序设计中，可采用下列三种方法实现输出和输入：
（1）通过 scanf 和 printf 语句；
（2）通过函数的参数显式传递；
（3）通过全局变量隐式传递。
试讨论这三种方法的优缺点。
答：（1）用 scanf 和 printf 直接进行输入输出的优点是形象、直观；
其缺点是需要对其进行格式控制，较为烦琐，如果出现错误，则会引起整个系统的崩溃，参数类型受限制，内存开销大。
（2）通过函数的参数传递进行输入输出，优点是便于实现信息的隐蔽，减少出错的可能；
其缺点是参数说明比较烦琐，内存开销大。
（3）通过全局变量的隐式传递进行输入输出最为方便，优点是只需修改变量的值即可，内存开销小；
其缺点是过多的全局变量使程序的维护较为困难，内容极易丢失。
7．设 n 为正整数。试确定下列各程序段中前置以记号@的语句的频度：
（1） i=1; k=0;
while（i＜=n－1）
{
@ k＋=10*i;
i＋＋;
}
（2）i=1; k=0;
do {
@ k＋=10*i;
i＋＋;
} while（i＜=n－1）;
（3）i=1;k=0;
while （i＜=n－1）
{
i＋＋;
@ k＋=10*i;

}

（4） k=0;

for（i=1;i＜=n;i＋＋）

{

for（j=i;j＜=n;j＋＋）

@ k＋＋;

}

（5）for（i=1;i＜=n;i＋＋）

{

for（j=1;j＜=i;j＋＋）

{

for（k=1;k＜=j;k＋＋）

@ x＋=delta;

}

（6）i=1; j=0;

while（i＋j＜=n）

{

@ if（i＞j） j＋＋;

else i＋＋;

}

（7）x=n; y=0; // n 是不小于 1 的常数

while（x＞=（y＋1）*（y＋1））

{

@ y＋＋;

}

（8） x=91;y=100;

while（y＞0）

{

@ if（x＞100）

{ x－=10; y－－; }

else x＋＋;

}

答：（1）n-1

（2）n-1

（3）n-1

（4）n＋（n-1）＋（n-2）＋……＋1= $\frac{n(n+1)}{2}$

（5）1＋（1＋2）＋（1＋2＋3）＋……＋（1＋2＋3＋……＋n）= $\sum_{i=1}^{n}\frac{i(i+1)}{2}$

$$= \frac{1}{2}\sum_{i=1}^{n} i(i+1) = \frac{1}{2}\sum_{i=1}^{n}(i^2+i) = \frac{1}{2}\sum_{i=1}^{n} i^2 + \frac{1}{2}\sum_{i=1}^{n} i$$

$$=\frac{1}{12}n(n+1)(2n+1)+\frac{1}{4}n(n+1)=\frac{1}{12}n(n+1)(2n+3)$$

（6）n

（7）$\lfloor\sqrt{n}\rfloor$向下取整

（8）1100

8．假设 n 为 2 的乘幂，并且 n＞2，试求下列算法的时间复杂度及变量 count 的值（以 n 的函数形式表示）。

```
int Time（int n） {
count=0; x=2;
while（x＜n/2） {
    x*=2; count＋＋;
}
return count;
}
```

答：因为执行条件是 x＜n/2，则循环语句 x*=2; count++;的执行频度是$o(\log_2 n)$。

count++共执行了$\log_2 n-2$次，所以 count=$\log_2 n-2$。

9．按增长率由小至大的顺序排列下列各函数：

2^{100}，（3/2）^{n}，（2/3）^{n}，（4/3）^{n}，n^n，$n^{3/2}$，$n^{2/3}$，$\sqrt{n}$，n!，n，Log_2n，$n/\log_n2$，$\log_2^2 n$，$\log_2(\log_2 n)$，$n\log_2 n$，$n^{\log 2n}$（常见函数的增长率示意图，如图 1-2 所示）。

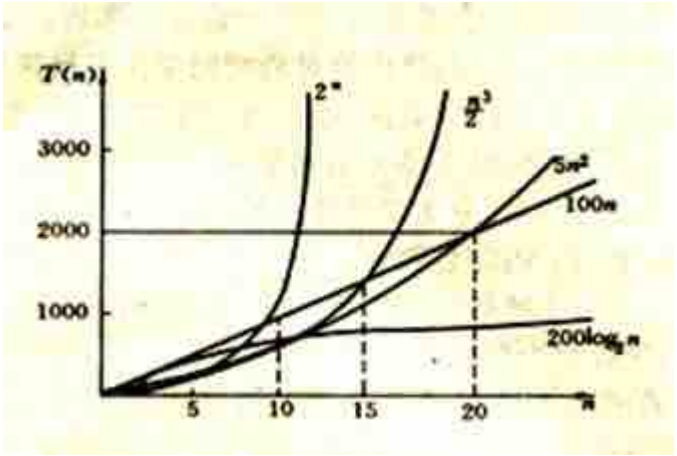


图 1-2

答：（2/3）^{n}＜2^{100}＜$\log_2(\log_2 n)$

＜Log_2n＜$\log_2^2 n$＜$\sqrt{n}$＜$n^{2/3}$＜$n/\log_n2$＜n＜$n\log_2n$＜$n^{3/2}$＜

（4/3）^{n}＜（3/2）^{n}＜$n^{\log 2n}$＜n!＜n^n。

10．已知有实现同一功能的两个算法，其时间复杂度分别为$O(2^n)$和$O(n^{10})$，假设现实中计算机可连续运算的时间为10^7秒（100 多天），每秒可执行基本操作（根据这些操作来估

算算法时间复杂度）10^5次。试问在此条件下，这两个算法可解问题的规模（即 n 值的范围）各为多少？哪个算法更适宜？请说明理由。

答：$2^n = 10^{12}$　n=40

$n^{10} = 10^{12}$　n=16

第一种算法更合适，因为对于同样的循环次数 n，第二种算法所花费的代价比较大。

11．设有以下三个函数：

$f(n) = 21n^4 + n^2 + 1000$，$g(n) = 15n^4 + 500n^3$，$h(n) = 500n^{3.5} + n\log n$

请判断以下断言正确与否：

（1）f（n）是 O（g（n））

（2）h（n）是 O（f（n））

（3）g（n）是 O（h（n））

（4）h（n）是 O（$n^{3.5}$）

（5）h（n）是 O（nlogn）

答：由分析可知，O（f（n））=O（g（n））=O（n^4）；O（h（n））=O（$n^{3.5}$），故：

（1）对　（2）错　（3）错　（4）对　（5）错

12．试设定若干 n 值，比较两函数n^2和$50n\log_2 n$的增长趋势，并确定 n 在什么范围内，函数n^2的值大于$50n\log_2 n$的值。

答：（1）n^2的增长趋势快，但在 n 较小的时候，$50n\log_2 n$的值较大。

（2）令$n^2 = 50n\log_2 n$，解得 n≈450，故当 n＞450 时，$n^2 > 50n\log_2 n$。

13．判断下列各对函数$f(n)$和$g(n)$，当$n \to \infty$时，哪个函数增长更快？

（1）$f(n) = 10n^2 + \ln\left(n! + 10^{n^3}\right)$，$g(n) = 2n^4 + n + 7$

（2）$f(n) = (\ln(n!) + 5)^2$，$g(n) = 13n^{2.5}$

（3）$f(n) = n^{2.1} + \sqrt{n^4 + 1}$，$g(n) = (\ln(n!))^2 + n$

（4）$f(n) = 2^{(n^3)} + (2^n)^2$，$g(n) = n^{(n^2)} + n^5$

答：（1）g（n）更快　（2）g（n）更快　（3）f（n）更快　（4）f（n）更快

14．试用数学归纳法证明：

（1）$\sum_{i=1}^{n} i^2 = n(n+1)(2n+1)/6 \quad (n \geq 0)$

（2）$\sum_{i=0}^{n} x^i = (x^{n+1} - 1)/(x - 1) \quad (x \neq 1, n \geq 0)$

（3） $\sum_{i-1}^{n} 2^{i-1} = 2^n - 1 \quad (n \geq 1)$

（4） $\sum_{i-1}^{n} (2i-1) = n^2 \quad (n \geq 1)$

答：（1）证明：①n＝1 时，1＝1×（1＋1）×（2×1＋1）/6＝1，等式成立；
②n＝2 时，1＋4＝2×（2＋1）×（2×2＋1）/6＝5，等式成立；
③设 n＝k（k＞=0）时，公式成立，即 1＋4＋9＋……＋k^2＝k（k＋1）（2k＋1）/6
则当 n＝k＋1 时，
1＋4＋9＋……＋k^2＋（k＋1）^{2}＝k（k＋1）（2k＋1）/6＋（k＋1）^{2}
＝（k＋1）×[2×（k^2）＋k＋6（k＋1）]/6
＝（k＋1）×[2×（k^2）＋7k＋6]/6
＝（k＋1）×（2k＋3）×（k＋2）/6
＝（k＋1）×[（k＋1＋1]×[2（k＋1＋1]/6
也满足公式。

综上所述， $\sum_{i-1}^{n} i^2 = n(n+1)(2n+1)/6 \quad (n \geq 0)$ 成立，得证。

（2）证明：①n=0 时，x^0=1，（x-1）/（x-1）=1，等式成立；
②n=1 时，1＋x=（x^2-1）/（x-1）=1＋x，等式成立；
③设 n=k（k＞=0）时等式成立，即 x^0＋x^1＋x^2＋……＋x^k=（x^{k+1}-1）/（x-1）
则当 n＝k＋1 时，
x^0＋x^1＋x^2＋…＋x^k＋x^{k+1}=（x^{k+1}-1）/（x-1）＋x^{k+1}
=[x^{k+1}-1）＋x^{k+1}（x-1）]/（x-1）
=[（x^{k+1}-1）＋x^{k+2}-x^{k+1}]/（x-1）
=（x^{k+2}-1）/（x-1）
也满足公式。

综上所述 $\sum_{i-0}^{n} x^i = (x^{n+1} - 1)/(x-1) \quad (x \neq 1, n \geq 0)$ 成立，得证。

（3）证明：①n=1 时，2^0=2^0=1，等式成立；
②n=2 时，2^0＋2^1=2^2-1，等式成立；
③设 n=k（k＞=1）时等式成立，即 2^0＋2^1＋…＋2^{k-1}=2^k-1
则当 n＝k＋1 时，
2^0＋2^1＋…＋2^{k-1}＋2^k=2^k-1＋2^k
=2×2^k-1
=2^{k+1}-1
也满足公式。

综上所述 $\sum_{i-1}^{n} 2^{i-1} = 2^n - 1 \quad (n \geq 1)$ 成立，得证。

（4）证明：①n=1 时，2-1=1，等式成立；
②n=2 时，（2×2-1）＋（2-1）=4=2^2，等式成立；

③设 n=k（k>=1）时等式成立，即（2×1-1）＋（2×2-1）＋…＋（2×k-1）=k^2
则当 n＝k＋1 时，
（2×1-1）＋（2×2-1）＋…＋（2×k-1）＋[2×（k＋1）-1]=k^2＋[2×（k＋1）-1]
=k^2＋2k＋1
=（k＋1）^{2}
也满足公式。

综上所述 $\sum_{i=1}^{n}(2i-1)=n^2 \quad (n\geq 1)$ 成立，得证。

【算法设计题】

15．试写一算法，自大至小依次输出顺序读入的三个整数 X，Y 和 Z 的值。

答：假设我们仍依 X、Y 和 Z 的次序输入这三个整数，则此题的目标是做到 X>=Y>=Z。在算法中应考虑对这 3 个元素做尽可能的比较和移动，如下述算法在最坏的情况下只需进行 3 次比较和 7 次移动。

```
void print_descending（int X，int Y，int Z）//按从大到小顺序输出三个整数
{
scanf（"%d"，&X）；
scanf（"%d"，&Y）；
scanf（"%d"，&Z）；
if（X＜Y）Exchange（X,Y）；
if（X＜Z）Exchange（X,Z）；
if（Y＜Z）Exchange（Y,Z）；
printf（"%d"，X）；
printf（"%d"，Y）；
printf（"%d"，Z）；
}    //print_descending
void Exchange（int x,int y）
{
int temp;
temp=x;
x=y;
y=temp;
}
```

16．已知 k 阶斐波那契序列的定义为

$f_0=0$，$f_1=0$，…，$f_{k-2}=0$，$f_{k-1}=1$；

$f_n=f_{n-1}+f_{n-2}+\cdots+f_{n-k}$，$n=k,k+1,\cdots$

试编写求 k 阶斐波那契序列的第 m 项值的函数算法，k 和 m 均以值调用的形式在函数参数表中出现。

答：

```
Status fib（int k，int m，int &f）//求 k 阶斐波那契序列的第 m 项的值 f
{
    int a[100];
```

```
    if（k＜2||m＜0） return FALSE;
if（m＜k-1） f=0;
else if （m==k-1） f=1;
else
    {
for（i=0;i＜=k-2;i++） a[i]=0;
a[k-1]=1; //初始化
    for（i=k;i＜=m;i++） //求出序列第 k 至第 m 个元素的值
    {
    sum=0;
    for（j=i-k;j＜i;j++） sum+=a[j];
    a[i]=sum;
    }
f=a[m];
}
return OK;
}
```

分析：k 阶斐波那契序列的第 m 项的值

$f[m]=f[m-1]+f[m-2]+\ldots+f[m-k]$

$=f[m-1]+f[m-2]+\ldots+f[m-k]+f[m-k-1]-f[m-k-1]$

$=2\times f[m-1]-f[m-k-1]$

所以上述算法的时间复杂度仅为 O（m）。如果采用递归设计，将达到 O（k^m）。即使采用暂存中间结果的方法，也将达到 O（m^2）。

17．假设有 A，B，C，D，E 五个高等院校进行田径对抗赛，各院校的单项成绩均已存入计算机，并构成一张表，表中每一行的形式为

项目名称　　性别　　校名　　成绩　　得分

编写算法，处理上述表格，以统计各院校的男、女总分和团体总分，并输出。

答：

```
typedef enum{A,B,C,D,E} SchoolName;
typedef enum{Female,Male} SexType;
typedef struct{
    char event[3]; //项目
    SexType sex;
    SchoolName school;
    int score;
} Component;
typedef struct{
    int MaleSum; //男团总分
    int FemaleSum; //女团总分
    int TotalSum; //团体总分
} Sum;
Component report[n];
Sum result[5];
```

算法过程体中主要结构：

```
for（i=0;i<n;i++）
{
对 result[report[i].school]进行处理;
}
for（s=A;s<=E;s++）
printf（……）；
例如：
typedef enum{A,B,C,D,E} SchoolName;
typedef enum{Female,Male} SexType;
typedef struct{
    char event[3]; //项目
    SexType sex;
    SchoolName school;
    int score;
} Component;
typedef struct{
    int MaleSum; //男团总分
    int FemaleSum; //女团总分
    int TotalSum; //团体总分
} Sum;
Sum SumScore（SchoolName sn,Component a[],int n）
{
    Sum temp;
    temp.MaleSum=0;
    temp.FemaleSum=0;
    temp.TotalSum=0;
    int i;
    for（i=0;i<n;i++）{
if（a[i].school==sn）{
if（a[i].sex==Male） temp.MaleSum+=a[i].score;
if（a[i].sex==Female） temp.FemaleSum+=a[i].score;
}
    }
    temp.TotalSum=temp.MaleSum+temp.FemaleSum;
    return temp;
}
```

18．试编写算法，计算$i!*2^i$（i=0，1，…，n-1）的值并存入数组 a[arrsize]的各个分量中。假设计算机中允许的整数最大值为 MAXINT，则当 n＞arrsize 或对某个$k(1 \le k \le n)$，使$k! \bullet 2^k > \text{MAXINT}$时，应按出错处理。注意选择你认为较好的出错处理方法。

答：注意 MAXINT 为计算机中允许出现的最大值，则在过程体中不能以计算机所得结果大于 MAXINT 作为判断出错的依据。

```
#include＜stdio.h＞
#define MAXINT 65535
#define ArrSize 100
int main（）
{
    int i，k;
    int a[ArrSize];
    printf（“Enter k:”）;
    scanf（“%d”，&k）;
    if（k＞ArrSize-1） exit（0）; //选用 exit（）作为出错处理方法
    for（i=0;i＜=k;i++）{
if（i==0） a[i]=1;
else{
if（2*i*a[i-1]＞MAXINT） exit（0）;    //选用 exit（）作为出错处理方法
else a[i]=2*i*a[i-1];
}
    }
    for（i=0;i＜=k;i++）{
if（a[i]＞MAXINT） exit（0）; //选用 exit（）作为出错处理方法
else printf（"%d\n"，a[i]）;
    }
    return 0;
}
```

19．试编写算法求一元多项式的值 $P_n(x)=\sum_{i=0}^{n}a_i x^i$ 的值 $P_n(x_0)$，并确定算法中每一语句的执行次数和整个算法的时间复杂度。注意选择你认为较好的输入和输出方法。本题的输入为 $a_i(i=0,1,\cdots,n)$，x_0 和 n，输出为 $P_n(x_0)$。

答：注意计算过程中不要对多项式中的每一项独立计算 x 的幂。

```
#include＜stdio.h＞
#define N 10
double polynomail（int a[]，int i，double x，int n）;
int main（）
{
double x;
    int n，i;
    int a[N];
printf（"Input the value of x:"）;
scanf（"%d"，&x）;
    printf（"Input number of terms:"）;
    scanf（"%d"，&n）;
    if（n＞N-1） exit（0）;
    printf（"输入多项式的系数 a[0]——a[n]:"）;
```

```
    for（i=0;i＜=n;i++）
    scanf（"%d",&a[i]）;
    printf （"The polynomail value is:%d\n"，polynomail（a，n，x，n））;
    return 0;
}
double polynomail（int a[]，int i，double x，int n）
{
    if（i＞0） return a[n-i]＋polynomail（a，i-1，x，n）*x;
    else return a[n];
}
```

本算法的时间复杂度为 O（n）。

1.3 考研真题与典型题详解

一、选择题

1．下列程常段的时间复杂度是（　　）。[2014 年联考真题]

```
count＝0;
for（k＝1;k<＝n; k*2）
for（j＝1;j<＝n;j+1）
count++;
```

A．O（$\log_2 n$）

B．O（n）

C．O（$n\log_2 n$）

D．O（n^2）

【答案】C 查看答案

【解析】外部循环的退出条件是 k>n，而对于 k，每次循环都执行 k＝k*2,所以循环次数为 $\log_2 n$；内部循环的退出条件是 j>n,对于 j，每次循环都执行 j＝j+1，所以每次循环次数为 n 次。所以此程序段的时间复杂度为 O（$n\log_2 n$），即选 C。

2．算法的计算量的大小称为计算的（　　）。[北京邮电大学 2000 研]

A．效率

B．复杂性

C．现实性

D．难度

【答案】B 查看答案

【解析】算法复杂度通常分为时间复杂度和空间复杂度，算法的计算量的大小可以用时间复杂度衡量，即可以称为计算的复杂度。

3．此程序的复杂度为（　　）。[大连理工 2008 研]

```
for(int i=0;i<m;i++)
for(int j=0;j<n;j++)
A[i][j]=i*j;
```

A．O（m^2）

B．O（n^2）

C．O（m×n）

D．O（m+n）

【答案】C 查看答案

【解析】时间复杂度是指基本操作重复执行次数的阶数 T（n）=O（f（n））。基本操作为 A[i][j]=i*j，该语句的执行次数与外层的两个循环次数相等，为 m×n，因此，算法的复杂度为 O（m×n）。

4．算法的计算量的大小称为计算的（　　）。[北京邮电大学 2000 研]

A．效率

B．复杂性

C．现实性

D．难度

【答案】B 查看答案

【解析】算法复杂度通常分为时间复杂度和空间复杂度，算法的计算量的大小可以用时间复杂度衡量，即可以称为计算的复杂度。

5．计算算法的时间复杂度是属于一种（　　）。[北京理工 2005 研]

A．事前统计的方法

B．事前分析估算的方法

C．事后统计的方法

D．事后分析估算的方法

【答案】B 查看答案

【解析】度量一个程序的执行时间通常有两种方法：

①事后统计。利用计算机内的计时功能，采用不同算法的程序可以用一组或多组相同的统计数据区分。其缺点，一是必须先运行依据算法编制的程序；二是所得时间的统计量依赖于计算机的硬件、软件等环境因素，有时容易掩盖算法本身的优劣。

②事前分析估计。一个高级语言程序在计算机上运行所消耗的时间。其运行消耗的时间取决于：依据的算法选用何种策略、问题的规模、程序语言、编译程序产生机器代码质量、机器执行指令速度。

算法的时间复杂度是指算法中基本操作重复执行的次数的阶数，因此计算算法的时间复杂度属于事前分析估算的方法。

6．以下属于逻辑结构的是（　　）。[西安电子 2001 研]

A．顺序表

B．哈希表

C．有序表

D．单链表

【答案】C 查看答案

【解析】顺序表是线性表的顺序存储结构；哈希表是存储结构；单链表是线性表的链式存储结构；有序表是指已经按照某一关键字排好序的线性表，它是逻辑结构。因此，C 项正确，而 ABD 三项都是数据元素在计算机中的存储结构，属于物理结构，而不是题目要求的逻辑结构。

7．以下数据结构中，哪一个是线性结构?（　　）[北方交通大学 2001 研]

A．广义表

B．二叉树

C．稀疏矩阵

D．串

【答案】D 查看答案

【解析】线性结构：数据元素之间存在一对一的关系。非线性结构指存在多对一或者一对多的关系。

8．下面说法错误的是（　　）。
（1）算法原地工作的含义是指不需要任何额外的辅助空间
（2）在相同的规模 n 下，复杂度 O（n）的算法在时间上总是优于复杂度 O（2n）的算法
（3）所谓时间复杂度是指最坏情况下，估算算法执行时间的二个上界
（4）同一个算法，实现语言的级别越高，执行效率就越低
A．（1）
B．（1），（2）
C．（1），（4）
D．（3）
【答案】C 查看答案
【解析】①一个可执行程序除了需要内存空间来寄存本身的指令、常数、变量和输入数据外，还需要额外空间，如果这个额外空间相对于问题的规模（输入数据）来说是个常数，称之为原地工作。
②对于相同的数据规模，O（n）的效率显然优于 O（2n）。
③对于 O（1）、O（log2n）、O（n）等，O 的形式定义为：若 f（n）是正数 n 的一个函数，则 O（f（n））表示存在一个正常数 M，使得当 $n \geq n_0$ 时满足｜O（f（n））｜小于等于 M×｜f（n）｜，即 O（f（n））给出了函数 f（n）的一个上界。
④编译链接后的最终的机器指令操作的次数越少，说明该语言在某种编译链接环境下效率越高，实际上即使同一种语言在不同的编译环境下，也有可能不同。
9．程序段
FOR i：=n-1 DOWNT0 1 DO
FOR j：=l TO i DO
IF A[j]>A[j+1]
THEN A[j]与 A[j+1]对换；
其中 n 为正整数，则最后一行的语句频度在最坏情况下是（　　）。
A．O（n）
B．O（nlogn）
C．O（n^3）
D．O（n^2）
【答案】D 查看答案
【解析】这个是冒泡排序，最坏的情况下需要进行 1+2+...+n-1 次交换，即时间复杂度是 O（n^2）。
二、填空题
1．抽象数据类型的定义仅取决于它的一组（1），而与（2）无关，即不论其内部结构如何变化，只要它的（3）不变，都不影响其外部使用。[山东大学 2001 研]
【答案】（1）逻辑特性查看答案
（2）在计算机内部如何表示和实现
（3）数学特性
2．数据结构是研讨数据的（1）和（2）以及它们之间的相互关系，并对与这种结构定义相应的（3），设计出相应的（4）。
【答案】（1）逻辑结构查看答案
（2）物理结构
（3）操作（运算）
（4）算法

3．在下面的程序段中，对 X 的赋值语句的频度为_____（表示为 n 的函数）。

```
FOR    i：=l TO n DO
FOR    j：=l TO i DO
FOR    k：=1 TO j DO
x：=x+delta；
```

【答案】1+（1+2）+（1+2+3）+…+（1+2+…+n）=n（n+1）（n+2）/ 6，即 O（n^3）查看答案

【解析】当 i=1 时，赋值语句就被执行了一次。当 i=2 时，赋值语句被执行了 1+2 次。当 i=3 时，赋值语句被执行了 1+2+3 次。……可以推出赋值语句总共被执行了 1+（1+2）+（1+2+3）+…+（1+2+…+n）=n（n+1）（n+2）/ 6 次。

4．对于给定的元素，可以构造出的逻辑结构有（1），（2），（3），（4）四种。

【答案】（1）集合查看答案

（2）线性结构

（3）树形结构

（4）图状结构（网状结构）

5．在下面的程序段中，对 x 的赋值语句的频度为_____（表示为 n 的函数）。

```
FOR i:=1 TO n DO
  FOR j:=1 TO i DO
    FOR k:=1 TO j DO
      x: =x+delta;
```

【答案】n（n+1）（n+2）/6 查看答案

【解析】对 x 的赋值语句的频度，即该语句被重复执行的次数为：

1+（1+2）+（1+2+3）+…+（1+2+…+n）=n（n+1）（n+2）/6。

6．已知如下程序段：

```
FOR i：=n DOWNTO 1 DO        {语句 1}
BEGIN
x：=x+1；      {语句 2}
FOR j：=n DOWNTO i DO    {语句 3}
Y：=y+1；      {语句 4}
END;
```

语句 l 执行的频度为__（1）___；语句 2 执行的频度为__（2）___；语句 3 执行的频度为__（3）___；语句 4 执行的频度为__（4）___。

【答案】（1）n+l 查看答案

（2）n

（3）n（n+3）/2

（4）n（n+1）/2

【解析】语句 1 执行到不符合条件情况下，执行了 n+1 次；当语句 1 不符合条件时不会执行语句 2，故语句 2 执行了 n 次；语句 3 每次都要执行到不符合条件，故执行次数为 2+3+4……+（n+1）即 n（n+3）/2；同样，当语句 3 不符合条件时不会执行语句 4，故语句 4 执行了 1+2+3……+n 次即 n（n+1）/2。

7．一个算法具有 5 个特性：（1）、（2）、（3）、有零个或多个输入、有一个或多个输出。

【答案】（1）有穷性查看答案

（2）确定性

（3）可行性

8．下面程序段中带下划线的语句的执行次数的数量级是_____。

i：=1；WHILE i<n DO i：=i+2；

【答案】n/2 查看答案

14．计算机执行下面的语句时，语句 s 的执行次数为_____。[南京理工大学 2000 研]

for（i=1；i<n-1；i++）

for（j=n；j>=i；j--）

s；

【答案】（n+3）（n-2）/ 2 查看答案

【解析】当 i=1 时，s 被执行了 n 次。当 i=n-2 是，s 被执行了 3 次。所以 s 总共被执行了 3+4+……+n=（n+3）（n-2）/ 2。

三、综合题

1．分析以下程序段中语句④的频度以及该程序段的时间复杂度。

```
for（i=0；i<n；i++）     //①
{
y=y+1；      //②
for（j=0；j<=2*n；j++）     //③
x++；      //④
}
```

答：语句的频度是指该语句被重复执行的次数。在本题中语句④执行的次数是内外两层循环的次数，即 n（2n+1）=$2n^2$+n，因此该语句的频度是 $2n^2$+n。

时间复杂度可以用算法中的基本操作重复执行的次数来度量，本程序段中的语句④即为基本操作，所以该算法的时间复杂度=$2n^2$+n=0（n^2）。

2．分析以下算法的时间复杂度。

```
void fun(int n)
{
    int i=0;s=0;
    while(s<n)
    {
        ++i;
        s=s+i;
    }
}
```

答：显然 n 为问题规模，基本操作为语句“++i”和“s=s+i”。变量 i 与 s 都从 0 开始，假设循环执行 m 次结束，即当循环 m 次时，s=n，则有

s1=1，s2=1+2=3，s3=1+2+3=6，…，Sm=m（m+1）/2（其中 Sm 为执行到第 m 次的时候 s 的值），则有 m（m+1）/ 2+K=n（K 为起修正作用的常数），由求根公式得

$$m=\frac{-1+\sqrt{8n+1-8k}}{2}$$

即 $$f(n)=\frac{-1+\sqrt{8n+1-8k}}{2}$$

由此可知时间复杂度为 $T(\mathrm{n})=O\left(\sqrt{n}\right)$

3．举一个数据结构的例子，叙述其逻辑结构、存储结构、运算三个方面的内容。

答：例如有一张学生成绩表，如表 1-1 所示，记录了一个班的学生各门课的成绩。这个表是一个数据结构。

表 1-1 学生成绩表

学号	姓名	成绩
1	王华	90
2	李明	84
.....	.....	.....

（1）逻辑结构

每个记录（包括学号，姓名，成绩）是指一个节点，对于整个表来说，只有一个开始节点（其前面无记录）和一个终端节点（其后面无记录），其他的节点则各有一个也只有一个前趋节点和后继节点（即它的前面和后面均有且仅只有一个记录）。

（2）存储结构

若用链式存储方式，节点的数据类型定义如下：

```
struct node
{ int no;    //存放学号的数据域
char name[10];    //存放姓名的数据域
int score;    //存放成绩的数据域
struct node*next;    //指向下一个节点的指针
}
```

（3）数据运算

在建表之后就要对这张表中的记录进行查询、修改、删除等操作。

4．以下算法的时间复杂度为多少?

```
void hanoi （int n,char x,char y,char z）
{
if （n= =l）
printf （"move %c to %c\n" , x, z）;
else
{
hanoi （n-l, x, z, y）;
printf （"move%cto%c\n", x, z）;
hanoi （n-l,y,x,z）;
}
}
```

答：设函数 hanoi （n…）的执行时间为 T（n），则 hanoi （n-1…）的执行时间为 T（n-1）。有：当 n=1 时，T（1）=1；当 n>1 时，T（n）=2T（n-1）+1。

所以有：

T（n）=2T（n-1）+1=2（2T（n-2）+1）+1

$=2^2T$（n-2）+1+2=2^2（2T（n-3）+1）+l+2^1

$=2^3T$（n-2）+l+2^1+2^2

=…

$=2^{n-1}T$（1）+1+2^1+2^2+……+2^{n-2}

$=2^{n-1}+2^{n-1}$-1

$=2^n$-l

=O（2^n）

5．假设一个不带头节点的单链表 h 中所有的节点数据域都为整数，设计一个递归算法求其中的最大值。

答：设单链表 h（以 h 为首节点指针）中最大值为 f（h），根据单链表的定义，h->next 也是一个单链表，则 f（h->next）为以 h->next 为首节点指针的单链表中的最大值，如图 l-1 所示。

f（h）求 a_1…a_n 中最大元素

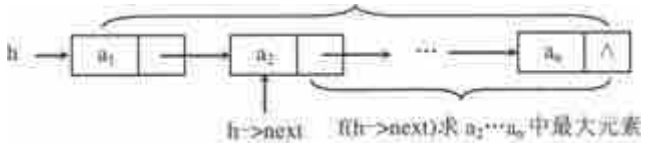


图 1-1 f（h）和 f（h->next）的功能

得到以下递归模型：

f（h）=h->data　//若 h->next=NULL 或单链表中只有一个节点

f（h）=MAX（h->data，f（h->next））　//其他情况

其中 MAX 表示求两个数的最大值。对应的递归算法如下：

```
int f（LinkList h）
{　int m;
if（h->next==NULL）
return h->data ；
else
{ m=f（h->next）；　　//递归调用
if（m>h->data）
return m;
else
return h->data;
}
}
```

第 2 章　线性表

2.1　复习笔记

一、线性表的类型定义

1．线性表的定义

线性表（linear_list）是最常用且最简单的一种数据结构。一个线性表是 n 个数据元素的有限序列。在稍复杂的线性表中，一个数据元素可以由若干个数据项（item）组成。在这种情况下，常把数据元素称为记录（record），含有大量记录的线性表又称文件（file）。

2．抽象数据类型线性表的定义

ADT List{

数据对象：$D = \{a_i \mid a_i \in ElemSet, i = 1, 2, \cdots, n, n \geq 0\}$

数据关系：$R1 = \{< a_{i-1}, a_i > \mid a_{i-1}, a_i \in D, i = 1, 2, \cdots, n\}$

基本操作：

InitList（&L）

操作结果：构造一个空的线性表 L。

DestroyList（&L）

初始条件：线性表 L 已存在。

操作结果：销毁线性表 L。

ClearList（&L）

初始条件：线性表 L 已存在。

操作结果：将 L 重置为空表。

ListEmpty（L）

初始条件：线性表 L 已存在

操作结果：若 L 为空表，则返回 TRUE，否则返回 FALSE。

ListLength（L）

初始条件：线性表 L 已存在。

操作结果：返回 L 中数据元素的个数。

GetElem（L，i，&e）

初始条件：线性表 L 已存在，1≤i≤ListLength（L）。

操作结果：用 e 返回 L 中的第 i 个数据元素的值。

LocateElem（L，e，compare（））

初始条件：线性表 L 已存在，compare（）是数据元素判定函数。

操作结果：返回 L 中第 1 个与 e 满足关系 compare（）的数据元素的位序。若这样的数据元素不存在，则返回值为 0。

PriorElem（L，cur_e，&pre_e）

初始条件：线性表 L 已存在

操作结果：若 cur_e 是 L 的数据元素，且不是第一个，则用 pre_e 返回它的前驱，否则操纵结果失败，next_e 无定义。

NextElem（L，cur_e，&next_e）

初始条件：线性表 L 已存在

操作结果：若 cur_e 是 L 的数据元素，且不是最后一个，则用 next_e 返回它的后继，否则操作失败，next_e 无定义。

ListInsert（&L，i，e）

初始条件：线性表 L 已存在，1≤i≤ListLength（L）+1。

操作结果：在 L 中第 i 个位置之前插入新的数据元素 e，L 的长度加 1.

ListDelete（&L，i，&e）

初始条件：线性表 L 已存在且非空，1≤i≤ListLength（L）。

操作结果：删除 L 的第 i 个数据元素，并用 e 返回其值，L 的长度减 1。

ListTraverse（L，visit（））

初始条件：线性表 L 已存在。

操作结果：依次对 L 的每个数据元素调用函数 visit（）。一旦 visit（）失败，则操作失败。

}ADT list

二、线性表的顺序表示和实现

1．顺序存储的特点方法

（1）顺序存储中，逻辑关系上相邻的两个元素在物理位置上也相邻，因此对表中相邻的元素 a_i 和 a_{i+1} 赋以相邻的存储位置 LOC（a_i）和 LOC（a_{i+1}）。如图 2-1 所示，所以只要确定了存储线性表的起始位置，线性表中任一数据元素都可随机存取，所以线性表的顺序存储结构是一种随机存取的存储结构；

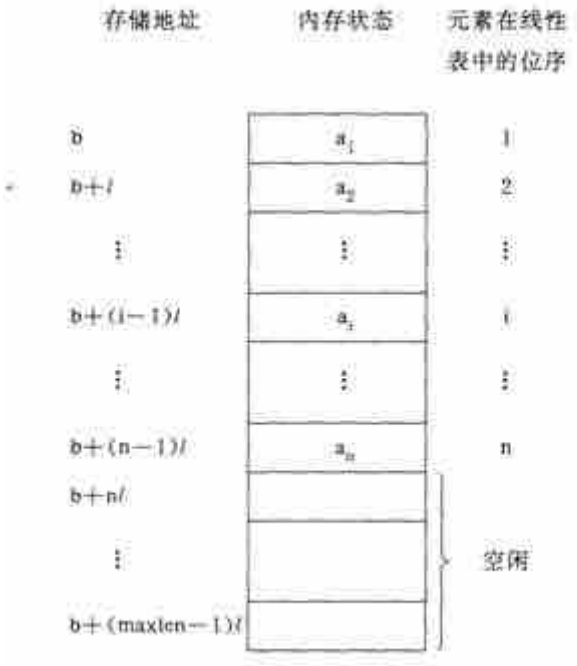


图 2-1 线性表的顺序存储结构示意图

（2）线性表的顺序存储结构是一个记录型的结构。数据元素的存储位置可用数组的下标值（即相对于线性表的起始位置的值）来表示；

（3）在顺序存储结构中，线性表的某些操作容易实现，如求表长的操作，线性表的长度即为 last 域的值。

3．线性存储的缺点

（1）在做插入或删除操作时，需移动大量元素；

（2）在给长度变化较大的线性表预先分配空间时，必须按最大空间分配，使存储空间不能得到充分利用，造成了空间的浪费；

（3）表的容量难以扩充。

三、线性表的链式表示和实现

链式存储结构，由于它不要求逻辑上相邻的元素在物理位置上也相邻，因此它没有顺序存储结构所具有的缺点。

1．定义

为了表示每个数据元素 a_i 与其直接后继元素 a_{i+1} 之间的逻辑关系，对数据元素 a_i 来说，除了存储其本身的信息之外，还需存储一个指示其直接后继元素的信息（即直接后继元素的

存储位置），这两部分信息组成数据元素 a_i 的存储映象，称为结点（Node）。它包括两个域：数据域和指针域，其中，数据域存储该数据元素信息；指针域存储直接后继元素的存储位置。指针域中存储的信息称做指针或链，n 个结点（a_i（$1 \le i \le n$）的存储映象）链结成一个链表，即为线性表（a_1，a_2，…，a_n）的链式存储结构，又由于此链表的每个结点中只包含一个指针域，又称线性链表或单链表。

2．线性链表特点

（1）它不要求逻辑上相邻的元素在物理位置上也相邻，因此可以用一组任意的存储单元存储线性表的数据元素（这组存储单元可以是连续的，也可以是不连续的）；

（2）用线性链表表示线性表时，数据元素之间的逻辑关系是由结点中的指针指示的；

（3）单链表是非随机存取的存储结构。

通常把链表画成用箭头相链接的结点的序列，结点之间的箭头表示链域中的指针。因为在使用链表时，关心的只是它所表示的线性表中数据元素之间的逻辑顺序，而不是每个数据元素在存储器中的实际存储的物理位置，如图 2-2 所示。

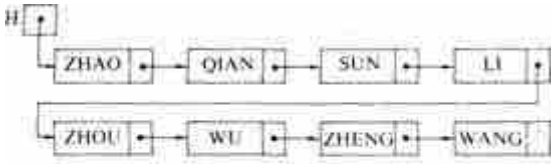
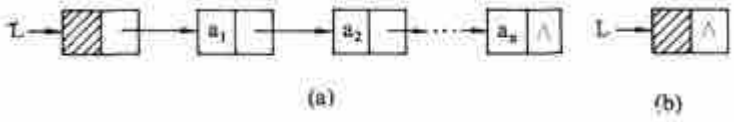


图 2-2 线性链表的逻辑状态

3．线性链表的操作

在单链表的第一个结点之前附设一个结点，称之为头结点。头结点的数据域可以不存储任何信息，也可存储如线性表的长度等类似的附加信息；头结点的指针域存储指向第一个结点的指针（即第一个元素结点的存储位置）。如图 2-3（a）所示，此时，单链表的头指针指向头结点。若线性表为空表，则头结点的指针域为“空”，如图 2-3（b）所示。

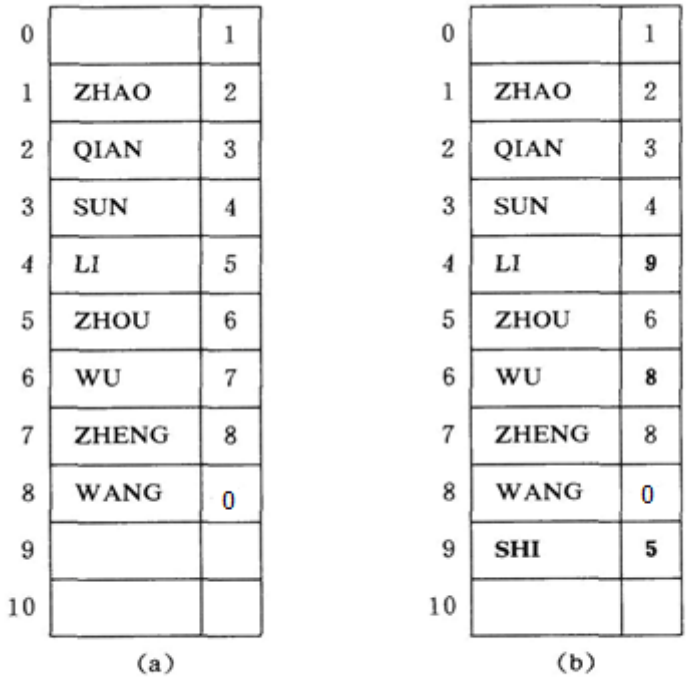


(a) 非空表； (b) 空表。

图 2-3 带头结点的单链表

4．静态链表

（1）这种存储结构需要预先分配一个较大的空间，但在做线性表的插入和删除操作时不需移动元素，仅需修改指针，故仍具有链式存储结构的主要优点。例如，图 2-4（b）展示了图 2-4（a）所示线性表在插入数据元素“SHI”和删除数据元素“ZHENG”之后的状况。为了和指针型描述的线性链表相区别，将这种用数组描述的链表称作静态链表。



0		1
1	ZHAO	2
2	QIAN	3
3	SUN	4
4	LI	5
5	ZHOU	6
6	WU	7
7	ZHENG	8
8	WANG	0
9		
10		

(a)

0		1
1	ZHAO	2
2	QIAN	3
3	SUN	4
4	LI	**9**
5	ZHOU	6
6	WU	**8**
7	ZHENG	8
8	WANG	0
9	**SHI**	**5**
10		

(b)

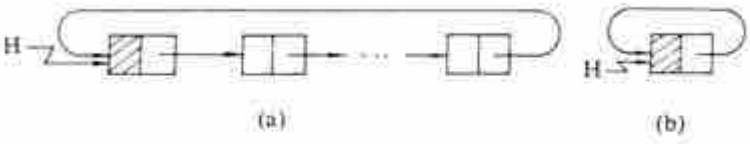
(a) 修改前的状态；(b) 修改后的状态

图 2-4 静态链表示例

（2）实现静态链表的插入和删除操作。首先将所有未被使用过以及被删除的分量用游标链成一个备用的链表，每当进行插入时便可从备用链表上取得第一个结点作为待插入的新结点；反之，在删除时将从链表中删除下来的结点链接到备用链表上。

5．循环链表

循环链表（circular linked list）是另一种形式的链式存储结构。它的特点是表中最后一个结点的指针域指向头结点，整个链表形成一个环。由此，从表中任一结点出发均可找到表中其它结点，如图 2-5 所示为单链的循环链表。类似地，还可有多重链的循环链表。



(a) 非空表；(b) 空表。

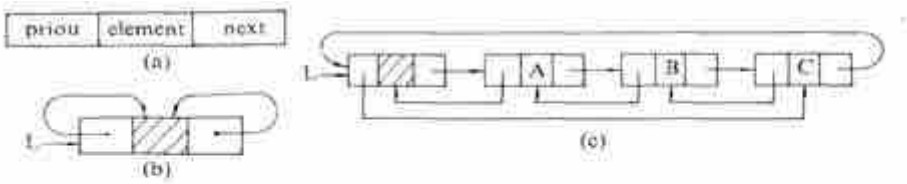
图 2-5 单循环链表

6．双向链表

在单链表中，NextElem 的执行时间为 O（1），而 PriorElem 的执行时间为为 O（n），为克服单链表这种单向性的缺点，可利用双向链表（double linked list）。

顾名思义，在双向链表的结点中有两个指针域，一个指向直接后继，另一个指向直接前驱。

双向链表也可以有循环链表，如图 2-6（c）所示，链表中存有两个环，图 2-6（b）所示为只有一个表头结点的空表。



（a）结点结构；（b）空的双向循环链表；
（c）非空的双向循环链表。

图 2-6 双向链表示例

在双向链表中，有些操作如：ListLength、GetElem、LocateElem 等仅需涉及一个方向的指针，则它们的算法描述和线性链表的操作相同，但在插入、删除时有很大的不同，在双向链表中需同时修改两个方向上的指针，图 2-7 和图 2-8 分别显示了删除和插入结点时指针修改的情况。

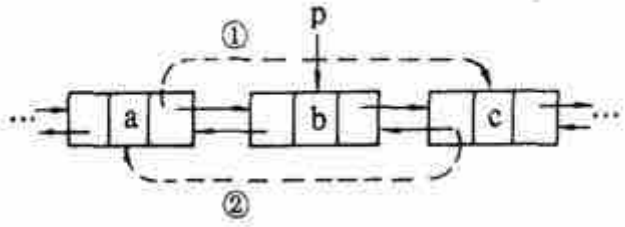


图 2-7 双向链表中删除节点时指针变化情况

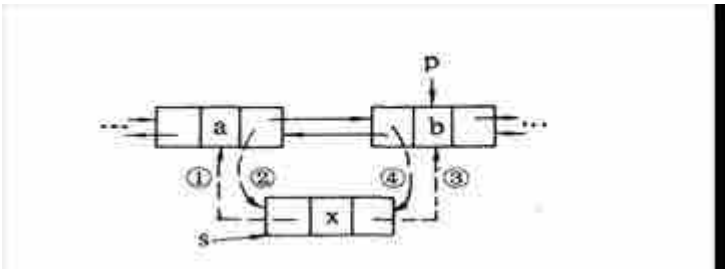


图 2-8 双向链表中插入结点时指针变化情况

四、一元多项式的表示及相加

1．一元多项式的表示

（1）一元多项式 P_x（x）按升幂表示

$$P_n(x)=p_0+p_1x+p_2x^2+\ldots+p_nX^n$$

（2）用线性表 P 表示

$$P=(p_0,\ p_1,\ p_2,\ \ldots,\ p_n)$$

2．一元多项式的相加

已知 Q_m（x）是一元 m 次多项式，同样可用线性表 Q 来表示：

$$Q=(q_0,\ q_1,\ q_2,\ \ldots,\ q_m)$$

不失一般性，设 m<n，则两个多项式相加的结果 $R_n(x)=P_n(x)+R_n(x)$ 可用线性表 R 表示：

$$R=(p_0+q_0,\ p_1+q_1,\ p_2+q_2,\ \ldots,\ p_m+q_m,\ p_{m+1},\ \ldots,\ p_n)$$

2.2 强化习题详解

【基础知识题】

1．描述以下三个概念的区别：头指针，头结点，首元结点（第一个元素结点）。

答：首元结点是指链表中存储第一个数据元素 a1 的结点.为了操作方便，通常在链表的受援结点之前附设一个结点，称为头节点，该节点的数据域中不存储线性表的数据元素，起作用是对链表进行操作时，可以对空表、非空表的情况以及对首元结点进行统一处理。头指针是指向链表中第一个结点的指针。若链表中附设头节点，则不管线性表是否为空表，头指针皆不为空，否则表示空表的链表的头指针为空。

2．填空题。

（1）在顺序表中插入或删除一个元素，需要平均移动　元素，具体移动的元素个数与　有关。

（2）顺序表中逻辑上相邻的元素的物理位置 紧邻。单链表中逻辑上相邻的元素的物理位置 紧邻。

（3）在单链表中，除了首元结点外，任一结点的存储位置由　指示。

（4）在单链表中设置头结点的作用是　。

答：（1）表中一半；元素在表中的位置。

（2）必定；不一定。

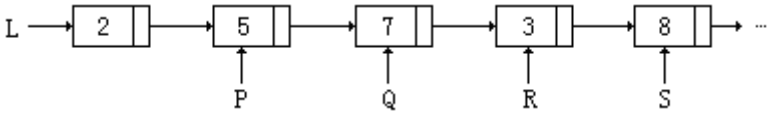
（3）其前驱结点的指针域的值。

（4）插入和删除首元结点时不用进行特殊处理。

3．在什么情况下用顺序表比链表好？

答：顺序表中逻辑关系上相邻的两个元素在物理位置上也相邻。因此当数据元素在物理位置上是连续存储的时候，或者需要对数据元素进行随机存储时，用顺序表比用链表好。

4．对以下单链表分别执行下列各程序段，并画出结果示意图。



（1）Q=P->next；

（2）L=P->next；

（3）R->data=P->data；

（4）R->data=P->next->data；

（5）P->next->next->next->ata=P->data；

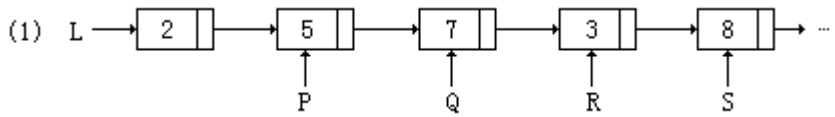
（6）T=P；

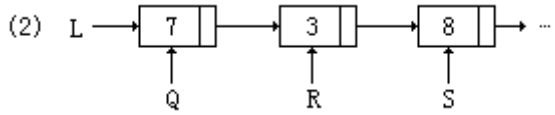
While（T!=NULL） {T->data=T->data*2；T=T->next；}

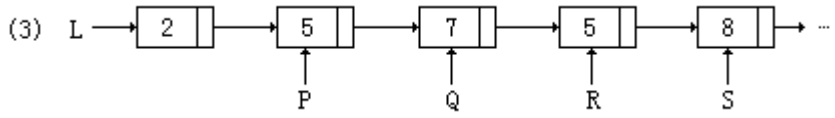
（7）T=P；

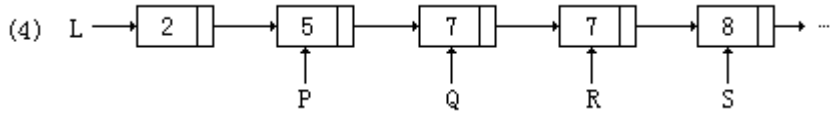
While（T->next!=NULL） {T->data=T->data*2；T=T->next；}

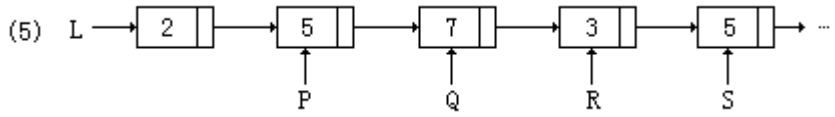
答：

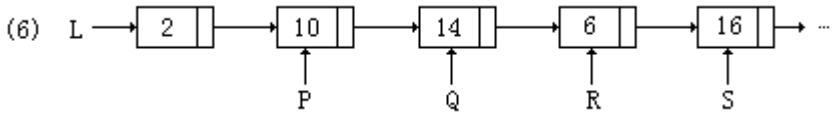


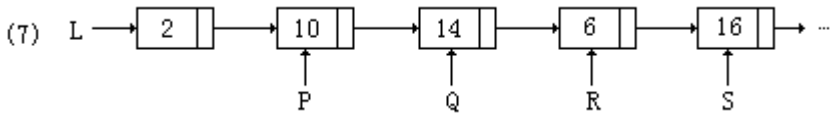








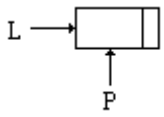


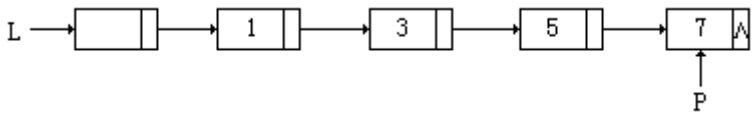


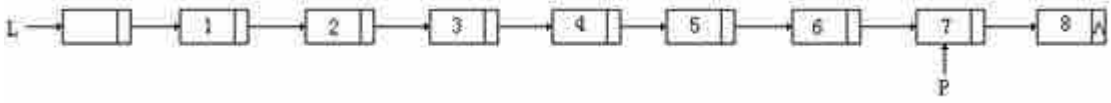
5．画出执行下列各行语句后各指针及链表的示意图。

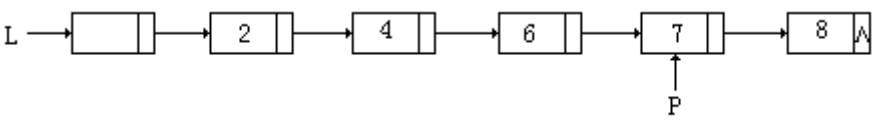
```
L=（LinkList）malloc（sizeof（LNode））；   P=L;
for（i=1;i<=4;i++）{
P->next=（LinkList）malloc（sizeof（LNode））；
P=P->next; P->data=i*2-1;
}
P->next=NULL;
for（i=4;i>=1;i--）  Ins_LinkList（L,i+1,i*2）；
for（i=1;i<=3;i++）  Del_LinkList（L,i）；
```

答：









6．已知 L 是无表头结点的单链表，且 P 结点既不是首元结点，也不是尾元结点，试从下列提供的答案中选择合适的语句序列。

a．在 P 结点后插入 S 结点的语句序列是_____。

b．在 P 结点前插入 S 结点的语句序列是_____。

c．在表首插入 S 结点的语句序列是_____。

d．在表尾插入 S 结点的语句序列是_____。

（1）P->next=S；

（2）P->next=P->next->next；

（3）P->next=S->next；

（4）S->next=P->next；

（5）S->next=L；

（6）S->next=NULL；

（7）Q=P；

（8）while（P->next!=Q）P=P->next；

（9）while（P->next!=NULL）P=P->next；

（10）P=Q；

（11）P=L；

（12）L=S；

（13）L=P；

答：a．（4）（1）；

b．（7）（11）（8）（4）（1）；

c．（5）（12）；

d．（9）（1）（6）。

7．已知 L 是带表头结点的非空单链表，且 P 结点既不是首元结点，也不是尾元结点，试从下列提供的答案中选择合适的语句序列。

a．删除 P 结点的直接后继结点的语句序列是_____。

b．删除 P 结点的直接前驱结点的语句序列是_____。

c．删除 P 结点的语句序列是_____。

d．删除首元结点的语句序列是_____。

e．删除尾元结点的语句序列是_____。

（1）P=P->next；

（2）P->next=P；

（3）P->next=P->next->next；

（4）P=P->next->next；

（5）while（P!=NULL）P=P->next；

（6）while（Q->next!=NULL）{ P=Q；　Q=Q->next；　}

（7）while（P->next!=Q）P=P->next；

（8）while（P->next->next!=Q）P=P->next；

（9）while（P->next->next!=NULL）P=P->next；

（10）Q=P；

（11）Q=P->next；

（12）P=L；

（13）L=L->next；

（14）free（Q）；

答：a. （11）（3）（14）；
b. （10）（12）（8）（11）（3）（14）；
c. （10）（12）（7）（3）（14）；
d. （12）（11）（3）（14）；
e. （9）（11）（3）（14）。
8. 已知 P 结点是某双向链表的中间结点，试从下列提供的答案中选择合适的语句序列。
a. 在 P 结点后插入 S 结点的语句序列是_____。
b. 在 P 结点前插入 S 结点的语句序列是_____。
c. 删除 P 结点的直接后继结点的语句序列是_____。
d. 删除 P 结点的直接前驱结点的语句序列是_____。
e. 删除 P 结点的语句序列是_____。
（1）P->next=P->next->next；
（2）P->priou=P->priou->priou；
（3）P->next=S；
（4）P->priou=S；
（5）S->next=P；
（6）S->priou=P；
（7）S->next=P->next；
（8）S->priou=P->priou；
（9）P->priou->next=P->next；
（10）P->priou->next=P；
（11）P->next->priou=P；
（12）P->next->priou=S；
（13）P->priou->next=S；
（14）P->next->priou=P->priou；
（15）Q=P->next；
（16）Q=P->priou；
（17）free（P）；
（18）free（Q）；
答：a. （7）（3）（6）（12）；
b. （8）（4）（5）（13）；
c. （15）（1）（11）（18）；
d. （16）（2）（10）（18）；
e. （14）（9）（17）。
9. 简述以下算法的功能。

```
（1） Status A（LinkedList L） { //L 是无表头结点的单链表
  if（L && L->next） {
  Q=L; L=L->next; P=L;
  while（P->next） P=P->next;
  P->next=Q;   Q->next=NULL;
  }
  return OK;
} //A
（2） void BB（LNode *s, LNode *q） {
```

```
    p=s;
    while（p->next!=q） p=p->next;
    p->next =s;
} //BB
void AA（LNode *pa, LNode *pb） {
    //pa 和 pb 分别指向单循环链表中的两个结点
    BB（pa,pb）;
    BB（pb,pa）;
} //AA
```

答：（1）L 的长度大于 1 时（条件判断语句“if（L&&L->next）”），将 L 的首元结点变成尾元结点。

（2）将单循环链表拆成两个单循环链表。

10．指出以下算法中的错误和低效（即费时）之处，并将它改写为一个既正确又高效的算法。

```
Status DeleteK（SqList &a,int i,int k）
{
//本过程从顺序存储结构的线性表 a 中删除第 i 个元素起的 k 个元素
if（i<1||k<0||i+k>a.length） return INFEASIBLE;//参数不合法
else {
for（count=1;count<k;count++）{
//删除第一个元素
for（j=a.length;j>=i+1;j- -） a.elem[j-i]=a.elem[j];
    a.length--;
}
return OK;
} //Delete OK
```

答：错误有两处：

（1）参数不合法的判别条件不完整。合法的入口参数为

（0<i<=a.length）∧（0<=k<=a.length-i）

（2）内层 for 循环语句错误，元素前移的次序错误。

低效之处：每次删除一个元素的策略。

程序改写为：

```
Status DeleteK（SqList &a,int i,int k）
{
    //从顺序存储结构的线性表 a 中删除从第 i 个元素起的 k 个元素
    //注意 i 的编号从 0 开始
    int j;
    if（i<0||i>a.length-1||k<0||k>a.length-i） return INFEASIBLE;
    for（j=0;j<=k;j++）
    a.elem[j+i-1]=a.elem[j+i+k-1];
    a.length=a.length-k;
    return OK;
}
```

【算法设计题】

11．设顺序表 va 中的数据元素递增有序。试写一算法，将 x 插入到顺序表的适当位置上，以保持该表的有序性。

答：

```
Status InsertOrderList（SqList &va,ElemType x）
{
   //在递增有序的顺序表 va 中插入元素 x，使其仍为顺序表的算法
   int i;
//如果表的长度已经等于表的容量，不能再进行插入，直接返回，否则会溢出
   if（va.length= =va.listsize）return（OVERFLOW）;
//从表尾依次往前循环比较
   for（i=va.length;i>0,x<va.elem[i-1];i--）
va.elem[i]=va.elem[i-1];
   va.elem[i]=x;
   va.length++;
   return OK;
}
```

12．设 $A=(a_1,\cdots,a_m)$ 和 $B=(b_1,\cdots,b_n)$ 均为顺序表，A' 和 B' 分别为 A 和 B 中除去最大共同前缀后的子表（例如，A=（x，y，y，z，x，z），B=（x，y，y，z，y，x，x，z），则两者中的最大共同前缀为（x，y，y，z），在两表中除去最大共同前缀后的子表分别为 $A'=(x,z)$ 和 $B'=(y,x,x,z)$）。若 $A'=B'=$空表，则 $A=B$；若 $A'=$空表，而 $B'\neq$空表，或者两者均不为空表，且 A' 的首元小于 B' 的首元，则 $A<B$；否则 $A>B$。试写一个比较 A，B 大小的算法（请注意：在算法中，不要破坏原表 A 和 B，并且，也不一定先求得 A' 和 B' 才进行比较）。

答：Status CompareOrderList（SqList &A,SqList &B）

```
//比较字符表 A 和 B,并用返回值 j 表示结果,值为 1,表示 A>B;值为-1,表示 A<B;值为零，
//表示 A=B
{
int i,k,j;
//将表 A 和表 B 较大的长度值赋给变量 k，以便在循环条件中使用
k=A.length>B.length?A.length:B.length;
for（i=0;i<k;i++）{
if（A.elem[i]>B.elem[i]） j=1;
if（A.elem[i]<B.elem[i]） j=-1;
}
if（A.length>k） j=1;
if（B.length>k） j=-1;
if（A.length==B.length） j=0;
return j;
}
```

13．试写一算法在带头结点的单链表结构上实现线性表操作 LOCATE（L,x）。

答：LNode* Locate（LinkList L,int x）

```
//链表上的元素查找,返回指针，该指针指向元素 x
```

```
{
for（p=L->next;p&&p->data!=x;p=p->next）;
return p;
}//Locate
```

14．试写一算法在带头结点的单链表结构上实现线性表操作 LENGTH（L）。

答：

```
//返回单链表的长度
int Length（LinkList L）
{
//判断指针 p 的后继只要不为空，说明还未达到链表表尾，继续循环判断
for（k=0,p=L;p->next;p=p->next,k++）;
return k;
}//Length
```

15．已知指针 ha 和 hb 分别指向两个单链表的头结点，并且已知两个链表的长度分别为 m 和 n。试写一算法将这两个链表连接在一起（即令其中一个表的首元结点连在另一个表的最后一个结点之后），假设指针 hc 指向连接后的链表的头结点，并要求算法以尽可能短的时间完成连接运算。请分析你的算法的时间复杂度。

答：

```
void ListConcat（LinkList ha,LinkList hb,LinkList &hc）
//把链表 hb 接在 ha 后面形成链表 hc
{
hc=ha;p=ha;
while（p->next） p=p->next;//循环，直到达到表 ha 的最后一个结点
//将 ha 最后一个结点的指针指向表 hb 的头指针，即将表 hb 链接在表 ha 后面
p->next=hb;
}//ListConcat
```

基本运算语句为“p=p->next;”，被重复执行的次数为链表 ha 的长度，因此时间复杂度为 T（n）=O（n）。

16．已知指针 la 和 lb 分别指向两个无头结点单链表中的首元结点。下列算法是从表 la 中删除自第 i 个元素起共 len 个元素后，将它们插入到表 lb 中第 i 个元素之前。试问此算法是否正确？若有错，请改正之。

```
Status DeleteAndInsertSub（LinkedList la,LinkedList lb,int i,int j,int len）
{
if（i<0||j<0||len<0） return INFEASIBLE;
p=la;  k=1;
while（k<i）{ p=p->next; k++;}
q=p;
while（k<=len）{ q=q->next; k++;}
s=lb; k=1;
while（k<j）{ s=s->next; k++;}
s->next=p; q->next=s->next;
return OK;
} //DeleteAndInsertSub
```

答：不正确。

```
Status DeleteAndInsertSub（LinkList &la,LinkList &lb,int i,int j,int len）
{
```

```
if（i<0||j<0||len<0） return INFEASIBLE;
//在 la 表中查找第 i 个结点
p=la; k=1; prev=NULL;
while（p&&k<i）
{//循环条件应该加上指针 p 即链表 la 的非空判断
prev=p;
p=p->next;
k++;
}
if（!p）return INFEASIBLE;
//在 la 表中查找第 i+len-1 个结点
q=p; k=1;
while（q&&k<len）
{ //循环条件应该加上指针 q 即链表 la 的非空判断
q=p->next;
k++;
}
if（!q） return INFEASIBLE;
//完成删除
if（!prev） la=q->next;
else prev->next=q->next;
//将从 la 中删除的结点插入到 lb 中
if（j==1）
{//j=1 的情况需要特殊处理
q->next=lb;
lb=p;
}
else{
s=lb;   k=1;
while（s&&k<j-1）
{ //循环条件应该加上指针 s 即链表 lb 的非空判断
   s=s->next;
   k++;
}
if（!s）return INFEASIBLE;
q->next=s->next;
s->next=p; //完成插入
}
return OK;
}
```

17．试写一算法，在无头结点的动态单链表上实现线性表操作 INSERT（L，i，b），并和在带头结点的动态单链表上实现相同操作的算法进行比较。

答：

```
Status Insert（LinkList &L,int i,int b）
//在无头结点链表 L 的第 i 个元素之前插入元素 b
```

```
{
p=L;
q=（LinkList*）malloc（sizeof（LNode））;//动态创建结点 q
q.data=b;//元素 b 即为结点 q 的数据域
if（i= =1）//考虑首元结点
{
q.next=p;L=q; //q 插入在链表头部
}
else
{
while（--i>1） p=p->next;
q->next=p->next;p->next=q; //插入在第 i 个元素的位置
}
}//Insert
```

头结点的作用是插入和删除操作时不必对首元结点做特殊处理，因此在头结点的动态单链表上实现删除操作不必单独考虑 i=1 时的特殊情况，可统一处理。

18．同 17 题要求。试写一算法，实现线性表操作 DELETE（L,i）。

答：

```
Status Delete（LinkList &L,int i）
//在无头结点链表 L 中删除第 i 个元素
{
p=L;
if（i= =1）
{//考虑首元结点
L=L->next; //删除第一个元素
free（p）;//释放存储空间
}
else
{
while（--i>1） p=p->next;
q=p->next
p->next=p->next->next; //删除第 i 个元素
free（q）;
}
}//Delete
```

19．已知线性表中的元素以值递增有序排列，并以单链表作存储结构。试写一高效的算法，删除表中所有值大于 mink 且小于 maxk 的元素（若表中存在这样的元素），同时释放被删结点空间，并分析你的算法的时间复杂度（注意，mink 和 maxk 是给定的两个参变量，它们的值可以和表中的元素相同，也可以不同）。

答：合法的入口条件只要求线性表不空。若 mink>maxk 则表明代删元素集为空集。

```
Status ListDelete_L（LinkList &L,ElemType mink,ElemType maxk）
{//删除元素按递增排列的链表 L 中值大于 mink 且小于 maxk 的所有元素
    LinkList p,q,prev=NULL;
if（mink>maxk） return ERROR;//[mink,maxk]区间错误，退出程序
p=L;
```

```
prev=p;//prev 为 p 的前驱
p=p->next;
while（p&&p->data<maxk）
{//p 的数据域要小于 maxk
if（p->data<=mink）
{// p 的数据域要小于 mink
    prev=p;
    p=p->next; //指针后移，继续比较
}
else{//如果同时 p 的数据域大于 mink
    prev->next=p->next;
    q=p;
    p=p->next;//删除 p 所指的元素结点
    free（q）;//释放存储空间
}
}
return OK;
}
```

时间复杂度为 T（n）=O（n^2）

20．同 19 题条件，试写一高效的算法，删除表中所有值相同的多余元素（使得操作后的线性表中所有元素的值均不相同），同时释放被删结点空间，并分析你的算法的时间复杂度。

答：

```
Status Delete_Equal（Linklist &L）    //删除元素按递增排列的链表 L 中所有值相同
//的元素
{
p=L->next;q=p->next; //p,q 指向相邻两元素，其中 q 为 p 的后继结点
while（p->next）
{
if（p->data!=q->data）
    {
p=p->next;
    q=p->next; //当相邻两元素不相等时,p 和 q 都向后移动一个结点位置
}
else
{//相邻两个元素相等
    while（q->data==p->data）
    {//比较 q 后面的元素是否还和 p 与 q 元素相等，如果相等，则删除
    r=q;
    q=q->next;
    free（r）;//释放存储空间
    }
    p->next=q;p=q;q=p->next; //当相邻元素相等时删除多余元素
}//else
```

}//while
}//Delete_Equal
时间复杂度为 T（n）=O（n^3）
21．试写一算法，实现顺序表的就地逆置，即利用原表的存储空间将线性表 $(a_1,a_2,\cdots,a_n)$ 逆置为 $(a_n,a_{n-1},\cdots,a_1)$。
答：//顺序表的逆置
```
Status ListOppose_Sq（SqList &L）
{
int i;
ElemType x;//链表元素 x，是一个中间变量，交换元素时用
for（i=0;i<L.length/2;i++）
{
//交换第 i 个元素和第 L.length-1-i 个元素，实现逆置
x=L.elem[i];
L.elem[i]=L.elem[L.length-1-i];
L.elem[L.length-1-i]=x;
}
return OK;
}
```
22．试写一算法，对单链表实现就地逆置。
答：将原连表中的头节点和第一个元素结点断开，先构成一个新的空表，然后将原链表中各节点，从第一个结点起插入这个新表的头部。
```
//链表的就地逆置;为简化算法,假设表长大于 2
void LinkList_reverse（Linklist &L）
{
   //定义 q 为 p 的后继，s 为 q 的后继
p=L->next;q=p->next;s=q->next;p->next=NULL;
while（s->next）
{
q->next=p;p=q;
q=s;s=s->next; //把 L 的元素逐个插入新表表头
}
q->next=p;s->next=q;L->next=s;
}//LinkList_reverse
```
23．设线性表 $A=(a_1,a_2,\cdots,a_m)$，$B=(b_1,b_2,\cdots,b_n)$，试写一个按下列规则合并 A，B 为线性表 C 的算法，即使得 $C=(a_1,b_1,\cdots,a_m,b_m,b_{m+1},\cdots,b_n)$ 当 $m\le n$ 时；

或者 $C=(a_1,b_1,\cdots,a_n,b_n,a_{n+1},\cdots,a_m)$ 当 $m>n$ 时。

线性表 A，B 和 C 均以单链表作存储结构，且 C 表利用 A 表和 B 表中的结点空间构成。注意：单链表的长度值 m 和 n 均未显式存储。

答：//将合并后的结果放在 C 表中，并删除 B 表

```
Status ListMerge_L（LinkList &A,LinkList &B,LinkList &C）
{
LinkList pa,pb,qa,qb;
pa=A->next;//pa 为链表 A 的首元结点
pb=B->next; //pb 为链表 B 的首元结点
C=A;
while（pa&&pb）{
qa=pa;
qb=pb;
pa=pa->next;
pb=pb->next;
qb->next=qa->next;
qa->next=qb; //将链表 B 中的元素链接到链表 A 后面
}
if（!pa）qb->next=pb;
pb=B;
free（pb）;//将链表 B 删除，释放表 B 的表头
return OK;
}
```

24．假设有两个按元素值递增有序排列的线性表 A 和 B，均以单链表作存储结构，请编写算法将 A 表和 B 表归并成一个按元素值递减有序（即非递增有序，允许表中含有值相同的元素）排列的线性表 C，并要求利用原表（即 A 表和 B 表）的结点空间构造 C 表。

答：采用“指针平移，一次扫描”的策略。

```
//将合并逆置后的结果放在 C 表中
Status ListMergeOppose_L（LinkList &A,LinkList &B,LinkList &C）
{
LinkList pa,pb,qa,qb;
pa=A;
pb=B;
qa=pa; // qa 保存 pa 的前驱指针
qb=pb;    // qb 保存 pb 的前驱指针
pa=pa->next;
pb=pb->next;
A->next=NULL;
C=A;
while（pa&&pb）{
if（pa->data<pb->data）{//若表 A 当前结点小于表 B 当前结点，        qa=pa;
    pa=pa->next;
    qa->next=A->next;
    A->next=qa;
}
```

```
else{//若表 B 当前结点小于表 A 当前结点
    qb=pb;
    pb=pb->next;
    qb->next=A->next;
A->next=qb;
}
}
while（pa）{//表 A 中仍有结点
qa=pa;
pa=pa->next;
qa->next=A->next;
A->next=qa;//将表 A 剩余的所有结点链接到 C 表
}
while（pb）{//表 B 中仍有结点
qb=pb;
pb=pb->next;
qb->next=A->next;//将表 B 剩余的所有结点链接到 C 表
A->next=qb;
}
pb=B;
free（pb）;//释放表 B 表头
return OK;
}
```

25．假设以两个元素依值递增有序排列的线性表 A 和 B 分别表示两个集合（即同一表中的元素值各不相同），现要求另辟空间构成一个线性表 C，其元素为 A 和 B 中元素的交集，且表 C 中的元素有依值递增有序排列。试对顺序表编写求 C 的算法。

答：

```
//将 A、B 求交后的结果放在 C 表中
//线性表从 0 开始存储
Status ListCross_Sq（SqList &A,SqList &B,SqList &C）
{
int i=0,j=0;
while（i<A.length && j<B.length）{
//如果表 A 中元素值较小，表 A 元素下标增 1，即向后移动一个位置
if（A.elem[i]<B.elem[j]）    i++;
else
    //如果表 A 中元素值较小，表 A 元素下标增 1，即向后移动一个位置
    if（A.elem[i]>B.elem[j]）    j++;
    else{
//表 A 和表 B 中的元素值相等，即得到 A 和 B 的交集元素，放到 C 中
    ListInsert_Sq（C,k,A.elem[i]）;
    i++;
    j++;
    }
}
```

```
return OK;
}
```
26．要求同 25 题。试对单链表编写求 C 的算法。

答：
```
//将 A、B 求交后的结果放在 C 表中，并删除 B 表
Status ListCross_L（LinkList &A,LinkList &B,LinkList &C）
{
LinkList pa,pb,qa,qb,pt;
pa=A;
pb=B;
qa=pa; // qa 保存 pa 的前驱指针
qb=pb;    // qb 保存 pb 的前驱指针
pa=pa->next;
pb=pb->next;
C=A;
while（pa&&pb）{
//如果链表 A 中数据域的值较小，指向链表 A 的指针向后移动一个位置
if（pa->data<pb->data）
{
    pt=pa;
    pa=pa->next;
    qa->next=pa;
    free（pt）;
}
else
//如果链表 B 中数据域的值较小，指向链表 B 的指针向后移动一个位置
    if（pa->data>pb->data）{
    pt=pb;
    pb=pb->next;
        qb->next=pb;
    free（pt）;
    }
    else{
//两个链表数据域的值相等，即为交集元素，放到 C 中
    qa=pa;
    pa=pa->next;
    }
}
while（pa）{//链表 A 有剩余结点
pt=pa;
pa=pa->next;
qa->next=pa;
free（pt）;//释放链表 A 的表头
}
while（pb）{//链表 B 有剩余结点
```

```
pt=pb;
pb=pb->next;
qb->next=pb;
free（pt）;//释放链表 B 的表头
}
pb=B;
free（pb）;//删除链表 B
return OK;
}
```

27．对 25 题的条件作以下两点修改，对顺序表重新编写求得表 C 的算法。

（1）假设在同一表（A 或 B）中可能存在值相同的元素，但要求新生成的表 C 中的元素值各不相同；

（2）利用 A 表空间存放表 C。

答：（1）

```
Status ListCrossDelSame_Sq（SqList &A,SqList &B,SqList &C） // A、B 求交，然后删//除相同元素，将结果放在 C 表中
{
int i=0,j=0,k=0;
while（i<A.length && j<B.length）{
if（A.elem[i]<B.elem[j]）    i++;//A 表中元素较小，A 下标增 1，向后移动
else
   if（A.elem[i]>B.elem[j]）    j++;//B 表中元素较小，B 下标增 1，向后移动
   else{
   if（C.length==0）{//如果 C 表为空表，直接依次插入
ListInsert_Sq（C,k,A.elem[i]）;
k++;
   }
else//否则，判断没有重复元素，再插入
if（C.elem[C.length-1]!=A.elem[i]）{
ListInsert_Sq（C,k,A.elem[i]）;
k++;
}
   i++;
   }
}
return OK;
}
```

（2）

```
// A、B 求交，然后删除相同元素，将结果放在 A 表中
Status ListCrossDelSame_Sq（SqList &A,SqList &B）
{
int i=0,j=0,k=0;
while（i<A.length && j<B.length）{
if（A.elem[i]<B.elem[j]）    i++;//A 表中元素较小，A 下标增 1，向后移动
else
```

```
    if（A.elem[i]>B.elem[j]）    j++;//B 表中元素较小，B 下标增 1，向后移动
    else{
    if（k==0）{//将相同元素放到 A 表中
A.elem[k]=A.elem[i];
k++;
    }
    else//判断是否是重复元素
if（A.elem[k]!=A.elem[i]）{
A.elem[k]=A.elem[i];
k++;
}
    i++;
    }
}
A.length=k;
return OK;
}
```

28．对 25 题的条件作以下两点修改，对单链表重新编写求得表 C 的算法。

（1）假设在同一表（A 或 B）中可能存在值相同的元素，但要求新生成的表 C 中的元素值各不相同；

（2）利用原表（A 表或 B 表）中的结点构成表 C，并释放 A 表中的无用的结点空间。

答：（1）

```
Status ListCrossDelSame_L（LinkList &A,LinkList &B,LinkList &C） // A、B 求交，结果放
//在 C 表中，并删除相同元素
{
LinkList pa,pb,qa,qb,pt;
pa=A;
pb=B;
qa=pa; // qa 保存 pa 的前驱指针
qb=pb;   // qb 保存 pb 的前驱指针
pa=pa->next;
pb=pb->next;
C=A;// C 指向链表 A
while（pa&&pb）
{
if（pa->data<pb->data）
{
    pt=pa;
    pa=pa->next;
    qa->next=pa;
    free（pt）;
}
else
    if（pa->data>pb->data）
```

```
{//如果 A 中元素值大于 B 中元素值
    pt=pb;
    pb=pb->next;
    qb->next=pb;
    free（pt）;//释放 B 中该元素结点
    }
    else{
    if（pa->data==qa->data）
{ //如果 B 中元素值大于 A 中元素值
pt=pa;
pa=pa->next;
qa->next=pa;
free（pt）; //释放 A 中该元素结点
    }
    else{
qa=pa;
pa=pa->next;
    }
    }
}
while（pa）
{
pt=pa;
pa=pa->next;
qa->next=pa;
free（pt）;
}
while（pb）
{
pt=pb;
pb=pb->next;
qb->next=pb;
free（pt）;
}
pb=B;
free（pb）;
return OK;
}
```

（2）// A、B 求交，结果放在 A 表中，并删除相同元素

```
Status ListCrossDelSame_L（LinkList &A,LinkList &B）
{
LinkList pa,pb,qa,qb,pt;
pa=A;
pb=B;
```

```
qa=pa; // qa 保存 pa 的前驱指针
qb=pb;    // qb 保存 pb 的前驱指针
pa=pa->next;
pb=pb->next;
while（pa&&pb）
{
if（pa->data<pb->data）
{//如果 A 中元素小于 B 中元素
    pt=pa;
    pa=pa->next;
    qa->next=pa;
    free（pt）;//释放 A 中该元素结点
}
else
    if（pa->data>pb->data）
{ //如果 B 中元素小于 A 中元素
    pt=pb;
    pb=pb->next;
    qb->next=pb;
    free（pt）; //释放 B 中该元素结点
    }
    else{
    if（pa->data==qa->data）
{
pt=pa;
pa=pa->next;
qa->next=pa;
free（pt）;
    }
    else{
qa=pa;
pa=pa->next;
    }
    }
}
while（pa）{
pt=pa;
pa=pa->next;
qa->next=pa;
free（pt）;
}
while（pb）{
pt=pb;
pb=pb->next;
```

qb->next=pb;
free（pt）;
}
pb=B;
free（pb）;
return OK;
}
29．已知 A，B 和 C 为三个递增有序的线性表，现要求对 A 表作如下操作：删去那些既在 B 表中出现又在 C 表中出现的元素。试对顺序表编写实现上述操作的算法，并分析你的算法的时间复杂度（注意：题中没有特别指明同一表中的元素值各不相同）。
答：void SqList_Intersect_Delete（SqList &A,SqList B,SqList C）
{
i=0;j=0;k=0;m=0;　//i 指示 A 中元素原来的位置,m 为移动后的位置
while（i<A.length&&j<B.length&& k<C.length）
{
if（B.elem[j]<C.elem[k]） j++;
else if（B.elem[j]>C.elem[k]） k++;
else　{ same=B.elem[j];　//找到了相同元素 same while（B.elem[j]= =same） j++;
while（C.elem[k]= =same） k++;　//j,k 后移到新的元素 while（i<A.length&&A.elem[i]
<same）　A.elem[m++]=A.elem[i++];　//需保留的元素移动到新位置
while（i<A.length&&A.elem[i]==same） i++; //跳过相同的元素　} }//while
while（i<A.length）　A.elem[m++]=A.elem[i++]; //A 的剩余元素重新存储。 A.length=m; }
// SqList_Intersect_Delete
分析:先从 B 和 C 中找出共有元素，记为 same，并记下位置，在 A 中从当前位置开始，凡小于 same 的元素均保留（存到新的位置），等于 same 的就跳过，到大于 same 时就再找下一个 same。时间复杂度为 T（n）=O（n^3）。
30．要求同 29 题。试对单链表编写算法，请释放 A 表中的无用结点空间。
答：// 在 A 中删除既在 B 中出现又在 C 中出现的元素，并释放 B、C
Status ListUnion_L（LinkList &A,LinkList &B,LinkList &C）
{
ListCross_L（B,C）;
ListMinus_L（A,B）;
}
Status ListMinus_L（LinkList &A,LinkList &B）//求集合 A-B，结果放在 A 表中，并删//除 B 表
{
LinkList pa,pb,qa,qb,pt;
pa=A;
pb=B;
qa=pa; // qa 保存 pa 的前驱指针
qb=pb;　// qb 保存 pb 的前驱指针
pa=pa->next;
pb=pb->next;
while（pa&&pb）{

```
if（pb->data<pa->data）
{//如果 B 中元素小于 A 中元素
    pt=pb;
    pb=pb->next;
    qb->next=pb;
    free（pt）;//释放 B 中该元素结点
}
else
    if（pb->data>pa->data）{ //如果 B 中元素大于 A 中元素
    qa=pa;
    pa=pa->next;
    }
    else{
    pt=pa;
    pa=pa->next;
    qa->next=pa;
    free（pt）; //释放 A 中该元素结点
    }
}
while（pb）{
pt=pb;
pb=pb->next;
qb->next=pb;
free（pt）;
}
pb=B;
free（pb）;//释放 B 的首元结点，即删除 B
return OK;
}
```

时间复杂度为 T（n）=O（n^3）

31．假设某个单向循环链表的长度大于 1，且表中既无头结点也无头指针。已知 s 为指向链表中某个结点的指针，试编写算法在链表中删除指针 s 所指结点的前驱结点。

答：

```
// 在单循环链表 s 中删除 s 的前驱结点
Status ListDelete_CL（LinkList &s）
{
LinkList p,q;
if（s= =s->next）return ERROR;//s 为空表
q=s;
p=s->next;//q 为 p 的前驱,p 为 s 的后继
while（p->next!=s）{
q=p;
p=p->next;
}
q->next=p->next;
```

free（p）; //删除并释放结点 p
return OK;
}
32．已知有一个单向循环链表，其每个结点中含三个域：prior，data 和 next，其中 data 为数据域，next 为指向后继结点的指针域，prior 也为指针域，但它的值为空（NULL），试编写算法将此单向循环链表改为双向循环链表，即使 prior 成为指向前驱结点的指针域。
答：Status DuLNode_Pre（DuLinkList &L）//完成双向循环链表结点的 prior 域
{
for（p=L;!p->next->prior;p=p->next） p->next->prior=p;
return OK;
}//DuLNode_Prior
33．已知由一个线性链表表示的线性表中含有三类字符的数据元素（如：字母字符、数字字符和其他字符），试编写算法将该线性表分割为三个循环链表，其中每个循环链表表示的线性表中均只含一类字符。在 2.34 至 2.36 题中，“异或指针双向链表”类型
XorLinkedList 和指针异或函数 XorP 定义为：
typedef struct XorNode {
char data;
struct XorNode *LRPtr;
} XorNode, *XorPointer;
typede struct { //无头结点的异或指针双向链表
XorPointer Left, Right; //分别指向链表的左侧和右端
} XorLinkedList;
XorPointer XorP（XorPointer p, XorPointer q）;
// 指针异或函数 XorP 返回指针 p 和 q 的异或（XOR）值
答：Status LinkList_Divide（LinkList &L,CiList &A,CiList &B,CiList &C）
//把单链表 L 的元素按类型分为三个循环链表.CiList 为带头结点的单循环链表类型
{
s=L->next;
A=（CiList*）malloc（sizeof（CiLNode））;p=A;//动态创建头结点
B=（CiList*）malloc（sizeof（CiLNode））;q=B;
C=（CiList*）malloc（sizeof（CiLNode））;r=C; //建立头结点
while（s）
{
if（isalphabet（s->data））//数据域为字母字符
{
p->next=s;p=s;
}
else if（isdigit（s->data））{//数字字符
q->next=s;q=s;
}
else//其它字符
{
r->next=s;r=s;
}

}//while
p->next=A;q->next=B;r->next=C; //完成循环链表
}//LinkList_Divide
34．假设在算法描述语言中引入指针的二元运算“异或”（用“⊕”表示），若 a 和 b 为指针，则 a⊕b 的运算结果仍为原指针类型，且
a⊕（a⊕b）=（a⊕a）⊕b=b
（a⊕b）⊕b=a⊕（b⊕b）=a
则可利用一个指针域来实现双向链表 L。链表 L 中的每个结点只含两个域：data 域和 LRPtr 域，其中 LRPtr 域存放该结点的左邻与右邻结点指针（不存在时为 NULL）的异或。若设指针 L.Left 指向链表中的最左结点，L.Right 指向链表中的最右结点，则可实现从左向右或从右向左遍历此双向链表的操作。试写一算法按任一方向依次输出链表中各元素的值。

答：Status TraversingLinkList（XorLinkedList &L,char d）
{
XorPointer p,left,right;
if（d=="l"||d=="L"）{
p=L.Left;//p 指向链表的最左结点
left=NULL;
while（p!=NULL）{
    VisitingData（p->data）;//循环遍历
    left=p;
    p=XorP（left,p->LRPtr）;
}
}
else
if（d=="r"||d=="R"）
{
    p=L.Right; //p 指向链表的最右结点
    right=NULL;
    while（p!=NULL）{
    VisitingData（p->data）; //循环遍历
    right=p;
    p=XorP（p->LRPtr,right）;
    }
}
else return ERROR;
return OK;
}
35．采用 34 题所述的存储结构，写出在第 i 个结点之前插入一个结点的算法。
答：Status Insert_XorLinkedList（XorLinkedList &L,int x,int i）
//在异或链表 L 的第 i 个元素前插入元素 x
{
p=L.left;pre=NULL;
r=（XorNode*）malloc（sizeof（XorNode））;

r->data=x;
if（i==1） //当插入点在最左边的情况
{
p->LRPtr=XorP（p.LRPtr,r）;
r->LRPtr=p;
L.left=r;
return OK;
}
j=1;q=p->LRPtr; //当插入点在中间的情况
while（++j<i&&q）
{
q=XorP（p->LRPtr,pre）;
pre=p;p=q;
}//while //在 p,q 两结点之间插入
if（!q） return INFEASIBLE; //i 不可以超过表长
p->LRPtr=XorP（XorP（p->LRPtr,q）,r）;
q->LRPtr=XorP（XorP（q->LRPtr,p）,r）;
r->LRPtr=XorP（p,q）; //修改指针
return OK;
}//Insert_XorLinkedList

36．采用 34 题所述的存储结构，写出删除第 i 个结点的算法。

答：Status Delete_XorLinkedList（XorlinkedList &L,int i） //删除异或链表 L 的第 i 个元素 { p=L.left;pre=NULL; if（i==1） //当删除的结点在链表的最左边 { q=p->LRPtr; q->LRPtr=XorP（q->LRPtr,p）; L.left=q;free（p）;//释放结点 p return OK; } j=1;q=p->LRPtr; while（++j<i&&q） { q=XorP（p->LRPtr,pre）; pre=p;p=q; }//while //找到待删结点 q if（!q） return INFEASIBLE; //i 不可以超过表长 if（L.right==q） //q 为最右结点的情况 { p->LRPtr=XorP（p->LRPtr,q）; L.right=p;free（q）; return OK; } r=XorP（q->LRPtr,p）; //q 为中间结点的情况,此时 p,r 分别为其左右结点 p->LRPtr=XorP（XorP（p->LRPtr,q）,r）; r->LRPtr=XorP（XorP（r->LRPtr,q）,p）; //修改指针 free（q）;//释放结点 q return OK; } //Delete_XorLinkedList

37．设以带头结点的双向循环链表表示的线性表 $L=(a_1,a_2,\cdots,a_n)$。试写一时间复杂度 O（n）的算法，将 L 改造为 $L=(a_1,a_3,\cdots,a_n,\cdots,a_4,a_2)$。

答：时间复杂度为 O（n），即基本运算语句执行的次数与链表表长同数量级。

void OEReform（DuLinkedList&L）//按 1,3,5,...4,2 的顺序重排双向循环链表 L 中的所有结点
{
p=L->next;//p 为 L 的首元结点
while（p->next!=L&&p->next->next!=L）
{
p->next=p->next->next;
p=p->next;
} //此时 p 指向最后一个奇数结点

```
if（p->next==L） p->next=L->pre->pre;
else p->next=L->pre;
p=p->next; //此时 p 指向最后一个偶数结点
while（p->pre->pre!=L）
{
p->next=p->pre->pre;
p=p->next;
}
p->next=L; //按题目要求调整了 next 链的结构,此时 pre 链仍为原状
for（p=L;p->next!=L;p=p->next） p->next->pre=p;
L->pre=p; //调整 pre 链的结构
}//OEReform
```

38．设有一个双向循环链表，每个结点中除有 prior，data 和 next 三个域外，还增设了一个访问频度域 freq。在链表被起用之前，频度域 freq 的值均初始化为零，而每当对链表进行一次 LOCATE（L,x）的操作后，被访问的结点（即元素值等于 x 的结点）中的频度域 freq 的值便增 1，同时调整链表中结点之间的次序，使其按访问频度非递增的次序顺序排列，以便始终保持被频繁访问的结点总是靠近表头结点。试编写符合上述要求的 LOCATE 操作的算法。

在 2.39 至 2.40 题中，稀疏多项式采用的顺序存储结构 SqPoly 定义为

```
typedef struct {
int coef;
int exp;
} PolyTerm;
typedef struct {
PolyTerm *data; //多项式的顺序存储结构
int length;
} SqPoly;
```

答：

```
DuLinkList ListLocate_DuL（DuLinkList &L,ElemType e）
{
DuLinkList p,q;
p=L->next;
while（p!=L && p->data!=e） p=p->next;
if（p==L） return NULL;//空表，返回
else{
p->freq++;
      p->pre->next=p->next; // 删除结点
p->next->pre=p->pre;
// 插入到合适的位置
q=L->next;
while（q!=L && q->freq>p->freq） q=q->next;
if（q==L）{//q 为头结点，在 q 后面插入
   p->next=q->next;
   q->next=p;
```

```
    p->pre=q->pre;
    q->pre=p;
}
else{
    // 在 q 之前插入
    p->next=q->pre->next;
    q->pre->next=p;
    p->pre=q->pre;
    q->pre=p;
}
return p;
}
}
```

39．已知稀疏多项式 $P_n(x)=c_1x^{e_1}+c_2x^{e_2}+\cdots+c_mx^{e_m}$，其中 $n=e_m>e_{m-1}>\cdots>e_1\geq 0$，$c_i\neq 0(i=1,2,\cdots,m)$，$m\geq 1$。试采用存储量同多项式项数 m 成正比的顺序存储结构，编写求 $P_n(x_0)$ 的算法（x_0 为给定值），并分析你的算法的时间复杂度。

答：

```
typedef struct{
int coef;
int exp;
} PolyTerm;
typedef struct{
PolyTerm *data;
int last;
} SqPoly;
// 建立一个多项式
Status PolyInit（SqPoly &L）
{
int i;
PolyTerm *p;
cout<<"请输入多项式的项数:";
cin>>L->last;
L->data=（PolyTerm *）malloc（L.last*sizeof（PolyTerm））;//动态创建结点
if（!L->data） return ERROR;
p=L->data;
for（i=0;i<L.last;i++）{
cout<<"请输入系数:";
cin>>p->coef;
cout<<"请输入指数:";
cin>>p->exp;
```

```
p++;
}
return OK;
}
// 求多项式的值
double PolySum（SqPoly &L,double x0）
{
double Pn,x;
int i,j;
PolyTerm *p;
p=L->data;
for（i=0,Pn=0;i<L.last;i++,p++）
{
for（j=0,x=1;j<p->exp;j++） x=x*x0;
Pn=Pn+p->coef*x;
}
return Pn;
}
```

时间复杂度为 T（n）=O（n^3）

40．采用 39 题给定的条件和存储结构，编写求 $P(x)=P_{n1}(x)-P_{n2}(x)$ 的算法，将结果多项式存放在新辟的空间中，并分析你的算法的时间复杂度。

在 41 至 42 题中，稀疏多项式采用的循环链表存储结构 LinkedPoly 定义为

```
typedef struct PolyNode {
PolyTerm data;
Struct PolyNode *next;
} PolyNode, *PolyLink;
typedef PolyLink LinkedPoly;
```

答：

```
Status PolyMinus（SqPoly &L,SqPoly &L1,SqPoly &L2） // 求两多项式的差
{
PolyTerm *p,*p1,*p2;
p=L->data;
p1=L1->data;
p2=L2->data;
int i=0,j=0,k=0;
while（i<L1->last&&j<L2->last）{
if（p1->exp<p2->exp）{//多项式 L1 的指数小于多项式 L2 的指数
    p->coef=p1->coef;
    p->exp=p1->exp;
    p++;    k++;
    p1++; i++;
}
else
```

```
    if（p1->exp>p2->exp）{//多项式 L1 的指数大于多项式 L2 的指数
    p->coef=-p2->coef;
    p->exp=p2->exp;
    p++; k++;
    p2++; j++;
    }
    else{
    if（p1->coef!=p2->coef）{ //多项式 L1 的系数小于多项式 L2 的系数
p->coef=（p1->coef）-（p2->coef）;
p->exp=p1->exp;
p++; k++;
    }
    p1++; p2++;
    i++; j++;
    }
}
if（i<L1->last）
while（i<L1->last）{
    p->coef=p1->coef;//新的多项式系数
    p->exp=p1->exp;//新的多项式指数
    p++; k++;
    p1++; i++;
}
if（j<L2->last）
while（j<L2->last）{
    p->coef=-p2->coef;
    p->exp=p2->exp;
    p++;    k++;
    p2++; j++;
}
L->last=k;
return OK;
}
```

时间复杂度为 T（n）=O（n^3）

41．试以循环链表作稀疏多项式的存储结构，编写求其导函数的算法，要求利用原多项式中的结点空间存放其导函数（多项式），同时释放所有无用（被删）结点。

答：

```
Status PolyDifferential（LinkedPoly &L）
{
LinkedPoly p,q,pt;
q=L;
p=L->next;
while（p!=L）{
if（p->data.exp==0）{//该多项式指数为 0，即为常数，求导后为 0
```

```
    pt=p;
    p=p->next;
    q->next=p;
    free（pt）;//该项的导函数为 0，释放该点
}
else{//该多项式系数等于系数乘以指数
    p->data.coef=p->data.coef*p->data.exp;
    p->data.exp--;//指数减 1
    q=p;
    p=p->next;
}
}
return OK;
}
```

42．试编写算法，将一个用循环链表表示的稀疏多项式分解成两个多项式，使这两个多项式中各自仅含奇次项或偶次项，并要求利用原链表中的结点空间构成这两个链表。

答：

```
// 将单链表 L 划分成 2 个单循环链表
Status ListDivideInto2CL（LinkedPoly &L,LinkedPoly &L1）
{
LinkedPoly p,p1,q,pt;
q=L;
p=L->next;
p1=L1;
while（p!=L）{
if（p->data.exp%2==0）{//偶次项
    pt=p;
    p=p->next;
    q->next=p;
    pt->next=p1->next;
    p1->next=pt;
    p1=p1->next;
}
else{
    q=p;
    p=p->next;
}
}
return OK;
}
```

2.3 考研真题与典型题详解

一、选择题

1．已知两个长度分别为 m 和 n 的升序链表，若将它们合并为一个长度为 m+n 的降序链表，则最坏情况下的时间复杂度是（　　）[2013 年联考真题]

A．O（n）
B．O（m*n）
C．O（min（m，n））
D．O（max（m，n））
【答案】D 查看答案
【解析】m 和 n 是两个升序链表□长度分别为 m 和 n，在合并过程中最坏的情况是两个链表中的元素依次进行比较，比较的次数是 m 和 n 中的最大值。
2．下述哪一条是顺序存储结构的优点？（　　）[江苏大学 2006 研]
A．插入运算方便
B．可方便地用于各种逻辑结构的存储表示
C．存储密度大
D．删除运算方便
【答案】C 查看答案
【解析】顺序存储结构插入删除操作较麻烦，顺序存储结构也不能用于各种逻辑结构的存储表示，只能表示线性的逻辑结构。
3．线性表是具有 n 个（　　）的有限序列（n>0）。
A．表元素　B．字符　C．数据元素　D．数据项　E．信息项
【答案】C 查看答案
【解析】一个线性表是 n 个数据元素的有限序列。至于每个数据元素的具体含义，在不同的情况下各不相同。
4．若某线性表最常用的操作是存取任一指定序号的元素和在最后进行插入和删除运算，则利用（　　）存储方式最节省时间。[哈尔滨工业大学 2001 研]
A．顺序表　B．双链表　C．带头结点的双循环链表　D．单循环链表
【答案】A 查看答案
【解析】线性表采用顺序表，便于进行存取任一指定序号的元素；线性表采用链表，便于进行插入和删除操作。但该题是在最后进行插入和删除运算，所以利用顺序表存储方式最节省时间。
5．设一个链表最常用的操作是在末尾插入结点和删除尾结点，则选用（　　）最节省时间。[江苏大学 2006 研]
A．带头结点的双循环链表
B．单循环链表
C．带尾指针的单循环链表
D．单链表
【答案】A 查看答案
【解析】要快速地在末尾插入元素，需要能立马得到末尾元素结点，故最好是循环链表。要快速地在末尾删除元素，需要立马得到末尾元素结点的前继结点，故最好是双向链表。综上可见为双循环链表。
6．静态链表中指针表示的是（　　）。[中南大学 2003 研]
A．下一元素的地址
B．内存储器的地址
C．下一元素在数组中的位置
D．左链或右链指向的元素的地址
【答案】C 查看答案
【解析】静态链表的一般结构为：struct static_list{ ElemType data；　int next；}

这种结构是预先分配一个较大的空间，类似于一次申请一个较大的数组，但是元素的增删操作都不会移动元素，只需要移动 next 成员就行。因此，静态链表中的指针实际上表示的就是下一个元素在数组中的位置。

7．在 n 个结点的线性表的数组实现中，算法的时间复杂性是 O（1）的操作是（　　）。[哈尔滨工业大学 2003 研]

A．访问第 i 个结点（1≤i＜=n）和求第 i 个结点的直接前驱（2≤i＜=n）

B．在第 i 个结点后插入一个新结点（1≤i＜=n）

C．删除第 i 个结点（1≤i＜=n）

D．以上都不对

【答案】A 查看答案

【解析】B、C 项操作复杂性均为 O（n），顺序表访问元素时间复杂度为 O（1）。7

8．若长度为 n 的线性表采用顺序存储结构，在其第 i 个位置插入一个新元素的算法的时间复杂度为（　　）（1＜=i＜=n+1）。

A．O（0）　B．O（1）　C．O（n）　D．O（n^2）

【答案】C 查看答案

【解析】线性表采用顺序存储结构，插入和删除操作的时间复杂度均为 O（n）。

9．对于双向循环链表，在 p 指针所指的结点之后插入 s 指针所指结点的操作应为（　　）。[北京工业大学 2004 研]

A．p->right=s；s->left=p；p->right->left=s；s->right=p->right；

B．p->right=s；p->right->left=s；s->left=p；s->right=p->right；

C．s->left=p；s->right=p->right；p->right=s；P->right->left=s；

D．s->left=p；s->right=p->right；P->right->left=s；P->right=s；

【答案】D 查看答案

【解析】先建立好 s 指向 p 和 p 的后继节点的链接，即 s->left=p；s->right=p->right；然后建立 p 的后继节点指向 s 的链接，即 p->right->left=s；最后，断开 p 与后继节点的链接，将 p 指向 s，即 p->right=s。

10．线性表的顺序存储结构是一种（　　）。[北京理工大学 2006 研]

A．随机存取的存储结构

B．顺序存取的存储结构

C．索引存取的存储结构

D．Hash 存取的存储结构

【答案】A 查看答案

【解析】线性表包括顺序存储结构和链式存储结构，顺序存储结构能够随机存取表中的元素，但插入和删除操作较麻烦，链式存储结构不能随机访问表中的元素，但是能够表示元素之间的先后次序，而且插入和删除操作较容易。

二、填空题

1．当线性表的元素总数基本稳定，且很少进行插入和删除操作，但要求以最快的速度存取线性表中的元素时，应采用____存储结构。[北方交通大学 2001 研]

【答案】顺序查看答案

【解析】顺序存储结构的存取操作比较方便，但插入和删除操作不如链式存储结构方便，而且需要连续的存储空间，由于该线性表的元素总数基本稳定，而且很少进行插入删除操作，为了更快的存取元素，顺序表更合适。

2．在一个长度为 n 的顺序表中第 i 个元素（1<=i<=n）之前插入一个元素时，需向后移动____个元素。[北京工商大学 2001 研]

【答案】n-i+1 查看答案
【解析】在第 i 个元素之前插入元素，需要移动第 i 个及以后的所有元素，因此需要向右移动 n-i+1 个元素。
3．在单链表中设置头结点的作用是____。[哈尔滨工业大学 2000 研]
【答案】有头结点后，插入元素和删除元素的算法统一了，不再需要判断是否在第一个元素之前插入和删除第一个元素。另外，不论链表是否为空，链表指针不变查看答案
【解析】在单链表中设置头节点的作用在于方便运算。
4．对于一个具有 n 个结点的单链表，在已知的结点半 p 后插入一个新结点的时间．复杂度为____，在给定值为 x 的结点后插入一个新结点的时间复杂度为____。[哈尔滨工业大学 2001 研]
【答案】O（1），O（n）查看答案
【解析】第一种情况只需直接修改指针的指向。第二种情况必须从头结点遍历找到 x 的结点。
5．在双向循环链表中，向 P 所指的结点之后插入指针 f 所指的结点，其操作是____、____、____、____。[中国矿业大学 2000 研]
【答案】f->next=p->next；f->prior=p；p->next->prior=f；p->next=f；查看答案
【解析】双向循环链表的插入操作。
6．顺序存储结构是通过____表示元素之间的关系的；链式存储结构是通过____表示元素之间的关系的。[北京理工大学 2001 研]
【答案】物理上相邻、指针查看答案
【解析】顺序存储结构是通过物理位置表示元素之间的关系的，链式存储结构通过指针表示元素之间的关系。
7．对于双向链表，在两个结点之间插入一个新结点需修改的指针共____个，单链表为____个。[南京理工大学 2000 研]
【答案】4、2 查看答案
【解析】双向链表插入新节点需要修改四个指针，单链表只需修改两个指针。
8．在单链表 L 中，指针 P 所指结点有后继结点的条件是____。[合肥工业大学 2001 研]
【答案】P->next!=NULL 查看答案
【解析】指针所指节点的指针域所指向的元素非空，说明该指针所指节点有后继结点。
三、综合题

1．已知一个整数序列 $A=(a_0,a_1\cdots\cdots a_{n-1})$，其中 $0\le a_i<n(0\le i\le n)$，若存在 $a_{p1}=a_{p2}=\cdots\cdots a_{pm}=x$ 且 $m>n/2(0\le p_k\le n,1\le k\le m)$，则称 x 为 A 的主元素。例如 $A=(0,5,5,3,5,7,5,5)$，则称 5 为主元素；又如 $A=(0,5,5,3,5,1,5,7)$ 则 A 中没有主元素。假设 A 中的 n 个元素保存在一个一维数组中，请设计一个尽可能高效的算法，找出 A 的主元素。若存在主元素，则输出该元素；否则输出-1。要求：
（1）给出算法的基本设计思想。
（2）根据设计思想，采用 C 或 C++或 Java 语言描述算法，关键之处给出注释。
（3）说明你所设计算法的时间复杂度和空间复杂度。[2013 年联考真题]

答：

（1）算法的策略是从前向后扫描数组元素，标记出一个可能成为主元素的元素 Num。然后重新计数，确认 Num 是否是主元素。

算法可分为以下两步：

①选取候选的主元素：依次扫描所给数组中的每个整数，将第一个遇到的整数 Num 保存到 c 中，记录 Num 的出现次数为 1；若遇到的下一个整数仍等于 Num，则计数加 1 否则计数减 1；当计数减到 0 时，将遇到的下一个整数保存到 c 中，计数重新记为 1，开始新一轮计数，即从当前位置开始重复上述过程，直到扫描完全部数组元素。

②判断 c 中元素是否是真正的主元素，再次扫描该数组，统计 c 中元素出现的次数，若大于 n/2，则为主元素；否则，序列中不存在主元素。

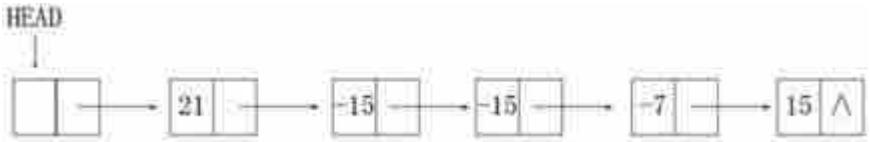
（2）算法实现如下：

```
int Majority （int A[]，int n）{
int i，c，count＝1；     //c 用来保存候选主元素，count 用来计数
c＝A[0]；     //设置 A[0]为候选主元素
for（i＝1；i<n；i++）{ //查找候选主元素
   if（A[i]＝＝c）
count++；     //为 A 中的候选主元素计数
   eles{
   if（count>0）     //处理不是候选主元素的情况
count--；
   else{ //更换候选主元素
c＝A[i]；
count＝1；
   }
   }
   }
   if（count>0）
   for（i＝count＝0；i<n；i++）{    //统计候选主元素的实际出现次数
if（A[i]＝＝c）
count++；
   }
   if（count>n/2） //确认候选主元素
   return c；
   else
   return -1；  //不存在主元素
}
```

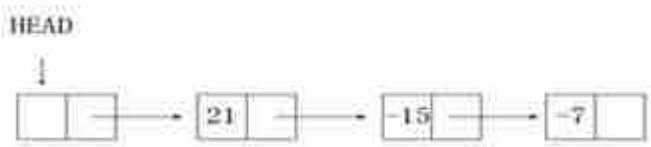
（3）时间复杂度为 O（n），空间复杂度为 O（1）。

2．用单链表保存 m 个整数，节点的结构为（data，link），且|data|＜n（n 为正整数）。现要求设计一个时间复杂度尽可能高效地算法，对于链表中绝对值相等的节点，仅保留第一次出现的节点而删除其余绝对值相等的节点。

例如若给定的单链表 head 如下



删除节点后的 head 为



要求:

（1）给出算法的基本思想

（2）使用 c 或 c++语言，给出单链表节点的数据类型定义。

（3）根据设计思想，采用 c 或 c++语言描述算法，关键之处给出注释。

（4）说明所涉及算法的时间复杂度和空间复杂度。[2015 年联考真题]

答：（1）算法思想：

定义一个大小为 n 的布尔数组 flag，初始时所有的元素都赋值为 false，用来标识遍历过程中是否出现元素绝对值为 flag 的节点。然后遍历链表，遍历过程中，每一个当前结点 data 域的绝对值所对应的 flag 位：若为真，则删除该结点；若为假（false），则将 flag 位置为真（true）。

（2）节点的数据结构定义如下：

```
typedef struct Node
{
Int data;
Struct Node * next;
}Node;
```

（3）bool flag[n]；　　//全局数组标志节点的绝对值是否出现过

```
Node * deleteABSEnqualNode（Node * head）
{
memset（flag, false, sizeof（flag））;
Node *pre ＝ head;
Node *p ＝ head->next;
while（ p !＝ NULL ）{
if（ flag[ abs（p->data） ] ）{ //如果此绝对值已经在节点值的绝对值中出现过则删除该节点
    pre->next ＝ p->next;
    delete p;
    p ＝ pre->next;
} else {//否则，将 flag 中对应的位置置为 true，并将指针指向下一个元素
flag[ abs（p->data） ] ＝ true;
p ＝ p->next;
}
}
return head;
```

}

（4）只遍历一次链表，所以时间复杂度为 O（m）（m 为单链表中元素的个数），申请大小为 n 的数组，所以空间复杂度为 O（n）（n 为节点绝对值的最大值）。

3．说明在线性表的链式存储结构中，头指针与头结点之间的根本区别；头结点与首元结点的关系。[厦门大学 2000 研]

答：在线性表的链式存储结构中，头指针指链表的指针，若链表有头结点则是链表的头结点的指针，头指针具有标识作用，故常用头指针冠以链表的名字。头结点是为了操作的统一、方便而设立的，放在第一元素结点之前，其数据域一般无意义（当然有些情况下也可存放链表的长度、用做监视哨等）。有头结点后，对在第一元素结点前插入结点和删除第一结点，其操作与对其他结点的操作统一了。而且无论链表是否为空，头指针均不为空。首元结点也就是第一元素结点，它是头结点后边的第一个结点。

4．有线性表（a_1，a_2，…，a_n），采用单链表存储，头指针为 H，每个结点中存放线性表中一个元素，现查找某个元素值等于 x 的结点。分别写出下面三种情况的查找语句。要求时间尽量少。

（1）线性表中元素无序。（2）线性表中元素按递增有序。（3）线性表中元素按递减有序。

答：设单链表带头结点，工作指针 p 初始化为 p=H->next。

（1）

```
while(p!=null && p->data!=X) p=p->next;
if(p==null) return(null);//查找失败
else return(p);//查找成功
```

（2）

```
while(p!=null && p->data<X ) p=p->next;
if(p==null || p->data>X) return(null);//查找失败
else return(p);
```

（3）

```
while(p!=null && p->data>X) p=p->next;
if(p==null || p->data<X) return(null); //查找失败
else return(p);  //查找成功
```

5．写算法将单链表 11 拆成二个链表，其中以 11 为头的链表保持原来向后的链接，另一个链表的头为 12，其链接方向与 11 相反，11 包含原链表的奇数序号的结点，12 包含原链表的偶数序号的结点。[东华大学 2004 研]

答：

```
LinkedList DisUnion(LinkedList L1,L2)
  //本算法将链表L1拆成L1和L2两个链表,L2链接方向与L1相反
{int i=0;
L2=(LNode*)malloc(sizeof(LNode));
 L2->next=null;      //空链表
  p=L1->next; pre=L1;
  while(p)
    {i++;
     if(i%2) {pre->next=p; pre=p; p=p->next;}            //奇数序号结点在L1中
     else{s=p->next; p->next=L2->next; L2->next=p; p=s;}//偶数序号结点逆置插入到L2中
    }
  pre->next=null; return L2;//置L1表尾
}
```

6．在数组 A[1．．n]中有 n 个数据，试建立一个带有头结点的循环链表，头指针为 h，要求链中数据从小到大排列，重复的数据在链中只保存一个。

答：

```
LinkedList creat(ElemType A[],int n)
//由含n个数据的数组A生成循环链表,要求链表有序并且无值重复结点
{LinkedList h;
h=(LinkedList)malloc(sizeof(LNode));//申请结点
h->next=h;  //形成空循环链表
for(i=0;i<n;i++)
   {pre=h;
   p=h->next;
   while(p!=h && p->data<A[i])

   {pre=p; p=p->next;}                    //查找A[i]的插入位置
  if(p==h || p->data!=A[i])               //重复数据不再输入
   {s=(LinkedList)malloc(sizeof(LNode));
    s->data=A[i]; pre->next=s; s->next=p;//将结点s链入链表中
   }
  }//for
 return(h);
}//算法结束
```

第 3 章　栈和队列

3.1　复习笔记

一、栈

1．抽象数据类型栈的定义

（1）栈的特点

栈（Stack）是被限定仅在表尾进行插入或删除操作的线性表。表尾端称为栈顶（top），表头端称为栈底（bottom），如图 3-1 所示。不含元素的空表称为空栈。栈的修改是按后进先出的原则进行的，因此，栈又称为后进先出（Last In First Out）的线性表（简称 LIFO 结构）。

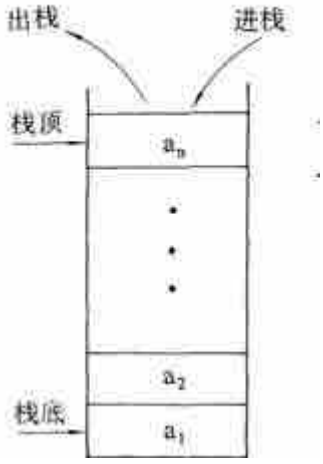


图 3-1 栈的示意图

（2）栈的抽象数据类型定义

ADT Stack{

数据对象：$D = \{a_i \mid a_i \in ElemSet, i = 1, 2, \cdots, n, n \geq 0\}$

数据关系：$R1 = \{< a_{i-1}, a_i > \mid a_{i-1}, a_i \in D, i = 2, \cdots, n\}$

约定 a_n 端为栈顶，a_1 端为栈底。

基本操作：

InitStack（&S）

操作结果：构造一个空栈 S。

DestroyStack（&S）

初始条件：栈 S 已存在。

操作结果：栈 S 被销毁。

ClearStack（&S）

初始条件：栈 S 已存在。

操作结果：将 S 清为空栈。

StackEmpty（S）

初始条件：栈 S 已存在。

操作结果：若栈 S 为空栈，则返回 TRUE，否则返回 FALSE。

StackLength（S）

初始条件：栈 S 已存在。

操作结果：返回 S 的元素个数，即栈的长度。

GetTop（S，&e）

初始条件：栈 S 已存在且非空。

操作结果：用 e 返回 S 的栈顶元素。

Push（&S，e）

初始条件：栈 S 已存在。

操作结果：插入元素 e 为新的栈顶元素。

Pop（&S，&e）

初始条件：栈 S 已存在且非空。

操作结果：删除 S 的栈顶元素，并用 e 返回其值。

StackTraverse（S，visit（））
初始条件：栈 S 已存在且非空。
操作结果：从栈底到栈顶依次对 S 的每个数据元素调用 visit（）。一旦 visit（）失败，则操作失败。
}ADT Stack
2．栈的表示和实现
（1）顺序栈
顺序栈即栈的顺序存储结构，是利用一组地址连续的存储单元依次存放自栈底到栈顶的数据元素，同时设指针 top 指示栈顶元素的当前位置。通常的习惯做法是以 top=0 表示空栈，鉴于 C 语言中数组的下标约定从 0 开始，则以 C 作描述语言时，如此设定会带来很大不便；另一方面，由于栈在使用过程中所需最大空间的大小难以估计，因此，一般来说，在初始化设空栈时不应限定栈的最大容量。一个合理的做法是：先为栈分配一个基本容量，然后在使用过程中，当栈的空间不够使用时再逐步扩大。
栈的初始化操作为：按设定的初始分配量进行一次存储分配，base 可称为栈底指针，在顺序栈中，它始终指向栈底的位置，若 base 的值为 NULL，则称栈结构不存在。称 top 为栈顶指针，其初值指向栈底，即 top=base 可作为栈空的标记，每当插入新的栈顶元素时，指针 top 增 1；删除栈顶元素时，指针 top 减 1，因此，非空栈的栈顶指针始终在栈顶元素的下一个位置上。
（2）链栈
由于栈的操作是线性表操作的特例，则链栈的操作易于实现，链栈如图 3-2 所示。

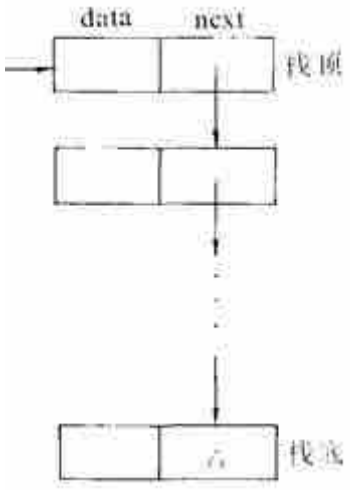


图 3-2 链栈示意图

二、栈的应用举例

1．数制转换
十进制 N 和其他 d 进制数的转换是计算机实现计算的基本问题，其解决方法很多，其中一个简单算法基于下列原理：
N=（N div d）×d+N mod d（其中：div 为整除计算，mod 为求余计算）
实现对于输入的任意一个非负十进制整数，打印输出与其等值的八进制数代码如下：
void conversion（）{
//对于输入的任意一个非负十进制整数，打印输出与其等值的八进制数
InitStack（）；

```
scanf（“%d”，N）；
while（N）{
Push（S，N%8）；
N=N%8；
}
while（！StackEmpty（S））{
Pop（S，e）；
printf（“%d”，e）；
}
```

2．括号匹配的检验

假设表达式中允许包括两种括号：圆括号和方括号，其嵌套的顺序随意，即（[]（））或[（[][]）]等为正确的格式，[（]）或（[（）]或（（）]）均为不正确的格式。检验括号是否匹配的方法可用“期待的急迫程度”这个概念来描述。

3．表达式求值

（1）算符优先法

算符优先法是根据算术四则运算优先关系的规定来实现对表达式的编译或解释执行的。

（2）算符间的优先关系。

将运算符和界限符统称为算符，它们构成的集合命名为 0P。

如表 3-1 所示定义了算符之间的优先关系。

表 3-1 算符间的优先关系

θ_1 \ θ_2	+	−	*	/	(	)	#
+	>	>	<	<	<	>	>
−	>	>	<	<	<	>	>
*	>	>	>	>	<	>	>
/	>	>	>	>	<	>	>
(	<	<	<	<	<	=	
)	>	>	>	>		>	>
#	<	<	<	<	<		=

（3）算法的实现

为实现算符优先算法，可以使用两个工作栈。一个称做 0PTR，用于寄存运算符；另一个称做 OPND，用于寄存操作数或运算结果。算法的基本思想是：

①首先置操作数栈为空栈，表达式起始符“#”为运算符栈的栈底元素。

②依次读入表达式中每个字符，该字符若是操作数则进 0PND 栈，若是运算符，则和 0PTR 栈的栈顶运算符比较优先权后作相应操作，直至整个表达式求值完毕（即 OPTR 栈的栈顶元素和当前读入的字符均为“#”）。

三、栈与递归的实现

1．递归的定义

一个直接调用自身或通过一系列过程调用语句间接地调用自身的过程，称做递归过程。

递归是程序设计过程中一个强有力的工具。其一，有许多数学函数是递归定义的，如大家熟悉的阶乘函数

$$Fact=\begin{cases}1 & n=0\\ n\bullet Fact(n-1) & n>0\end{cases}$$

2 阶 Fibonacci 数列

$$Fib(n)=\begin{cases}0 & n=0\\ 1 & n=1\\ Fib(n-1)+Fib(n-2) & other\end{cases}$$

和 Ackerman 函数

$$Ack(m,n)=\begin{cases}n+1 & m=0\\ Ack(m-1,1) & n=0\\ Ack(m-1,Ack(m,n-1)) & other\end{cases}$$

等；其二，有的数据结构，如二叉树、广义表等，由于结构本身固有的递归特性，则它们的操作可递归地描述；其三，还有一类问题，虽然问题本身没有明显的递归结构，单用递归求解比迭代求解更简单，如八皇后问题、Hanoi 塔问题等。

2．递归函数的执行过程

与汇编程序设计中主程序和子程序之间的链接及信息交换相类似，在高级语言编制的过程中，调用函数和被调用函数之间的链接及信息交换需通过栈来进行。

一个递归函数的运行过程类似于多个函数的嵌套调用，只是调用函数和被调用函数是同一个函数，因此，和每次调用相关的一个重要的概念是递归函数运行的“层次”。假设调用该递归函数的主函数为第 0 层，则从主函数调用递归函数为进入的第 1 层；从第 i 层递归调用本函数为进入的“下一层”，即第 i+1 层。反之，退出第 i 层递归应返回至“上一层”，即第 i-1 层。为了保证递归函数的正确执行，系统需设立一个“递归工作栈”作为整个递归函数运行期间使用的数据存储区。每一层递归所需信息构成一个“工作记录”，其中包括所有的实在参数、所有的局部变量以及上一层的返回地址。每进入一层递归，就产生一个新的工作记录压入栈顶。每退出一层递归，就从栈顶弹出一个工作记录，则当前执行层的工作记录必是递归工作栈栈顶的工作记录，称这个记录为“活动记录”，并称指示活动记录的栈顶指针为“当前环境指针”。

四、队列

1．抽象数据类型队列的定义

（1）队列的特点

队列（Queue）是一种先进先出（First In First Out，缩写为 FIFO）的线性表。它只允许在表的一端插入元素，而在另一端删除元素。允许插入的一端叫做队尾（rear），允许删除的一端则称为队头（front）。

（2）队列的抽象数据类型定义

ADT Queue{

数据对象：$D=\{a_i \mid a_i \in ElemSet, i=1,2,\cdots,n, n\geq 0\}$

数据关系：$R1=\{<a_{i-1},a_i> \mid a_{i-1},a_i \in D, i=2,\cdots,n\}$

基本操作：

InitQueue（&Q）

操作结果：构造一个空队列 Q。

DestroyQueue（&Q）

初始条件：队列 Q 已存在。

操作结果：队列 Q 被销毁，不再存在。

ClearQueue（&Q）
初始条件：队列 Q 已存在。
操作结果：将 Q 清为空队列。
QueueEmpty（Q）
初始条件：队列 Q 已存在。
操作结果：若 Q 为空队列，则返回 TRUE，否则 FALSE。
QueueLength（Q）
初始条件：队列 Q 已存在。
操作结果：返回 Q 的元素个数，即队列的长度。
GetHead（Q，&e）
初始条件：Q 为非空队列。
操作结果：插入元素 e 为 Q 的新的队头元素。
EnQueue（&Q，e）
初始条件：队列 Q 已存在。
操作结果：插入元素 e 为 Q 的新的队尾元素。
DeQueue（&Q，&e）
初始条件：Q 为非空队列。
操作结果：删除 Q 的对头元素，并用 e 返回其值。
QueueTraverse（Q，visit（））
初始条件：Q 已存在且非空。
操作结果：从队头到队尾，依次对 Q 的每个元素调用函数 visit（）。一旦
visit（）失败，则操作失败。
}ADT Queue
（3）双端队列
双端队列（Deque）是限定插入和删除操作在表的两端进行的线性表。这两端分别称作端点 1 和端点 2（如图 3-3 所示）。在实际使用中，还有输出受限的双端队列（即一个端点允许插入和删除，另一个端点只允许插入）和输入受限的双端队列（即一个端点允许插入和删除，另一个端点只允许删除）。

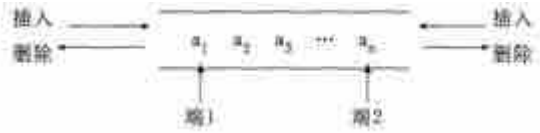


图 3-3 双端队列

2．链队列—队列的链式表示和实现
对于在使用中数据元素变动较大的数据结构，用链式存储结构比顺序存储结构更有利。队列也是一种链式存储结构。
（1）定义
用链表表示的队列简称为链队列，如图 3-4 所示。一个链队列需要两个分别指示队头和队尾的指针（分别称为头指针和尾指针）才能唯一确定。为了操作方便起见，通常给链队列添加一个头结点，并令头指针指向头结点。

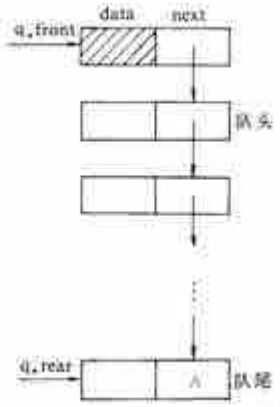


图 3-4 链队列示意图

（2）特点

空的链队列的判决条件为头指针和尾指针均指向头结点。链队列的操作即为单链表的插入和删除操作的特殊情况，只是尚需要修改尾指针和头指针。

3．循环队列--队列的顺序存储结构

队列的顺序存储结构中，除了用一个能容纳最大容量元素的向量以外，还需要两个指针分别指示队头元素和队尾元素，并约定尾指针指向队尾元素在队列中的当前位置。为了在 C 语言中描述方便起见，在此我们约定：初始化建空队列时，令 front==rear=0，每当插入新的队列尾元素时，“尾指针增 1”；每当删除队列头元素时，“头指针增 1”。因此，在非空队列中，头指针始终指向队列头元素，而尾指针始终指向队列尾元素的下一个位置。

3.2 强化习题详解

【基础知识题】

1．若按如图 3-1 所示的铁道进行车厢调度（注意：两侧铁道均为单向行驶道），则请回答：

（1）如果进栈的车厢序列为 123，则可能得到的出站车厢序列是什么？

（2）如果进栈的车厢序列为 123456，则能否得到 435612 和 135426 的出站序列，并请说明为什么不能得到或者如何得到（即写出以‘S’表示进栈和以‘X’表示出栈的栈操作序列）。

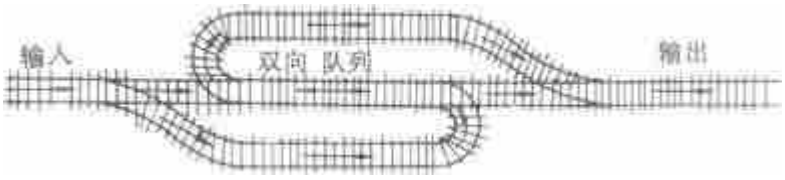


图 3-1

答：

（1） 123，132，213，231，321。

（2） ①可以得到 135426 的出栈序列，方法为：1 进栈，1 出栈，输出为 1；2 进栈，3 进栈，3 出栈，输出为 1 3；4 进栈，5 进栈，5 出栈，输出为 1 3 5； 4 出栈，输出为 1 3 5 4； 2 出栈，输出为 1 3 5 4 2；6 进栈，6 出栈，输出为 1 3 5 4 2 6。

②不能得到 435612 的出栈序列。因为 4356 出栈说明 12 已经在栈中，1 不可能先于 2 出栈。

2．简述栈和线性表的差别。

答：线性表是具有相同特性的数据元素的一个有限序列，可以进行随机插入和删除操作；
栈是限定仅在表尾进行插入或删除操作的线性表，具有后进先出的特点。
栈是特殊的线性表，其操作是线性表基本操作的子集。

3．写出下列程序段的输出结果（栈的元素类型 SElemType 为 char）。

```
void main（）
{
Stack S;
char x,y;
InitStack（S）;
x= "c"; y= "k";
Push（S,x）;
Push（S, "a"）;
Push（S,y）;
Pop（S,x）;
Push（S, "t"）;
Push（S,x）;
    Pop（S,x）;
Push（S, "s"）;
while（!StackEmpty（S））
{
Pop（S,y）;
printf（y）;
}
printf（x）;
}
```

答：stack（根据进栈和出栈的顺序依次写出字符。）

4．简述以下算法的功能（栈的元素类型 SElemType 为 int）。

```
（1） status algo1（Stack S）
    {
int i,n,A[255];
n=0;
while（!StackEmpty（S））
    {
n++;
Pop（S,A[n]）;
    }
for（i=1;i<=n;i++）
Push（S,A[i]）;
}
（2） status algo2（Stack S,int e）
{
    Stack T;
int d;
    InitStack（T）;
```

```
    while（!StackEmpty（S））{
    Pop（S,d）;
    if（d!=e） Push（T,d）;
    }
    while（!StackEmpty（T））{
    Pop（T,d）;
    Push（S,d）;
    }
}
```

答：（1）栈中的数据元素逆置（根据对栈的操作具有后进先出的特点，元素依次出栈后再次入栈，会实现逆序）。

（2）利用栈 T 辅助查找栈中为 e 的元素，将其从栈中清除（先将元素全部出栈，如果元素不是 e，则再次进栈，从而可以将栈中为 e 的元素清除）。

5．假设以 S 和 X 分别表示入栈和出栈的操作，则初态和终态均为空栈的入栈和出栈的操作序列可以表示为仅由 S 和 X 组成的序列。称可以操作的序列为合法序列（例如，SXSX 为合法序列，SXXS 为非法序列）。试给出区分给定序列为合法序列或非法序列的一般准则，并证明：两个不同的合法（栈操作）序列（对同一输入序列）不可能得到相同的输出元素（注意：在此指的是元素实体，而不是值）序列。

答：判别给定序列 T 是否合法的充分必要条件是：

$$[N_{si}(T)+N_{xi}(T)=l]\wedge(\forall i)(1\le i\le l\rightarrow N_{SI}(T)\ge N_{xi}(T))$$

其中：

N_{si}（T）表示序列 T 中前 i 个字符构成的子序列中‘S’的数目；

N_{xi}（T）表示序列 T 中前 i 个字符构成的子序列中‘X’的数目。

l 为序列的长度。

6．试证明：若借助栈由输入序列 12…n 得到的输出序列为 $p_1p_2\cdots p_n$（它是输入序列的一个排列），则在输出序列中不可能出现这样的情形：存在着 i<j<k 使 $p_j<p_k<p_i$。

答：因为输入序列是从小到大排列的，所以若 $p_j<p_k<p_i$，则可以理解为通过输入序列 $p_j\ p_k\ p_i$ 可以得到输出序列 $p_i\ p_j\ p_k$，设 p_j=1，p_k=2，p_i=3，由栈的后进先出特点可知，通过输入序列 123 是无法得到 312 的。所以不可能存在着 i<j<k 使 $p_j<p_k<p_i$。

7．试推导求解 n 阶梵塔问题至少要执行的 move 操作的次数。

答：设至少要执行 M（n）次 move 操作，则

$$M(n)=\begin{cases}1 & n=1\\ 2M(n-1)+1 & n>1\end{cases}$$

解此差分方程可求得解为 $M(n)=2^n-1$。

【算法设计题】

8．试将下列递推过程改写为递归过程。

```
void ditui（int n）
```

```
{
int i;
i = n;
while（i>1）
    printf（i--）;
}
```
答：
```
void ditui（int j）
{
if（j>1）{
printf（j）;
ditui（j-1）;
}
return;
}
```
9．试将下列递归过程改写为非递归过程。
```
void test（int &sum）
{
intx;
scanf（x）;
if（x==0） sum=0;
else
{
test（sum）;
sum+=x;
}
printf（sum）;
}
```
答：
```
void test（int &sum）
{
Stack s;
InitStack（s）;
int x;
do{
scanf（x）;
Push（s,x）;
}while（x>0）;
while（!StackEmpty（s））{
Pop（s,x）;
sum+=x;
printf（sum）;
}
DestoryStack（s）;
```

}
10．简述队列和栈这两种数据类型的相同点和差异处。
答：相同点：栈和队列都是操作受限的线性表。
不同点：
①栈是一种后进先出的线性表，且只允许在栈顶进行插入和删除操作。
②队列是一种先进先出的线性表，且只允许在队头进行删除操作，在队尾进行插入操作。
11．写出以下程序段的输出结果（队列中的元素类型 QElemType 为 char）。

```
voidmain（）
{
QueueQ;
InitQueue（Q）;
charx = "e", y = "c";
EnQueue（Q,"h"）;
EnQueue（Q,"r"）;
EnQueue（Q,y）;
DeQueue（Q,x）;
EnQueue（Q,x）;
DeQueue（Q,x）;
EnQueue（Q,"a"）;
While（!QueueEmpty（Q））
{
DeQueue（Q,y）;
printf（y）;
}
printf（x）;
}
```

答：char（根据队列先进先出的操作特点，按照入队和出队顺序依次写出元素即可。）
12．简述以下算法的功能（栈和队列的元素类型均为 int）。

```
voidalgo3（Queue &Q）
{
StackS;
intd;
InitStack（S）;
while（!QueueEmpty（Q））
{
DeQueue（Q,d）;
Push（S,d）;
}
while（!StackEmpty（S））
{
Pop（S,d）;
EnQueue（Q,d）;
}
}
```

答：该算法充分利用栈后进先出的特点和队列先进先出的特点，功能是借助栈，将队列中的数据元素进行逆置。

13．若以 1234 作为双端队列的输入序列，试分别求出满足以下条件的输出序列：

（1）能由输入受限的双端队列得到，但不能由输出受限的双端队列得到的输出序列。

（2）能由输出受限的双端队列得到，但不能由输入受限的双端队列得到的输出序列。

（3）既不能由输入受限的双端队列得到，也不能由输出受限的双端队列得到的输出序列。

答：（1）4132

（2）4213

（3）4231

14．假设以顺序存储结构实现一个双向栈，即在一维数组的存储空间中存在着两个栈，它们的栈底分别设在数组的两个端点。试编写实现这个双向栈 tws 的三个操作：初始化 inistack（tws）、入栈 push（tws,i,x）和出栈 pop（tws,i）的算法，其中 i 为 0 或 1，用以分别指示设在数组两端的两个栈，并讨论按过程（正/误状态变量可设为变参）或函数设计这些操作算法各有什么优缺点。

答：

typedef struct{ Elemtype *base[2]; Elemtype *top[2]; }BDStacktype; //双向栈类型

Status inistack（BDStacktype &tws,int m）//初始化一个大小为 m 的双向栈 tws {

tws.base[0]=（Elemtype*）malloc（sizeof（Elemtype））； tws.base[1]=tws.base[0]+m;

tws.top[0]=tws.base[0]; tws.top[1]=tws.base[1]; return OK; }//inistack

Status push（BDStacktype &tws,int i,Elemtype x）//x 入栈,i=0 表示低端栈,i=1 表示高端栈 {

if（tws.top[0]>tws.top[1]） return OVERFLOW; //注意此时的栈满条件 if（i==0）

*tws.top[0]++=x; //x 入栈，进入低端栈 else if（i==1） *tws.top[1]--=x; //x 入栈，进入高端栈 else return ERROR; return OK; }//push

Status pop（BDStacktype &tws,int i,Elemtype &x）//x 出栈,i=0 表示低端栈,i=1 表示高端栈 {

if（i= =0） { if（tws.top[0]= =tws.base[0]）

return OVERFLOW;//出栈时的栈满条件 x=*--tws.top[0];//x 从低端栈出栈 } else if（i==1） { if（tws.top[1]= =tws.base[1]）

return OVERFLOW; x=*++tws.top[1];//x 从高端栈出栈 } else return ERROR; return OK; }

//pop

15．假设如题 1 所示火车调度站的入口处有 n 节硬席或软席车厢（分别以 H 和 S 表示）等待调度，试编写算法，输出对这 n 节车厢进行调度的操作（即入栈或出栈操作）序列，以使所有的软席车厢都被调整到硬席车厢之前。

答：

void Train_arrange（char *train）//这里用字符串 train 表示火车,"H"表示硬席,"S"表示软席。

{

q=train; p=train; InitStack（s）； while（*p） { if（*p=="H"） push（s,*p）; //把"H"存入栈中 else *（q++）=*p; //把"S"调到前部，即软席车厢调整到硬席车厢之前 p++; } while（!StackEmpty（s）） { pop（s,c）; *（q++）=c; //把"H"接在后部，即硬席车厢直接顺序接到后面 } }//Train_arrange

16．试写一个算法，识别一次读入的一个以@为结束符的字符序列是否为形如‘序列 1&序列 2’模式的字符序列。其中序列 1 和序列 2 中都不含字符‘&’，且序列 2 是序列 1 的逆序列。例如，‘a+b&b+a’是属该模式的字符序列，而‘1+3&3-1’则不是。

答：

int IsReverse（ ）//判断输入的字符串中"&"前和"&"后部分是否为逆串,是则返回 1,否则返回 0
{ InitStack（s）; while（（e=getchar（ ））!="&"） { if（e=='@'） return 0;//不允许在'&'之前出现'@' push（s,e）; } while（ （e=getchar（ ））!="@"）//字符序列结束标识 { if（StackEmpty（s）） return 0; pop（s,c）; if（e!=c） return 0;//序列 2 是序列 1 的逆序列 } if（!StackEmpty（s）） return 0; return 1; }//IsReverse

17．试写一个判别表达式中开、闭括号是否配对出现的算法。

答：

（1）使用栈：

```
bool BracketCorrespondency（char a[]）
{
int i=0;
Stack s;
InitStack（s）;
ElemType x;
while（a[i]）{
switch（a[i]）{
case "（":
   Push（s,a[i]）;break;
case "[":
   Push（s,a[i]）; break;
case "）":
   GetTop（s,x）;
   if（x=="（"） Pop（s,x）;
   else return FALSE; break;
   case "]"：
   GetTop（s,x）;
if（x=="["） Pop（s,x）;
   else return FALSE; break;
}
   i++;
}
return true;
}
```

（2）不使用栈：

Status Bracket_Test（char *str）//判别表达式中小括号是否匹配 { count=0; for（p=str;*p;p++） { if（*p= ="（"） count++; else if（*p= ="）"） count--; if （count<0） return ERROR;// count<0，说明"）"的个数多于"（"的个数，表达式中小括号不匹配 } if（count） return ERROR; // count>0，说明"（"的个数多于"）"的个数，表达式中小括号不匹配 return OK; }//Bracket_Test

18．假设一个算术表达式中可以包含三种括号：圆括号"（"和"）"，方括号"["和"]"和花括号"{"和"}"，且这三种括号可按任意的次序嵌套使用（如：…[…{…}…[…]…]…[…]…（…）…）。编写判别给定表达式中所含括号是否正确配对出现的算法（已知表达式已存入数据元素为字符的顺序表中）。

答：

```
Status AllBrackets_Test（char *str）//判别表达式中三种括号是否匹配 { InitStack（s）;
for（p=str;*p;p++） { if（*p==" （" || *p=="[" || *p=="{"） push（s,*p）; else if（*p==")
"||*p=="]"||*p=="}"） { if（StackEmpty（s）） return ERROR;//此时栈空说明括号不
匹配 pop（s,c）; if（*p==")"&&c!= "（"） return ERROR; if（*p=="]"&&c!=
"["） return ERROR; if（*p=="}"&&c!= "{"） return ERROR; //必须与当前栈顶括号匹配 } }
//for if（!StackEmpty（s）） return ERROR;//栈中元素有剩余，说明括号没有完全匹配
return OK; }//AllBrackets_Test
```

19．假设以二维数组 g（1…m, 1…n）表示一个图像区域，g[i,j]表示该区域中点（i,j）所具颜色，其值为从 0 到 k 的整数。编写算法置换点（i_0,j_0）所在区域的颜色。约定和（i_0,j_0）同色的上、下、左、右的邻接点为同色区域的点。

答：

```
typedef struct { int x; int y; } coordinate;
void Repaint_Color（int g[m][n],int i,int j,int color） //把点（i,j）相邻区域的颜色置换为
color
{ old=g[i][j];
InitQueue（Q）; EnQueue（Q,{i,j}）; while（!QueueEmpty（Q）） {
DeQueue（Q,a）; x=a.x; y=a.y; if（x>1） if（g[x-1][y]==old） { g[x-1][y]=color;//将左
邻点颜色置换为 color EnQueue（Q,{x-1,y}）; //修改左邻点的颜色 } if（y>1）
if（g[x][y-1]==old） { g[x][y-1]=color; //将上邻点颜色置换为 color EnQueue（Q,{x,y-1}）
; //修改上邻点的颜色 } if（x<m） if（g[x+1][y]==old） { g[x+1][y]=color; //将右
邻点颜色置换为 color EnQueue（Q,{x+1,y}）; //修改右邻点的颜色 } if（y<n）
if（g[x][y+1]==old） { g[x][y+1]=color; //将下邻点颜色置换为 color EnQueue（Q,{x,y+1}）
; //修改下邻点的颜色 } }//while }//Repaint_Color
```

20．假设表达式由单字母变量和双目四则运算符构成。试写一个算法，将一个通常书写形式且书写正确的表达式转换为逆波兰表达式。

答：

```
void NiBoLan（char *str,char *new） //把中缀表达式 str 转换成逆波兰式 new，即后缀表达
式
{ p=str;q=new; //为方便起见,设 str 的两端都加上了优先级最低的特殊符号 InitStack（s）;
//s 为运算符栈 while（*p） { if（*p 是字母） *q++=*p; //直接输出 else {
c=gettop（s）; if（*p 优先级比 c 高） push（s,*p）;//将比栈顶运算符优先级高的压入运
算符栈，即栈顶的运
//算符优先级最高
else { while（gettop（s）优先级不比*p 低） { pop（s,c）;*（q++）=c; }//while
push（s,*p）; //运算符在栈内遵循越往栈顶优先级越高的原则 }//else }//else p++; }//while }
//NiBoLan
```

21．如题 20 的假设条件，试写一个算法，对以逆波兰式表示的表达式求值。

答：

```
int GetValue_NiBoLan（char *str）//对逆波兰式求值 { p=str;
InitStack（s）; //s 为操作数栈 while（*p） { if（*p 是数） push（s,*p）; else {
pop（s,a）;pop（s,b）;//从栈顶弹出两个操作数 r=compute（b,*p,a）; //假设 compute 为
执行双目运算的过程 push（s,r）;//将运算结果作为新的操作数压入栈顶 }//else p++; }
//while pop（s,r）;return r; }//GetValue_NiBoLan
```

22．如题 20 的假设条件，试写一个算法，判断给定的非空后缀表达式是否为正确的逆波兰表达式（即后缀表达式），如果是，则将它转化为波兰式。

答：

Status NiBoLan_to_BoLan（char *str,stringtype &new）//把逆波兰表达式 str 转换为波兰式 new { p=str;
Initstack（s）; //s 的元素为 stringtype 类型 while（*p） { if（*p 为字母） push（s,*p）; else { if（StackEmpty（s）） return ERROR; pop（s,a）; if（StackEmpty（s）） return ERROR;//操作数不匹配 pop（s,b）; c=link（link（*p,b）,a）;//连接操作 push（s,c）; }//else p++; }//while pop（s,new）; if（!StackEmpty（s）） return ERROR; return OK; }//NiBoLan_to_BoLan 分析:本题中暂不考虑串的具体操作的实现,而将其看作一种抽象数据类型 stringtype,对其可以进行连接操作:c=link（a,b）。

23．试编写如下定义的递归函数的递归算法，并根据算法画出求 g（5,2）时栈的变化过程。

$$g(m,n)=\begin{cases}0 & m=0,n\geq 0\\ g(m-1,2n)+n & m>0,n\geq 0\end{cases}$$

答：

Status g（int m,int n,int &s）//求递归函数 g 的值 s { if（m==0&&n>=0） s=0; else if（m>0&&n>=0） s=n+g（m-1,2*n）; else return ERROR; return OK; }//g

假设主函数的返回地址是 0，递归函数三条语句的返回地址是 1、2、3，求 g（5,2）时栈的变化过程如表 1 所示：

表 1

函数返回地址	m	n
3	0	64
3	1	32
3	2	16
3	3	8
3	4	4
0	5	2

24．试写出求递归函数 F（n）的递归算法，并消除递归：

$$F(n)=\begin{cases}n+1 & n=0\\ n\cdot F\left(n/2\right) & n>0\end{cases}$$

答：

Status F_recursive（int n,int &s）//递归算法 { if（n<0） return ERROR; if（n= =0） s=n+1; else { F_recurve（n/2,r）;//递归调用本身 s=n*r; } return OK; }//F_recursive
Status F_nonrecursive（int n,int s）//非递归算法，利用栈和循环 { if（n<0） return ERROR; if（n==0） s=n+1; else { InitStack（s）; //s 的元素类型为 struct {int a;int b;}
while（n!=0） { a=n;b=n/2; push（s,{a,b}）; n=b; }//while s=1;
while（!StackEmpty（s）） { pop（s,t）; s*=t.a; }//while } return OK; }
//F_nonrecursive

25．求解平方根 $\sqrt{A}$ 的迭代函数定义如下：

$$sqrt(A,p,e)=\begin{cases} p & |p^2-A|<e \\ sqrt\left(A,\frac{1}{2}\left(p+\frac{A}{p}\right),e\right) & |p^2-A|\geq e \end{cases}$$

其中，p 是 A 的近似平方根，e 是结果允许误差。试写出相应的递归算法，并消除递归。

答：

float Sqrt_recursive（float A,float p,float e）//求平方根的递归算法 { if（abs（p^2-A）<=e）return p; else return sqrt_recurve（A,（p+A/p）/2,e）; }//Sqrt_recurve

float Sqrt_nonrecursive（float A,float p,float e）//求平方根的非递归算法 { while（abs（p^2-A）>=e） p=（p+A/p）/2; return p; }//Sqrt_nonrecu rsive

26．已知 Ackerman 函数的定义如下：

$$akm(m,n)=\begin{cases} n+1 & m=0 \\ akm(m-1,1) & m\neq 0,n=0 \\ akm(m-1,akm(m,n-1)) & m\neq 0,n\neq 0 \end{cases}$$

（1） 写出递归算法；

（2） 写出非递归算法；

（3） 根据非递归算法，画出求 akm（2,1）时栈的变化过程。

答：

（1）递归算法

```
unsigned int akm（unsigned int m,unsigned int n）
{
unsigned int g;
if（m= =0）
return n+1;
else
if（n= =0） return akm（m-1,1）;
else{
   g=akm（m,n-1）;
   return akm（m-1,g）;
}
}
```

（2）非递归算法：

```
int akm1（int m,int n）
{
Stack s;
InitStack（s）;
ElemType e,e1,d;
e.mval=m;
e.nval=n;
Push（s,e）;
do{
while（e.mval）{
   while（e.nval）{
```

```
    e.nval--;
    Push（s,e）;//元素依次进栈
    }
    e.mval--;
    e.nval=1;
}
if（StackLength（s）>1）{
    e1.nval=e.nval;
    Pop（s,e）;//出栈
    e.mval--;
    e.nval=e1.nval+1;
}
}while（StackLength（s）!=1||e.mval!=0）;//循环结束条件
return e.nval+1;//m=0 时，akm（m,n）=n+1
}
```

（3）栈的变化过程：

```
0,akm（2,1）    0 2 1   g=akm（2,0）
1,akm（2,0）    6 2 0   akm=akm（m-1,1）=akm（1,1）
2,akm（1,1）    4 1 1   g=akm（m,n-1）=akm（1,0）
3,akm（1,0）    6 1 0   akm=akm（m-1,1）=akm（0,1）
4,akm（0,1）    4 0 1   akm=n+1=2    退栈
0,akm（2,1）    0 2 1   g=akm（2,0）
1,akm（2,0）    6 2 0   akm=akm（m-1,1）=akm（1,1）
2,akm（1,1）    4 1 1   g=akm（m,n-1）=akm（1,0）
3,akm（1,0）    6 1 0   akm=akm（m-1,1）=akm（0,1）=2 退栈
0,akm（2,1）    0 2 1   g=akm（2,0）
1,akm（2,0）    6 2 0   akm=akm（m-1,1）=akm（1,1）
2,akm（1,1）    4 1 1   g=akm（m,n-1）=akm（1,0）=2; akm=akm（m-1,g）=akm（0,2）;
3,akm（0,2）    7 0 2   akm=n+1=3 退栈
0,akm（2,1）    0 2 1   g=akm（2,0）
1,akm（2,0）    6 2 0   akm=akm（m-1,1）=akm（1,1）
2,akm（1,1）    4 1 1   g=akm（m,n-1）=akm（1,0）=2; akm=akm（m-1,g）=akm（0,2）=3;
退栈
0,akm（2,1）    0 2 1   g=akm（2,0）
1,akm（2,0）    6 2 0   akm=akm（m-1,1）=akm（1,1）=3 退栈
0,akm（2,1）    0 2 1   g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3）    6 1 3   g=akm（1,2）; akm（m-1,g）
2,akm（1,2）    6 1 2   g=akm（1,1）; akm（m-1,g）
3,akm（1,1）    6 1 1   g=akm（1,0）; akm（m-1,g）
4,akm（1,0）    6 1 0   akm=akm（0,1）
5,akm（0,1）    4 0 1   akm（0,1）=2 退栈
0,akm（2,1）    0 2 1   g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3）    6 1 3   g=akm（1,2）; akm（m-1,g）
2,akm（1,2）    6 1 2   g=akm（1,1）; akm（m-1,g）
```

3,akm（1,1） 6 1 1 g=akm（1,0）; akm（m-1,g）
4,akm（1,0） 6 1 0 akm=akm（0,1）=2 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3） 6 1 3 g=akm（1,2）; akm（m-1,g）
2,akm（1,2） 6 1 2 g=akm（1,1）; akm（m-1,g）
3,akm（1,1） 6 1 1 g=akm（1,0）=2; akm（m-1,g）=akm（0,2）
4,akm（0,2） 7 0 2 akm=n+1=3 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3） 6 1 3 g=akm（1,2）; akm（m-1,g）
2,akm（1,2） 6 1 2 g=akm（1,1）; akm（m-1,g）
3,akm（1,1） 6 1 1 g=akm（1,0）=2; akm（m-1,g）=akm（0,2）=3 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3） 6 1 3 g=akm（1,2）; akm（m-1,g）
2,akm（1,2） 6 1 2 g=akm（1,1）=3; akm（m-1,g）=akm（0,3）
3,akm（0,3） 7 0 3 akm=n+1=4 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3） 6 1 3 g=akm（1,2）; akm（m-1,g）
2,akm（1,2） 6 1 2 g=akm（1,1）=3; akm（m-1,g）=akm（0,3）=4 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3） 6 1 3 g=akm（1,2）=4; akm（m-1,g）=akm（0,4）
2,akm（0,4） 7 0 4 akm=n+1=5 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）
1,akm（1,3） 6 1 3 g=akm（1,2）=4; akm（m-1,g）=akm（0,4）=5 退栈
0,akm（2,1） 0 2 1 g=akm（2,0）=3; akm=akm（1,3）=5 退栈
akm（2,1）=5;

27．假设以带头结点的循环链表表示队列，并且只设一个指针指向队尾元素结点（注意不设头指针），试编写相应的队列初始化、入队列和出队列的算法。

答：

void InitCiQueue（CiQueue &Q）//初始化循环链表表示的队列 Q { Q=（CiLNode*）malloc（sizeof（CiLNode））;//动态创建循环链表结点 Q->next=Q; }//InitCiQueue
void EnCiQueue（CiQueue &Q,int x）//把元素 x 插入循环链表表示的队列 Q,Q 指向队尾元//素,Q->next 指向头结点,Q->next->next 指向队头元素
{ p=（CiLNode*）malloc（sizeof（CiLNode））; p->data=x; p->next=Q->next; Q->next=p; //直接把 p 加在 Q 的后面 Q=p; //修改尾指针 }
Status DeCiQueue（CiQueue &Q,int x）//从循环链表表示的队列 Q 头部删除元素 x { if（Q==Q->next） return INFEASIBLE; //队列已空 p=Q->next->next; x=p->data; Q->next->next=p->next; free（p）;//释放结点 return OK; }

28．如果希望循环队列中的元素都能得到利用，则需设置一个标志域 tag，并以 tag 的值为 0 和 1 来区分尾指针和头指针值相同时的队列状态是“空”还是“满”。试编写与此结构相应的入队列和出队列的算法，并从时间和空间角度讨论设标志和不设标志这两种方法的使用范围（如当循环队列容量较小而队列中每个元素占的空间较多时，哪一种方法较好）。

答：

#define MaxQSize 4
typedef int ElemType;

```
typedef struct{
ElemType *base;
int front;
int rear;
Status tag;
}Queue;
Status InitQueue（Queue &q）
{
q.base=new ElemType[MaxQSize];
if（!q.base） return FALSE;
q.front=0;
q.rear=0;
q.tag=0;
return OK;
}
Status EnQueue（Queue &q,ElemType e）
{
if（q.front= =q.rear&&q.tag）
return FALSE;//队列的头指针和尾指针指向同一结点，并且队列状态为“满”，
   //则元素不能入队
else{
q.base[q.rear]=e;
q.rear=（q.rear+1）%MaxQSize;//循环队列计算尾指针的位置
if（q.rear= =q.front）q.tag=1;//入队操作后如果头指针和尾指针指向同一个结点，
   //说明队列已满
}
return OK;
}
Status DeQueue（Queue &q,ElemType &e）
{
if（q.front==q.rear&&!q.tag）return FALSE;//队列的头指针和尾指针指向同一结点，
   //并且队列状态为“空”，没有可出队的元素
else{
e=q.base[q.front];
q.front=（q.front+1）%MaxQSize; //循环队列计算头指针的位置
q.tag=0;//出队操作后，队列可以继续入队
}
return OK;
}//设标志节省存储空间，但运行时间较长。不设标志则正好相反。
```

29．假设将循环队列定义为:以域变量 rear 和 length 分别指示循环队列中队尾元素的位置和内含元素的个数。试给出此循环队列的队满条件，并写出相应的入队列和出队列的算法（在出队列的算法中要返回队头元素）。

答：

设 head 指示循环队列中队头元素元素的位置，则有

head=（（q.rear+MAXLEN）-q.length+1）%MAXLEN
其中 MAXLEN 为队列可用的最大空间。

```
#define MaxQSize 4
typedef int ElemType;
typedef struct{
ElemType *base;
int rear;
int length;
}Queue;
Status InitQueue（Queue &q）
{
q.base=new ElemType[MaxQSize];
if（!q.base） return FALSE;
q.rear=0;
q.length=0;
return OK;
}
Status EnQueue（Queue &q,ElemType e）
{
if（（q.rear+1）%MaxQSize==（q.rear+MaxQSize-q.length）%MaxQSize）//入队操作，头指针与
   //尾指针指向同一结点，队列满
return FALSE;
else{
q.base[q.rear]=e;
q.rear=（q.rear+1）%MaxQSize;//循环队列计算尾指针位置
q.length++;
}
return OK;
}
Status DeQueue（Queue &q,ElemType &e）
{
if（（q.rear+MaxQSize-q.length）%MaxQSize==q.rear）//出队操作，头指针与尾指针指向同一个结点，队列空
return FALSE;
else{
e=q.base[（q.rear+MaxQSize-q.length）%MaxQSize];//队头元素
q.length--;
}
return OK;
}
```

30．假设称正读和反读都相同的字符序列为“回文”，例如，‘abba’和‘abcba’是回文，‘abcde’和‘ababab’则不是回文。试写一个算法判别读入的一个以‘@’为结束符的字符序列是否是“回文”。

答：
int Palindrome_Test（）//判别输入的字符串是否回文序列,是则返回 1,否则返回 0 {
InitStack（S）;
InitQueue（Q）; while（（c=getchar（））!="@"）//字符串结束标志 { Push（S,c）;
EnQueue（Q,c）; //同时使用栈和队列两种结构 } while（!StackEmpty（S））{ Pop（S,a）;
//出栈元素为字符串末尾的元素
DeQueue（Q,b））;//出队元素为字符串开头的元素 if（a!=b） return ERROR; } return OK; }

31．试利用循环队列编写求 k 阶菲波那契序列中前 n+1 项的算法，要求满足：

$f_n \le \max$ 而 $f_{n+1} > \max$，其中 max 为某个约定的常数。（注意：本题所用循环队列的容量仅为 k，则在算法执行结束时，留在循环队列中的元素应是所求 k 阶菲波那契序列中的最后 k 项）
答：
void GetFib_CyQueue（int k,int n）//求 k 阶斐波那契序列的前 n+1 项 {
InitCyQueue（Q）; //其 MAXSIZE 设置为 k for（i=0;i<k-1;i++） Q.base[i]=0;
Q.base[k-1]=1; //给前 k 项赋初值 for（i=0;i<k;i++） printf（"%d",Q.base[i]）;
for（i=k;i<=n;i++） { m=i%k;sum=0; for（j=0;j<k;j++）
sum=sum+Q.base[（m+j）%k]; Q.base[m]=sum; //求第 i 项的值存入队列中并取代已无用的第一项 printf（"%d",sum）; } }
32．在顺序存储结构上实现输出受限的双端循环队列的入列和出列（只允许队头出列）算法。设每个元素表示一个待处理的作业，元素值表示作业的预计时间。入队列采取简化的短作业优先原则，若一个新提交的作业的预计执行时间小于队头和队尾作业的平均时间，则插入在队头，否则插入在队尾。
答：
typedef struct{
ElemType *base;
int front;
int rear;
Status tag;
int MaxSize;
}DQueue;//定义队列的抽象数据类型
Status InitDQueue（Dqueue &q,int size）
{
q.MaxSize=size;
q.base=new ElemType[q.MaxSize];
if（!q.base） return FALSE;//初始化操作失败
q.front=0;
q.rear=0;
q.tag=0;
return OK;
}
Status EnDQueue（Dqueue &q,ElemType e）
{

```
if（q.front==q.rear&&q.tag） return FALSE;//队列已满，无法进行入列操作
if（q.front==q.rear&&!q.tag）    //空队列
{
q.base[q.rear]=e;
q.rear=（q.rear+1）%q.MaxSize;
if（q.rear==q.front）q.tag=1;
}
else   //非空非满，若一个新提交的作业的预计执 //行时间小于队头和队尾作业的平均时间，则插入在队头，否则插入在队尾。
{
if（e<（q.base[q.front]+q.base[（q.rear+q.MaxSize-1）%q.MaxSize]）/2）
{
//从队头入队
q.front=（q.front+q.MaxSize-1）%q.MaxSize;//计算循环队列的头指针
q.base[q.front]=e;
if（q.rear==q.front）q.tag=1;//判断队列是否已满
}
else   //从队尾入队
{
q.base[q.rear]=e;
q.rear=（q.rear+1）%q.MaxSize;//计算循环队列的尾指针
if（q.rear==q.front）q.tag=1; //判断队列是否已满
}
}
return OK;
}
Status DeDQueue（Dqueue &q,ElemType &e）
{
if（q.front==q.rear&&!q.tag）return FALSE;//空队列，无法进行出列操作
else{ //  非空队列
e=q.base[q.front];
q.front=（q.front+1）%q.MaxSize;//计算头指针位置
q.tag=0;//设置队列不满的标志
}
return OK;
}
```

3.3　考研真题与典型题详解

一、选择题

1．循环两列放在一维数组 A[0…M-1]中，end1 指向队头元素，end2 指向队尾元素的后一个位置。假设队列两端均可进行入队和出队操作，队列中最多能容纳 M-1 个元素。初始时为空，下列判断队空和队满的条件中，正确的是（　　）。[2014 年联考真题]

A．队空：end1＝＝end2；　　队满：end1＝＝（end2+1）modM

B．队空：end1＝＝end2;　队满：end2＝＝（end1+1）mod（M-1）

C．队空：end2＝＝（end1+1）modM；　队满：end1＝＝（end2+1）modM

D．队空：end1＝＝（end2+1）modM;　队满：end2＝＝（end1+1）mod（M-1）

【答案】A 查看答案

【解析】在循环队列中，在少用一个元素空间的前提下，可约定入队前，测试尾指针在循环意义下加 1 后是否等于头指针，若相等，则队满。而队空的条件还是首尾指针是否相等。

2．一个栈的入栈序列为 1,2,3,……,n，其出栈序列是 P_1，P_2，P_3……P_n。若，则 P_2＝3，则 P_3 可能取值的个数是（　　）[2013 年联考真题]

A．n-3

B．n-2

C．n-1

D．无法确定

【答案】C 查看答案

【解析】除了 3 本身以外,其他的值均可以取到,因此可能取值的个数为 n-1。

3．假设栈初始为空，将中缀表达式 $a/b+(c*d-e*f)/g$ 转换为等价后缀表达式的过程中，当扫描到 f 时，栈中的元素依次是（　　）。[2014 年联考真题]

A．$+(*-$

B．$+(-*$

C．$/+(*-*$

D．$/+-*$

【答案】B 查看答案

【解析】中缀表达式转后缀表达式遵循以下原则：

①遇到操作数，直接输出；

②栈为空时，遇到运算符，入栈；

③遇到左括号，将其入栈；

④遇到右括号，执行出栈操作，并将出栈的元素输出，直到弹出栈的是左括号，左括号不输出；

⑤遇到其他运算符"+""-""*""/"时，弹出所有优先级大于或等于该运算符的栈顶元素，然后将该运算符入栈；

⑥最终将栈中的元素依次出栈，输出。

所以扫描到’/’，入栈‘描到’+’,由于’+’优先级比’/’低，所以将’/’弹出，’+’入栈；扫描到’*’,优先级比’+’高，入栈；扫描到’（‘,入栈;扫描到’-‘,将栈中优先级更高的’*’弹出，‘-’ 入栈;扫描到’*’,优先级比’-‘高，入栈。所以扫描到 f 的时候，栈中元素为：$+(-*$

4．已知程序如下：

```
int S（int n）
{
return （n<=0） ? 0 : s（n-1）+n;
}
void main（）
```

{
cout<<S（1）;
}
程序运行时使用栈来保存调用过程的信息，自栈底到栈顶保存的信息依次对应的是（）。[2015 年联考真题]

A．main（）->S（1）->S（0）

B．S（0）->S（1）->main（）

C．main（）->S（0）->S（1）

D．S（1）->S（0）->main（）

【答案】A 查看答案

【解析】函数 S（int n）是一个递归函数：①当实际参数小于等于零时则返回 0，并终止递归；②当实际参数大于零时则递归调用 S（n-1），并将 S（n-1）的结果加上 n 作为返回值。程序从 main（）函数开始，首先调用 main（）函数；在 main（）函数中调用 S（1）函数时，将 main（）函数的上下文保存到栈中，并进入函数 S（1）；由于函数 S（1）的实际参数大于零，需要调用 S（0），故将 S（1）函数的上下文保存到栈中，进入 S（0）；在 S（0）中，实际参数小于等于零，递归终止。

5．对于栈操作数据的原则是（　　）。[青岛大学 2001 研]

A．先进先出

B．后进先出

C．后进后出

D．不分顺序

【答案】B 查看答案

【解析】先进先出是队列操作数据的原则。先进后出是栈操作数据的原则,栈限定在表尾进行插入和删除。

6．一个栈的输入序列为 1，2，3，…，n，若输出序列的第一个元素是 n，输出第 i（1≤i≤n）个元素是（　　）。

A．不确定　B．n-i+1　C．i D．n-i

【答案】B 查看答案

【解析】对于栈操作数据的原则：后进先出。最后进入的元素先出栈，因为第一个出来的元素是 n，则前边的元素必定都在栈里，第 i 个出栈的元素是 n-i+1。

7．有六个元素 6，5，4，3，2，1 顺序入栈，问下列哪一个不是合法的出栈序列（　　）。[北方交通大学 2001 研]

A．543612　B．453126　C．346521　D．234156

【答案】C 查看答案

【解析】根据栈的后进先出的特点，对于 C 选项中前两个元素得出栈顺序可以看出，4 在 5 和 6 前先出栈，又根据入栈顺序，4 在 5 和 6 后入栈，因此 4 出栈时，5 和 6 必定在栈内，且 5 在 6 之上，所以出栈时 5 要比 6 先出栈。

8．向一个栈顶指针为 h 的带头结点的链栈中插入指针 S 所指的结点时，应执行（　　）。[北京理工大学 2005 研]

A．h->next=s；

B．s->next=h；

C．s->next=h；h->next=s；

D．s->next=h-next；h->next=s；

【答案】D 查看答案

【解析】本题是向一个链栈中插入结点，可从头结点后插入。先将 s 结点指向第一个头结点之后的结点之前，再将头结点指向 s 结点。

9．输入序列为 ABC，可以变为 CBA 时，经过的栈操作为（　　）。

A．push，pop，push，pop，push，pop

B．push，push，push，pop，pop，pop

C．push，push，pop，pop，push，pop

D．push，pop，push，push，pop，pop

【答案】B 查看答案

【解析】根据输入序列和输出序列可知，输入序列全部进栈，然后再出栈。从中可以看出，push 的数目始终大于等于 pop 的数目。

10．一个递归算法必须包括（　　）。[武汉大学 2000 研]

A．递归部分　B．终止条件和递归部分　C．迭代部分　D．终止条件和迭代部分

【答案】B 查看答案

【解析】递归算法是把问题转化为规模缩小了的同类问题的子问题。然后递归调用函数（或过程）来表示问题的解。递归算法首先判断是否满足终止条件，如果满足，递归结束，如果不满足，执行递归部分的代码，因此一个递归算法包括终止条件和递归部分。

11．执行完下列语句段后，f 值为（　　）。[浙江大学 2000 研]

```
int  f(int x)
{ return  ((x>0) ? x* f(x-1):2);}
int i  ;
i =f(f(1));
```

A．2　B．4　C．8　D．无限递归

【答案】B 查看答案

【解析】该程序使用了递归调用，由题知，f（0）=2；f（1）=1*f（0）=2；f（2）=2*f（1）=4；所以结果为 4。

12．对于循环队列（　　）。[北京理工大学 2005 研]

A．无法判断队列是否为空

B．无法判断队列是否为满

C．队列不可能满

D．以上说法都不是

【答案】D 查看答案

【解析】，循环队列也会出现队列满的情况，并且循环队列也可以判断是否为空或满。至少可以通过两种方法进行判断：①另设一个布尔变量来区别队列是空还是满；②队满时，（rear+1）==font。

13．执行（　　）操作时，需要使用队列做辅助存储空间。[华中科技大学 2006 研]

A．查找哈希（Hash）表

B．广度优先搜索网

C．前序（根）遍历二叉树

D．深度优先搜索网

【答案】B 查看答案

【解析】查找哈希表不需要辅助存储空间，前序遍历二叉树和深度优先搜索网需要使用栈做辅助存储空间，广度优先搜索树需要队列做辅助存储空间。

二、填空题

1．在一个具有 n 个单元的顺序栈中，假定以地址高端（即下标为 n 的单元）作为栈底，以 top 作为栈顶指针，则当向栈中压入一个元素时，top 的变化是 top=_____。[江苏大学 2004 研]

【答案】top-1 查看答案

【解析】由于栈底在地址高端，栈中压入一个元素时，栈顶向地址底端移动一个单位，所以 top-1。

2．当两个栈共享一存储区时，栈利用一维数组 stack（1，，1）表示，两栈顶指针为 top[1]与 top[2]，则当栈 1 空时，top[1]为_____，栈 2 空时，top[2]为_____，栈满时为_____。

【答案】0；n+1；top[1]+l=top[2]查看答案

【解析】共享栈的栈底在共享存储区的两端，当栈满时栈顶相邻。

3．多个栈共存时，最好用_____作为存储结构。[南京理工大学 2001 研]

【答案】链式存储结构查看答案

【解析】链式存储结构可以随时分配内存，不需要固定的连续的内存空间，所有多个栈共存时，最好用链式存储结构。

4．顺序栈用 data[1．．n]存储数据，栈顶指针是 top，则值为 x 的元素入栈的操作是_____。[合肥工业大学 2001 研]

【答案】if（top!=n）data[++top]=x；查看答案

5．设 a=6，b=4，c=2，d=3，e=2，则后缀表达式 abc-／de*+的值为_____。[南京邮电学院 2004 研]

【答案】9 查看答案

【解析】考察由后缀表达式转换成正规表达式。

6．区分循环队列的满与空，只有两种方法，它们是_____和_____。[北京邮电大学 2001 研]

【答案】牺牲一个存储单元；设标记查看答案

【解析】牺牲一个存储单元即让 font 指针指向第一个元素的前的一个位置，当队空时，则 font 和 rear 重合；当队满时，则 rear 在 head 的前一个位置。设标记即初始化当队空时，设置一个标志位 flag=0；当元素进队时，flag=1；当元素出队时，flag=0。如果队头指针和队尾指针重合时，若 flag= 0，则队空；若 flag=1，则队满。

7．设循环队列存放在向量 sq．data[0：M]中，则队头指针 sq．front 在循环意义下的出队操作可表示为_____，若用牺牲一个单元的办法来区分队满和队空（设队尾指针 sq．rear），则队满的条件为_____。

【答案】sq．front=（sq．front+1）％（M+1）；查看答案

return（sq．data[sq．front]）；

（sq．rear+1）％（M+1）==sq．front；

8．下面程序的功能是用递归算法将一个整数按逆序存放到一个字符数组中。如 123 存放成 321。请填空：[浙江大学 2004 研]

```
#include<stdio.h>
void convert(char *a, int n)
{   int i;
    if(i=n/10)  convert(_____________,i);
    *a=_____________;
}
main(    )
     {int number;  char str[10]="";
     scanf("%d",&number);
     convert(str,number);    puts(str);
}
```

【答案】a+1，n%10 查看答案

【解析】通过递归算法，首先找到最高位的值，将其放到 str 对应的数组中，依次反向获取从高位到地位的值，将其放到数组中，完成了将整数逆序放到一个字符数组中。

三、综合题

1．假设以 S 和 X 分别表示入栈和出栈操作，则对初态和终态均为空的栈操作可由 S 和 X 生成的序列表示（如 SXSX）。

（1）试指出判别给定序列是否合法的一般规则；

（2）两个不同合法序列（对同一输入序列）能否得到相同的输出元素序列?如能得到，请举列说明。

答：（1）通常有两条规则。第一是给定序列中 S 的个数和 X 的个数相等；第二是从给定序列的开始，到给定序列中的任一位置，S 的个数要大于或等于 X 的个数。

（2）可以得到相同的输出元素序列。例如，输入元素为 A，B，C，则两个输入的合法序列 ABC 和 BAC 均可得到输出元素序列 ABC。对于合法序列 ABC，使用 S×S×S×操作序列；对于合法序列 BAC，使用 SS××S×操作序列。

2．设输入序列为 a，b，c，d，试写出借助一个栈可得到的两个输出序列和两个不能得到的输出序列。[北京科技大学 2001 研]

答：n 个元素的排列有 n!种，但借助栈结构，n 个入栈元素只可得到 l／（n+1）（（2n）！／（n!* n!））种出栈序列，这个数小于 n!。本题 4 个元素，可有 14 种出栈序列，abcd 和 dcba 就是其中两种；有 1O（4!-14=10）种排列得不到，其中，dabc 和 adbc 是不可能得到的两种。

3．已知 Q 是一个非空队列，S 是一个空栈。仅用队列和栈的 ADT 函数和少量工作变量，使用 PASCAL 或 C 语言编写一个算法，将队列 Q 中的所有元素逆置。栈的 ADT 函数有：

```
makeEmpty(s:stack);                    //置空栈
push(s:stack;value:datatype);          //新元素value入栈
pop(s:stack):datatype;                 //出栈,返回栈顶值
isEmpty(s:stack):Boolean;              //判栈空否
```

队列的 ADT 函数有：

```
enQueue(q:queue:value:datatype);   //元素value进队
deQueue(q:queue):datatype;         //出队列,返回队头值
isEmpty(q:queue):boolean;          //判队列空否
```

答：

```
void Invert(queue Q)
//Q是一个非空队列，本算法利用空栈S和已给的几个栈和队列的ADT函数，将队列Q中的元素逆置
 {makEmpty(S);                                    //置空栈
  while(!isEmpty(Q))                              //队列Q中元素出队
   {value=deQueue(Q);  push(S,value);  } //将出队元素压入栈中
  while(!isEmpty(S)) //栈中元素出栈
   {value=pop(S);   enQueue(Q,value); }  //将出栈元素入队列 Q
  }//算法Invert 结束
```

4．设整数序列 a_1，a_2，a_3，…，a_n，给出求解最大值的递归程序。[南京航空航天大学 2000 研]

答：

```
int MaxValue (int a[],int n)
//设整数序列存于数组a中，共有n个，本算法求解其最大值
{if(n==1) max=a[1];
 else if a[n]>MaxValue(a,n-1) max=a[n];
      else max=MaxValue(a,n-1);
 return(max);
 }
```

第 4 章　串

4.1　复习笔记

一、串类型的定义

1．串的逻辑结构定义

串（String）（或字符串），是由零个或多个字符组成的有限序列。一般记为

$S='a_1 a_2 \cdots a_n'$　　（n≥0）

其中，S 是串的名，用单引号括起来的字符序列是串的值；a_i（1≤i≤n）可以是字母、数字或其它字符；串中字符的数目 n 称为串的长度，零个字符的串称为空串（null string），它的长度为零。

串中任意个连续的字符组成的子序列称为该串的子串。包含子串的串相应地称为主串。通常称字符在序列中的序号为该字符在串中的位置。子串在主串中的位置则以子串的第一个字符在主串中的位置表示。

当且仅当这两个串的长度相等并且对应位置的字符全部相同，才能称两个串是相等的。

串必须用一对单引号括起来，单引号本身不属于串，它的作用只是为了避免与变量名或数值常量混淆而已。

2．串的抽象数据类型的定义

ADT String{

数据对象：$D = \{a_i \mid a_i \in CharacterSet, i = 1, 2, \cdots, n, n \geq 0\}$

数据关系：$R_1 = \{< a_{i-1}, a_i > \mid a_{i-1}, a_i \in D, i = 2, \cdots, n\}$

基本操作：StrAssign（&T，chars）

初始条件：chars 是字符串常量。

操作结果：生成一个其值等于 chars 的串 T。

StrCopy（&T，S）

初始条件：串 S 存在。

操作结果：由串 S 复制得串 T。
StrEmpty（S）
初始条件：串 S 存在。
操作结果：若 S 为空串，则返回 TRUE，否则返回 FALSE。
StrCompare（S，T）
初始条件：串 S 和 T 存在。
操作结果：若 S>T，则返回值>0；若 S=T，则返回值=0；若 S<T，则返回值<0。
StrLength（S）
初始条件：串 S 存在。
操作结果：返回 S 的元素个数，称为串的长度。
ClearString（&S）
初始条件：串 S 存在。
操作结果：将 S 清为空串。
Concat（&T，S1，S2）
初始条件：串 S1 和 S2 存在。
操作结果：用 T 返回由 S1 和 S2 联接而成的新串。
SubString（&Sub，S，pos，len）
初始条件：串 S 存在，1≤pos≤StrLength（S）且 0≤len≤StrLength（S）-pos+1。
操作结果：用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。
Index（S，T，pos）
初始条件：串 S 和 T 存在，T 是非空串，1≤pos≤StrLength（S）。
操作结果：若主串 S 中存在和串 T 值相同的子串，则返回它在主串 S 中第 pos 个字符之后第一次出现的位置；否则函数值为 0。
Replace（&S，T，V）
初始条件：串 S，T 和 V 存在，T 是非空串。
操作结果：用 V 替换主串 S 中出现的所有与 T 相等的不重叠的子串。
StrInsert（&S，pos，T）
初始条件：串 S 和 T 存在，1≤pos≤StrLength（S）+1。
操作结果：在串 S 的第 pos 个字符之前插入串 T。
StrDelete（&S，pos，len）
初始条件：串 S 存在，1≤pos≤StrLength（S）-len+1。
操作结果：从串 S 中删除第 pos 个字符起长度为 len 的子串。
DestroyStrig（&S）
初始条件：串 S 存在。
操作结果：串 S 被撤销。
}ADT String

二、串的表示和实现

如果在程序设计语言中，串只是作为输入或输出的常量出现，则只需存储此串的串值，即字符序列即可串有三种机内表示方法：定长顺序存储表示、堆分配存储表示、串的块链存储表示。

1．定长顺序存储表示

（1）定长数组的描述

类似于线性表的顺序存储表示，用一组连续的存储单元存储串值的字符序列。在串的定长顺序存储结构中，按照预定义的大小，为每个定义的串变量分配一个固定长度的存储区。

（2）串长度的表示方法

①如上述定义描述的那样，以下标为 0 的数组分量存放串的实际长度，如 PASCAL 语言中的串类型采用这种表示方法。

②在串值后面加一个不计入串长的结束标记字符，如在有的 C 语言中以“\0”表示串值的终结。

2．堆分配存储表示

这种存储表示的特点是，仍以一组地址连续的存储单元存放串值字符序列，但它们的存储空间是在程序执行过程中动态分配所得。在 C 语言中，存在一个称之为“堆”的自由存储区。并由 C 语言的动态分配函数 malloc（）和 free（）来管理。利用 malloc（）为每个新产生的串分配一块实际串长所需的存储空间，若分配成功，则返回一个指向起始地址的指针，作为串的基址。

3．块的串链存储表示

和线性表的链式存储结构相类似，也可采用链表方式存储串值。由于串结构的特殊性—结构中的每个数据元素是一个字符，则用链表存储串值时，存在一个“结点大小”的问题，即每个结点可以存放一个字符，也可以存放多个字符。例如，图 4-1（a）是结点大小为 4（即每个结点存放四个字符）的链表，图 4-1（b）是结点大小为 1 的链表。当结点大于 1 时，由于串长不一定是结点大小的整倍数，则链表中的最后一个结点不一定全被串值占满，此时通常补上“#”或其它的非串值字符（通常“#”不属于串的字符集，是一个特殊符号）。

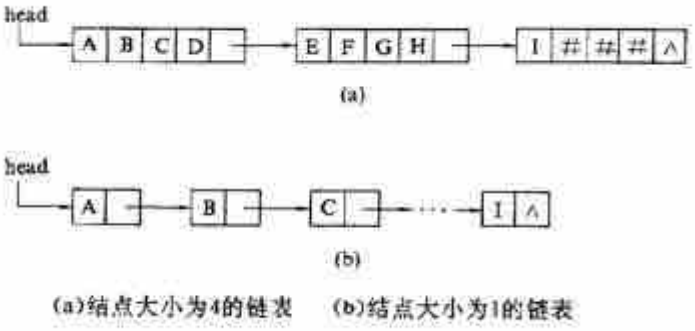


图 4-1 串值的链表存储方式

尾指针是为了便于进行连接操作，但应注意连接时需处理第一个串尾的无效字符。

在链式存储方式中，结点大小的选择和顺序存储方式的格式选择一样都很重要，它直接影响着串处理的效率。链式存储的存储密度小（如结点大小为 1 时），运算处理方便，然而，存储空间占用量大。如果在串处理过程中需进行内、外存数据交换的话，则会因为内外存交换操作过多而影响处理的总效率。

三、串的模式匹配算法

1．求子串位置的定位函数 Index（S，T，pos）

子串的定位操作通常称作串的模式匹配（其中 T 称为模式串），是各种串处理系统中最重要的操作之一。根据定长顺序存储结构，写出的不依赖于其他串操作的匹配算法如下。

```
int Index（SString S，SString T，int pos）{
//返回子串 T 在主串 S 中第 pos 个字符之后的位置。若不存在，则函数值为 0.
//其中，T 非空，1≤pos≤StrLength（S）。
i=pos；    j=1；
while（i<=S[0]&&j<=T[0]）{
if（S[i]==T[j]）{++i；  ++j；}
```

```
else（j>T[0]） return i-T[0]；
}
if（j>T[0]） return i-T[0]；
else return 0；
}
```

算法的基本思想是：从主串 S 的第 pos 个字符起和模式的第一个字符比较之。若相等，则继续逐个比较后续字符；否则从主串的下一个字符起再重新和模式的字符比较之。以此类推，直至模式 T 中的每个字符依次和主串 S 中的一个连续的字符序列相等，则称匹配成功，函数值为和模式 T 中第一个字符相等的字符在主串 S 中的序号，否则称匹配不成功，函数值为零。

2．模式匹配的一种改进算法

这种改进算法是 D.E.Knuth 与 V.R.Pratt 和 J.H.Morris 同时发现的，因此称之为克努特一莫里斯一普拉特算法（简称为 KMP 算法），此算法可以在 O（n+m）的时间数量级上完成串的模式匹配操作。

改进之处在于：每当一趟匹配过程中出现字符比较不等时，不需回溯指示主串的指针 i，而是利用已经得到的“部分匹配”的结果将模式向右“滑动”尽可能远的一段距离后，继续进行比较。

KMP 算法的最大特点是指示主串的指针不需回溯，整个匹配过程中，对主串仅需从头至尾扫描一遍。这对处理从外设输入的庞大文件很有效，可以边读入边匹配，而无需回头重读。

四、串操作应用举例

1．文本索引

文本编辑程序是一个面向用户的系统服务程序，广泛用于源程序的输入和修改，甚至用于报刊和书籍的编辑排版以及办公室的公文书信的起草和润色。文本编辑的实质是修改字符数据的形式和格式。

2．建立词索引表

信息检索是计算机应用的重要领域之一。由于信息检索的主要操作是在大量的存放在磁盘上的信息中查询一个特定的信息，为了提高查询效率，一个重要的问题是建立一个好的索引系统。在实际系统中，按书名检索并不方便，因为很多内容相似的书籍其书名不一定相同。因此较好的办法是建立“书名关键词索引”。

4.2 强化习题详解

【基础知识题】

1．简述空串和空格串（或称空格符串）的区别。

答：（1）空格串是指一个或多个空格字符（ASCII 码为 20H）组成的串，串值为所包含的空格；而空串中没有任何字符，串值为 NULL。

（2）空串的长度为零；空格串的长度为所包括的空格字符的个数。

2．对于串的各个基本操作：串赋值、串比较、求串长、串连接、求子串、定位操作 INDEX、置换操作 REPLACE（s，t，v）、插入操作 INSERT（s，pos，t）和删除操作 DELETE（s，pos，len），讨论是否可由其他基本操作构造而得？如何构造？

答：串赋值（StrAssign）、串比较（StrCompare）、求串长（StrLength）、串连接（Concat）、求子串（SubString）这五种基本操作构成串类型的最小操作子集。

定位操作 INDEX（s，t）、置换操作 REPLACE（s，t，v）、插入操作 INSERT（s，pos，t）和删除操作 DELETE（s，pos，len）均可由上述五种基本操作构造而得。如插入操作可由串比较和串赋值构造而来。其算法如下所示：

```
viod repl（Sting &s,String v）
{
k=Index（s,t）;
if（k）
{
StrAssign（temp,’’）;
n=StrLength;
m=StrLength（s）;
while（k）{
StrAssign（temp,Concat（temp,SubString（s,1,k-1）,v）;
m-=（k-1）-n;
StrAssign（s,SubString（s,k+m,m））;
k=Index（s,t）;
}
StrAssign（s,Concat（temp,s））;
}
}
```

3．设 s="I AM A STUDENT"，t="GOOD"，q="WORKER"。

求:StrLength（s），StrLength（t），SubString（s,8,7），SubString（t,2,1），Index（s,"A"），Index（s,t），Replace（s,"STUDENT",q），Concat（SubString（s,6,2），Concat（t,SubString（s,7,8））））。

答：

StrLength（s）=14

StrLength（t）=4

SubString（s,8,7）= "STUDENT"

SubString（t,2,1）= "O"

Index（s,"A"）=3

Index（s,t）=0

Replace（s,"STUDENT",q）= "I AM A WORKER"

Concat（SubString（s,6,2），Concat（t,SubString（s,7,8）））= "A GOOD STUDENT"

4．已知下列字符串

a="THIS"，f="A SAMPLE"，c="GOOD"，d="NE"，b=" "，

s=Concat（a,Concat（SubString（f,2,7），Concat（b,SubString（a,3,2））），

t=Replac（f,SubString（f,3,6）,c），

u=Concat（SubString（c,3,1）,d），　g="IS"

v=Concat（s,Concat（b,Concat（t,Concat（b,u））））），

试问: s，t，v，StrLength（s），Index（v,g），Index（u,g）各是什么？

答：

s=’THIS SAMPLE IS’

t=’A GOOD’

v=’THIS SAMPLE IS A GOOD ONE’

StrLength（s）=14

Index（v,g）=3

Index（u,g）=0

5．试问执行以下函数会产生怎样的输出结果？

```
Void demonstrate（）{
StrAssign（s,"THIS IS A BOOK"）;
Replace（s,SubString（s,3,7）,"ESE ARE"）;
StrAssign（t,Concat（s,"S"））;
StrAssign（u,"XYXYXYXYXYXY"）;
StrAssign（v,SubString（u,6,3））;
StrAssign（w,"W"）;
Printf（"t=",t,"v=",v,"u=",Replace（u,v,w,））;
}//demonstrate
```

答：

t="THESE ARE BOOKS"

v="YXY"

u="XWXWXW"

6．已知;s="（XYZ）+*",t="（X+Z）*Y"试利用连接,求子串和置换等基本运算,将 s 转化为 t。

答：

算法设计：

```
s1=SubString（s,3,1）
s2=SubString（s,6,1）
Replace（s,s1,s2）
Concat（s3,s,s1）
Concat（t,SubString（s3,1,5）,SubString（s3,7,2））
```

7．令 s="aaab",t="abcabaa",u="abcaabbabcabaacbacba"..试分别求出它们的 next 函数值和 nextrval 函数值。

答：

（1）next[j]=0123 nextrval[j]=0003

（2）next[j]=0111232 nextrval[j]=0110132

（3）next[j]=011122312345322112l1 nextrval[j]=01102130110532210210

8．在以链表存储串值时,存储密度是结点大小和串长的函数。假设每个字符占一个字节,每个指针占 4 个字节,每个结点的大小为 4 的整数倍.已知串长的分布函数为 f（l）,求结点大小为 4k,串长为 l 时的存储密度 d（4k,l）（用公式表示）。

答：每个结点大小为 4k，存储密度为 $$d(4k,l)=\frac{l}{4(k+1)\bullet\left\lfloor\frac{l}{4k}\right\rfloor}$$。

【算法设计题】

在编写 10 至 14 题的算法时,请采用 StringType 数据类型;

StringType 是串的一个抽象数据类型,它包含以下五种基本操作:

```
Void StrAssign（StringType &t,StringType s）
//将 s 的值赋给 t。s 的实际参数可以是串变量或者串常量（如:"abcd"）。
int StringCompare（StringType s,StringType t）
//比较 s 和 t。若 s>t,返回值>0,若 s=t,返回值=0;若 s<T,返回值<.0。
int StrLength（StringType s）
```

返回 s 中的元素个数，即该串的长度。
StringType Concat（StringType s,StringType t）
//返回由 s 和 t 连接而成的新串
StringType SubString（StringType s,int start ,int len）
//当 1≤start≤ StrLength（s） 且 0≤len ≤StrLength（s）-start+1 时返回 s 中第 start 个字符起长度为//len 的子串,否则返回空串.
10．编写对串求逆的递推算法。
答：
```
void Reverse（StringType &s）     //求 s 的逆串
/* Reverse s by iteration. */
{
int i=0,j=StrLength（s）-1;
char temp;
while（i<=j）     ////交换头尾元素
{    temp=s[i];
s[i]=s[j];
s[j]=temp;
i++;
j--;
}
}
```
11．编写算法,求得所有包含在串 s 中而不包含在串 t 中的字符（s 中重复的字符只选一个）构成的新串 r,以及 r 中每个字符在 s 中第一次出现的位置。
答：
```
void String_Subtract（Stringtype s,Stringtype t,Stringtype &r）//求所有包含在串 s 中而 t 中没有的
   //字符构成的新串 r
{
StrAssign（r, " "）;
for（i=1;i<=Strlen（s）;i++）
{
StrAssign（c,SubString（s,i,1））;
for（j=1;j<i&&StrCompare（c,SubString（s,j,1））;j++）; //判断 s 的当前字符 c 是否第一次
   //出现
if（i==j）
{
   for（k=1;k<=Strlen（t）&&StrCompare（c,SubString（t,k,1））;k++）; //判断当前字符是
//否包含在 t 中
   if（k>Strlen（t）） StrAssign（r,Concat（r,c））;
}
}//for
}//String_Subtract
```
12．编写一个实现串的置换操作 Replace（&S,T,V）的算法。
答：

int Replace（Stringtype &S,Stringtype T,Stringtype V）;//将串 S 中所有子串 T 替换为 V,并返
//回置换次数
{
for（n=0,i=1;i<=Strlen（S）-Strlen（T）+1;i++） //注意 i 的取值范围
if（!StrCompare（SubString（S,i,Strlen（T））,T）） //找到了与 T 匹配的子串，分别把 T
的前
//面和后面部分保存为 head 和 tail
{
StrAssign（head,SubString（S,1,i-1））;//T 的前面部分 head
StrAssign（tail,SubString（S,i+Strlen（T）,Strlen（S）-i-Strlen（T）+1））;//T 的后面部分
tail
StrAssign（S,Concat（head,V））;//将串 V 连接到 head，并赋给串 S
StrAssign（S,Concat（S,tail））; //把 head,V,tail 连接为新串
i+=Strlen（V）; //当前指针跳到插入串以后
n++;
}//if
return n;
}//Replace
13．编写算法,从串 s 中删除所有和串 t 相同的子串。
答：
int Delete_SubString（Stringtype &s,Stringtype t）//从串 s 中删除所有与 t 相同的子串,并返
//回删除次数
{
for（n=0,i=1;i<=Strlen（s）-Strlen（t）+1;i++）
if（!StrCompare（SubString（s,i,Strlen（t））,t） //找到与 t 相同的子串
{
StrAssign（head,SubString（S,1,i-1））;
StrAssign（tail,SubString（S,i+Strlen（t）,Strlen（s）-i-Strlen（t）+1）;
StrAssign（S,Concat（head,tail））;//把 head,tail 连接为新串，此时与 t 相同的子串已删除
n++;//删除次数加 1
}//if
return n;
}//Delete_SubString
14．利用串的基本操作以及栈和集合的基本操作,编写"由一个算术表达式的前缀式求后缀
式"的递推算法（假设前缀式不含语法错误）。
答：
Status NiBoLan_to_BoLan（Stringtype str,Stringtype &new）/*利用栈把前缀表达式 str 转换
为后缀式 new*/
{
Initstack（s）; //s 的元素为 Stringtype 类型
for（i=1;i<=Strlen（str）;i++）
{
r=SubString（str,i,1）;//循环判断每个字符
if（r 为字母） push（s,r）;//如果是字母字符，即操作数，则压入栈 s

```
else
{
   if（StackEmpty（s）） return ERROR;
   pop（s,a）;//如果是运算符，则弹出栈顶操作数 a
   if（StackEmpty（s）） return ERROR;
   pop（s,b）; /*如果是运算符，则弹出栈顶操作数 b，两次弹出的操作数进行运算，并将运算结果存入 b 中*/
   StrAssign（t,Concat（r,b））;
   StrAssign（c,Concat（t,a））; //把算符 r，子前缀表达式 a,b 连接为新子前缀表达式 c
   push（s,c）;
}
}//for
pop（s,new）;
if（!StackEmpty（s）） return ERROR;
return OK;
}//NiBoLan_to_BoLan
```

在编写 15 至 20 题的算法时不允许调用串的基本操作。

15．编写算法,实现串的基本操作 StringAssign（&T,chars）。

答：

```
void StrAssign（Stringtype &T,char chars&#）//用字符数组 chars 给串 T 赋值
{
for（i=0,T[0]=0;chars[i];T[0]++,i++） T[i+1]=chars[i];
}//StrAssign
```

16．编写算法,实现串的基本操作 StrCompare（S,T）。

答：

```
char StrCompare（Stringtype s,Stringtype t）/*串的比较，s>t 时返回正数，s=t 时返回 0，s<t 时返回负数*/
{
for（i=1;i<=s[0]&&i<=t[0]&&s[i]==t[i];i++）;//逐个比较串 s 和 t 的每个字符
if（i>s[0]&&i>t[0]） return 0;
else if（i>s[0]） return -t[i];
else if（i>t[0]） return s[i];
else return s[i]-t[i];
}//StrCompare
```

17．编写算法,实现串的基本操作 Replace（&S,T,V）。

答：

```
int String_Replace（Stringtype &S,Stringtype T,Stringtype V）;/*将串 S 中所有子串 T 替换为 V,并返回置换次数*/
{
for（n=0,i=1;i<=S[0]-T[0]+1;i++）
{
for（j=i,k=1;T[k]&&S[j]==T[k];j++,k++）;
if（k>T[0]） //找到了与 T 匹配的子串，分三种情况处理：
{
```

```
    if（T[0]==V[0]）
    for（l=1;l<=T[0];l++） //新子串长度与原子串相同时，直接替换
    S[i+l-1]=V[l];
    else if（T[0]<V[0]） //新子串长度大于原子串时，先将串 S 后面部分后移
    {
    for（l=S[0];l>=i+T[0];l--）
S[l+V[0]-T[0]]=S[l];
    for（l=1;l<=V[0];l++）
S[i+l-1]=V[l];
    }
    else //新子串长度小于原子串时，先将串 S 后面部分前移
    {
    for（l=i+V[0];l<=S[0]+V[0]-T[0];l++）
S[l]=S[l-V[0]+T[0]];
    for（l=1;l<=V[0];l++）
S[i+l-1]=V[l];
    }
    S[0]=S[0]-T[0]+V[0];
    i+=V[0];n++;
}//if
}//for
return n;
}//String_Replace
```

18．编写算法,求串 s 所含不同字符的总数和每种字符的个数。

答：

```
typedef struct {
char ch;
int num;
} mytype;
void StrAnalyze（Stringtype S）//统计串 S 中字符的种类和每种字符的个数
{
mytype T[MAXSIZE]; //用结构体数组 T 存储统计结果
for（i=1;i<=S[0];i++）
{
c=S[i];j=0;
while（T[j].ch&&T[j].ch!=c） j++; //查找当前字符 c 是否已记录过
if（T[j].ch） T[j].num++;
else T[j]={c,1};
}//for
for（j=0;T[j].ch;j++）
printf（"%c: %d\n",T[j].ch,T[j].num）;
}//StrAnalyze
```

19．在串的定长顺序存储结构上直接实现 11 题要求的算法。

答：

```
void Subtract_String（Stringtype s,Stringtype t,Stringtype &r）/*求所有包含在串 s 中而 t 中没有的字符构成的新串 r*/
{
r[0]=0;
for（i=1;i<=s[0];i++）
{
c=s[i];
for（j=1;j<i&&s[j]!=c;j++）;
if（i==j）  //判断 s 的当前字符 c 是否第一次出现
{
   for（k=1;k<=t[0]&&t[k]!=c;k++）;
   if（k>t[0]） r[++r[0]]=c; //判断当前字符是否包含在 t 中
}
}//for
}//Subtract_String
```

20．编写算法,从 s 中删除所有和串相同的子串。

答：

```
int SubString_Delete（Stringtype &s,Stringtype t）/*从串 s 中删除所有与 t 相同的子串，并返回删除次数*/
{
for（n=0,i=1;i<=s[0]-t[0]+1;i++）
{
for（j=1;j<=t[0]&&s[i+j-1]==t[i];j++）;
if（j>m） //找到了与 t 匹配的子串
{
   for（k=i;k<=s[0]-t[0];k++） s[k]=s[k+t[0]]; //左移删除
   s[0]-=t[0];n++;
}
}//for
return n;
}//Delete_SubString
```

21．假设以结点大小为 1 且附设头结点的链表结构表示串,试编写实现下列六种串的基本操作 StrAssign

答：

```
typedef struct
{
char ch;
LStrNode *next;
} LStrNode,*LString; //链串结构
void StringAssign（LString &s,LString t）//把串 t 赋值给串 s
{
s=malloc（sizeof（LStrNode））;//动态创建结点
for（q=s,p=t->next;p;p=p->next）
{
```

```
r=（LStrNode*）malloc（sizeof（LStrNode））;
r->ch=p->ch;
q->next=r;q=r;
}
q->next=NULL;
}//StringAssign
void StringCopy（LString &s,LString t）/*把串 t 复制为串 s。与前一个赋值操作的区别在于,串 s 也已存在*/
{
for（p=s->next,q=t->next;p&&q;p=p->next,q=q->next）
{
p->ch=q->ch; pre=p;//pre 是 p 的前驱结点
}
while（q）
{
p=（LStrNode*）malloc（sizeof（LStrNode））;
p->ch=q->ch;
pre->next=p; pre=p;
}
p->next=NULL;//尾结点
}//StringCopy
char StringCompare（LString s,LString t）/*串的比较,s>t 时返回正数；s=t 时返回 0；s<t 时返回负数*/
{
for（p=s->next,q=t->next;p&&q&&p->ch==q->ch;p=p->next,q=q->next）;//s 与 t 中的元素逐个比较
if（!p&&!q） return 0;
else if（!p） return -（q->ch）;
else if（!q） return p->ch;
else return p->ch-q->ch;
}//StringCompare
int StringLen（LString s）//求串 s 的长度（元素个数）
{
for（i=0,p=s->next;p;p=p->next,i++）;//指针依次向后移动，直到表尾
return i;
}//StringLen
LString * Concat（LString s,LString t）//连接串 s 和串 t 形成新串，并返回指针
{
p=malloc（sizeof（LStrNode））;//动态创建指针结点
for（q=p,r=s->next;r;r=r->next）
{
q->next=（LStrNode*）malloc（sizeof（LStrNode））;
q=q->next;
q->ch=r->ch; //复制串 s
```

```
}//for
for（r=t->next;r;r=r->next）
{
q->next=（LStrNode*）malloc（sizeof（LStrNode））;
q=q->next;
q->ch=r->ch; //复制串 t
}//for
q->next=NULL; 此时指针 q 指向串 s 和串 t 形成的新串的尾部
return p;//指针 p 即新串的头指针
}//Concat
LString * Sub_String（LString s,int start,int len）//返回一个串,其值等于串 s 从 start 位置起长为 len 的子串
{
p=malloc（sizeof（LStrNode））;q=p;
for（r=s;start;start--,r=r->next）; //找到 start 所对应的结点指针 r
for（i=1;i<=len;i++,r=r->next）
{
q->next=（LStrNode*）malloc（sizeof（LStrNode））;
q=q->next;
q->ch=r->ch;
} //复制串 t
q->next=NULL;
return p;//p 为指向所求串的头指针
}//Sub_String
```

22．假设以块链结构表示串,试编写将串 s 插入到串 t 中某个字符之后的算法,（若串 t 中不存在此字符,则将串 s 连接在串的末尾）。

答：

```
void LString_Concat（LString &t,LString &s,char c）/*用块链存储结构,把串 s 插入到串 t 的字符 c 之后*/
{
p=t.head;
while（p&&!（i=Find_Char（p,c））） p=p->next; //查找字符 c
if（!p） //没找到
{
t.tail->next=s.head;
t.tail=s.tail; //把 s 连接在 t 的后面
}
else
{
q=p->next;
r=（Chunk*）malloc（sizeof（Chunk））; //将包含字符 c 的节点 p 分裂为两个
for（j=0;j<i;j++） r->ch[j]="#"; //原结点 p 包含 c 及其以前的部分
for（j=i;j<CHUNKSIZE;j++） //新结点 r 包含 c 以后的部分
{
```

```
    r->ch[j]=p->ch[j];
    p->ch[j]="#"; //将 p 的后半部分和 r 的前半部分的字符改为无效字符"#"
}
p->next=s.head;
s.tail->next=r;
r->next=q; //把串 s 插入到结点 p 和 r 之间
}//else
t.curlen+=s.curlen; //修改串长
s.curlen=0;
}//LString_Concat
int Find_Char（Chunk *p,char c）//在某个块中查找字符 c,如找到则返回位置，否则返回 0
{
for（i=0;i<CHUNKSIZE&&p->ch[i]!=c;i++）;
if（i==CHUNKSIZE） return 0;
else return i+1;
}//Find_Char
```

23．假设以块链结构作为串的存储结构.试编写判别给定串是否是具有对称性的算法,并要求算法的时间复杂度为 O（StrLength（S））。

答：

```
int LString_Palindrome（LString L）/*判断以块链结构存储的串 L 是否为回文序列，是则返回 1,否则返回 0*/
{
InitStack（S）;
p=S.head;i=0;k=1; //i 指示元素在块中的下标，k 指示元素在整个序列中的序号（从 1 开始）

for（k=1;k<=S.curlen;k++）
{
if（k<=S.curlen/2） Push（S,p->ch[i]）; //将串前半段的字符入串
else if（k>（S.curlen+1）/2）
{
    Pop（S,c）; //将后半段的字符与串中的元素逐个相匹配
    if（p->ch[i]!=c） return 0; //失配
}
if（++i==CHUNKSIZE） //转到下一个元素，当为块中最后一个元素时，转到下一块
{
    p=p->next;
    i=0;
    }
}//for
return 1; //成功匹配
}//LString_Palindrome
```

在编写 24 到 26 题的算法时,请采用堆分配存储表示。

24．试写一算法,在串的堆存储结构上实现基本操作 Concat（&T,s1.s2）。

答：

```
void HString_Concat（HString s1,HString s2,HString &t）/*将堆结构表示的串 s1 和 s2 连接为新串 t*/
{
if（t.ch） free（t.ch）;//将串 t 清空以存储新串
t.ch=malloc（（s1.length+s2.length）*sizeof（char））;
for（i=1;i<=s1.length;i++） t.ch[i-1]=s1.ch[i-1];//s1 作为新串 t 的前半部分
for（j=1;j<=s2.length;j++,i++） t.ch[i-1]=s2.ch[j-1];//s2 作为新串 t 的后半部分
t.length=s1.length+s2.length;//新串的长度
}//HString_Concat
```

25．试写一算法,实现堆存储结构的串的置换操作 Replace（&S,T,V）。

答：

```
int HString_Replace（HString &S,HString T,HString V）//堆结构串上的置换操作，返回置换次数
{
for（n=0,i=0;i<=S.length-T.length;i++）
{
for（j=i,k=0;k<T.length&&S.ch[j]==T.ch[k];j++,k++）;
if（k==T.length） //找到了与 T 匹配的子串，分三种情况处理：
{
if（T.length==V.length）
for（l=1;l<=T.length;l++） //新子串长度与原子串相同时，直接替换
    S.ch[i+l-1]=V.ch[l-1];
else if（T.length<V.length） //新子串长度大于原子串时，先将串 S 的后半部分往后移
{
for（l=S.length-1;l>=i+T.length;l--）
    S.ch[l+V.length-T.length]=S.ch[l];
for（l=0;l<V.length;l++）
    S[i+l]=V[l];
}
else //新子串长度小于原子串时，先将串 S 的后半部分往前移
{
for（l=i+V.length;l<S.length+V.length-T.length;l++）
    S.ch[l]=S.ch[l-V.length+T.length];
for（l=0;l<V.length;l++）
    S[i+l]=V[l];
}
S.length+=V.length-T.length;
i+=V.length;n++;
}//if
}//for
return n;
}//HString_Replace
```

26．试写一算法,实现堆存储结构的串的插入操作 StrIsert（&S,pos,T）。

答：

```
Status HString_Insert（HString &S,int pos,HString T）/*把 T 插入堆结构表示的串 S 的第 pos 个字符之前*/
{
if（pos<1） return ERROR;//第 1 个位置前无法进行插入
if（pos>S.length） pos=S.length+1;//当插入位置大于串长时，看作添加在串尾
S.ch=realloc（S.ch,（S.length+T.length）*sizeof（char））;
for（i=S.length-1;i>=pos-1;i--）
S.ch[i+T.length]=S.ch[i]; //后移为新插入的字符串让出位置
for（i=0;i<T.length;i++）
S.ch[pos+i-1]=T.ch[pos]; //插入串 T
S.length+=T.length;
return OK;
}//HString_Insert
```

27．当以定长顺序结构表示串时,可如下所述改进定位函数的算法;先将模式串 t 中的第一个字符和最后一个字符与主串 s 中相应的字符比较,在两次比较都相等之后,再依次从 t 的第二个字符起逐个比较,这样做可以克服算法 Index（算法 4.5）在模式串"ak b"（ak 表示连续 k 个字符"a" ）在主串"an b"（k≤n）中的定位函数时产生的弊病,试编写上述改进算法,并比较这两种算法在作 Index（"an b","ak b"）运算时所需进行的字符间的比较次数。

答：

```
int Index_New（Stringtype s,Stringtype t）//改进的定位算法
{
i=1;j=1;
while（i<=s[0]&&j<=t[0]）
{
if（（j!=1&&s[i]==t[j]）||（j==1&&s[i]==t[j]&&s[i+t[0]-1]==t[t[0]]））
{ //当 j==1 即匹配模式串的第一个字符时，需同时匹配其最后一个
    i=i+j-2;
    j=1;
}
else//如果字符不匹配，指示主串的指针和指示匹配串的指针同时后移
{
    i++;
    j++;
}
}//while
if（j>t[0]） return i-t[0];
}//Index_New
```

28．假设以结点大小为 1,（带头结点）的链表的结构表示串,则在利用 next 函数值进行匹配时,在每个结点中需要设三个域:数据域 chdata,指针域 succ 和指针域 next.其中 chdata 域存放一个字符,succ 域存放指向同一链表中后继结点的指针;next 域在主串中存放指向同一链表中前驱结点的指针,在模式串中,存放指向当该结点的字符与主串中的字符不等时,模式串中下一个应进行比较的字符的 next 函数值为零,则其 next 域的值应指向头结点,试按上述定义的结构改写模式串的 next 函数值的算法。

答：

```
void LGet_next（LString &T）//链串上的 get_next 算法
{
p=T->succ;p->next=T;q=T;
while（p->succ）
{
if（q==T || p->data==q->data）//找到匹配的字符
{
    p=p->succ;
    q=q->succ;
    p->next=q;
}
else q=q->next;
}//while
}//LGet_next
```

29．试按 28 题定义的结构改写串匹配的改写算法（KMP 算法）。

答：

```
LStrNode * LIndex_KMP（LString S,LString T,LStrNode *pos）/*链串上的 KMP 匹配算法，返回值为匹配的子串头指针*/
{
p=pos;q=T->succ;
while（p&&q）
{
if（q==T||p->chdata==q->chdata）//字符匹配，指示主串和指示子串的指针同时后移
{
    p=p->succ;
    q=q->succ;
}
else q=q->next;
}//while
if（!q）
{
for（i=1;i<=Strlen（T）;i++）
    p=p->next;
return p;
} //发现匹配后，要往回找子串的头
return NULL;
}//LIndex_KMP
```

30．假设以定长顺序存储结构表示串,试设计一个算法,求串 s 中出现的第一个最长重复子串及其位置,并分析你的算法的时间复杂度。

答：

```
void Get_LRepSub（Stringtype S）//求 S 的最长重复子串的位置和长度
{
for（maxlen=0,i=1;i<S[0];i++）//串 S2 向右移 i 格
{
```

```
for（k=0,j=1;j<=S[0]-i;j++）/*j 为串 S2 的当前指针,此时串 S1 的当前指针为 i+j,两指针同步移动*/
{
   if（S[j]==S[j+i]） k++; //用 k 记录连续相同的字符数
   else k=0; //失配时 k 归零
   if（k>maxlen） //发现了比以前发现的更长的重复子串
   {
   lrs1=j-k+1;lrs2=mrs1+i;maxlen=k; //作记录
   }
}//for
}//for
if（maxlen）
{
printf（"Longest Repeating Substring length:%d\n",maxlen）;
printf（"Position1:%d Position 2:%d\n",lrs1,lrs2）;
}
else printf（"No Repeating Substring found!\n"）;
}//Get_LRepSub
```

31．假设定长顺序存储结构表示串,试设计一个算法,求串 s 和串 t 的一个最长公共子串,并分析你的算法的时间复杂度,若要求第一个出现的最长公共子串（既它在串 s 和串 t 的最左边的 位置上出现）和所有的最长公共子串,讨论你的算法能否实现。

答：

```
void Get_LPubSub（Stringtype S,Stringtype T）//求串 S 和串 T 的最长公共子串位置和长度
{
if（S[0]>=T[0]）
{
StrAssign（A,S）;StrAssign（B,T）;
}
else
{
StrAssign（A,T）;StrAssign（B,S）;
} //为简化设计，令 S 和 T 中较长的那个为 A，较短的那个为 B
for（maxlen=0,i=1-B[0];i<A[0];i++）
{
if（i<0） //i 为 B 相对于 A 的错位值,向左为负,左端对齐为 0，向右为正
{
   jmin=1;jmax=i+B[0];
}//B 有一部分在 A 左端的左边
else if（i>A[0]-B[0]）
{
   jmin=i;jmax=A[0];
}//B 有一部分在 A 右端的右边
else
{
```

```
    jmin=i;jmax=i+B[0];
}//B 在 A 左右两端之间.
//以上是根据 A 和 B 不同的相对位置确定 A 上需要匹配的区间（与 B 重合的区间）的端点:
jmin,jmax.
for（k=0,j=jmin;j<=jmax;j++）
{
    A[j]==B[j-i]） k++;
    else k=0;
    if（k>maxlen）
    {
    lps1=j-k+1;lps2=j-i-k+1;maxlen=k;
    }
}//for
}//for
if（maxlen）
{
if（S[0]>=T[0]）
{
    lpsS=lps1;lpsT=lps2;
}
else
{
    lpsS=lps2;lpsT=lps1;
} //将 A,B 上的位置映射回 S,T 上的位置
printf（"Longest Public Substring length:%d\n",maxlen）;
printf（"Position in S:%d Position in T:%d\n",lpsS,lpsT）;
}//if
else printf（"No Repeating Substring found!\n"）;
}//Get_LPubSub
```

4.3 考研真题与典型题详解

一、选择题

1．已知字符串 S 为“abaabaabacacaabaabcc”，模式串 t 为“abaabc”，采用 KMP 算法进行匹配，第一次出现“失配”（s[i]!＝t[i]）时，i＝j＝5，则下次开始匹配时，i 和 j 的值分别是（）。[2015 年联考真题]

A．i＝1，j＝0

B．i＝5，j＝0

C．i＝5，j＝2

D．i＝6，j＝2

【答案】C 查看答案

【解析】模式匹配（KMP）算法对普通的暴力匹配的改进在于：每当匹配过程中匹配失败时，主串（本题为 S）的指针（i）不需要回溯，而是利用已经得到的“部分匹配”的结果将模式串（t）向右“滑动”尽可能远的一段距离后，继续进行比较。模式串“滑动”的距离是由模式串（t）本身决定的，即 t 的子串 t[0…j-1]中前缀串和后缀串相等的最长长度。本题中第一次失配 i＝5，字串为“abaab”，其相等且最长的前后缀为“ab”，一次下一个 j＝2。

2．设有两个串 S1 和 S2，求 S2 在 S1 中首次出现的位置的运算称作（　　）。[中南大学 2005 研]

A．求子串　B．判断是否相等　C．模型匹配　D．连接

【答案】C 查看答案

【解析】常用的串的基本操作有七种，INDEX（s,t）是其中的定位函数，这种运算就是所说的模式匹配。

3．设 S 为一个长度为 n 的字符串，其中的字符各不相同，则 S 中的互异的非平凡子串（非空且不同于 S 本身）的个数为（　　）。

A．2n-1

B．n^2

C．（n^2 / 2）+（n / 2）

D．（n^2 / 2）+（n / 2）-1

【答案】D 查看答案

【解析】因为长度 n 的字符串，字符各异，连续 2 个、3 个等等的子字符都不相同，所以个数为（n+2）（n-1）/2，答案就为（n^2 / 2）+（n / 2）-1。

4．串的长度是指（　　）。[北京工商大学 2001 研]

A．串中所含不同字母的个数

B．串中所含字符的个数

C．串中所含不同字符的个数

D．串中所含非空格字符的个数

【答案】B 查看答案

【解析】串中字符的数目 n 称为字符的长度，不必考虑其中单个字符是否相等。

5．已知串 S=′aaab′，其 Next 数组值为（　　）。

A．0123　B．1123　C．123l　D．1211

【答案】A 查看答案

【解析】KMP 算法的 next 数组建立的原则

$$next[j]=\begin{cases}0 & \text{当 } j=1 \text{ 时}\\ \max\{k \mid 1<k<j \text{ 且 } 'p_1\cdots p_{k-1}'='p_{j-k+1}\cdots p_{j-1}'\} & \text{当此集合不空时}\\ 1 & \text{其它情况}\end{cases}$$

二、填空题

1．模式串 P=′abaabcac′的 next 函数值序列为____。[西安电子科技大学 2001 研]

【答案】01122312 查看答案

2．空格串是指____，其长度等于____。[西安电子科技大学 2001 研]

【答案】由空格字符（ASCII 值 32）所组成的字符串；空格个数查看答案

3．组成串的数据元素只能是____。

【答案】字符查看答案

4．设 T 和 P 是两个给定的串，在 T 中寻找等于 P 的子串的过程称为____，又称 P 为____。

【答案】模式匹配；模式串查看答案

5．实现字符串拷贝的函数 strcpy 为：

```
void strcpy(char *s , char *t) / *copy t to s* /
{ while (_____________)
}
```

【答案】*s++=*t++或（*s++=*t++）!=′\0′查看答案

6．完善算法：求 KMP 算法．next 数组。

```
PROC get _next(t:string,VAR next:ARRAY[1..t.len] OF integer);
BEGIN
 j:=1; k:=__(1)__;  next[1]:=0;
 WHILE j<t.len DO
 IF k=0 OR t.ch[j]=t.ch[k] THEN BEGIN j:=j+1; k:=k+1; next[j]:=k;END
                           ELSE k:=__(2)__;
```

END;

【答案】（1）0 （2）next[k]查看答案

三、综合题

1．两个字符串 s1 和 s2 的长度分别为 m 和 n。求这两个字符串最大共同子串算法的时间复杂度为 T（m，n）。估算最优的 T（m，n），并简要说明理由。

答：最优的 T（m，n）是 D（n）。串 S2 是串 S1 的子串，且在 S1 中的位置是 1。开始求出最大公共子串的长度恰是串 S2 的长度。一般情况下，T（m，n）=D（m*n）。

2．给出模式串′abacabaaad′在 KMP 算法中的 next 和 nextval 数组。[北京邮电大学 2000 研]

答：模式串的 next 函数定义如下：

$$next[j]=\begin{cases}0, & \text{当 } j=1 \text{ 时}\\ \max\{k \mid 1<k<j \text{ 且 } ' p_1\cdots p_{k-1}' = ' p_{j-k+1}\cdots p_{j-1}' \}, & \text{当此集合不空时}\\ 1, & \text{其他情况}\end{cases}$$

根据此定义，可求解模式串′abacabaaad′的 next 和 nextval 值如下：

j	1	2	3	4	5	6	7	8	9	10
模式串	a	b	a	c	a	b	a	a	a	d
$next[j]$	0	1	1	2	1	2	3	4	2	2
$nextval[j]$	0	1	0	2	0	1	0	4	2	2

3．在改进了的（无回溯）字符串模式匹配中，要先求 next 数组的值。下面是求 nextval 值的算法。

```
TYPE SAR=ARRAY[1..m] OF INTEGER;
     PTY=ARRAY[1..m] OF CHAR;
PROCEDURE next2(P:PTY;VAR NEXTVAL:SAR);
{在模式P中求nextval数组的值}
1 BEGIN
2 J:=1;NEXTVAL[1]:=0;K:=0
3 REPEAT
4   IF (K=0) OR (P[J]=P[K])
5     THEN [ J:=J+1;K:=K+1;
6            IF P[J]=P[K]
7              THEN NEXTVAL[J]:=NEXTVAL[K]
8              ELSE NEXTVAL[J]:=K ]
9     ELSE K:=NEXTVAL[K]
10  UNTIL J=m
11 END;
```

算法中第 4 行有 P[J]=P[K]，第六行中也有 P[J]=P[K]。两处比较语句相同。请分析说明此两处比较语句的含义是什么?分析此算法在最坏情况下的时间复杂度是多少?

答：第 4 行的 P[J]=P[K]语句是测试模式串的第 J 个字符是否等于第 K 个字符，如是，则指针 J 和 K 均增加 1，继续比较。第 6 行的 p[J]=p[K]语句的意义是，当第 J 个字符在模式匹配中失配时，若第 K 个字符和第 J 个字符不等，则下个与主串匹配的字符是第 K 个字符；否则，若第 K 个字符和第 J 个字符相等，则下个与主串匹配的字符是第 K 个字符失配时的下一个（即 NEXTVAL[K]）。

该算法在最坏情况下的时间复杂度 O（m^2）。

4．设 s、t 为两个字符串，分别放在两个一维数组中，m、n 分别为其长度，判断 t 是否为 s 的子串。

如果是，输出子串所在位置（第一个字符），否则输出 0（注：用程序实现）。[中科院研究生院 2003 研]

答：判断字符串 t 是否是字符串 s 的子串，称为串的模式匹配，其基本思想是对串 s 和 t 各设一个指针 i 和 j，i 的值域是 0．．m—n，j 的值域是 0．．n-1。初始值 i 和 j 均为 0。模式匹配从 S0#和 t0#开始，若 S0#=t0#，则 i 和 j 指针增加 l，若在某个位置 s_i!=t_j，则主串指针 i 回溯到 i=i-j+1，j 仍从 0 开始，进行下一轮的比较，直到匹配成功（j>n-1），返回子串在主串的位置（i-j）。

否则，若 i>m-n 则为匹配失败。

```
int index(char s[],t[],int m,n)
//字符串s和t用一维数组存储,其长度分别为m和n
//本算法求字符串t在字符串s中的第一次出现,如匹配成功输出子串在s中的位置,否则输出0
{int i=0,j=0;
 while(i<=m-n && j<=n-1)
   if(s[i]==t[j]){i++;j++;}     //对应字符相等,指针后移
      else {i=i-j+1;j=0;}       //对应字符不相等,i回溯,j仍为0
   if(i<=m-n && j==n) {printf("t在s串中位置是%d",i-n+1);return(i-n+1);}//匹配成功
else  return(0); //匹配失败
}//算法index结束
main ()//主函数
{char s[],t[]; int m,n,i;
 scanf("%d%d",&m,&n);   //输入两字符串的长度
 scanf("%s",s);         //输入主串
 scanf("%s",t);         //输入子串
 i=index(s,t,m,n);
}//程序结束
```

5．写出一个递归算法来实现字符串逆序存储。

答：实现字符串的逆置并不难，但本题“要求不另设串存储空间”来实现字符串逆序存储，即第一个输入的字符最后存储，最后输入的字符先存储，使用递归能够实现这一功能，先存储后输入的数据，然后反序存储后输入的数据。

```
void InvertStore(char A[])
  //字符串逆序存储的递归算法
{char ch;
 static int i = 0;   //需要使用静态变量
 scanf ("%c",&ch);
```

```
if(ch!= '.')          //规定'.'是字符串输入结束标志
    {InvertStore(A);
    A[i++] = ch; //字符串逆序存储
  }
A[i] = '\0';          //字符串结尾标记
}//结束算法InvertStore
```

第 5 章 数组和广义表

5.1 复习笔记

一、数组的定义

1．抽象数据类型数组定义

ADT Array{

数据对象：j_i=0,…,b_i-1，i=1,2，…，n，

D={ $a_{j_1 j_2 \cdots j_n}$ |n（>0）称为数组的维数，b_i 是数组第 i 维的长度，j_i 是数组元素的第 i 维下标，

$a_{j_1 j_2 \cdots j_n} \in$ ElemSet}

数据关系：R={R1，R2，…，Rn}

Ri={< $a_{j_1 \cdots j_i \cdots j_n}$, $a_{j_1 \cdots j_{i+1} \cdots j_n}$ >|0≤j_k≤b_k-1,1≤k≤n 且 k≠i，0≤j_i≤b_i-

2， $a_{j_1 \cdots j_i \cdots j_n}$, $a_{j_1 \cdots j_{i+1} \cdots j_n} \in$ D，i=2，…，n}

基本操作：

InitArray（&A，n，bound1，…，boundn）

操作结果：若数组 n 和各维长度合法，则构造相应的数组 A，并返回 OK。

DestroyArray（&A）

操作结果：销毁数组 A。

Value（A，&e，index1，…，indexn）

初始条件：A 是 n 维数组，e 为元素变量，随后是 n 个下标值。

操作结果：若个下标不超界，则 e 复制为所指定的 A 的元素值，并返回 OK。

Assign（&A，e，index1，…，indexn）

初始条件：A 是 n 维数组，e 为元素变量，随后是 n 个下标值。

操作结果：若下标不越界，则将 e 的值赋给所指定的 A 的元素，并返回 OK。

} ADT array

2．数组和线性表的关系

可以把二维数组看成是这样一个定长线性表：它的每个数据元素也是一个定长线性表。图 5-1（a）所示是一个二维数组，以 m 行 n 列的矩阵形式表示。它可以看成是一个线性表。

A=（α_0，α_1，…，α_p）（p=m-1 或 n-1）

其中每一个数据元素 α_j 是一个列向量的线性表

α_j=（a_{0j},a_{2j}，…，$a_{m-1,\ j}$） 1≤j≤n-1

（如图 5-1（b）所示）或者 α_i 是一个行向量的线性表

α_i=（a_{i1}，a_{i2}，…，$a_{i,\ n-1}$） 1≤i≤m-1

（如图 5-1（c）所示）。在 C 语言中，一个二维数组可以用其分量类型为一维数组的数组来定义。

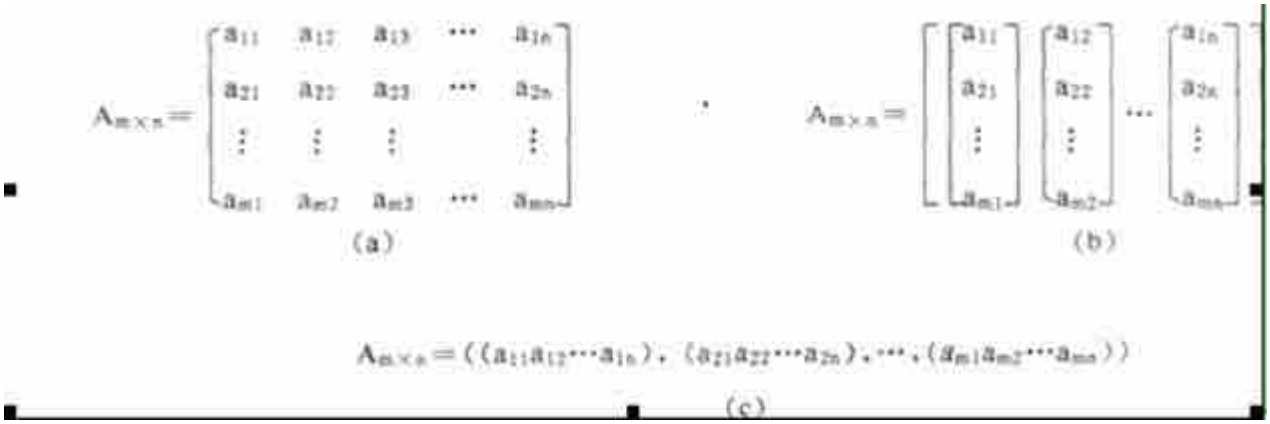


图 5-1 二维数组图例

（a）矩阵形式表示；（b）列向量的一维数组；（c）行向量的一维数组。

数组一旦被定义，它的维数和维界就不再改变。因此，除了结构的初始化和销毁之外，数组只有存取元素和修改元素值的操作。

二、数组的顺序表示和实现

1．二维数组的两种存储方式

由于数组一般不作插入或删除操作，也就是说，一旦建立了数组，则结构中的数据元素个数和元素之间的关系就不再发生变动，因此采用顺序存储结构。二维数组可有两种存储方式：一种以列序为主序（column-major-order）的存储方式，如图 5-2（a）所示；另一种是以行序为主序（row-major-order）的存储方式，如图 5-2（b）所示。

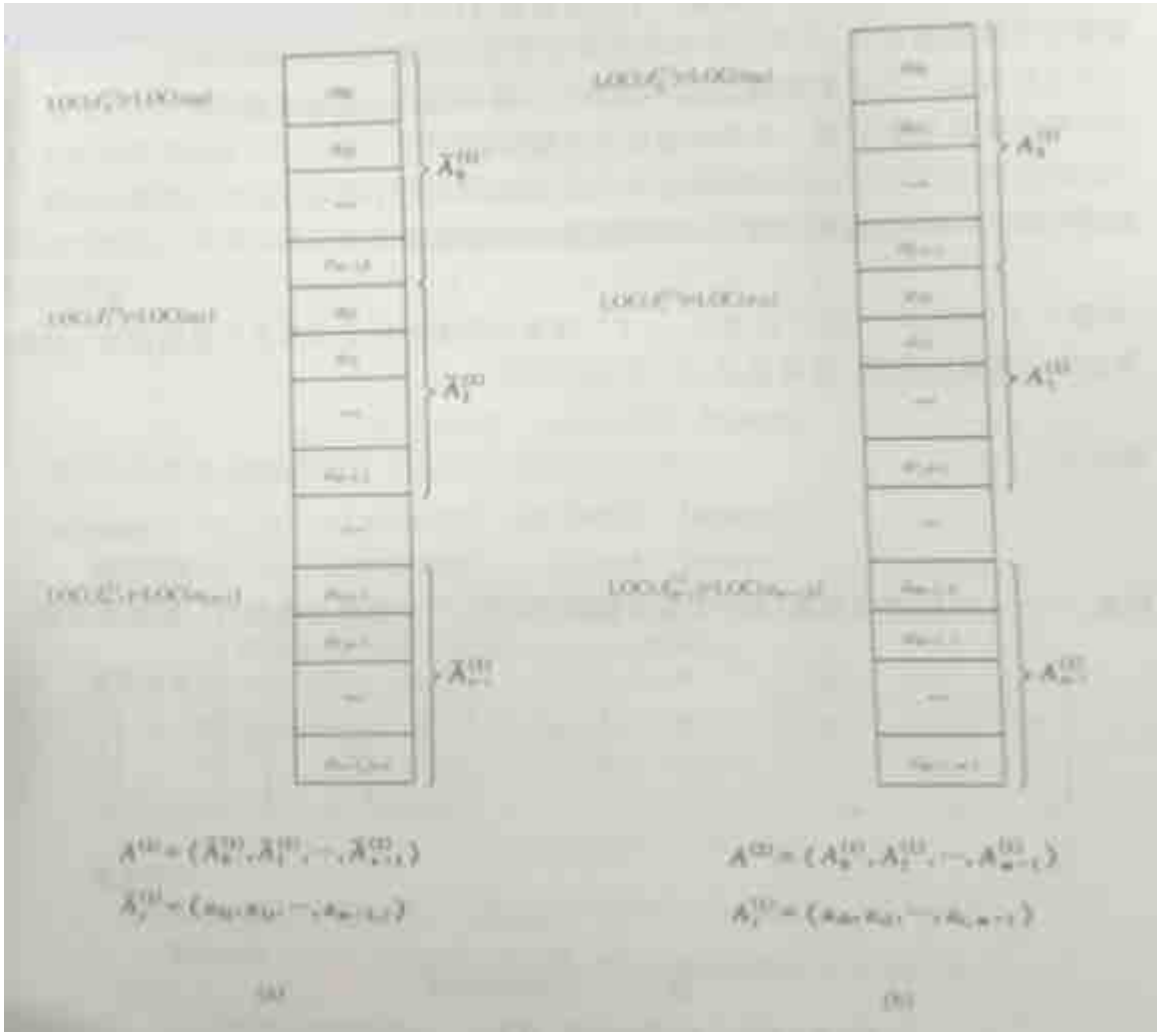


图 5-2 二维数组的两种存储方式

2．数组元素的存储位置

（1）数组元素存储位置的计算

以行序为主序的存储结构为例，n 维数组中的数据元素存储位置的计算公式为：

$$LOC(j_1,j_2,\cdots,j_n)=LOC(0,0,\cdots,0)+(b_2\times\cdots b_n\times j_1+b_3\times\cdots\times j_2$$

$$+\cdots+b_n\times j_{n-1}+j_n)L=LOC(0,0,\cdots,0)+(\sum_{i=1}^{n-1}\prod_{k=j+1}^{n}b_k+j_n)L$$

可缩写成

$$LOC(j_1,j_2,\cdots,j_n)=LOC(0,0,\cdots,0)+\sum_{i=1}^{n}c_ij_i$$

其中 $c_n=L$，$c_{i-1}=b_i\times c_i, 1<i\leq n$。

（2）随机存储结构

数组元素的存储位置是其下标的线性函数，一旦确定了数组的各维的长度，c_i 就是常数。由于计算各个元素存储位置的时间相等，所以存取数组中任一元素的时间相等。我们称具有这一特点的存储结构为随机存储结构。

三、矩阵的压缩存储

在数值分析中经常出现一些阶数很高的矩阵，同时在矩阵中有许多值相同的元素或者是零元素。为了节省存储空间，可以对这类矩阵进行压缩存储。即为多个值相同的元只分配一个存储空间，对零元不分配空间。

假若值相同的元素或者零元素在矩阵中的分布有一定的规律，则我们称此类矩阵为特殊矩阵；反之，称为稀疏矩阵。

1．特殊矩阵

（1）对称矩阵

若一个 n 阶矩阵 A 中的元满足下述性质 $a_{ij}=a_{ji}$（$1\le i,j\le n$），则称其为 n 阶对称矩阵。

（2）对称矩阵的存储

若将每一对对称元分配一个存储空间，则可将 n^2 个元压缩存储到 $n(n+1)/2$ 个元的空间中。以一维数组 sa[k]为例， sa[k]中存储的元素如图 5-3 所示。



图 5-3 对矩阵的压缩存储

其中 sa[k]和矩阵元 a_{ij} 之间存在的对应关系如下：

$$k=\begin{cases}\dfrac{i(i-1)}{2}+j & \text{当 } i\ge j\\ \dfrac{j(j-1)}{2}+i & \text{当 } i<j\end{cases}$$

（3）特殊矩阵的特点

上述对称矩阵的存储方式也适用于三角矩阵和对角矩阵。一般统称的特殊矩阵中，非零元的分布都有一个明显的规律，从而可将其压缩存储到一维数组中，并找到每个非零元在一维数组中的对应关系。

2．稀疏矩阵

（1）稀疏矩阵的定义

稀疏矩阵的概念只能凭人们的直觉来了解。假设在 m×n 的矩阵中，有 t 个元素不为零。令

$\delta=\dfrac{t}{m\times n}$，称 δ 为矩阵的稀疏因子。通常认为 $\delta\le 0.05$ 时称为稀疏矩阵。

（2）三元组顺序表

①三元组顺序表的定义

假设以顺序存储结构来表示三元组表，则可得稀疏矩阵的一种压缩存储方式——我们称之为三元组顺序表。

②三元组顺序表的运算

压缩存储结构下矩阵的两种运算：

a．转置；

b．相乘。

（3）十字链表

①十字链表的定义

同一行的非零元听过 right 域链接成一个线性链表，同一列的非零元通过 down 域链接成一个线性链表，每一个非零元既是某个行链表中的一个结点，又是某个列链表中的一个结点，整个矩阵构成了一个十字交叉的链表，故称这样的存储结构为十字链表。

②采用十字链表的原因

当矩阵非零元的位置或个数经常变动时，三元组就不适合于作稀疏矩阵的存储结构。

③十字链表的特点

链表中，每个非零元用一个结点表示，结点中除了表示非零元的三元组外（row，col，val）还有两个链域：向下域（down）和向右域（right），如图 5-4 所示。

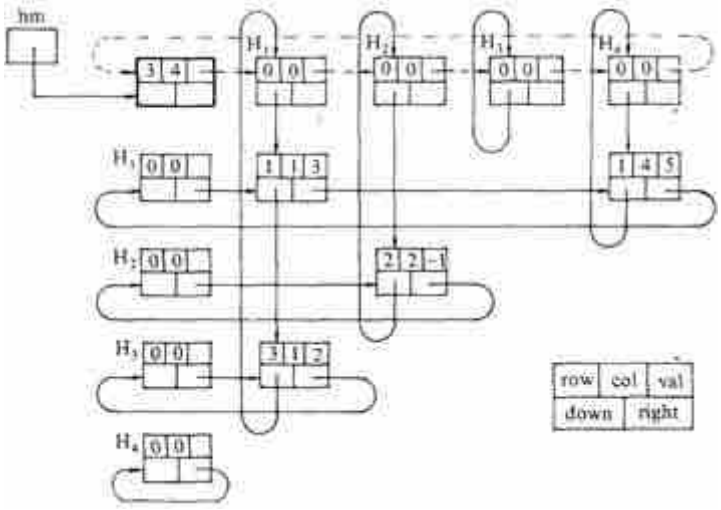


图 5-4 稀疏矩阵的十字链表

四、广义表

1．广义表的定义

广义表是线性表的推广。也有人称其为列表（1ists，用复数形式以示与统称的表 list 的区别）。广义表一般记作

$$LS=(\alpha_1,\alpha_2,\cdots,\alpha_n)$$

LS 是广义表$(\alpha_1,\alpha_2,\cdots,\alpha_n)$的名称，n 是它的长度。在线性表的定义中，$\alpha_i(1\le i\le n)$只限于是单个元素。而在广义表中，$\alpha_i$可以是单个元素，也可以是广义表，分别称为广义表 LS 的原子和子表。

2．广义表的举例

广义表的定义是一个递归的定义。

（1）A=（ ）——A 是一个空表，它的长度为零。

（2）B=（e）——广义表 B 只有一个单元素 e，它的长度为 1。

（3）C=（a，（b，c，d））——广义表 C 的长度为 2，两个元素分别为单元素 a 和子表（b，c，d）。

（4）D=（A，B，C）——广义表 D 的长度为 3，三个元素都是子表表。

（5）E=（a，E）——这是一个递归的表，它的长度为 2，E 相当于一个无限的列表。

3．列表的三个重要结论：

（1）列表的元素可以是子表，而子表的元素还可以是子表……。由此，列表是一个多层次的结构，可以用图形象地表示。

（2）列表可为其它列表所共享。

（3）列表可以是一个递归的表，即列表也可以是其本身的一个子表。

五、广义表的存储结构

广义表中的数据元素可以具有不同的结构（或是单元素，或是列表），因此难以用顺序存储结构表示，通常采用链式存储结构，每个数据元素可用一个结点表示。

1．设定结点的结构

若列表不空，则可分解成表头和表尾；反之，一对确定的表头和表尾可唯一确定列表。由此，一个表结点可由三个域组成：标志域、指示表头的指针域和指示表尾的指针域，而元素结点只需两个域：标志域和值域。结点的结构说明如图 5-5 所示。

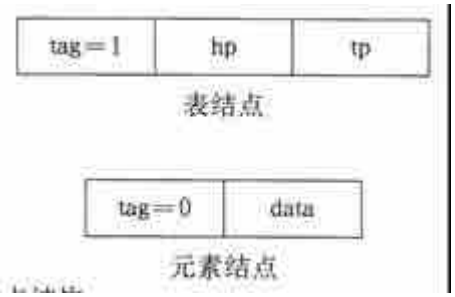


图 5-5 结点的结构

2．广义表的存储结构

广义表的存储结构如图 5-6 所示。

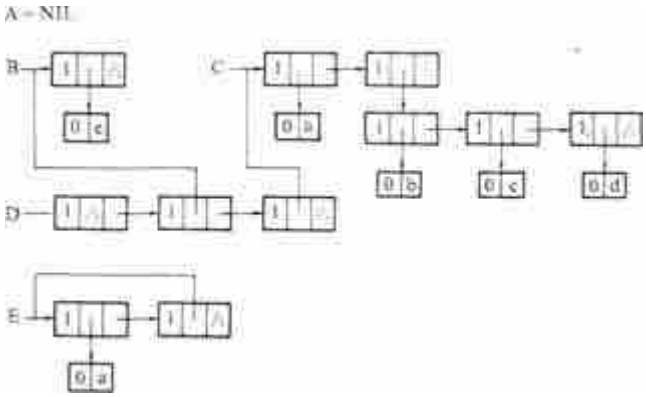


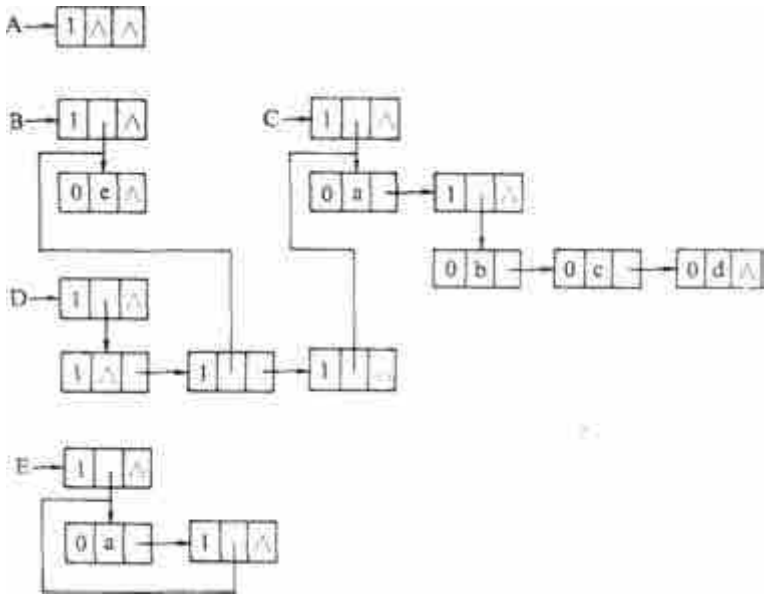
图 5-6 广义表的存储结构示例

3．其他存储结构

广义表的另一种结点结构的链表示列表，如图 5-7 和图 5-8 所示。



图 5-7 另一种存储结构的结点



5-8 广义表的另一种存储结构

六、m 元多项式的表示

广义表中的数组元素可以是另外一个广义表，一个 m 元多项式的表示是广义表的典型应用。

对任何一个 m 元多项式都可进行如此操作：先分解出一个主变元，随后再分解出第二个变元，等等。由此，一个 m 元的多项式首先是它的主变元的多项式，而其系数又是第二变元的多项式，由此可用广义表来表示 m 元多项式。

七、广义表的递归算法

1．求广义表的深度

广义表的深度定义为广义表中括弧的重数，是广义表的一种度量。求广义表深度的递归函数如下所示。

```
int GListDepth（GList L）{
//采用头尾链表存储结构，求广义表 L 的深度
if（!L） return 1；
if（L->tag==ATOM） retrun 0；
for（max=0，pp=L；pp；pp=pp->ptr.tp）{
dep=GListDepth（pp->ptr->hp）；
if（dep>max） max=dep；
}
return max+1；
}
```

2．复制广义表

复制广义表只需复制其表头和表尾，然后合成即可。复制广义表的递归算法如下所示。

```
Status CopyGList（GList &T，GList L）{
if（!L） T=NULL；
else{
if（!T=（GList）malloc（sizeof（GLNode））） exit（OVERFLOW）；
T->tag=L->tag；
if（L->tag==ATOM） T->atom=L->atom；
```

```
else{
CopyGList（T->ptr.hp，L->ptr.hp）；
CopyGList（T->ptr.tp，L->ptr.tp）；
}
}
return OK；
}
```

3．建立广义表的存储结构

建广义表存储结构的递归过程如图 5-11 所示。

```
Status CreateGList（GList &L，SString S）{
if（StrCompare（S,emp））  L=NULL；
else{
if（!L=（GList）malloc（sizeof（GLNode））） exit（OVERFLOW）；
if（StrLength（S）==1）{L->tag=ATOM；L->atom=S；}
else{
L->tag=LIST；p=L；
SubString（sub，S，2，StrLength（S）-2）；
do{
sever（sub，sub）；
CreateGList（p->ptr.hp，hsub）；q=p；
if（!StrEmpty（sub））{
if（!（p=（GLNode *）malloc（sizeof（GLNode））））
exit（OVERFLOW）；
p->tag=LIST；  q->ptr.tp=p；
}
}while（！StrEmpty（sub））；
q->ptr.tp=NULL；
}else
}
return OK；
}
Status sever（SString &str，SString &hstr）{
n=StrLength（str）；i=0；k=0；
do{
++i；
SubString（ch，str，i，1）；
if（ch==’（’） ++k；
else if（ch==’）’） --k；
}while（i<n&&（ch!=’，’||k!=0））；
if（i<n）{
SubString（hstr，str，1，i-1）；SubString（str，str，i+1，n-i）；
else{StrCopy（hstr，str）；  ClearString（str）；}
}
}
```

5.2 强化习题详解

【基础知识题】

1．假设有二维数组 A6×8，每个元素用相邻的 6 个字节存储，存储器按字节编址。已知 A 的起始存储位置（基地址）为 1000，计算:

（1）数组 A 的体积（即存储量）;

（2）数组 A 的最后一个元素 a[5][7]的第一个字节的地址;

（3）按行存储时，元素 a[1][4]的第一个字节的地址;

（4）按行存储时，元素 a[4][7]的第一个字节的地址;

答：（1）数组 A 的体积：6×8×6=288 Byte

（2）LOC（5,7）=1000+（5×8+7）×6=1282

（3）LOC（1,4）=1000+（1×8+4）×6=1072

（4）LOC（4,7）=1000+（7×6+4）×6=1276

2．假设按低下标优先存储整数数组 A9×3×5×8 时，第一个元素的字节地址是 100，每个整数占四个字节，问下列元素的存储地址是什么？

（1）a0000 （2）a1111 （3）a3125 （4）a8247

答：（1）LOC（0,0,0,0）=100

（2）LOC（1,1,1,1）=100+（1×3×5×8 + 1×5×8 + 1×8 + 1）×4=776

（3）LOC（3,1,2,5）=100+（3×3×5×8 + 1×5×8 + 2×8 + 5）×4=1784

（4）LOC（8,2,4,7）=100+（8×3×5×8 + 2×5×8 + 4×8 + 7）×4=4416

3．按高下标优先存储方式（以最右的下标为主序），顺序列出数组 A9×3×5×8 中所有元素 aijkl，为了简化表达，可以只例出（i,j,k,l）的序列。

答：（0,0,0,0） （1,0,0,0） （0,1,0,0） （1,1,0,0） （0,0,1,0） （1,0,1,0） （0,1,1,0）
（1,1,1,0）
（0,0,2,0） （1,0,2,0） （0,1,2,0） （1,1,2,0） （0,0,0,1） （1,0,0,1） （0,1,0,1）
（1,1,0,1）
（0,0,1,1） （1,0,1,1） （0,1,1,1） （1,1,1,1） （0,0,2,1） （1,0,2,1） （0,1,2,1）
（1,1,2,1）
（0,0,0,2） （1,0,0,2） （0,1,0,2） （1,1,0,2） （0,0,1,2） （1,0,1,2） （0,1,1,2）
（1,1,1,2）
（0,0,2,2） （1,0,2,2） （0,1,2,2） （1,1,2,2）

4．将下列公式改写为一个等式的形式。

$$k_Array = (D^{(k)}, R^{(k)}) \quad (1 \leqslant k \leqslant n)$$

其中，

$$D^{(k)} = \left\{ A^{(k)}_{[i_k]} \;\middle|\; \begin{array}{l} c[n-k+1] \leqslant i[k] \leqslant d[n-k+1] \\ k=1 \text{ 时 } A^{(k)}_{[i_k]} \in D_0 \\ k>1 \text{ 时 } A^{(k)}_{[i_k]} \text{ 是 } k-1 \text{ 维数组} \end{array} \right\}$$

$$R^{(k)} = \{AR\}$$

$$AR = \left\{ < A^{(k)}_{[i_k]}, A^{(k)}_{[i_k]+1} > \;\middle|\; \begin{array}{l} c[n-k+1] \leqslant i[k] \leqslant d[n-k+1]-1 \\ A^{(k)}_{[i_k]}, A^{(k)}_{[i_k]+1} \in D \end{array} \right\}$$

答：

$$k = \frac{a(a+1)}{2} + b$$

-1

其中，a=Max{i,j}，b=Min{i,j}

5．设有上三角矩阵（a_{ij}）n×n，将其上三角元素逐行存于数组 B[m]中（m 充分大），使得 B[k]=a_{ij} 且 k=f_1（i）+ f_2（j） +c 。试推导出函数 f_1，f_2 和常数 c （要求 f_1 和 f_2 中不含常数项）。

答：

$$k = ni - (n - j) - \frac{i(i-1)}{2} - 1$$

（i<=j）

$$f_1(i) = (n + \frac{1}{2})i - \frac{1}{2}i^2, \quad f_2(j) = j, \quad c = -(n+1)$$

6．设有三对角矩阵（a_{ij}）n×n，将其上三角元素逐行存于数组 B[3][n]中,使 B[3][n]的元素 B[u][v]= a_{ij},试推导出从（i,j）到（u,v）的下标变换公式。

答：u=i-j+1， v=j-1

7．设有三对角矩阵（a_{ij}）n×n，将其上三角元素逐行存于数组 B[3n-2]中，使得 B[3n-2]中的元素 B[k]=a_{ij} ,求:

用 i ,j 表示 k 的下标变换公式;

用 k 表示 i,j 的下标变换公式。

答：（1） k=2（i-1）+j-1 （$|i-j| \le 1$）

（2） i=（k+1） DIV 3 + 1， （$0 \le k \le 3n-1$）

j= k+1-2（i-1）=k+1-2 （k DIV 3）

8．已知 A 为稀疏矩阵，试从空间和时间角度比较采用两种不同的存储结构（二维数组和三元组表）完成求累加运算的优缺点。

答：设稀疏矩阵为 n 行 n 列，其中的非零元为 m 个，且 m 远小于n^2。

①从时间上来说，采用二维数组存储稀疏矩阵,要将所有的矩阵元素累加起来，需要用一个两层的嵌套循环,做n^2-1 次加法运算，时间复杂度为 O（n^2）；而用三元组,将所有的矩阵元素累加起来只需将三元组顺序表扫描一遍,做 m-1 次加法运算,时间复杂度为 O（m）。

②从空间上来说，用二维数组需要n^2个基本存储单元,空间复杂度为 O（n^2），而三元组需要 m 个基本存储单元外加 2m 个整型存储单元，空间时间复杂度为 O（m）。由于n^2远远大于 m，故实际存储空间也较大。

9．求下列广义表操作的结果:

GetHead【（p,h,w）】;

GetTail【（b,k,p,h）】;

GetHead【（a,b）,（c,d）】;

GetTail【（a,b）,（c,d）】;

GetHead【GetTail【（a,b）,（c,d）】】;

GetTail【GetHead【（a,b）,（c,d）】】;

GetHead【GetTail【GetHead【（a,b）,（c,d）】】】;

GetTail【GetHead【GetTail【（a,b），（c,d）】】】;
注意:【】是函数的符号。

答：

（1） GetHead【（p,h,w）】= p

（2） GetTail【（b,k,p,h）】= （k,p,h）

（3） GetHead【（（a,b），（c,d））】= （a,b）

（4） GetTail【（（a,b），（c,d））】= （（c,d））

（5） GetHead【GetTail【（（a,b），（c,d））】】= GetHead【（（c,d））】= （c,d）

（6） GetTail【GetHead【（（a,b），（c,d））】】= GetTail【（a,b）】= （b）

（7） GetHead【GetTail【GetHead【（（a,b），（c,d））】】】= GetHead【（b）】= b

（8） GetTail【GetHead【GetTail【（（a,b），（c,d））】】】= GetTail【（c,d）】= （d）

10．利用广义表的 GetHead 和 GetTail 操作写出如上题的函数表达式，把原子 banana 分别从下列广义表中分离出来。

L1=（apple,pear,banana,orange）;

L2=（（apple,pear），（banana,orange））;

L3=（（（apple），（pear），（banana），（orange）））;

L4=（apple,（pear），（（banana））,（（（orange））））;

L5=（（（（apple））），（（pear））,（banana）,orange）;

L6=（（（（apple）,pear）,banana）,orange）;

L7=（apple,（pear,（banana）,orange））;

答：

（1） GetHead【GetTail【GetTail【L_1】】】;

（2） GetHead【GetHead【GetTail【L_2】】】;

（3） GetHead【GetHead【GetTail【GetTail【GetHead【L_3】】】】】;

（4） GetHead【GetHead【GetHead【GetTail【GetTail【L_4】】】】】;

（5） GetHead【GetHead【GetTail【GetTail【L_5】】】;

（6） GetHead【GetTail【GetHead【L_6】】】;

（7） GetHead【GetHead【GetTail【GetHead【GetTail【L_7】】】】】;

11．按图 5-1 所示结点结构，画出下列广义表的存储结构图，并求它的深度。

（1）（（（））,a,（（b,c），（）,d），（（（e））））

（2）（（（（a）,b））,（（（）,d），（e,f））

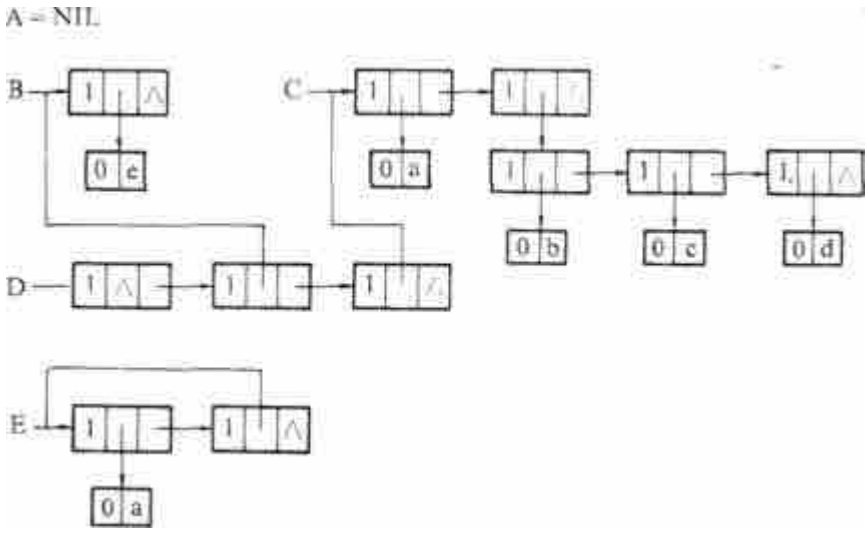
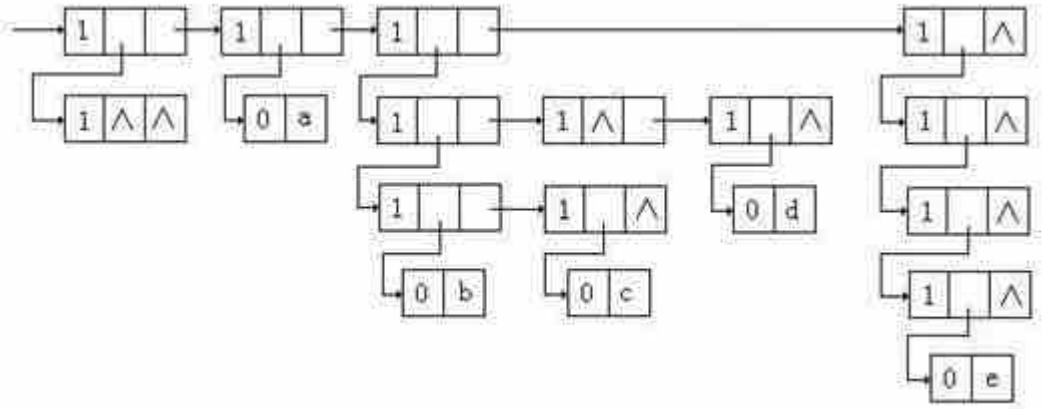


图 5-1 广义表的存储结构示例

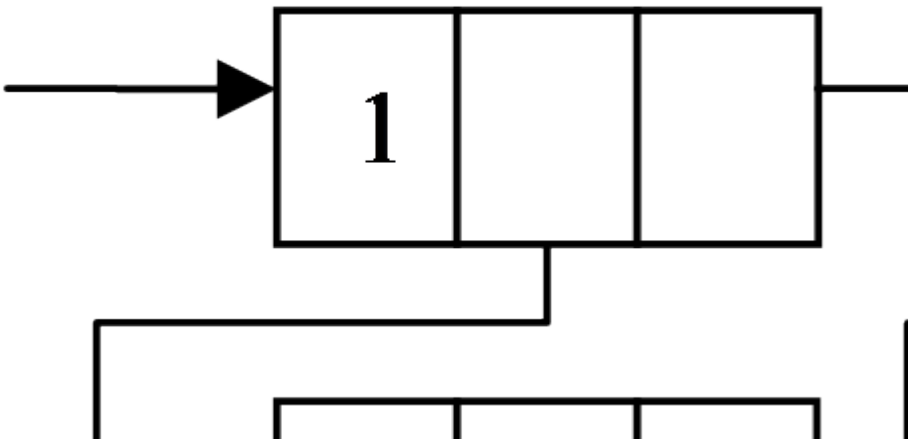
答：

（1）（（（））,a,（（b,c）,（）,d）,（（（e））））



该广义表的深度为 4

（2）（（（（a）,b））,（（（）,d）,（e,f））



该广义表的深度为 4

【算法设计题】

12．已知图 5-2 与图 5-3 所示为广义表的存储结构图，其节点结构与 11 题相同。写出各图表示的广义表。

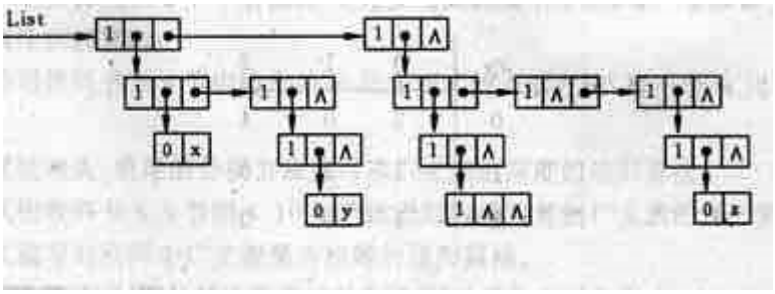


图 5-2

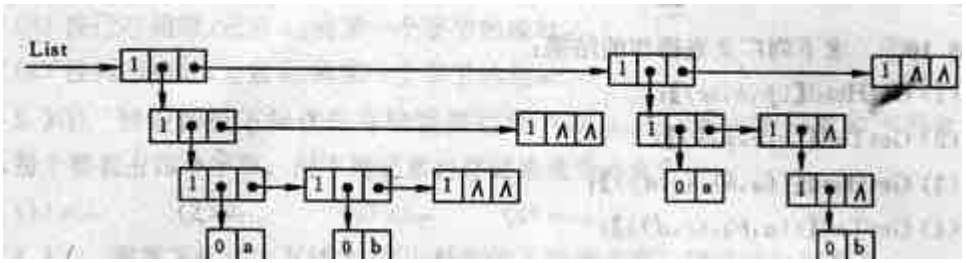


图 5-3

答：

（1） List=（（x,（y））,（（（））,（）,（z）））

（2） List=（（（a,b,（））,（））,（a,（b））,（））

13．已知等差数列的第一项为 a1，公差为 d，试写出该数列前 n 项的和 S（n）（n≥0）的递归定义。

答：$s(n)=s(n-1)+a_n=s(n-1)+a_1+(n-1)d$　　（n>=1）

```
ElemType s（int i）
{
if（i>1）
    return s（i-1）+a1+（i-1）*d;//数列元素个数大于 1
else
    return a1;//只有一个元素
}
```

14．写出求给定集合的幂集的递归定义。

答：该算法为求幂集的递归法（C 语言程序）,用数组实现.

```
void Getset（int i,char ch1[4],char ch2[4]）{
k=strlen（ch1）;
if （i>=4） printf（"%s,",ch1）;
else {
ch1[k]=ch2[i-1];Getset（i+1,ch1,ch2）;
ch1[--k]="\0";Getset（i+1,ch1,ch2）;
}
}
```

15．试利用 C 语言中的增量运算"++"和减量运算"--"写出两个非负整数 a 和 b 相加的递归定义。

答：

$$add(a,b)=\begin{cases} a & b=0 \\ add(++a,--b) & b>0 \end{cases}$$

```
int add（int a,int b）
{
if（a<0&&b<0） error;
if（b= =0） sum=a;
else
sum=add（++a,--b）;
}
```

16．已知顺序表 L 含有 n 个整数，试以函数形式分别写出下列运算的递归算法:

求表中的最大整数;

求表中的最小整数;

求表中的 n 个整数之和;

求表中的 n 个整数之积;

求表中的 n 个整数的平均值;

答：

```
public int RtMax（int[] L） //求表中的最大整数，返回值为 max
    {
    int max = 0;
    for （int i = 0;i < L.Length - 1; i++）
    {
    if （L[j] > L[j + 1]）
        {
    max = L[j];
    L[j] = L[j + 1];
    L[j + 1] = max ;
    }
    else
    {
max = L[j + 1];
    }
    }
    return max;
    }
public int RtMin（int[] L） //求表中的最小整数，返回值为 min
    {
    int min = 0;
    for （int j = 0; j < L.Length - 1; j++）
    {
    if （L[j] < L[j + 1]）
    {
    min = L[j];
```

```
    L[j] = L[j + 1];
     L[j + 1] = min;
     }
     else
     {
     min = L[j + 1];
     }
     }
     return min;
     }
public int RtSum（int[] L）        //求表中 n 个整数之和
{
     int sum=0;
     for（int i=0;i<L.length;i++）
     {
     sum+=L[i];//累加
     }
     return sum;
}
public int RtProduct（int[] L） //求表中 n 个整数之积
{
     int product=0;
     for（int i=0;i<L.length;i++）
     {
     product = product *L[i];//累积
     }
     return product ;
}
public float RtAvg（int[] L） //求表中 n 个整数平均值
{
     int sum=0;
     for（int i=0;i<L.length;i++）
     {
     sum +=L[i];
     }
     return sum/L.Length;
}
```

17．试设计一个算法，将数组 A_n 中的元素 A[0]至 A[n-1]循环右移 k 位，并要求只用一个元素大小的附加存储，元素移动或交换次数为 O（n）。

答：

算法的基本思想是将数组分成 k 组，将第一组与第二组进行两两交换，再将第二组与第三组进行两两交换，…，总共需进行 n-k 次交换。注意最后一组可能出现不足 k 个元素的情况，此时最后一组为剩余元素加第一组的前几个元素共 k 个元素构成最后一组。

```
void RRMove（ElemType A[],int k,int n）
```

```
{
ElemType e;
int i=0,j,p;
while（i<n-k）{
p=i/k+1;
for（j=0;j<k;j++）    //k 组数据依次与其后面那组数据交换
{
   e=A[j];
   A[j]=A[（p*k+j）%n];
A[（p*k+j）%n]=e;
   i++;//累计交换次数
}
}
}
```

18．若矩阵 $A_{n\times m}$ 中的某个元素 a_{ij} 是第 i 行中的最小值，同时又是第 j 列中的最大值，则称此元素为该矩阵中的一个马鞍点。假设以二维数组存储矩阵 $A_{n\times m}$，试编写求出矩阵中所有马鞍点的算法，并分析 你的算法在最坏情况下的时间复杂度。

答：

```
void Get_Saddle（int A[m][n]）//求矩阵 A 中的马鞍点
{
for（i=0;i<m;i++）
{
for（min=A[i][0],j=0;j<n;j++）
   if（A[i][j]<min） min=A[i][j]; //求每一行中的最小值
for（j=0;j<n;j++）
   if（A[i][j]==min） //判断这些最小值是否是鞍点
   {
for（flag=1,k=0;k<m;k++）
   if（min<A[k][j]） flag=0;//不是鞍点
if（flag）
   printf（"Found a saddle element!\nA[%d][%d]=%d",i,j,A[i][j]）;
   }
}//for
}//Get_Saddle
```

19．类似于以一维数组表示一元多项式，以 m 维数组:（aj1 j2 … jm），0≤jm≤n,i=1,2…,m,表示 m 元多项式,数组元素 ae1 e2 … em 表示多项式中 x1e1 x2e2 … xmem 的系数。例如，和二元多项式 x2 +3xy+4y2 -x+2 相应的二维数组为

xy	0	1	2
0	2	0	4
1	-1	3	0
2	1	0	0

试编写一个算法将 m 维数组表示的 m 元多项式以常规表示的形式（按降幂顺序）输出。可以将其中一项 ck x1e1 x2e2…xmem 印成 ck x1Ee 1，x2Ee 2,x3Ee 3…xmEe2 其中 m，ck

和 ej （j=1,2…m）印出他们具体的值），当 ck 或 ej （j=1,2…m）为 1 时，ck 的值或"E"和 ej 的值可省略不印。

答：

```
int exps[MAXSIZE]; //exps 数组用于存储某一项各变元的指数
int maxm,n; //maxm 指示变元总数，n 指示一个变元的最高指数
void Print_Poly_Descend（int *a,int m）/*按降幂顺序输出 m 元多项式的项，各项的系数已经按照题目要求存储于 m 维数组中，数组的头指针为 a，即数组 a 的首地址*/
{
maxm=m;
for（i=m*n;i>=0;i--） //按降幂次序,可能出现的最高项次数为 m*n
    Get_All（a,m,i,0）; //确定并输出所有次数为 i 的项
}//Print_Poly_Descend
void Get_All（int *a,int m,int i,int seq）//递归求出所有和为 i 的 m 个自然数
{
if（seq==maxm） Print_Nomial（a,exps）; //若全部求完，输出该项
else
{
    min=i-（m-1）*n; //当前数不能小于 min
    if（min<0） min=0;
    max=n<i?n:i; //当前数不能大于 max
    for（j=min;j<=max;j++）
    {
exps[seq]=j; //依次取符合条件的数
Get_All（a,m-1,i-j,seq+1）; //取下一个数
    }
}//else
exps[seq]=0; //返回
}//Get_All
void Print_Nomial（int *a,int exps[ ]）//输出一个项，该项各变元的指数已经存储在数组 exps 中
{
pos=0;
for（i=0;i<maxm;i++） //求出该项的系数在 m 维数组 a 中低下标优先的存储位置 pos
{
    pos*=n;
    pos+=exps[i];
}
coef=*（a+pos）; //取得该系数 coef
if（!coef） return; //该项系数 coef 为 0 时，无需输出
else if（coef>0） printf（"+"）; //系数 coef 为正时打印加号
else if（coef<0） printf（"-"）; //系数 coef 为负时打印减号
if（abs（coef）!=1） printf（"%d",abs（coef））; //当系数 coef 的绝对值不为 1 时打印出系数
for（i=0;i<maxm;i++）
```

```
    if（exps[i]） //打印该项各变元及其系数
    {
printf（"x"）;
printf（"%d",i）;
printf（"E"）;
if（exps[i]>1） printf（"%d",exp[i]）; //系数为 1 时无需打印
    }
}//Print_Nomial
```

20．假设稀疏矩阵 A 和 B 均以三元数组顺序表示作为存储结构,试写出矩阵相加的算法，另设三元组表 C 存放结果矩阵。

答：

```
void TSMatrix_Add（TSMatrix A,TSMatrix B,TSMatrix &C）//三元数组表示的稀疏矩阵加法
{
C.mu=A.mu;C.nu=A.nu;C.tu=0;
pa=1;pb=1;pc=1;
for（x=1;x<=A.mu;x++） //依次对矩阵的每一行进行加法
{
    while（A.data[pa].i<x） pa++;
    while（B.data[pb].i<x） pb++;
    while（A.data[pa].i==x&&B.data[pb].i==x）//寻找出行列值都相等的元素
    {
    if（A.data[pa].j==B.data[pb].j）
    {
ce=A.data[pa].e+B.data[pb].e;
if（ce） //和不为 0
{
C.data[pc].i=x;
C.data[pc].j=A.data[pa].j;
C.data[pc].e=ce;
pa++;pb++;pc++;//三个数组的下标同时后移
}
    }//if
    else if（A.data[pa].j>B.data[pb].j） //若 A 的列下标大于 B 的列下标，将 B 的元素赋给 C
    {
C.data[pc].i=x;
C.data[pc].j=B.data[pb].j;
C.data[pc].e=B.data[pb].e;
pb++;pc++;//B 和 C 的列下标同时后移
    }
    else //否则将 A 的元素赋给 C
    { C.data[pc].i=x;
C.data[pc].j=A.data[pa].j;
C.data[pc].e=A.data[pa].e
pa++;pc++;    //A 和 C 的列下标同时后移
```

```
    }
    }//while
    while（A.data[pa]= =x） //将 A 中剩余的元素插入（第 x 行）
    {
C.data[pc].i=x;
C.data[pc].j=A.data[pa].j;
C.data[pc].e=A.data[pa].e
pa++;pc++;
    }
    while（B.data[pb]==x）  //将 B 中剩余的元素插入（第 x 行）
    {
C.data[pc].i=x;
C.data[pc].j=B.data[pb].j;
C.data[pc].e=B.data[pb].e;
pb++;pc++;
    }
}//for
C.tu=pc;
}//TSMatrix_Add
```

21．假设系数矩阵 A 和 B 均以三元数组顺序表示作为存储结构，试写出满足以下条件的矩阵相加的算法，假设三元组顺序表 A 的空间足够大，将矩阵 B 加到矩阵 A 上，不增加 A，B 之外的附加空间，你的算法能否达到 O（m+n）的时间复杂度。其中 m 和 n 分别为 A，B 矩阵中非零的数目。

答：

```
void TSMatrix_Addto（TSMatrix &A,TSMatrix B）//将三元组矩阵 B 加到 A 上
{
for（i=1;i<=A.tu;i++）
    A.data[MAXSIZE-A.tu+i]=A.data[i];//把 A 的所有元素都移到尾部以空出位置
pa=MAXSIZE-A.tu+1;pb=1;pc=1;
for（x=1;x<=A.mu;x++） //依次对矩阵的每一行进行加法
{
    while（A.data[pa].i<x） pa++;
    while（B.data[pb].i<x） pb++;
    while（A.data[pa].i==x&&B.data[pb].i==x）//行列值都相等的元素
    {
if（A.data[pa].j==B.data[pb].j）//列标相同，计算元素值的和
{
    ne=A.data[pa].e+B.data[pb].e;
    if（ne） //和不为 0
    {
    A.data[pc].i=x;
    A.data[pc].j=A.data[pa].j;
    A.data[pc].e=ne;
    pa++;pb++;pc++;
```

```
    }
}//if
else if（A.data[pa].j>B.data[pb].j） //A 矩阵的下标大，将 B 矩阵中元素插入到 A 中
{
    A.data[pc].i=x;
    A.data[pc].j=B.data[pb].j;
    A.data[pc].e=B.data[pb].e;
    pb++;pc++;
}
else
{//否则 A 中元素保持不变
    A.data[pc].i=x;
    A.data[pc].j=A.data[pa].j;
    A.data[pc].e=A.data[pa].e;
    pa++;pc++;
}
    }//while
    while（A.data[pa]==x） //将 A 中剩余的元素插入（第 x 行）
    {
A.data[pc].i=x;
A.data[pc].j=A.data[pa].j;
A.data[pc].e=A.data[pa].e;
        pa++;pc++;
    }
    while（B.data[pb]==x） //将 B 中剩余的元素插入（第 x 行）
    {
A.data[pc].i=x;
A.data[pc].j=B.data[pb].j;
A.data[pc].e=B.data[pb].e;
pb++;pc++;
    }
}//for
A.tu=pc;
for（i=A.tu;i<MAXSIZE;i++） A.data[i]={0,0,0}; //清除原来的 A 矩阵中记录
}//TSMatrix_Addto
```

22．三元组顺序表的一种变型是从三元组顺序表中去掉行下标域得到二元组顺序表,另设一个行起始向量。其每个分量是二元数组顺序表的一个下标值，指示该行中第一个非零元素在二元组顺序表中的起始位置，试编写一个算法,由矩阵元素的下标值 i，j 求矩阵元素，试讨论这种方法和三元顺序表相比有什么优缺点。

答：

```
typedef struct{
    int j;
    int e;
} DSElem; //二元组数据类型
```

```
typedef struct{
    DSElem data[MAXSIZE];
    int cpot[MAXROW];//这个向量存储每一行在二元组中的起始位置
    int mu,nu,tu;
} DSMatrix; //二元组矩阵类型
Status DSMatrix_Locate（DSMatrix A,int i,int j,int &e）//求二元组矩阵的元素 A[i][j]的值 e
{
for（s=A.cpot[i];s<A.cpot[i+1]&&A.data[s].j!=j;s++）;/*注意查找范围为：s<A.cpot[i+1]
&&A.data[s].j!=j*/
if（s<A.cpot[i+1]&&A.data[s].j==j） //找到了元素 A[i][j]
{
    e=A.data[s];
    return OK;
}
return ERROR;
}//DSMatrix_Locate
```

23．若将稀疏矩阵 A 的非零元素以行为主序的顺序存于一维数组 V 中,并用二维数组 B 表示 A 中的相应元素是否为零元素（以 0 和 1 分别表示零元素和非零元素）。

例如

15 0 0 22

A= 0 -6 0 0

91 0 0 0

可用 V=（15,22,-6,9）和

1 0 0 1

B = 0 1 0 0 表示

1 0 0 0

试编写一算法,实现上述表示法中实现矩阵相加的运算,并分析你的算法时间复杂度。

答：

```
typedef enum{0,1} bool;
typedef struct{
    int mu,nu;
    int elem[MAXSIZE];
    bool map[mu][nu];
} BMMatrix; //用位图表示的矩阵类型
void BMMatrix_Add（BMMatrix A,BMMatrix B,BMMatrix &C）//位图矩阵的加法
{
C.mu=A.mu;C.nu=A.nu;
pa=1;pb=1;pc=1;
for（i=0;i<A.mu;i++） //每一行的相加
    for（j=0;j<A.nu;j++） //每一列的相加
    {
if（A.map[i][j]&&B.map[i][j]&&（A.elem[pa]+B.elem[pb]））//结果不为 0
{
    C.elem[pc]=A.elem[pa]+B.elem[pb];
```

```
    C.map[i][j]=1;
    pa++;pb++;pc++;//三个矩阵的列下标同时后移
}
else if（A.map[i][j]&&!B.map[i][j]）//矩阵 A 元素不为 0，矩阵 B 元素为 0
{
    C.elem[pc]=A.elem[pa];//将 A 中元素插入到 C 中
    C.map[i][j]=1;
    pa++;pc++;
}
else if（!A.map[i][j]&&B.map[i][j]）/*矩阵 A 元素为 0，矩阵 B 元素不为 0，将 B 中元素插入到 C 中*/
{
    C.elem[pc]=B.elem[pb];
    C.map[i][j]=1;
    pb++;pc++;
}
    }
}//BMMatrix_Add
```

24．试编写一个以三元组形式输出用十字链表表示的稀疏矩阵中非零元素及其下标的算法。

答：

```
void Print_OLMatrix（OLMatrix A）//以三元组格式输出十字链表表示的矩阵
{
for（i=0;i<A.mu;i++）
{
    if（A.rhead[i]）
for（p=A.rhead[i];p;p=p->right） //逐次遍历每一个行链表
printf（"%d %d %d\n",i,p->j,p->e）;
}
}//Print_OLMatrix
```

25．试按十字链表存储表示编写将稀疏矩阵 B 加到稀疏矩阵 A 上的算法。

答：

```
void OLMatrix_Add（OLMatrix &A,OLMatrix B）//把十字链表表示的矩阵 B 加到 A 上
{
for（j=1;j<=A.nu;j++） cp[j]=A.chead[j]; //向量 cp 存储每一列最后一个元素的当前指针
for（i=1;i<=A.mu;i++）
{
    pa=A.rhead[i];pb=B.rhead[i];pre=NULL;
    while（pb）
    {
    if（pa==NULL || pa->j>pb->j） //新插入一个结点
    {
    p=（OLNode*）malloc（sizeof（OLNode））;
    if（!pre） A.rhead[i]=p;
```

```
    else pre->right=p;
    p->right=pa;pre=p;
    p->i=i;p->j=pb->j;p->e=pb->e; //插入行链表中
    f（!A.chead[p->j]）
    {
A.chead[p->j]=p;
p->down=NULL;
    }
    else
    {
while（cp[p->j]->down） cp[p->j]=cp[p->j]->down;
p->down=cp[p->j]->down;
cp[p->j]->down=p;
    }
    cp[p->j]=p; //插入列链表中
    }//if
    else if（pa->j<pb->j）
    {
pre=pa;
pa=pa->right;
    } //pa 右移一步
    else if（pa->e+pb->e）
    {
pa->e+=pb->e;
pre=pa;pa=pa->right;
pb=pb->right;
    } //pa 与 pb 所指向的元素值直接相加
    else
    {
if（!pre） A.rhead[i]=pa->right;
else pre->right=pa->right;
p=pa;pa=pa->right; //从行链表中删除
if（A.chead[p->j]==p）
A.chead[p->j]=cp[p->j]=p->down;
else cp[p->j]->down=p->down; //从列链表中删除
free （p）;
    }//else
    }//while
}//for
}//OLMatrix_Add
```

26．试按 m 元多项式的表示方法，写一个求 m 元多项式中第一变元的偏导数的算法。

答：

```
void MPList_PianDao（MPList &L）//对广义表存储结构的多元多项式求第一变元的偏导
{
```

```
for（p=L->hp->tp;p&&p->exp;pre=p,p=p->tp）
{
   if（p->tag） Mul（p->hp,p->exp）;
   else p->coef*=p->exp; //把系数乘在本结点或其下属结点上
   p->exp--;
}
pre->tp=NULL;
if（p） free （p）; //删除可能存在的常数项
}//MPList_PianDao
void Mul（MPList &L,int x）//运用递归算法,多元多项式 L 乘以 x
{
for（p=L;p;p=p->tp）
{
   if（!p->tag） p->coef*=x;
   else Mul（p->hp,x）;
}
}//Mul
```

27．采用 m 元多项式的表示方法，写一个求 m 元多项式相加算法。

答：

```
void MPList_Add（MPList A,MPList B,MPList &C）/*采用广义表存储结构的多项式相加的
递归算法*/
{
C=（MPLNode*）malloc（sizeof（MPLNode））;
if（!A->tag&&!B->tag） //原子项,可直接相加
{
  C->coef=A->coef+B->coef;
   if（!C->coef）//系数等于 0
   {
free（C）;
C=NULL;
   }
}//if
else if（A->tag&&B->tag） //两个多项式相加
{
   p=A;q=B;pre=NULL;
   while（p&&q）
   {
if（p->exp==q->exp）//指数相等
{
   C=（MPLNode*）malloc（sizeof（MPLNode））;
   C->exp=p->exp;
   MPList_Add（A->hp,B->hp,C->hp）;
   pre->tp=C;pre=C;
   p=p->tp;q=q->tp;
```

```
}
else if（p->exp>q->exp）//A 中多项式项的指数较大
{
    C=（MPLNode*）malloc（sizeof（MPLNode））;
    C->exp=p->exp;
   C->hp=A->hp;
    pre->tp=C;pre=C;
    p=p->tp;
}
else    //B 中多项式项的指数较大
{
    C=（MPLNode*）malloc（sizeof（MPLNode））;
    C->exp=q->exp;
    C->hp=B->hp;
    pre->tp=C;pre=C;
    q=q->tp;
}
    }//while
    while（p）
    {
C=（MPLNode*）malloc（sizeof（MPLNode））;
C->exp=p->exp;
C->hp=p->hp;
pre->tp=C;pre=C;
p=p->tp;
    }
    while（q）
    {
C=（MPLNode*）malloc（sizeof（MPLNode））;
C->exp=q->exp;
C->hp=q->hp;
pre->tp=C;pre=C;
q=q->tp;
    } //将其同次项分别相加得到新的多项式,原理见第二章多项式相加一题
}//else if
else if（A->tag&&!B->tag） //多项式和常数项相加
{
    x=B->coef;
    for（p=A;p->tp->tp;p=p->tp）;
    if（p->tp->exp==0） p->tp->coef+=x; //当多项式中含有常数项时,加上常数项
    if（!p->tp->coef）
    {
free（p->tp）;
p->tp=NULL;
```

```
    }
    else
    {
q=（MPLNode*）malloc（sizeof（MPLNode））;
q->coef=x;q->exp=0;
q->tag=0;q->tp=NULL;
p->tp=q;
    } //否则新建常数项,下同
}//else if
else
{
    x=A->coef;
    for（p=B;p->tp->tp;p=p->tp）;
    if（p->tp->exp==0） p->tp->coef+=x;
    if（!p->tp->coef）
    {
free（p->tp）;
p->tp=NULL;
    }
    else
    {
q=（MPLNode*）malloc（sizeof（MPLNode））;
q->coef=x;q->exp=0;
q->tag=0;q->tp=NULL;
p->tp=q;
    }
}//else
}//MPList_Add
```

28．试按表头、表尾、的分析方法重写广义表的深度的递归算法。

答：

```
int GList_Getdeph（GList L）//求广义表深度的递归算法
{
if（!L->tag） return 0; //原子深度为 0
else if（!L） return 1; //空表深度为 1
m=GList_Getdeph（L->ptr.hp）+1;//用表头分析方法得到的广义表深度
n=GList_Getdeph（L->ptr.tp）;//用表尾分析方法得到的广义表深度
return m>n?m:n;//取最大值
}//GList_Getdeph
```

29．试按图 5-10 所示结点结构编写复制广义表的递归算法。



图 5-10 列表的另一种结点结构

答：

```
void GList_Copy（GList A,GList &B）//复制广义表的递归算法
{
if（!A->tag） //当结点为原子时，直接复制
{
   B->tag=0;
   B->atom=A->atom;
}
else //当结点为子表时
{
   B->tag=1;
   if（A->ptr.hp）
   {
B->ptr.hp=malloc（sizeof（GLNode））;
GList_Copy（A->ptr.hp,B->ptr.hp）;
   } //复制表头
   if（A->ptr.tp）
   {
B->ptr.tp=malloc（sizeof（GLNode））;
GList_Copy（A->ptr.tp,B->ptr.tp）;
   } //复制表尾
}//else
}//GList_Copy
```

30．试编写判别两个广义表是否相等的递归算法。

答：

```
int GList_Equal（GList A,GList B）/*判断广义表 A 和 B 是否相等,是则返回 1,否则返回 0。
广义表相等可分三种情况:*/
{
if（!A&&!B） return 1; //空表是相等的
if（!A->tag&&!B->tag&&A->atom==B->atom） return 1;//原子的值相等
if（A->tag&&B->tag）
   if（GList_Equal（A->ptr.hp,B->ptr.hp）&&GList_Equal（A->ptr.tp,B->ptr.tp））
return 1; //表头表尾都相等
return 0;
}//GList_Equal
```

31．试编写递归算法,输出广义表所有原子项及其所在的层次。

答：

```
void GList_PrintElem（GList A,int layer）/*递归输出广义表的原子及其所在层次,layer 表示
当前层次*/
{
if（!A） return;//空表
if（!A->tag） printf（"%d %d\n",A->atom,layer）;
else
{
   GList_PrintElem（A->ptr.hp,layer+1）;
```

```
    GList_PrintElem（A->ptr.tp,layer）; //注意尾表与原表是同一层次
}
}//GList_PrintElem
```

32．试编写递归算法,逆转广义表中的数组元素。

例如:将广义表：（a,（（b,c）,（））,（（（d）,e）,f））

逆转为:（（f,（e,（d））），（（），（c,b））,a）.

答：

```
void GList_Reverse（GList A）//递归逆转广义表 A
{
GLNode *ptr[MAX_SIZE];
if（A->tag&&A->ptr.tp） //当 A 不为原子且表尾非空时才需逆转
{
    for（i=0,p=A;p;p=p->ptr.tp,i++）
    {
if（p->ptr.hp） GList_Reverse（p->ptr.hp）;//递归逆转各子表
ptr[i]=p->ptr.hp;
    }
    for（p=A;p;p=p->ptr.tp） //重新按逆序排列各子表的顺序
p->ptr.hp=ptr[--i];
}
}//GList_Reverse
```

33．试编写递归算法,删除广义表中所有值等于 x 的原子项。

答：

```
void GList_DelElem（GList &A,int x）//从广义表 A 中删除所有值为 x 的原子
{
if（A&&A->ptr.hp）
{
    if（A->ptr.hp->tag） GList_DelElem（A->ptr.hp,x）;//递归删除表头中所有值为 x 的原子
    else if（!A->ptr.hp->tag&&A->ptr.hp->atom==x）
    {
q=A;
A=A->ptr.tp; //删去元素值为 x 的表头
free（q）;
GList_DelElem（A,x）;//递归删除子表中所有值为 x 的原子
    }
}
if（A&&A->ptr.tp） GList_DelElem（A->ptr.tp,x）;//递归删除表尾中所有值为 x 的原子
}//GList_DelElem
```

34．试编写算法，依次从左到右输出广义表中底 l 层的原子项。

例如:广义表（a,（b,（c））,（d））中的第一层的原子项为 a；b 和 d 为第二层的原子项；c 为第三层的原子项。

答：

```
void GList_PrintElem_LOrder（GList A）//按层序输出广义表 A 中的所有元素
{
```

```
InitQueue（Q）;
for（p=L;p;p=p->ptr.tp） EnQueue（Q,p）;//依次入队
while（!QueueEmpty（Q））
{
   DeQueue（Q,r）;//r 为对头元素
   if（!r->tag） printf（"%d",r->atom）;//若为原子项，则输出
   else
for（r=r->ptr.hp;r;r=r->ptr.tp） EnQueue（Q,r）;//否则将其表头元素插入到队尾
}//while
}//GList_PrintElem_LOrder
```

5.3 考研真题与典型题详解

一、选择题

1．设有一个 10 阶的对称矩阵 A，采用压缩存储方式，以行序为主存储，a_{11} 为第一元素，其存储地址为 1，每个元素占一个地址空间，则 a_{85} 的地址为（　　）。[燕山大学 2001 研]

A．13

B．33

C．18

D．40

【答案】B 查看答案

【解析】对于对称矩阵,$a_{i,j}=a_{j,i}$。为了节省存储空间，为多个相同的元素只分配一个存储空间。对于对称矩阵，元素下表之间的对应关系为：当 i>=j 时，k=i（i-1）/2+j -1；当 i< =j 时，k=j（j-1）/2+i-1。其中 k 相当于地址空间的标号，i 为行号，j 为列号。因为第一个元素存储地址为 1，所以最后计算的 k 需要加 1。所以 a_{85} 的存储位置为 8*（8-1）/2+5=33。

2．设有数组 A[i，j]，数组的每个元素长度为 3 字节，i 的值为 1 到 8，j 的值为 1 到 10，数组从内存首地址 BA 开始顺序存放，当用以列为主存放时，元素 A[5，8]的存储首地址为（　　）。

A．BA+141

B．BA+180

C．BA+222

D．BA+225

【答案】B 查看答案

【解析】在计算中，可以考虑按照列存放时，A[5,8]在内存的位置，比较容易计算元素的首地址。比如 A[5，8]顺序存放时，它是第 7*8+5=61 个元素，由于首地址为 BA，所以它的存储首地址为 BA+（61-1）*3=180+BA。

3．数组 A[0．.5，0．.6]的每个元素占 5 个字节，将其按列优先次序存储在起始地址为 1000 的内存单元中，则元素 A[5，5]的地址是（　　）。[南京理工大学 2001 研]

A．1175　B．1180　C．1205　D．1210

【答案】A 查看答案

【解析】按列优先次序存储在起始地址为 1000 的内存单元中，元素 A[5，5]的地址是 1000+5*5*6+5*5=1175。

4．稀疏矩阵一般的压缩存储方法有两种，即（　　）。[江苏大学 2005 研]

A．二维数组和三维数组

B．三元组和散列

C．三元组和十字链表

D．散列和十字链表

【答案】C 查看答案

【解析】稀疏矩阵一般的压缩方法为三元组表和十字链表。三元组表就是将非零元素及其对应的行和列构成一个三元组（行标，列标，值）。十字链表相比三元组表而言，主要是对每个结点增加了两个链域。如果数组经常运算时，会产生大量数据元素的移动，此时，采用链表存储结构更为恰当。

5.A[N，N]是对称矩阵，将下三角（包括对角线）以行序存储到一维数组 T[N（N+1） / 2]中，则对任一上三角元素 a[i][j]对应 T[k]的下标 k 是（　　）。[青岛大学 2002 研]

A．i（i-1） / 2+j　B．j（j-1） / 2+i　C i（j-i） / 2+1 D．j（i-1） / 2+1

【答案】B 查看答案

【解析】将 n 阶对称矩阵存入一维数组中，一维数组的大小需为 n（n+1） / 2。

对 n 阶对称矩阵 A 以行序为主序方式将其下三角形的元素（包括主对角线上所有元素）依次存放于一维数组 B[1…（n（n+1）） / 2]中，当 i<j 时，i 与 k 的关系为 j*（j-1） / 2+i。

6．对矩阵压缩存储是为了（　　）。[中南大学 2005 研]

A．方便运算　B．方便存储　C．提高运算速度　D．减少存储空间

【答案】D 查看答案

【解析】压缩存储也就是对那些没用的元素不进行存储或者对那些具有一定规律的相同元素放在一个存储空间，目的就是为了节省空间。

7．广义表 A=（a，b，（c，d），（e，（f,g））），则式子

Head（Tail（Head（Tail（Tail（A））））））的值为（　　）。

A．（g）　B．（d）　C．c D．d

【答案】D 查看答案

【解析】head 操作就是得到广义表中第一个的原子。tail 操作就是得到除第一个原子外剩下元素构成的表。也就是 tail 得到的元素需要在外层再加一个（）。

二、填空题

1．数组的存储结构采用_____存储方式。

【答案】顺序存储结构查看答案

2．设数组 a[1．.50，1．.80]的基地址为 2000，每个元素占 2 个存储单元，若以行序为主序顺序存储，则元素 a[45，68]的存储地址为__（1）____；若以列序为主序顺序存储，则元素 a[45，68]的存储地址为___（2）____。[华中理工大学 2000 研]

【答案】（1）9174　（2）8788 查看答案

【解析】设一个元素的行标为 i，列标为 j。若以行序为主存储顺序，则它的存储地址为 2000+（（i-1）*80＋ j －1 ）＊ 2 。若以列序为主存储顺序，则它的存储地址为 2000+（（j-1）*50+i-1）*2。

3．广义表 A=（（（a，b），（C，d，e））），取出 A 中的原子 e 的操作是：_____。

【答案】*head (tail (tail (head (tail (head (A))))))*查看答案

【解析】广义表的表头指表的第一个元素，表尾指表中除了第一个元素外其余所有的元素构成的表。

4．假设一个 15 阶的上三角矩阵 A 按行优先顺序压缩存储在一维数组 B 中，则非零元素 $A_{9,9}$ 在 B 中的存储位置 k=_____。（注：矩阵元素下标从 1 开始）[北京工商大学 2001 研]

【答案】93 查看答案

【解析】对于上三角矩阵，k=（i-1）（2n-i+2）/2+（j-i）+1。将 i=j=9,n=15 代入得 93。

5．利用广义表的 GetHead 和 GetTail 操作，从广义表 L=（（apple，pear），（banana，orange））中分离出原子 banana 的函数表达式是_____。[山东大学 2001 研]

【答案】$GetHead\ (GetHead\ (GetTail\ (L)))$查看答案

【解析】GetHead 是取表的表头，GetTail 是取表的表尾。

三、综合题

1．数组 A[1．.8，-2．.6，0．.6]以行为主序存储，设第一个元素的首地址是 78，每个元素的长度为 4，试求元素 A[4，2，3]的存储首地址。

答：元素 A[4，2，3]的存储首地址为 958。三维数组以行为主序存储，其元素地址公式为：

$$LOC(A_{ijk}) = LOC(A_{c1c2c3}) + [(i-c_1)V_2V_3 + (j-c_2)V_3 + (k-c_3)]*L$$

其中，c_i，d_i 是各维的下界和上界，$V_i=d_i-c_i+1$ 是各维元素个数，L 是一个元素所占的存储单元数。

2．设有三维数组 A[-2：4，0：3，-5：1]按列序存放，数组的起始地址为 1210，试求 A（1，3，-2）所在的地址。

答：1210+108L（公式：

$$LOC(A[1,3,-2]) = 1210+[(k-c_3)v_2v_1 + (j-c_2)v_1+(i-c_1)]*L$$

（设每个元素占 L 个存储单元））

3．对一个有 t 个非零元素的 A_{mn} 矩阵，用 B[0．．t][1．.3]的数组来表示，其中第 0 行的三个元素分别为 m，n，t，从第一行开始到最后一行，每行表示一个非零元素；第一列为矩阵元素的行号，第二列为其列号，第三列为其值。对这样的表示法，如果需要经常进行该操作——确定任意一个元素 A[i][j]在 B 中的位置并修改其值，应如何设计算法可以使时间得到改善?

答：题中矩阵非零元素用三元组表存储，查找某非零元素时，按常规要从第一个元素开始查找，属于顺序查找，时间复杂度为 O（n）。若使查找时间得到改善，可以建立索引，将各行行号及各行第一个非零元素在数组 B 中的位置（下标）放入一向量 C 中。若查找非零元素，可先在数组 C 中用折半查找到该非零元素的行号，并取出该行第一个非零元素在 B 中的位置，再到 B 中顺序（或折半）查找该元素，这时时间复杂度为 O（logn）。

4．特殊矩阵和稀疏矩阵哪一种压缩存储后失去随机存取的功能?为什么?[北京邮电大学 2001 研]

答：特殊矩阵指值相同的元素或零元素在矩阵中的分布有一定规律，因此可以对非零元素分配单元（对值相同元素只分配一个单元），将非零元素存储在向量中，元素的下标 i 和 j 和该元素在向量中的下标有一定规律，可以用简单公式表示，仍具有随机存取功能。而稀疏矩阵是指非零元素和矩阵容量相比很小（t<<m*n），且分布没有规律。用十字链表作存储结构自然失去了随机存取的功能。即使用三元组表的顺序存储结构，存取下标为 i 和 j 的元素时，要扫描三元组表，下标不同的元素，存取时间也不同，最好情况下存取时间为 O（1），最差情况下是 O（n），因此也失去了随机存取的功能。

5．给定 n×m 矩阵 A[a．．b，c．．d]，并设

$$A[i,j] \leqslant A[i,j+1] \quad (a \leqslant i \leqslant b,\ c \leqslant j \leqslant d-1)$$

A[i，j]≤A[i+1，j]（a≤i≤b-1，c≤j≤d）。设计一算法判定 x 的值是否在 A 中，要求时间复杂度为 O（m+n）。[东南大学 2005、2001 研]

答：

```
void search(ElemType A[][], int a,b,c,d, ElemType x)
 //n*m矩阵A,行下标从a到b,列下标从c到d,本算法查找x是否在矩阵A中
 {i=a; j=d; flag=0;       //flag是成功查到x的标志
  while(i<=b && j>=c && !flag)
    if(A[i][j]==x) flag=1;
    else if(A[i][j]>x) j--; else i++;
   if(flag) printf("A[%d][%d]=%d\n",i,j,x);          //假定x为整型
   else printf("矩阵A中无%d 元素\n",x);
  }算法search结束。
```

第 6 章　树和二叉树

6.1　复习笔记

一、树的定义和基本术语

1．定义

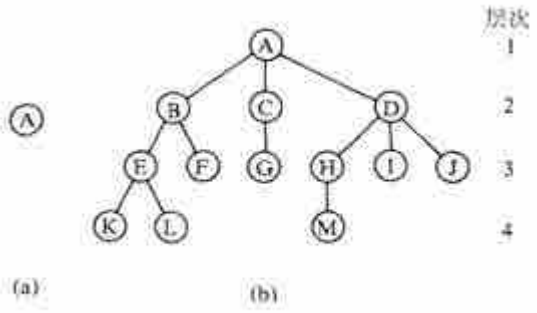
树（tree）是 n（n≥0）个结点的有限集。

在一棵非空树中：

（1）有且仅有一个特定的称为根（root）的结点；

（2）当 n>1 时，其余结点可分为 m（m>0）个互不相交的有限集 T_1，T_2，…，T_m，其中每一个集合本身又是一棵树，并且称为根的子树（subtree）。

例如，在图 6-1 中，（a）是只有一个根结点的树；（b）是有 l3 个结点的树，其中 A 是根，其余结点分成三个互不相交的子集：T_1={B，E，F，K，L}，T_2={C，G}，T_3=（D，H，I，J，M}；T_1、T_2 和 T_3 都是根 A 的子树，且本身也是一棵树。对树的定义可用图 6-1 表示：



(a) 只有根结点的树；(b) 一般的树。

图 6-1　树的示例

2．树的形式化定义

树是一种数据结构

Tree=（D，R）

其中：D 是具有相同特性的数据元素的集合；若 D 只含一个数据元素，则 R 为空集，否则 R 是 D 上某个二元关系 H 的集合，即 R={H}。H 为如下描述的二元关系：

（1）在 D 中存在唯一的称为根的数据元素 root，它在关系 H 下无前驱；

（2）若 D={root}≠Φ，则存在 D={root}的一个划分 D_1，D_2，…，D_m（m>0），对任意一对 j≠k（1≤j，k≤m）有 $D_j \cap D_k = \Phi$，且对任意的 i（1≤i≤m），存在唯一数据元素 $x_i \in D_i$，有（root，x_i）∈H；

（3）对应于 D={root}的划分，H={（root，x_1），…，（root，x_m））有唯一的一个划分 H_1，H_2，…，H_m（m>0），对任意 j≠k（1≤j，k≤m）有 $H_j \cap H_k = \Phi$，且对任意的 i（1≤i≤m），H_i 是 D_i 上的二元关系，（D_i，{H_i}）是一棵符合本定义的树，称为根 root 的子树。

3．树的表示形式

如图 6-2 所示为图 6-1（b）中树的各种表示形式。其中：

（a）是以嵌套集合（即一些集合的集体，对于其中任何两个集合，或者不相交，或者一个包含另一个）的形式表示的；

（b）是以广义表的形式表示的，根作为由子树森林组成的表的名字写在表的左边；

（c）用的是凹入表示法（类似书的编目）。

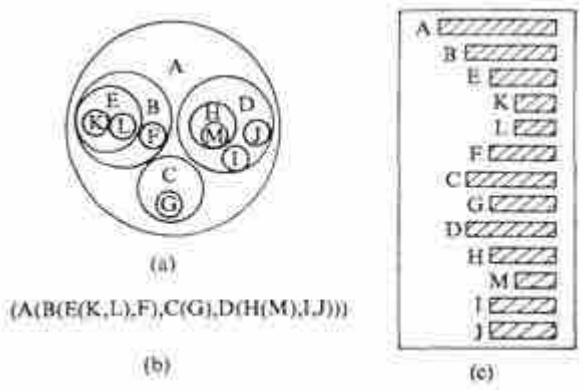


图 6-2 树的其他三种表示法

4．树结构中的基本术语

（1）结点：树的结点包含一个数据元素及若干指向其子树的分支。

（2）结点的度（degree）：结点拥有的子树个数称为结点的度。

（3）树的度（degree）：树的度是树内各结点的度的最大值。

（4）叶子（或终端结点）：度为 0 的结点称为叶子（1eaf）或终端结点。度不为 0 的结点称为非终端结点或分支结点。除根结点之外，分支结点也称为内部结点。

（5）结点的孩子、双亲：结点的子树的根称为该结点的孩子（child），相应地，该结点称为孩子的双亲（parent）。

（6）结点的兄弟（sibing）：同一双亲的孩子之间互称为兄弟。

（7）树的深度（depth）：树中结点的最大层次称为树的深度（depth）或高度。

（8）结点的祖先：从根到该节点所经分支上的所有节点。

（9）结点的子孙：以该节点为根的子树中的任一结点。

（10）结点的层次：从根开始定义，根为第一层，根的孩子为第二层。

（11）结点的堂兄弟：其双亲在同一层的结点互为堂兄弟。

（12）森林（forest）： m（m≥0）棵互不相交的树的集合。对树中每个结点而言，其子树的集合即为森林。

（13）有序数与无序树：如果将树中结点的各子树看成从左至右是有次序的（即不能互换），则称该树为有序数，否则称为无序树。有序数中最左边的子树称为第一个孩子，最右边的称为最后一个孩子。

任何一棵树是一个二元组 Tree=（root，F），其中：root 是数据元素，称做树的根结点；F 是 m（m≥0）棵树的森林，F=（T_1，T_2，…，T_m），其中T_i=（r_i，F_i）称做根 root 的第 i 棵子树；当 i≠0 时，在根结点和其子树森林之间存在下列关系：

$$RF=\{<\text{root}，\ r_i>i=1,\ldots m,m>0\}$$

二、二叉树

1．二叉树的定义

（1）定义

二叉树（binary tree）是另一种树型结构，它的特点是每个结点至多只有二棵子树（即二叉树中不存在度大于 2 的结点），并且二叉树的子树有左右之分，其次序不能任意颠倒。

（2）形式化定义

二叉树是一种数据结构：

BinaryTree=（D，R）

其中：D 是具有相同特性的数据元素的集合；若 D=Φ，则 R=Φ，称 BinaryTree 为空的二叉树；若 D≠Φ，则 R 是 D 上某个二元关系 H 的集合，即 R={H}，且

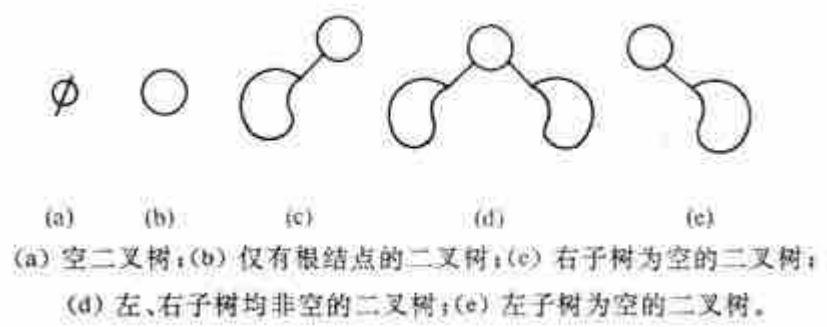
①D 中存在唯一的称为根的元素 r，它在关系 H 下无前驱；

②若 D-{r}≠Φ，则 D-{r}={D_l，D_r}，$D_l\cap D_r=\Phi$；

③若$D_l\neq\Phi$，则在 n 中存在唯一的元素x_i，<r，x_i>∈H，且存在D_r上的关系H_1∈H；若$D_r\neq\Phi$，则在D_r中存在唯一的元素x_r，（r，x_r）∈H，且存在D_r上的关系H_r∈H；H={（r，x_i），（r，x_r），H_l，H_r}；

④（D_l，H_l）是一棵符合本定义的二叉树，称为根 r 的左子树，（D_r，H_r）是一棵符合本定义的二叉树，称为根 r 的右子树。

二叉树可以有五种基本形态，如图 6-3 所示。



(a) 空二叉树；(b) 仅有根结点的二叉树；(c) 右子树为空的二叉树；
(d) 左、右子树均非空的二叉树；(e) 左子树为空的二叉树。

图 6-3 二叉树的五种基本形态

前面引入的有关树的术语也都适用于二叉树。

前面关于树的基本操作也都适用于二叉树，只需将 T（树）改成 BT（二叉树）即可。

2．二叉树的性质

性质 l：在二叉树的第 i 层上至多有 2^{i-1} 个结点（i≥1）。

性质 2：深度为 k 的二叉树至多有 2^k -l 个结点，（k≥1）。

性质 3：对任何一棵二叉树 T，如果其终端结点数为 n_0，度为 2 的结点数为 n_2，则 $n_0=n_2+1$。

3．完全二叉树和满二叉树

（1）满二叉树的定义和特点

定义：一棵深度为 k 且有 2^k-1 个结点的二叉树称为满二叉树。

特点：每一层上的结点数都是最大结点数。

（2）完全二叉树的定义和特点

定义：深度为 k、有 n 个结点的二叉树，当且仅当其每一个结点都与深度为 k 的满二叉树中编号从 1 至 n 的结点一一对应时，称之为完全二叉树。

特点：①叶子结点只可能在层次最大的两层上出现；②对任一结点，若其右分支下的子孙的最大层次为 l，则其左分支下的子孙的最大层次必为 l 或 l+1。

（3）二叉树性质的扩充

性质 4：具有 n 个结点的完全二叉树的深度为 $\lfloor \log_2 n \rfloor + 1$。

性质 5：如果对一棵有 n 个结点的完全二叉树（其深度为 $\lfloor \log_2 n \rfloor + 1$）结点按层序编号（从第 l 层到第 $\lfloor \log_2 n \rfloor + 1$ 层，每层从左到右），则对任一结点 i（1≤i≤n），有

①如果 i=1，则结点 i 是二叉树的根，无双亲；如果 i>1，则其双亲 PARENT（i）是结点 $\lfloor i/2 \rfloor$。

②如果 2i>n，则结点 i 无左孩子（结点 i 为叶子结点）；否则其左孩子 LCHILD（i）是结点 2i。

③如果 2i+l>n，则结点 i 无右孩子；否则其右孩子 RCHILD（i）是结点 2i+1。

4．二叉树的存储结构

（1）顺序存储结构

用一组地址连续的存储单元存储二叉树的数据元素。

顺序存储结构仅适合于完全二叉树，普通二叉树如果采用顺序存储结构，会造成存储空间的浪费。

（2）链式存储结构

设计不同的结点结构可构成形式不同的链式存储结构。表示二叉树的链表中的结点至少包含三个域：数据域和左、右指针域。有时为了便于找到结点的双亲，还可以在结点结构中增加一个指向双亲结点的指针域，利用这两种结点所得的二叉树的存储结构分别称之二叉链表和三叉链表。如图 6-4 所示链表存储结构分类。

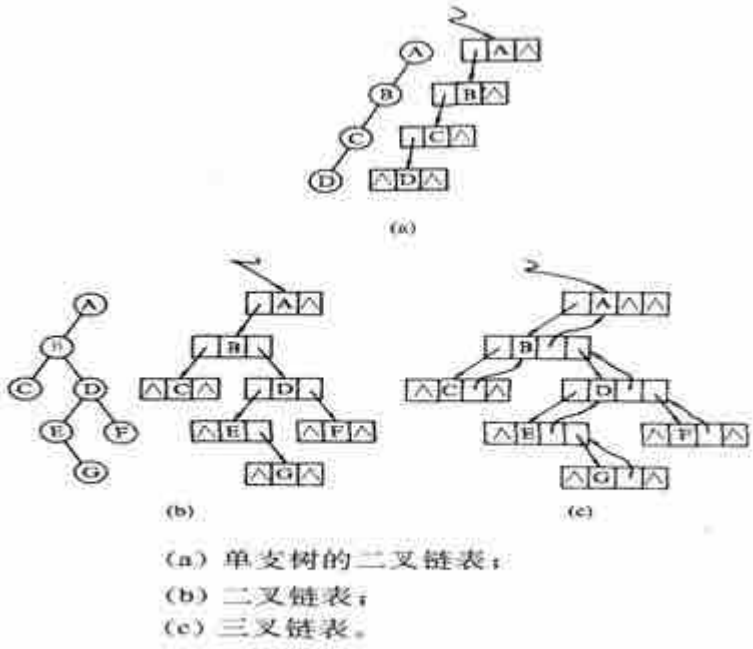


图 6-4 链表存储结构

三、遍历二叉树和线索二叉树

1．遍历二叉树

（1）定义：按某条搜索路径巡访树中每个结点，使得每个结点均被且仅被访问一次。“访问”的含义很广，可以是对结点做各种处理，如输出结点的信息等。

（2）遍历分类：先（根）序遍历，中（根）序遍历和后（根）序遍历。

①先序遍历二叉树的操作定义为：

若二叉树为空，则空操作；否则

a．访问根结点；

b．先序遍历左子树；

c．先序遍历右子树。

②中序遍历二叉树的操作定义为：

若二叉树为空，则空操作；否则

a．中序遍历左子树；

b．访问根结点；

c．中序遍历右子树。

③后序遍历二叉树的操作定义为：

若二叉树为空，则空操作；否则

a．后序遍历左子树；

b．后序遍历右子树；

c．访问根结点。

（3）时间复杂度与空间复杂度

不论按哪一种方法进行遍历，对含 n 个结点的二叉树，其时间复杂度均为 O（n）。所需辅助空间为遍历过程中栈的最大容量，即树的深度，最坏情况下为 n，则空间复杂度也为 O（n）。

2．线索二叉树

（1）定义

若结点有左子树，则其 lchild 域指示其左孩子，否则令 lchild 域指示其前驱；若结点有右子树，则其 rchild 域指示其右孩子，否则令 rchild 域指示其后继。为了避免混淆，需改变结点结构，增加两个标志域

lchild　ltag　data　rtag　rchild

其中：

$$ltag=\begin{cases}0 & \text{lchild 域指示结点的左孩子}\\ 1 & \text{lchild 域指示结点的前驱}\end{cases}$$

$$rtag=\begin{cases}0 & \text{rchild 域指示结点的右孩子}\\ 1 & \text{rchild 域指示结点的后继}\end{cases}$$

以这种结点结构构成的二叉链表作为二叉树的存储结构，称作线索链表，其中指向结点前驱和后继的指针，称作线索。加上线索的二叉树称之为线索二叉树（threaded binary tree）。例如图 6-5（a）所示为中序线索二叉树，与其对应的中序线索链表如图 6-5（b）所示。其中实线为指针（指向左、右子树），虚线为线索（指向前驱和后继）。对二叉树以某种次序遍历使其变为线索二叉树的过程称作线索化。

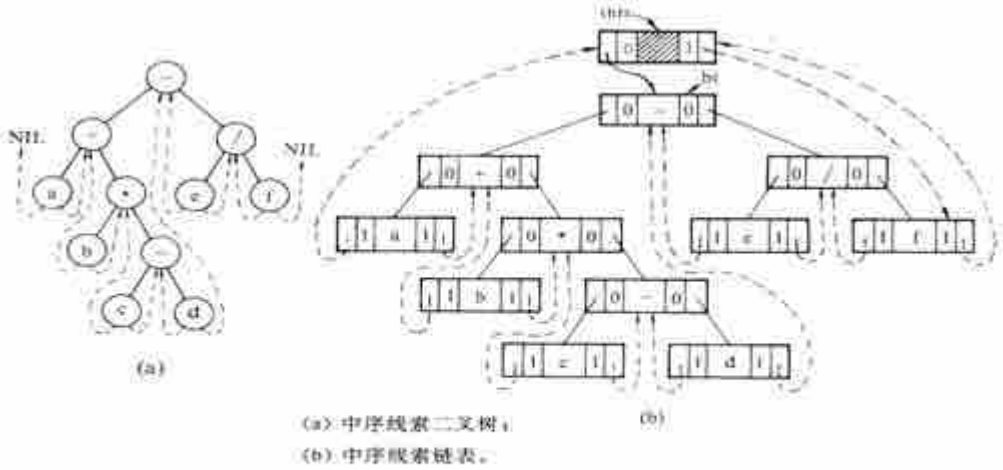


图 6-5 线索二叉树及其存储结构

（2）遍历

在线索树上进行遍历，只要先找到序列中的第一个结点，然后依次找结点后继直至其后继为空时为止。

①在后序线索树中找结点后继较复杂些，可分三种情况：

a．若结点 X 是二叉树的根，则其后继为空；

b．若结点 X 是其双亲的右孩子或是其双亲的左孩子且其双亲没有右子树，则其后继即为双亲结点；

c．若结点 X 是其双亲的左孩子，且其双亲有右子树，则其后继为双亲的右子树上按后序遍历列出的第一个结点。

②在中序线索二叉树上遍历二叉树，虽则时间复杂度亦为 O（n），但常数因子要小，且不需要设栈。因此，若在某程序中所用二叉树需经常遍历或查找结点在遍历所得线性序列中的前驱和后继，则应采用线索链表作存储结构。

四、树和森林

1．树的存储结构

大量的应用中常使用多种形式的存储结构来表示树，这里介绍三种常用的链表结构。

（1）双亲表示法

如图 6-6 所示为一颗树及其双亲表示的存储结构。这种存储结构利用了每个结点（除根以外）只有唯一的双亲的性质。PARENT（T，x）操作可以在常量时间内实现，反复调用 PARENT 操作，直到遇见无双亲的结点时，便找到了树的根，这就是 ROOT（x）操作的执行过程。但是，在这种表示法中，求结点的孩子需要遍历整个向量。

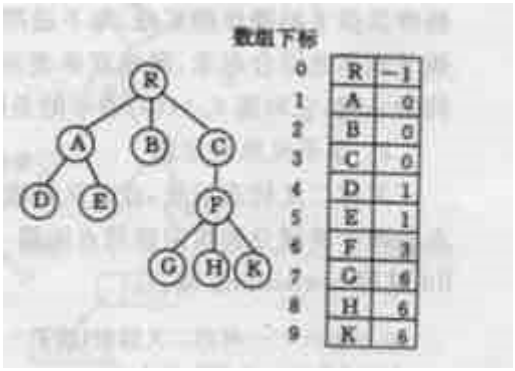


图 6-6 树的双亲表示法示例

（2）孩子表示法

由于树中每个结点可能有多棵子树，则可用多重链表，即每个结点有多个指针域，其中每个指针指向一棵子树的根结点，此时链表中的结点可以有如下两种结点格式：

Data childl child2 … Childd

Data degree childl child2 … childd

①若采用第一种结点格式，则多重链表中的结点是同构的，其中 d 为树的度。由于树中很多结点的度小于 d，所以链表中有很多空链域，空间较浪费，在一棵有 n 个结点度为 k 的树中必有 n（k-l）+1 个空链域。

②若采用第二种结点格式，则多重链表中的结点是不同构的，其中 $\bar{d}$ 为结点的度，degree 域的值等于 $\bar{d}$。此时，虽能节约存储空间，但操作不方便。

图 6-7（a）是图 6-6 中的树的孩子表示法。与双亲表示法相反，孩子表示法便于那些涉及孩子的操作的实现，却不适用于 PARENT（T，x）操作。通常把双亲表示法和孩子表示法结合起来，即将双亲向量和孩子表头指针向量合在一起。图 6-7（b）就是这种存储结构的一例，它和图 6-7（a）表示的是同一棵树。

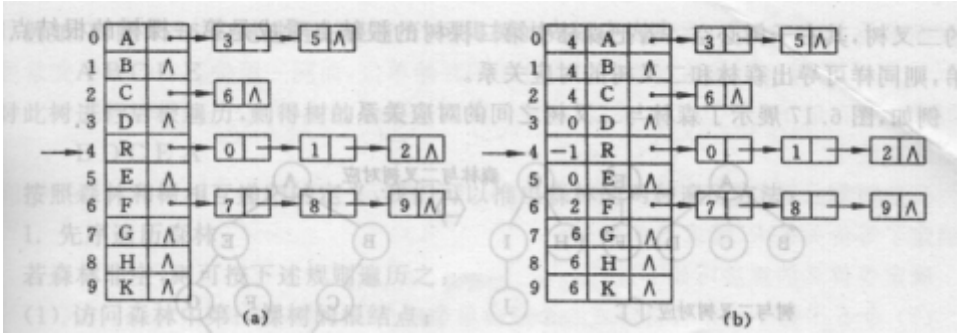


图 6-7 图 6-6 的树的另外两种表示方式

（a）孩子链表；　（b）带双亲的孩子链表

（3）孩子兄弟表示法

又称二叉树表示法，或二叉链表表示法,即以二叉链表作树的存储结构。链表中结点的两个链域分别指向该结点的第一个孩子结点和下一个兄弟结点，分别命名为 firstchild 域和 nextsibling 域。

图 6-8 是图 6-6 中的树的孩子兄弟链表。利用这种存储结构便于实现各种树的操作,首先易于实现找结点孩子等的操作。

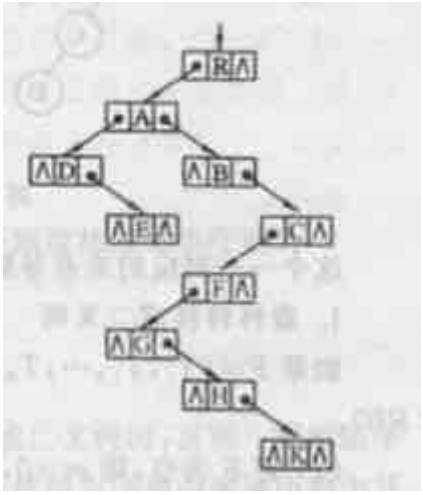


图 6-8 图 6-6 中树的二叉链表表示法

2．森林与二叉树的转换

由于二叉树和树都可用二叉链表作为存储结构，则以二叉链表作为媒介可导出树与二叉树之间的一个对应关系。即给定一个树，可以找到唯一的一颗二叉树与之对应，从物理间的一个对应结构来看，它们的二叉链表是相同的，只是解释不同而已。图 6-9 直观地展示了树与二叉树之间的对应关系。

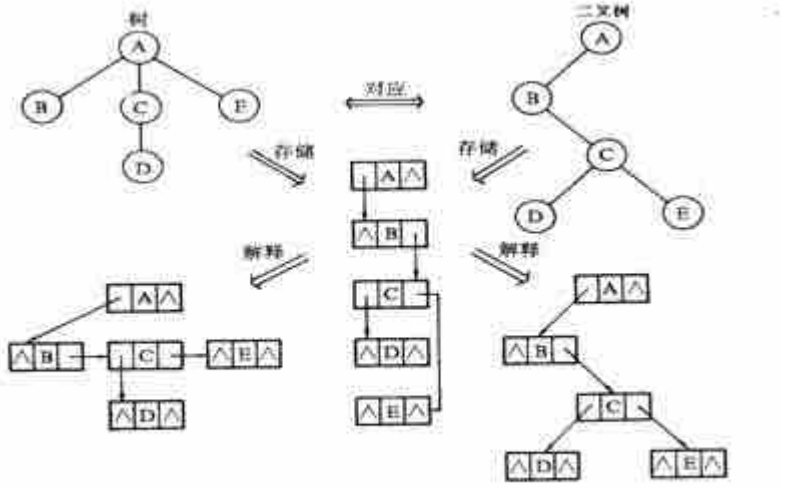


图 6-9 树与二叉树的对应关系示例

从树的二叉链表表示定义可知，任何一棵和树对应的二叉树，其右子树必空。若把森林中第二棵树的根结点看成是第一棵树的根结点的兄弟，则同样可导出森林和二叉树的对应关系。

如图 6-10 所示森林与二叉树之间的对应关系。

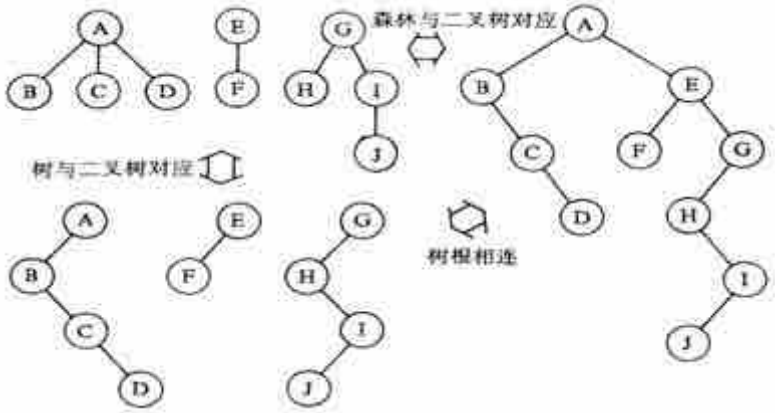


图 6-10 森林与二叉树之间的对应关系

这个一一对应关系导致森林或树与二叉树可以相互转换，其形式定义如下：

（1）森林转换成二叉树

如果 F={T_1，T_2，…，T_m}是森林，则可按如下规则转换成一棵二叉树 B=（root，LB，RB）。

①若 F 为空，即 m=0，则 B 为空树；

②若 F 非空，即 m≠0，则 B 的根 root 即为森林中第一棵树的根 ROOT（T_1）；B 的左子树 LB 是从T_1中根结点的子树森林F_1={T_{11}，T_{12}，…，T_{1m1}}转换而成的二叉树；其右子树 RB 是从森林F'={T_2，T_3，…，T_m）转换而成的二叉树。

（2）二叉树转换成森林

如果 B=（root，LB，RB）是一棵二叉树，则可按如下规则转换成森林 F={T_1，T_2，…，T_m}。

①若 B 为空，则 F 为空；

②若 B 非空，则 F 中第一棵树 T 的根 ROOT（T_1）即为二叉树 B 的根 root；T_1中根结点的子树森林F_1是由 B 的左子树 LB 转换而成的森林；F 中除T_1之外其余树组成的F'={T_2，T_3，…，T_m}是由 B 的右子树 RB 转换而成的森林。

3．树和森林的遍历

由树结构的定义可引出遍历树的两种次序方法：一种是先根（次序）遍历树，即：先访问树的根结点，然后依次先根遍历根的每棵子树；另一种是后根（次序）遍历，即：先依次后根遍历每棵子树，然后访问根结点。

按照森林和树的相互递归定义，可以推出森林的两种遍历方法：

（1）先序遍历森林

若森林非空，则可按下述规则遍历之：

①访问森林中第一棵树的根结点；

②先序遍历第一棵树中根结点的子树森林；
③先序遍历除去第一棵树之后剩余的树构成的森林。
（2）中序遍历森林
若森林非空，则可按下述规则遍历之；
①中序遍历森林中第一棵树的根结点的子树森林；
②访问第一棵树的根结点；
③中序遍历除去第一棵树之后剩余的树构成的森林。

五、树与等价问题

1．等价关系
如果集合 S 中的关系 R 是自反的、对称的和传递的，则称它为一个等价关系。
2．等价类
若 R 是集合 S 的一个等价关系，则由这个等价关系可产生这个集合的惟一划分。即可以按 R 将 S 划分为若干不相交的子集 S_1，S_2，…，它们的并即为 S，则这些子集 S_i，便称为 S 的 R 等价类。
划分等价类需对集合进行三个操作：①构造只含单元素的集合；②判定某个单元素所在集合；③将两个互不相交的集合归并为一个集合。

根据 MFSet 类型中定义的操作 FIND（S，x）和 MERGE（S_i，S_i）的特点，可利用树型结构表示集合。约定：以森林 F=（T_1，T_1，…，T_1）表示 MFSet 型的集合 S，森林中的每一棵树 T；（i=1，2，…，n）表示 S 中的一个元素——子集 S_i（$S_i \subset$S，i=1，2，…，n），树中每个结点表示集合中的一个单元数 x，同时令每个结点中含有一个指向其双亲的指针，并约定根结点的单元数兼作子集的名称。

六、赫夫曼树及其应用

赫夫曼树又称最优树，是一类带权路径长度最短的树。
1．最优二叉树（赫夫曼树）
从树中一个结点到另一个结点之间的分支构成这两个结点之间的路径，路径上的分支数目称做路径长度。树的路径长度是从树根到每一结点的路径长度之和。
结点的带权路径长度为从该结点到树根之间的路径长度与结点上的权的乘积。树的带权路径长度为树中所有叶子结点的带权路径长度之和，通常记作 $WPL=\sum_{k=1}^{n} W_K t_K$。

假设有 n 个权值{w_1，w_2，…，w_n}，构造一棵有 n 个叶子结点的二叉树，每个叶子结点带权为 w_i，则其中带权路径长度 WPL 最小的二叉树称做最优二叉树或赫夫曼树。
赫夫曼（Huffman）最早给出了一个带有一般规律的算法，俗称赫夫曼算法。现叙述如下：

（1）根据给定的 n 个权值{w_1，w_2，…，w_n}构成 n 棵二叉树的集合 F={T_1，T_2，…，T_n}，其中每棵二叉树 T_i 中只有一个带权为 w_i 的根结点，其左右子树均空。
（2）在 F 中选取两棵根结点的权值最小的树作为左右子树构造一棵新的二叉树，且置新的二叉树的根结点的权值为其左、右子树上根结点的权值之和。

（3）在 F 中删除这两棵树，同时将新得到的二叉树加入 F 中。
（4）重复（2）和（3），直到 F 只含一棵树为止。最后得到的这棵树便是赫夫曼树。

2．赫夫曼编码

若要设计长短不等的编码，任一个字符的编码都不是另一个字符的编码的前缀，这种编码称做前缀编码。可以利用二叉树来设计二进制的前缀编码，得到的二进制前缀编码称为哈夫曼编码。前缀编码示例如图 6-11 所示。

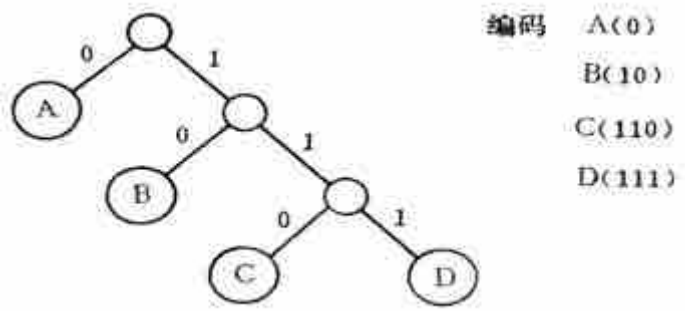


图 6-11 前缀编码示例

七、回溯法与树的遍历

1．回溯法的定义

回溯法是设计递归过程的一种重要方法，它的求解过程实质上是一个先序遍历一棵“状态树”的过程，只是这棵树不是遍历前预先建立的，而是隐含在遍历过程中。

2．回溯法示例

求四皇后问题的所有合法布局。

图 6-12 展示求解过程中棋盘状态的变化情况。这是一棵四叉树，树上每个结点表示一个局部布局或一个完整的布局。根结点表示棋盘的初始状态：棋盘上无任何棋子。每个（皇后）棋子都有四个可选择的位置，但在任何时刻，棋盘的合法布局都必须满足三个约束条件，即任何两个棋子都不占据棋盘上的同一行、或者同一列、或者同一对角线。如图 6-12 中除结点 a 之外的叶子结点都是不合法的布局。

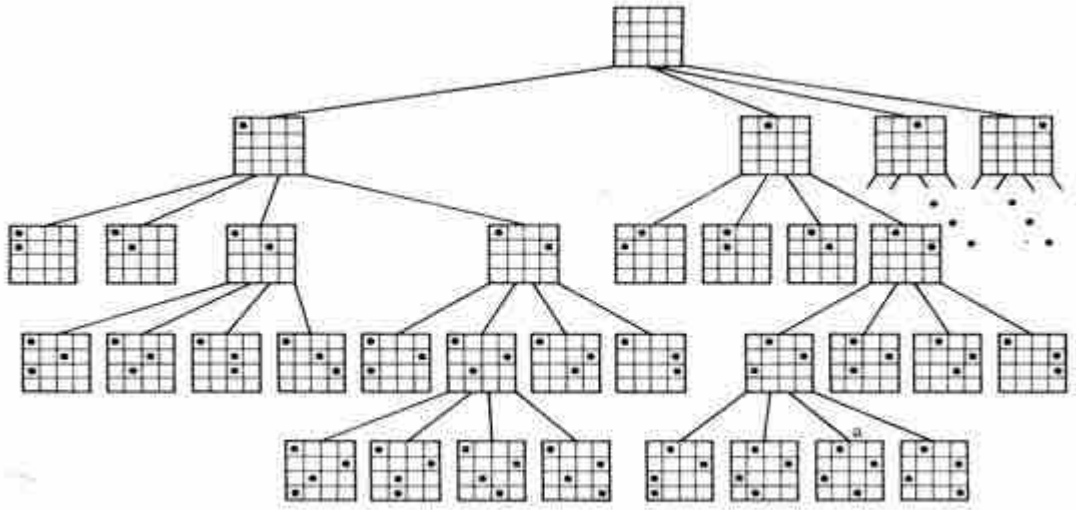


图 6-12 四皇后问题的棋盘状态图

求所有合法布局的过程即为在上述约束条件下先根遍历图 6-12 的状态树的过程。遍历结点的操作为：判别棋盘上是否已得到一个完整的布局（即棋盘上是否已摆上四个棋子），若是，则输出该布局；否则依次先根遍历满足约束条件的各棵子树，即首先判断该子树的根的布局是否合法，若合法，则先根遍历该子树，否则剪去该子树分支。

八、树的计数

1．二叉树的相似

称二叉树 T 和T'相似是指：二者都为空树或者二者均不为空树，且它们的左右子树分别相似。

2．二叉树的等价

称二叉树 T 和T'等价是指：二者不仅相似，而且所有对应结点上的数据元素均相同。

3．二叉树的计数问题

树的计数问题是指：具有 n 个结点的不同形态的树有多少棵。二叉树的计数问题就是讨论具有 n 个结点、互不相似的二叉树的数目 b_n。

在 n 很小的情况下，可直观地得到：b_0=l 为空树；b_1=l 是只有一个根结点的树；b_2=2 和 b_3=5，它们的形态分别如图 6-13（a）和图 6-13（b）所示。

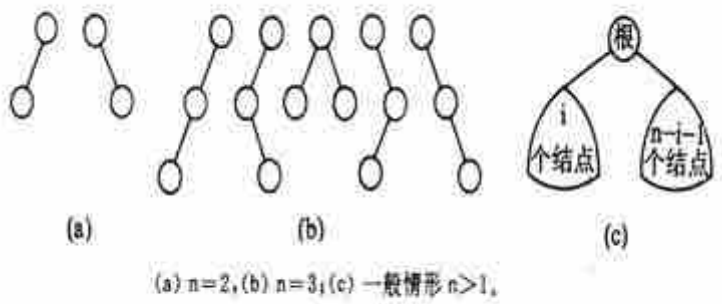


图 6-13 二叉树的形态

一般情况下，一棵具有 n（n>1）个结点的二叉树可以看成是由一个根结点、一棵具有 i 个结点的左子树、和一棵具有 n-i-1 个结点的右子树组成（如图 6-13（c）所示），其中 0≤i≤n-1。由此可得下列递推公式：

$$\begin{cases} b_0 = 1 \\ b_n = \sum_{i=0}^{n-1} b_i b_{n-i-1} \quad n \geqslant 1 \end{cases}$$

可以利用生成函数来讨论这个递推公式。

有如下两条结论：

（1）给定结点的前序序列和中序序列，可唯一确定一棵二叉树。

（2）具有 n 个结点的不同形态的树的数目和具有 n-1 个结点互不相似的二叉树的数目相同。

$$b_0, b_1, \cdots, b_n, \cdots$$

定义生成函数

$$B(z) = b_0 + b_1 z + b_2 z^2 + \cdots + b_n z^n + \cdots$$
$$= \sum_{k=0}^{\infty} b_k z^k$$

因为

$$B^2(z) = b_0 b_0 + (b_0 b_1 + b_1 b_0) z + (b_0 b_2 + b_1 b_1 + b_2 b_0) z^2 + \cdots$$
$$= \sum_{p=0}^{\infty} \left(\sum_{i=0}^{p} b_i b_{p-i} \right) z^p$$

根据(6-7)

$$B^2(z) = \sum_{p=0}^{\infty} b_{p+1} z^p$$

由此得

$$zB^2(z) = B(z) - 1$$

即

$$zB^2(z) - B(z) + 1 = 0$$

解此二次方程得

$$B(z) = \frac{1 \pm \sqrt{1 - 4z}}{2z}$$

由初值 $b_0 = 1$，应有 $\lim_{z \to 0} B(z) = b_0 = 1$

所以

$$B(z) = \frac{1 - \sqrt{1 - 4z}}{2z}$$

利用二项式展开

$$(1 - 4z)^{\frac{1}{2}} = \sum_{k=0}^{\infty} \binom{\frac{1}{2}}{k} (-4z)^k$$

当 $k = 0$ 时，式(6-10)的第一项为1，故有

$$B(z) = \frac{1}{2} \sum_{k=1}^{\infty} \binom{\frac{1}{2}}{k} (-1)^{k-1} 2^{2k} z^{k-1}$$
$$= \sum_{m=0}^{\infty} \binom{\frac{1}{2}}{m+1} (-1)^m 2^{2m+1} z^m$$
$$= 1 + z + 2z^2 + 5z^3 + 14z^4 + 42z^5 + \cdots$$

https://www.cnblogs.com/kohler21/

$$b_n = \binom{\frac{1}{2}}{n+1}(-1)^n 2^{2n+1}$$

$$= \frac{\frac{1}{2}\left(\frac{1}{2}-1\right)\left(\frac{1}{2}-2\right)\cdots\left(\frac{1}{2}-n\right)}{(n+1)!}(-1)^n 2^{2n+1}$$

$$b_n = \frac{1}{n+1}\cdot\frac{(2n)!}{n!n!} = \frac{1}{n+1}C_{2n}^{n}$$

因此，含有 n 个结点的不相似的二叉树有 $\frac{1}{n+1}C_{2n}^{n}$ 棵。

6.2 强化习题详解

【基础知识题】

1．已知一棵树边的集合为{<I,M>, <I,N>, <E,I>, <B,E>, <B,D>, <A,B>, <G,J>, <G,K>, <C,G>, <C,F>, <A,C>}，请画出这棵树，并回答下列问题：

（1）哪个是根结点？

（2）哪些是叶子结点？

（3）哪个是结点 G 的双亲？

（4）哪些是结点 G 的祖先？

（5）哪些是结点 G 的孩子？

（6）哪些是结点 E 的子孙？

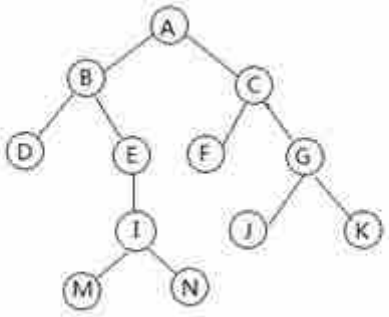
（7）哪那些是结点 E 的兄弟？哪些是结点 F 的兄弟？

（8）结点 B 和 N 的层次号分别是什么？

（9）树的深度是多少？

（10）以结点 C 为根的子树的深度是多少？

答：



（1）A 是根结点。

（2）M，N，D，J，K，F 是叶子结点。

（3）C 是 G 的双亲。

（4）A，C 是 G 的祖先。
（5）G 的孩子是 J，K。
（6）E 的子孙是 I，M，N。
（7）E 的兄弟结点是 D，F 的兄弟结点是 G 和 H’。
（8）结点 B 在第二层，结点 N 在第五层。
（9）树的深度为 5。
（10）以 C 为根的子树的深度为 3。

2．一棵度为 2 的树与一棵二叉树有何区别？

答：二叉树是一种树型结构，特点是每个结点至多只能有二棵子树，即每个结点的度最大为 2，并且二叉树的子树有左右之分，顺序不能任意颠倒；但度为 2 的树则未必有序，在一棵普通的树中若某结点只有一个孩子，就无需区分其左右次序，而在二叉树中即使是一个孩子也有左右之分。

3．试分别画出具有 3 个结点的树和 3 个结点的二叉树的所有不同形态。

答：含有 3 个结点的树有两种形态：如图 6-1 中（1）和（2）所示；而含有 3 个结点的二叉树可能有下列 5 种形态：如图 6-2 中（一），（二），（三），（四），（五）所示。

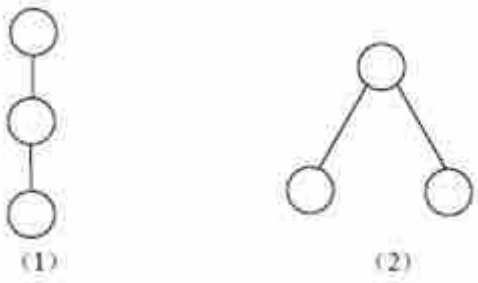


图 6-1

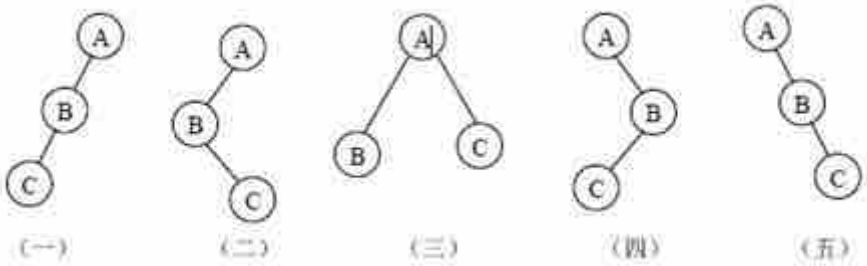


图 6-2

4．一棵深度为 H 的满 k 叉树有如下性质：第 H 层上的结点都是叶子结点，其余各层上每个结点都有 k 棵非空子树。如果按层次顺序从 1 开始对全部结点编号，问：

（1）各层的结点数目是多少？
（2）编号为 p 的结点的父结点（若存在）的编号是多少？
（3）编号为 p 的结点的第 i 个儿子结点（若存在）的编号是多少？
（4）编号为 p 的结点有右兄弟的条件是什么？其右兄弟的编号是多少？

答：（1）第一层得结点数目为 1，第二层的结点数目为 k，第三层得结点数目为 k^2，第四层的结点数目为 k^3，可见每层的结点从上往下形成等比数列。所以第 n 层得结点为 k^{n-1}，深度为 H，因此 n<=H。

（2）分两种情况。

①p=1 时，则该结点没有父结点。

②p≠1 时，如果 p 是其双亲的最小的孩子（右孩子），则 p 减去根结点的一个结点，应是 k 的整数倍，该整数即为所在的组数，每一组为一棵满 k 叉树，正好应为双亲结点的编号。如果 p 是其双亲的最大的孩子（左孩子），则 p+k-1 为其最小的弟弟，再减去一个根结点 p+k-2，除以 k，即为其双亲结点的编号。

p 结点的父结点编号用 i 表示。综合来说，对于 p 是左孩子的情况，i=（p+k-2）/k；对于 p 是右孩子的情况，i=（p-1）/k。如果左孩子的编号为 p，则其右孩子编号必为 p+k-1，所以，其双亲结点的编号为

$$i = \left\lfloor \frac{p + k - 2}{k} \right\rfloor$$

（向下取整，如 1.5 向下取整为 1）

（3）结点 p 的右孩子的编号为 kp+1，左孩子的编号为 kp+1-k+1=k（p-1）+2，第 i 个孩子的编号为 k（p-1）+2+i-1=kp-k+i+1。

（4）当（p-1）%k≠0 时，结点 p 有右兄弟，其右兄弟的编号为 p+1。

5．已知一棵度为 k 的树中有 n_1 个度为 1 的结点，n_2 个度为 2 的结点，…，n_k 个度为 k 的结点，问该树中有多少个叶子结点？

答：根据树的定义，在一颗树中，除树根结点外，每个结点有且仅有一个前驱结点，即每个结点与指向它的一个分支一一对应，所以除树根结点之外的结点树等于所有结点的分支数，即度数，从而可得树中的结点数等于所有结点的度数加 1。总结点数为

$$1 + n_1 + 2n_2 + 3n_3 + \ldots + kn_k$$

其中，1 表示根结点，而度为 0 的结点数就是叶子结点的数目，就应为总结点数减去度不为 0 的结点数的总和，即

$$n_0 = 1 + n_1 + 2n_2 + 3n_3 + \ldots + kn_k - (n_1 + n_2 + n_3 + \ldots + n_k) = 1 + \sum_{i=1}^{k} (i-1)n_i$$

6．已知在一棵含有 n 个结点的树中，只有度为 k 的分支结点和度为 0 的叶子结点。试求该树含有的叶子结点数目。

答：设度为 k 的结点个数为 n_k，利用题 5 的结论，则总结点数为 $n = 1 + n_k k$。叶子结点即度为 0 的结点的数目应等于总结点数减去度不为 0 的结点的数目，即

$$n_0 = n - n_k = n - \frac{n-1}{k}$$

7．一棵含有 n 个结点的 k 叉树，可能达到的最大深度和最小深度各为多少？

答：能达到最大深度的树是单支树，其深度为结点数 n。

满 k 叉树的深度最小，其深度为

$$\lfloor \log_k (n(k-1)+1) \rfloor$$

证明：设具有 n 个结点的 k 叉树的深度为 h，若在该树中前 h-1 层都是满的，即每一层的结点数都等于

$$k^{i-1} (1 \le i \le h-1)$$

个，第 h 层（即最后一层）的结点数可能满，也可能不满，则该树具有最小的深度。其深度 h 的计算公式为：

$$\frac{k^{h-1}-1}{k-1} < n \le \frac{k^h-1}{k-1}$$

可变换为 $k^{h-1} < n(k-1)+1 \le k^h$

以 k 为底取对数后得 $h-1 < \log_k(n(k-1)+1) \le h$

即 $\log_k(n(k-1)+1) \le h < \log_k(n(k-1)+1)+1$

因 h 只能是整数，所以 h=$\lceil \log_k(n(k-1)+1) \rceil$。

因此得到具有 n 个结点的一般 k 叉树的最小深度为 h=$\lfloor \log_k(n(k-1)+1) \rfloor$。

例如，对于二叉树，求最小深度的计算公式为 h=$\lfloor \log_2(n+1) \rfloor$,若 n=20，则最小深度为 5；对于三叉树，求最小深度的计算公式为 h=$\lfloor \log_3(2n+1) \rfloor$，若 n=20，则最小深度为 4。

8．证明：一棵满 k 叉树上的叶子结点数 n_0 和非叶子结点数 n_1 之间满足以下关系：

$$n_0 = (k-1)n_1 + 1$$

答：一棵深度为 h 的满 k 叉树，最后一层（深度为 h）的结点数（叶子结点数）为 $n_0 = k^{h-1}$，其总结点数为 $\frac{k^h-1}{k-1}$，则非叶子结点数 $n_1 = \frac{k^h-1}{k-1} - n_0 = \frac{kn_0-1}{k-1} - n_0$，从而得

$$n_0 = (k-1)n_1 + 1$$

9．试分别推导含有 n 个结点和含 n_0 个叶子结点的完全三叉树的深度 H。

答：

（1）根据完全三叉树的定义

其深度的计算公式为：

$\frac{k^{H-1}-1}{k-1} < n \le \frac{k^H-1}{k-1}$，可变换成：$3^{H-1}-1 < 2n \le 3^H-1$

即：$2n+1 \le 3^H < 3(2n+1)$，对不等式取对数：$\log_3(2n+1) \le H < 1+\log_3(2n+1)$

因为 H 只能取整数，H=$\lfloor \log_3(2n+1) \rfloor$

（2）三叉树的第 i 层上至多有 3^{i-1} 个结点。树中结点的最大层次为树的深度。设含 n_0 个叶子结点的完全三叉树的深度为 H。

则 $3^{H-2} \le n_0 \le 3^{H-1}$，所以 H=$\lfloor \log_3 n_0 \rfloor$+1

10．对于那些所有非叶子结点均含有左右子数的二叉树：
（1）试问：有 n 个叶子结点的树中共有多少个结点？

（2）试证明：$\sum_{i=1}^{n} 2^{-(l_i-1)}=1$，其中 n 为叶子结点的个数，$l_i$ 表示第 i 个叶子结点所在的层次（设根节点所在层次为 1）。

答：

（1）设 n_1 为非叶子结点数，则总结点数为 $1+2n_1$，叶子结点数为 $n=n_1+1$，所以总结点数为 $2n-1$。

（2）用归纳法证明。

①当 n=1 时，i=1，说明二叉树只有一个叶子结点，则整棵树只有一个根结点，$l_1=1$，$2^{-(l_1-1)}=1$，结论成立。

②设当 n=k 时（即有 n 个叶子结点）也成立，即

$2^{-(l_1-1)}+2^{-(l_2-1)}+\ldots+2^{-(l_p-1)}+\ldots+2^{-(l_k+1)}=1$，则当 n=k+1 时，现假设增加一个叶子结点，这意味着在某叶子结点 p 上新生两个叶子结点，而结点 p 则成为非叶子结点，可见，总结点数增 2，叶子结点数增 1。此时，所有叶子结点是原结点除去 p，然后加上两个深度为 l_p+1 的新叶子结点，由此，

$$2^{-(l_1-1)}+2^{-(l_2-1)}+\ldots+2^{-(l_{p-1}-1)}+2^{-(l_{p+1}-1)}+\ldots+2^{-(l_k+1)}+2^{-(l_p+1-1)}+2^{-(l_p+1-1)}$$

$$=2^{-(l_1-1)}+2^{-(l_2-1)}+\ldots+2^{-(l_p-1)}+\ldots+2^{-(l_k+1)}=1$$

则当 n=k+1 时，也成立。

综上：$\sum_{i=1}^{n} 2^{-(l_i-1)}=1$ 成立。

11．在二叉树的顺序存储结构中，实际上隐含着双亲的信息，因此可和三叉链表对应。假设每个指针域占 4 个字节，每个信息域占 k 个字节。试问：对于一棵有 n 个结点的二叉树，且在顺序存储结构中最后一个结点的下标为 m，在什么条件下顺序存储结构比三叉链表更节省空间？

答：采用三叉链表结构，需要 n+（k+4×3）即 n（k+12）个字节的存储空间。采用顺序存储结构，需要 mk 个字节的存储空间，则当 mk<n（k+12）时，即 $k<\frac{12n}{m-n}$ 时，采用顺序存储比采用三叉链表更节省空间。

12．对题 3 所得各种形态的二叉树，分别写出前序、中序和后序遍历的序列。

答：

	（一）	（二）	（三）	（四）	（五）
前序	ABC	ABC	ABC	ABC	ABC
中序	CBA	BCA	BAC	ACB	ABC
后序	CBA	CBA	BCA	CBA	CBA

13．假设 n 和 m 为二叉树中两结点，用 1、0 或 Φ（分别表示肯定、恰恰相反或不一定）填写表 1：

表 1

已知 \ 问	前序遍历时 n 在 m 前？	中序遍历时 n 在 m 前？	后序遍历时 n 在 m 前？
n 在 m 左方			
n 在 m 右左方			
n 是 m 祖先			
n 是 m 子孙			

注：如果（1）离 a 和 b 最近的共同祖先 p 存在，且（2）a 在 p 的左子树中，b 在 p 的右子树中，则称 a 在 b 的左方（即 b 在 a 的右方）。

答：

已知 \ 问	前序遍历时 n 在 m 前？	中序遍历时 n 在 m 前？	后序遍历时 n 在 m 前？
n 在 m 左方	1	1	1
n 在 m 右方	0	0	0
n 是 m 祖先	1	Φ	0
n 是 m 子孙	0	Φ	1

14．找出所有满足下列条件的二叉树：

（a）它们在先序遍历和中序遍历时，得到的结点访问序列相同；

（b）它们在后序遍历和中序遍历时，得到的结点访问序列相同；

（c）它们在先序遍历和后序遍历时，得到的结点访问序列相同。

答：（a）只有右分支（不含左子树的单分支树。

（b）只有左分支（不含右子树）的单分支树。

（c）只有根结点（即不含左子树，也不含右子树的二叉树）的二叉树。

15．请对如图 2 所示的二叉树进行后序线索化，为每个空指针建立相应的前驱或后继线索。

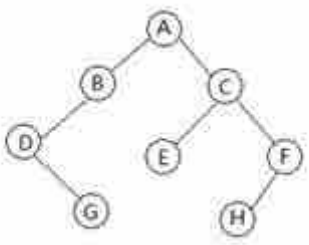
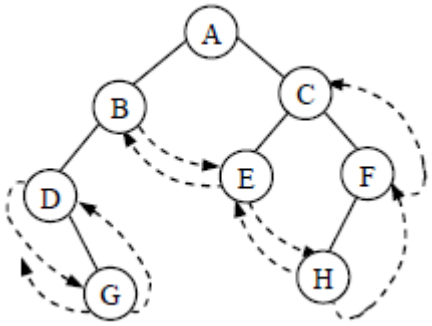


图 2

答：图中所给的二叉树后序遍历序列为：GDBEHFCA，线索化过程如下：



16．将下列二叉链表改为先序线索链表（不画出树的形态）。

1	2	3	4	5	6	7	8	9	10	11	12	13	14
Info	A	B	C	D	E	F	G	H	I	J	K	L	M	N
Ltag														
Lchild	2	4	6	0	7	0	10	0	12	13	0	0	0	0
Rtag														
Rchild	3	5	0	0	8	9	11	0	0	0	14	0	0	0

答：

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Info	A	B	C	D	E	F	G	H	I	J	K	L	M	N
Ltag	0	0	0	1	0	1	0	1	0	0	1	1	1	1
Lchild	2	4	6	2	7	3	10	14	12	13	13	9	10	11
Rtag	0	0	1	1	0	0	0	1	1	1	0	1	1	1
Rchild	3	5	6	5	8	9	11	3	12	13	14	0	11	8

17．阅读下列算法，若有错，则改正之。

```
BiTree InSucc（BiTree q） /*已知 q 是指向中序线索二叉树上某个结点的指针，本函数返回指向*q 的后继的指针。*/
{
r=q->rchild;
if（!r->ltag）
while（!r->ltag） r=r->rchild;
return r;
} // InSucc
```

答：其错误在于中序遍历应先访问其左子树，可做如下修改：

```
BiTree InSucc（BiTree q） /*已知 q 是指向中序线索二叉树上某个结点的指针，本函数返回指向*q 的后继的指针。*/
{
r=q->rchild;
if（!r->ltag）
while（!r->ltag） r=r->lchild;//此处先访问其左子树
return r;
} // InSucc
```

18．试讨论，能否在一棵中序全线索二叉树上查找给定结点*p 在后序序列中的后继。

答：如果 p 是根结点，则其后继为空，否则需查找 p 的双亲结点。从 p 结点开始中序线索遍历：

①如果某结点的左指针域等于 p，说明该结点是 p 的双亲结点，且 p 是它的左孩子；

②如果某结点的右指针域等于 p，说明该结点是 p 的双亲结点，且 p 是它的右孩子；

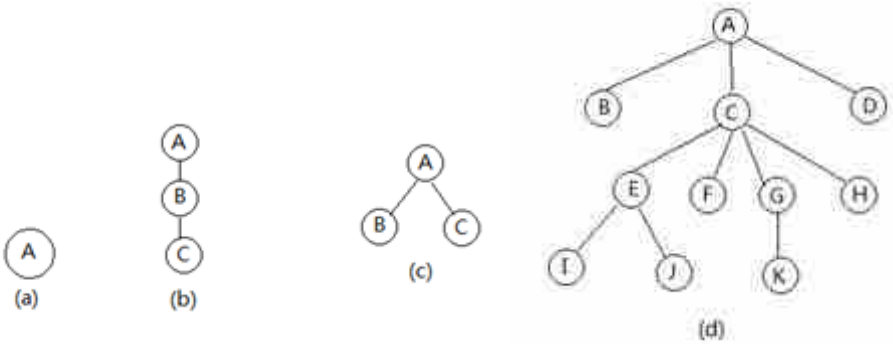
如此即可确定访问次序：

①若 p 是右孩子，则在后序序列中其后继是双亲结点；

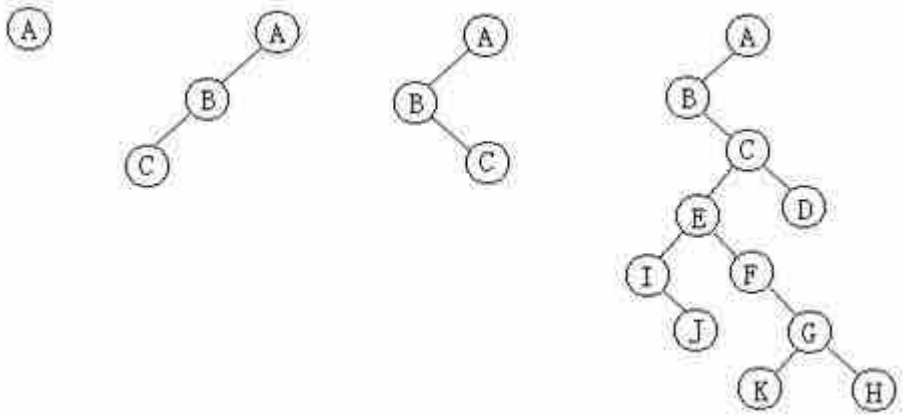
②若 p 是左孩子，则在后序序列中其后继是其兄弟最左下的子孙，如果兄弟不存在，其后继是其双亲结点。

因此，在一棵中序全线索二叉树上可以查找到给定结点在后序序列中的后继。

19．分别画出和下列树对应的各个二叉树：



答：根据二叉树和树的转化规则，可以画出图中所给的树对应的二叉树形态。

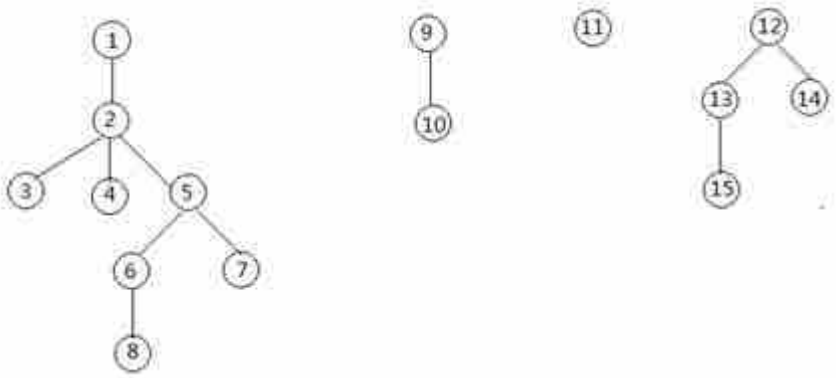


20．将下列森林转换为相应的二叉树，并分别按以下说明进行线索化。

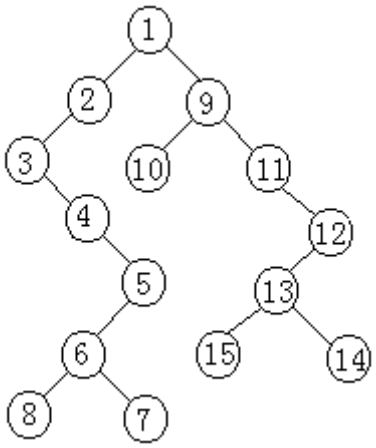
（1）先序前驱线索化；

（2）中序全线索化前驱线索和后继线索；

（3）后序后继线索化。



答：根据森林与二叉树的转换规则，先将图中的森林转换为二叉树：



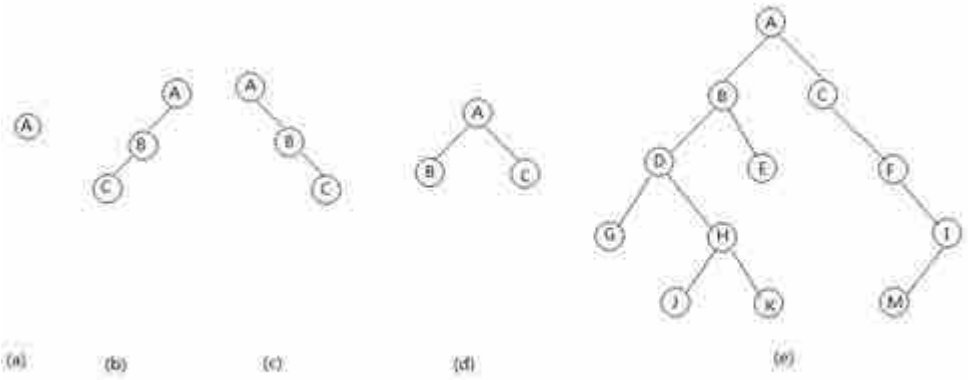
（1）先序序列：1 2 3 4 5 6 8 7 9 10 11 12 13 15 14

（2）中序序列：3 4 8 6 7 5 2 1 10 9 11 15 14 13 12

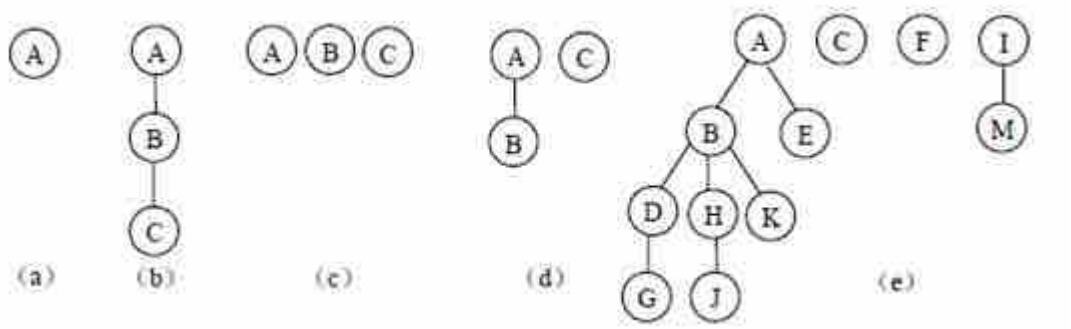
（3）后序序列：8 7 6 5 4 3 2 10 15 14 13 12 11 9

1

21．画出下列二叉树对应的森林：



答：根据二叉树和森林的转换规则，图中二叉树对应的森林如下：



22．对于 19 题中所给出的各树分别求出以下遍历序列：
（1）先根序列；（2）后根序列。

答：

（1）

（a）先根序列：A

（b）先根序列：A B C

（c）先根序列：A B C

（d）先根序列：A B C E I J F G K H D

（2）

（a）后根序列：A

（b）后根序列：C B A

（c）后根序列：B C A

（d）后根序列：B I J E F K G H C D A

23．画出和下列已知序列对应的树 T：
树的先根次序访问序列为 GFKDAIEBCHJ；
树的后根次序访问序列为 DIAEKFCJHBG。

答：树的先根序列为 GFKDAIEBCHJ，后根序列为 DIAEKFCJHBG，可以先转化成二叉树，再通过二叉树转换成树。注意二叉树的先根序列与等价树的先根序列相同，二叉树的中序序列对应着树的后根序列。

GFKDAIEBCHJ 为所求二叉树的先序序列，DIAEKFCJHBG 为二叉树的中序序列。通过观察先序序列，G 为二叉树的根结点，再由中序序列，G 的左子树序列为 DIAEKFCJHB，右子为空。可以表示成如下形式：

G（DIAEKFCJHB，NULL）

对于子树先序序列为 FKDAIEBCHJ，中序序列为 DIAEKFCJHB，显然子树根为 F。再由中序序列可以看到，F 的左子树是 DIAEK，右子树为 CJHB。进一步表示成：

G（F（DIAEK,CJHB）,NULL）

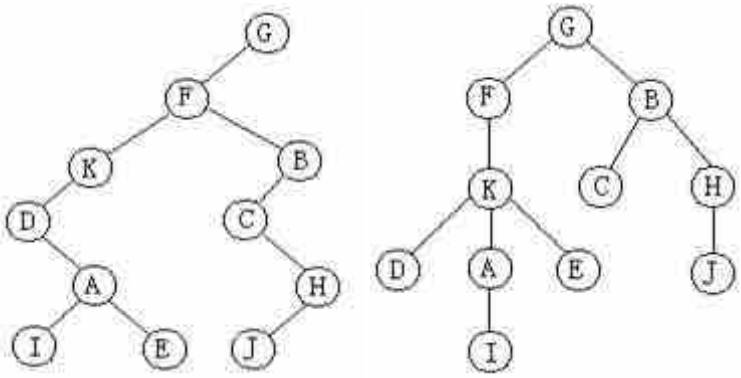
对于 DIAEK（中序表示），先序为 KDAIE，K 为根，左子为 DIAE，右子为空；对于 CJHB，B 为根，左子为 CJH，右子为空。进一步表示成：

G（F（K（DIAE,NULL）,B（CJH,NULL））,NULL）

G（F（K（D（NULL,IAE）,NULL）,B（C（NULL,JH）,NULL））,NULL）

G（F（K（D（NULL,A（I,E））,NULL）,B（C（NULL,H（J,NULL））,NULL））,NULL）

由此画出二叉树，进而画出树。



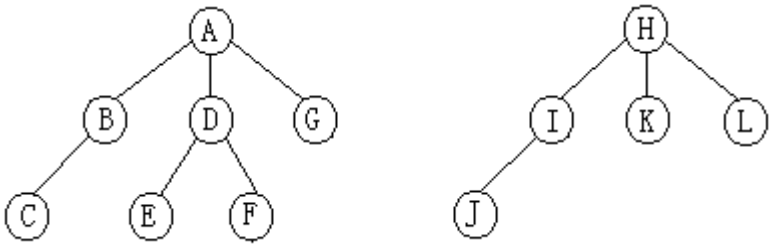
24．画出和下列已知序列对应的森林 F：

森林的先序次序访问序列为 ABCDEFGHIJKL；

森林的后序次序访问序列为 CBEFDGAJIKLH；

答：本题应注意下列转换：

树	森林	二叉树
先根	先序	先序
后根	中序	中序



25．证明：在结点数多于 1 的哈夫曼树中不存在度为 1 的结点。

答：用归纳法证明。

①当 n=2 时，要使其成为最优二叉树，必须使两个结点都成为叶子结点。

②设 n=k 时成立，则当 n=k+1 时，要使其成为最优，必须用 k 个结点的哈夫曼树与第 k+1 个结点组成一个新的最优二叉树，所以 n=k+1 时也成立。

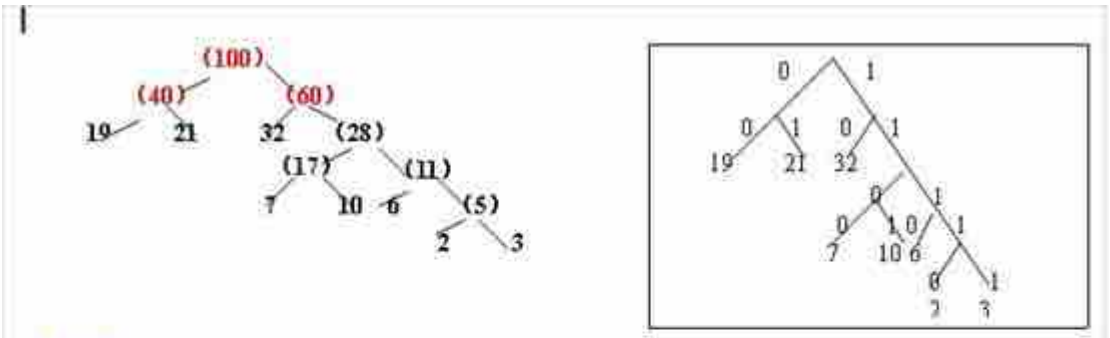
综上所述，在结点数多于 1 的哈夫曼树中不存在度为 1 的结点。

26．假设用于通信的电文仅由 8 个字母组成，字母在电文中出现的频率分别为 0.07,0.19,0.02,0.06,0.32,0.03,0.21,0.10.试为这 8 个字母设计哈夫曼编码。使用 0~7 的二进制表示形式是另一种编码方案。对于上述实例，比较两种方案的优缺点。

答：

（1）采用哈夫曼编码：不妨设这 8 个结点为 A、B、C、D、E、F、G、H，其相应的权为 7、19、2、6、32、3、21、10。

以下为由题意得到的哈夫曼树：



方案比较：

字母编号	对应编码	出现频率
1	1100	0.07
2	00	0.19
3	11110	0.02
4	1110	0.06
5	10	0.32
6	11111	0.03
7	01	0.21
8	1101	0.10

字母编号	对应编码	出现频率
1	000	0.07
2	001	0.19
3	010	0.02
4	011	0.06
5	100	0.32
6	101	0.03
7	110	0.21
8	111	0.10

方案 1：WPL=2（0.19+0.32+0.21）+4（0.07+0.06+0.10）+5（0.02+0.03）
=1.44+0.92+0.25=2.61

方案 2：WPL＝3（0.19+0.32+0.21+0.07+0.06+0.10+0.02+0.03）=3

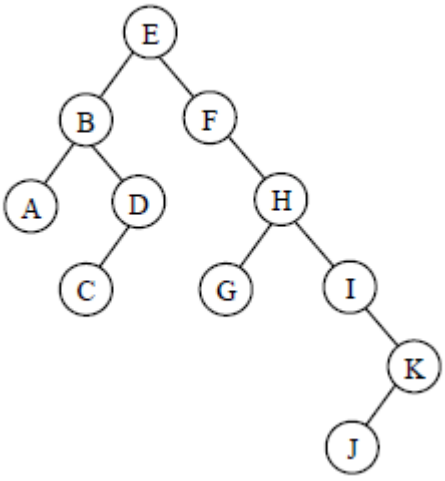
结论：哈夫曼编码优于等长二进制编码。

采用哈夫曼编码的优点是，编码的总长度为最小，数据处理时消耗的资源少。缺点是，编码过程中需要构造哈夫曼树。

采用二进制表示的编码优点是：编码简单，直接将数值转换为对应的二进制表示。缺点是，编码的数据比较长，在处理大量数据时消耗的资源多。

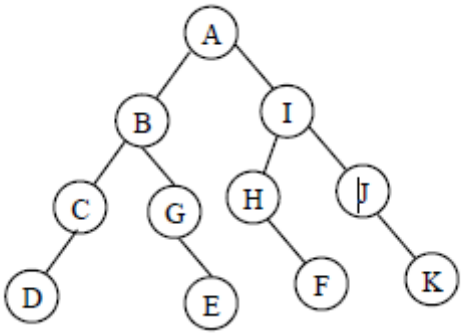
27．假设一颗二叉树的先序序列为 EBADCFHGIKJ 和中序序列为 ABCDEFGHIJK。请画出该树。

答：由先序序列可知二叉树的根结点是 E，再根据中序序列，从而得到序列 ABCD 是二叉树的左子树，序列 FGHIJK 是二叉树的右子树，依次递归判断，可画出整棵二叉树：



28．假设一棵二叉树的中序序列为 DCBGEAHFIJK 和后序序列 DCEGBFHKJIA。请画出该树。

答：根据后序序列可知，二叉树的根结点是 A，再根据中序序列，从而得到序列 DCBGE 在二叉树的左子树中，序列 HFIJK 在二叉树的右子树中，依次递归判断，可以画出整棵二叉树：



29．假设一棵二叉树的层序序列 ABCDEFGHIJ 和中序序列 DBGEHJACIF。请画出该树。
答：根据层序序列和中序序列的遍历规则，画出该树：

30．证明：树中结点 u 是结点 v 的祖先，当且仅当在先序序列中 u 在 v 之前，且在后序序列中 u 在 v 之后。
答：①先序序列的遍历顺序是：根结点、左孩子、右孩子；
②后序序列的遍历顺序是：左孩子、右孩子、根结点。
u 是结点 v 的祖先，即 v 是 u 的子孙。结合先序序列和后序序列的遍历顺序，可知：在先序序列中 u 在 v 之前，且在后序序列中 u 在 v 之后。
31．证明：由一棵二叉树的先序序列和中序序列可唯一确定这棵二叉树。
答：由先序序列可以首先确定整棵二叉树的根结点，然后在中序序列里根结点将它分为两个部分,根左边部分为左子树结点，根右边部分为右子树结点。根结点的两棵子树的根也会唯一确定,依次递归判断,所有子树的根都唯一确定,整棵二叉树就是唯一的。
32．证明：如果一棵二叉树的先序序列是 u_1，u_2，…，u_n，中序序列是 u_{p1}，u_{p2}，…，u_{pn}，则序列 1,2，…，n 可以通过一个栈得到序列 $p_1,p_2,\ldots,p_n$；反之，若以上述中的结论作为前提，则存在一棵二叉树，若其前序序列是 $u_1,u_2,\ldots,u_n$，则其中序序列为 u_{p1}，u_{p2}，…，u_{pn}。
答：由先序序列可以首先确定整棵二叉树的根结点，然后在中序序列里根结点将它分为两个部分,根左边部分为左子树结点，根右边部分为右子树结点。根结点的两棵子树的根也会唯一确定,依次递归判断,所有子树的根都唯一确定,整棵二叉树就是唯一的。
一棵二叉树的先序序列和中序序列可唯一确定这棵二叉树，因此序列 1,2，…，n 可以通过一个栈得到序列 $p_1,p_2,\ldots,p_n$，其逆命题也成立。
【算法设计题】

33．假定用两个一位数组 L[n+1]和 R[n+1]作为有 n 个结点的二叉树的存储结构，L[i]和 R[i]分别指示结点 i（i=1,2,…,n）的左孩子和右孩子，0 表示空。试写一个算法判别结点 u 是否为结点 v 的子孙。

答：

```
int Descendant （int u,int v）/*在孩子存储结构上判断 u 是否 v 的子孙,是则返回 1,否则返回 0*/
{
if（u==v） return 1;
if（L[v]==0）{//左子树不存在
if（R[v]==0）//右子树也不存在
    return 0;
else{//右子树存在，继续访问右子树
    if（Visit（u,R[v]）） return 1; //这是个递归算法，右子树中存在目标
    else return 0;
}
}
else //左子树存在
{
if（Visit（u,L[v]））//左子树中存在目标
    return 1;
else{//左子树中没有目标，需访问右子树
    if（R[v]==0）//没有右子树
    return 0;
    else{// 右子树存在，继续访问右子树
    if（Visit（u,R[v]）） return 1;
    else return 0;
    }
}
}
}
```

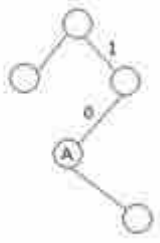
34．同 33 题的条件。先由 L 和 R 建立一维数组 T[n+1]，使 T 中第 i（i=1,2,…,n）个分量指示结点 i 的双亲，然后写判别结点 u 是否为结点 v 的子孙的算法。

答：

```
int Descendant （int u,int v）
{
int Nv;
Nv=NumElem（v）; //返回结点 v 的下标
    if（Nv==-1） exit（-2）; //结点 v 不存在，退出
//下面的判断过程与 6.33 基本一致
    if（u==v） return 1;
    if（L[Nv]==0） //左子树不存在
{
if（R[Nv]==0）//右子树也不存在
    return 0;
```

```
    else//右子树存在，继续访问右子树
{
if（Descendant （u,R[Nv]）） return 1;
else return 0;
    }
}
else//左子树存在
{
if（Descendant （u,L[Nv]））//左子树中存在目标
return 1;
else//左子树中没有目标，需访问右子树
{
if（R[Nv]==0）// 没有右子树
return 0;
else//右子树存在，继续访问右子树
{
if（Descendant （u,R[Nv]）） return 1;
else
return 0;
}
}
}
}
//返回结点在数组 T 中的下标
int NumElem（int x）
{
int i=0;
while（i<N&&T[i]!=x） i++;
if（T[i]==x） return i;
else return -1;
}
```

35．假设二叉树中左分支的标号为“0”，右分支的标号为“1”，并对二叉树增设一个头结点，令根结点为其右孩子，则从头结点到树中任一结点所经分支的序列为一个二进制序列，可认作是某个十进制数的二进制表示。例如，右图所示二叉树中，和结点 A 对应的二进制序列为“110”，即十进制整数 6 的二进制表示。已知一棵非空二叉树以顺序存储结构表示，试写一尽可能简单的算法，求出与在树的顺序存储结构中下标值为 i 的结点对应的十进制整数。



答：

```
#define N 8
char T[N]={"0","a","b","c","d","e","f","g"};
//用顺序数组存储，0 为头结点，a 为根结点，b 为左子树，c 为右子树...
//若某结点编号为奇数，则它必为其双亲的右孩子，否则为左孩子，则它必为其双亲的右孩//子，否则为左孩子。编号为 k 的结点的双亲结点的编号为 k/2。
int NumTree（char c）;//求结点在树中的编号
int Exp（int a,int b）;//求 a 的 b 次方
int NumElem（char c）;//返回元素在数组中的下标
int main（）
{
char c;
cout<<"请输入结点的值：";
cin>>c;
cout<<NumTree（c）<<endl;
return 0;
}
int NumTree（char c）// 求结点在树中的编号
{
int k,i=0;
int Code[N];
k=NumElem（c）; //返回结点 c 的下标
if（k==-1） exit（-2）; //结点 c 不存在，退出
while（k）{
//如果 k 为偶数，记录一个代码 1
if（k%2==1） Code[i]=1;
else Code[i]=0;
k=k/2;
i++;
}
int x=0,j;
for（j=0;j<i;j++）
x=x+Code[j]*Exp（2,j）;//将二进制转换为对应的十进制整数
return x;
}
//返回值为 a 的 b 次方
int Exp（int a,int b）
{
int i;
int x=1;
for（i=0;i<b;i++）
x=x*a;
return x;
}
```

```
int NumElem（char c） //返回结点在数组 T 中的下标
{
int i=0;
while（i<N&&T[i]!=c） i++
if（T[i]==c） return i;
else return -1;
}
```

36．若已知两棵二叉树 B1 和 B2 皆为空，或者皆不空且 B1 的左、右子树和 B2 的左、右子树分别相似，则称二叉树 B1 和 B2 相似。试编写算法，判别给定两棵二叉树是否相似。

答：

```
Status SimilarTree（BiTree& T1,BiTree& T2）
{
if（!T1）{//T1 是空树
if（!T2） return TRUE;//T2 是空树
else return FALSE;
}
else{//T1 是非空树
if（!T2） return FALSE;
else{//T2 是非空树
    if（SimilarTree（T1->lchild,T2->lchild）//递归判断左子树
    && SimilarTree（T1->rchild,T2->rchild））//递归判断右子树
    return TRUE;
    else return FALSE;
}
}
}
```

37．试利用栈的基本操作写出先序遍历的非递归形式的算法。

答：利用栈的后进先出特点。

```
void PreOrder_Nonrecursive（Bitree T）//先序遍历二叉树的非递归算法 {
InitStack（S）; Push（S,T）; //根指针进栈 while（!StackEmpty（S）） {
while（Gettop（S,p）&&p） { visit（p->data）; push（S,p->lchild）; } //向左走到尽头，遍历左子树 pop（S,p）; if（!StackEmpty（S）） { pop（S,p）; push（S,p->rchild）; //向右一步 } }//while }//PreOrder_Nonrecursive
```

38．同 37 题条件，写出后序遍历的非递归算法（提示：为分辨后序遍历时两次进栈的不同返回点，需在指针进栈时同时将一个标志进栈）。

答：

```
typedef struct { BTNode* ptr; enum {0,1,2} mark; } PMType; //有 mark 域的结点指针类型
void PostOrder_Stack（BiTree T）//后续遍历二叉树的非递归算法,用栈 { PMType a;
InitStack（S）; //S 的元素为 PMType 类型 Push （S,{T,0}）; //根结点入栈
while（!StackEmpty（S）） { Pop（S,a）; switch（a.mark） { case 0: Push（S,{a.ptr,1}）; //修改 mark 域 if（a.ptr->lchild） Push（S,{a.ptr->lchild,0}）; //访问左子树 break;
case 1: Push（S,{a.ptr,2}）; //修改 mark 域 if（a.ptr->rchild） Push（S,{a.ptr->rchild,0}）
```

; //访问右子树　　break; case 2:　　visit（a.ptr）; //访问结点,返回 }　　}//while }
//PostOrder_Stack

39．假设在二叉链表的结点中增设两个域：双亲域（parent）以指示其双亲结点；标志域（mark 取值 0…2）以区分在遍历过程中达到该结点时应继续向左或向右或访问该结点。试以此存储结构编写不用栈进行后序遍历的递推形式的算法。

答：

```
void PostOrder( BiTree root )
{
  // 设二叉树的结点中含有四个域：mark，parent，lchild，rchild。其中，
  // mark 域的初值均为零，指针 root 指向根结点，后序遍历此二叉树。
    p=root;
    while (p)
      switch (p->mark) {

                case 0:
                  p->mark=1;
                  if (p->lchild) p=p->lchild ;
                  break;
                case 1:
                  p->mark=2;
                  if (p->rchild) p=p->rchild ;
                  break;
                case 2:
                  p->mark=0;
                  visit( *p);
                  p=p->parent;
                  break;
                default: ;
              }
  } // PostOrder
```

40．若在二叉链表的结点中增设一个双亲域以指示其双亲结点，则在遍历过程中能否不设栈？试以此存储结构编写不设栈进行中序遍历的递推形式的算法。

答：

typedef struct {　　int data;　　PBTNode *lchild;　　PBTNode *rchild;　　PBTNode *parent; }
PBTNode,PBitree; //有双亲指针域的二叉树结点类型
void Inorder_Nonrecursive（PBitree T）//不设栈非递归中序遍历有双亲指针的二叉树 {
p=T;　　while（p->lchild） p=p->lchild; //向左走到尽头　　while（p）　　{ visit（p）;//访问结点 if（p->rchild） //寻找中序后继：当有右子树时 { p=p->rchild; while（p->lchild）
p=p->lchild; //后继就是在右子树中向左走到尽头 }
else if（p->parent->lchild==p） p=p->parent; /*当自己是双亲的左孩子时后继就是双亲*/

else/*当自己是双亲的右孩子时后继就是向上返回直到遇到自己是在其左子树中的祖先*/ { p=p->parent; while（p->parent&&p->parent->rchild==p） p=p->parent; p=p->parent; } } //while }//Inorder_Nonrecursive

41．编写递归算法，在二叉树中求位于先序序列中第 k 个位置的结点的值。

答：

```
Status PONodeK（TElemType& e,int& i,int k,BiTree& T） /*求谓语先序序列中第 k 各位置的结点的值。e 中存放结点的返回值，i 为计数器。*/
{
if（T） //二叉树非空
{  i++;
if（i==k） e=T->data;
else{
   PONodeK（e,i,k,T->lchild）;//递归判断左子树
   PONodeK（e,i,k,T->rchild）;//递归判断右子树
}
}
return OK;
}
```

42．编写递归算法，计算二叉树中叶子结点的数目。

答：

```
Status POLeafNodeNum（int& i,BiTree& T） //i 记录二叉树中叶子结点的个数
{
if（T） //二叉树非空
{
if（!T->lchild && !T->rchild） i++;
POLeafNodeNum（i,T->lchild）;//递归判断左子树
POLeafNodeNum（i,T->rchild）;//递归判断右子树
}
return OK;
}
```

43．编写递归算法，将二叉树中所有结点的左右子树相互交换。

答：

```
Status ExchangeBiTree（BiTree& T） //按先序交换二叉树的左右子树
{
BiTree p;//交换操作的中间变量
if（T）{//二叉树非空，交换左右子树
p=T->lchild;
T->lchild=T->rchild;
T->rchild=p;
ExchangeBiTree（T->lchild）;//递归交换左子树中的左右子树
ExchangeBiTree（T->rchild）;//递归交换右子树中的左右子树
}
return OK;
}
```

44．编写递归算法：求二叉树中以元素值为 x 的结点为根的子树的深度。

答：

```
Status ChildTreeDepth（BiTree& T,TElemType x,int& depth） /*求二叉树中以元素值为 x 的结点为根的子树的深度*/
{
BiTree T1;
if（PreOrderLocate（T,x,T1）） //找到二叉树中以元素值为 x 的结点为根的子树 T1
{
depth=BiTDepth（T1）;//求出 T1 的深度
return OK;
}
else return ERROR;
}
Status PreOrderLocate（BiTree& T,TElemType x,BiTree& T1） /*按先序在树中查找根为 x 的子树，T1 为指向子树根的指针*/
{
if（T）{
if（T->data==x）{
    T1=T;
    return OK;
}
else{
    if（PreOrderLocate（T->lchild,x,T1））//递归判断左子树
    return OK;
    else{
    if（PreOrderLocate（T->rchild,x,T1））//递归判断右子树
return OK;
    else return ERROR;
    }
}
}
else return ERROR;
}
int BiTDepth（BiTree& T） //求二叉树的深度
{
int ldep,rdep;
if（!T） return 0;
else{
ldep=BiTDepth（T->lchild）+1;//左子树的深度
rdep=BiTDepth（T->rchild）+1;//右子树的深度
return ldep>rdep?ldep:rdep;
}
}
```

45．编写递归算法：对于二叉树中每一个元素值为 x 的结点，删去以它为根的子树，并释放相应的空间。

答：

```
Status DelChildTree（BiTree& T,TElemType x） //删除以元素值为 x 的结点为根的子树
{
if（T）{
if（T->data==x） //在树 T 中找到元素值为 x 的结点
{
   DelBTree（T）;//删除二叉树
   T=NULL;
   return OK;
}
else//没找到元素值为 x 的结点
{   if（DelChildTree（T->lchild,x））//递归删除左子树中元素值为 x 的结点为根的子树
   return OK;
   else//递归删除右子树中元素值为 x 的结点为根的子树
{
   if（DelChildTree（T->rchild,x））
return OK;
   else return ERROR;
   }
}
}
else return ERROR;
}
Status DelBTree（BiTree& T） //删除二叉树
{
if（T） //二叉树非空，递归删除左子树和右子树
{
DelBTree（T->lchild）;
DelBTree（T->rchild）;
delete T;
return OK;
}
else return ERROR;
}
```

46．编写复制一棵二叉树的非递归算法。

答：

```
Status CopyBiTree（BiTree& T,BiTree& T1） //复制一棵二叉树
{
BiTree p;
if（T）{
p=new BiTNode;//创建结点
if（!p） return ERROR;
```

```
p->data=T->data;//复制结点
T1=p;
CopyBiTree（T->lchild,T1->lchild）;//递归复制左子树
CopyBiTree（T->rchild,T1->rchild）;//递归复制右子树
}
else{//如果 T 是空树，那么直接复制，T1 也是空树
T1=T;
}
return OK;
}
```

47．编写按层次顺序（同一层自左至右）遍历二叉树的算法。

答：借助队列先进先出的特点对二叉树进行层次遍历。

```
typedef BiTree QElemType;
#include "c:\Yin\include\Queue.h"
Status LevelOrderTraverse（BiTree& T,Status （*Visit）（TElemType e））
{
QElemType p;
Queue q;
InitQueue（q）;
if（T） EnQueue（q,T）;
while（!QueueEmpty（q））{
DeQueue（q,p）;
Visit（p->data）;
if（p->lchild） EnQueue（q,p->lchild）;
if（p->rchild） EnQueue（q,p->rchild）;
}
return OK;
}
```

48．已知在二叉树中，*root 为根结点，*p 和*q 为二叉树中两个结点，试编写求距离它们最近的共同祖先的算法。

答：

```
Status MinComAncst（BiTree& T,TElemType& e,TElemType *p,TElemType *q）
//在二叉树 T 中求结点*p 和*q 距离最近的共同最小祖先 e
{
if（!T） return ERROR;
BiTree T1,T2,pt=NULL;
if（!CopyBiTree（T,T1））  return ERROR;
if（!CopyBiTree（T,T2））  return ERROR;
if（!PathTree（T1,p））  //求根结点到结点 p 的路径树 T1
return ERROR;
else ShowBiTree（T1）;
cout<<endl;
if（!PathTree（T2,q））  //求根结点到结点 q 的路径树 T2
return ERROR;
```

```
else ShowBiTree（T2）;
cout<<endl;
while（T1 && T2 && T1->data==T2->data）{
pt=T1;
if（T1->lchild）{
    T1=T1->lchild;
    T2=T2->lchild;
}
else{
    if（T1->rchild）{
    T1=T1->rchild;
    T2=T2->rchild;
    }
}
}
if（!pt） return ERROR;
else{
e=pt->data;
return OK;
}
}
Status PathTree（BiTree& T,TElemType *p）
//在二叉树 T 中求根到结点 p 的路径树，该操作将剪去除路径之外的所有分支
{
if（!T || !p） return ERROR;
if（T->data==*p） //找到目标，删除目标的左右子树
{
if（T->lchild） DelBiTree（T->lchild）;
if（T->rchild） DelBiTree（T->rchild）;
return OK;
}
else//没找到目标，继续递归查找
{
if（PathTree（T->lchild,p））{//目标在左子树中，删除右子树
    if（T->rchild） DelBiTree（T->rchild）;
    return OK;
}
else
    if（PathTree（T->rchild,p）） //目标在右子树中，删除左子树
{
    if（T->lchild） DelBiTree（T->lchild）;
    return OK;
    }
    else return ERROR;//找不到目标
```

}
}
49．编写算法判别给定二叉树是否为完全二叉树。
答：
Status CompleteBiTree（BiTree &T）
//d 表示左子树的深度减去右子树的深度所得到的差值，如果该二叉树为完全二叉树，则 d 的范围在[0,1]内。
{
int d;
if（T）{
d=BiTDepth（T->lchild）-BiTDepth（T->rchild）;
if（d<0 || d>1） return ERROR;//不符合完全二叉树的特点
else{
if（CompleteBiTree（T->lchild） &&
CompleteBiTree（T->rchild）） return OK;
else return ERROR;
}
}
else return OK;
}
50．假设以三元组（F，C，L/R）的形式输入一棵二叉树的诸边（其中 F 表示双亲结点的标识，C 表示孩子结点标识，L/R 表示 C 为 F 的左孩子或右孩子），且在输入的三元组序列中，C 是按层次顺序出现的。设结点的标识是字符类型。F="^"时 C 为根结点标识，若 C 也为"^"，则表示输入结束。例如，15 题所示二叉树的三元组序列输入格式为：
^AL
ABL
ACR
BDL
CEL
CFR
DGR
FHL
^^L
试编写算法，由输入的三元组序列建立二叉树的二叉链表。
答：
Status CreateBitree_Triplet（Bitree &T）//输入三元组建立二叉树 { if（getchar（）!="^"） return ERROR; T=（BTNode*）malloc（sizeof（BTNode））;//动态创建根结点 p=T;p->data=getchar（）; getchar（）;//滤去多余的回车字符 InitQueue（Q）; EnQueue（Q,T）; while（（parent=getchar（））!="^"&&（child=getchar（）） &&（side=getchar（）） { while（QueueHead（Q）!=parent&&!QueueEmpty（Q）） DeQueue（Q,e）; if（QueueEmpty（Q）） return ERROR; //未按层次顺序输入 p=QueueHead（Q）; q=（BTNode*）malloc（sizeof（BTNode））;//动态申请结点 if（side=="L"） p->lchild=q; else if（side=="R"） p->rchild=q; else return ERROR; //格式不正确 q->data=child; EnQueue（Q,q）; } return OK; }//CreateBitree_Triplet

51．编写一个算法，输出以二叉树表示的算术表达式，若该表达式中含有括号，则在输出时应添上。

答：使用中序遍历，输出表达式。

```
Status ShowBiTExpress（BiTree& T）
{
if（T）{
if（T->lchild）{//访问左子树
    if（Low（T->lchild->data,T->data））{
    cout<<"（";
    ShowBiTExpress（T->lchild）;
    cout<<"）";
    }
    else ShowBiTExpress（T->lchild）;
}
cout<<T->data;//访问结点
if（T->rchild）{//访问右子树
    if（Low（T->rchild->data,T->data））{
    cout<<"（";
    ShowBiTExpress（T->rchild）;
    cout<<"）";
    }
    else ShowBiTExpress（T->rchild）;
}
}
return OK;
}
Status Low（char a,char b） //运算符的判断
{
if（（a=="+" || a=="-"）&&（b=="*" || b=="/"））
return TRUE;
else return FALSE;
}
```

52．一棵二叉树的繁茂度定义为各层结点数的最大值与树的高度的乘积。试写一算法，求二叉树的繁茂度。

答：

```
int BiTreeThrive（BiTree& T）
{
int i,d,m[20];
d=BiTDepth（T）;//树的深度，即高度
BiTree p=T;
Stack s1,s2;
InitStack（s1）;//初始化栈 s1 存储第 i 层的结点
InitStack（s2）;//初始化 s2 存储第 i+1 层的结点
for（i=0;i<20;i++）{
```

```
m[i]=0; //每层结点个数
}
if（p） Push（s1,p）;
else return 0;
for（i=0;i<d;i++）{
if（!StackEmpty（s1） && StackEmpty（s2））{
   while（!StackEmpty（s1））{
   Pop（s1,p）; m[i]++;//s1 中存放第 i 层的结点
   if（p->lchild） Push（s2,p->lchild）;//s2 中存放第 i+1 层结点
   if（p->rchild） Push（s2,p->rchild）;
   }
}
else{
   if（StackEmpty（s1） && !StackEmpty（s2））{
   while（!StackEmpty（s2））{
Pop（s2,p）; m[i]++;
if（p->lchild） Push（s1,p->lchild）;
if（p->rchild） Push（s1,p->rchild）;
   }
   }
}
}
int max=m[0];
for（i=0;i<d;i++）
if（max<m[i]） max=m[i];
return max*d;
}
```

53．试编写算法，求给定二叉树上从根结点到叶子结点的一条其路径长度等于树的深度减一的路径（即列出从根结点到该叶子结点的结点序列），若这样的路径存在多条，则输出路径终点（叶子结点）在“最左”的一条。

答：

```
Status MaxPathBiTree（BiTree& T） //所有从根到叶子最长路径树
{
if（T）{
if（BiTDepth（T）-BiTDepth（T->lchild）!=1）
   DelBiTree（T->lchild）;//将不符合的无关分支删除
else
   MaxPathBiTree（T->lchild）;//递归判断左子树中根到叶子的最长路径
if（BiTDepth（T）-BiTDepth（T->rchild）!=1）
   DelBiTree（T->rchild）;
else
   MaxPathBiTree（T->rchild）;//递归判断右子树中根到叶子的最长路径
}
return OK;
```

```
}
Status LMaxPathBiTree（BiTree& T） //从根到叶子最长路径中最左方的路径树
{
if（T）{
if（BiTDepth（T）-BiTDepth（T->lchild）==1）{
    DelBiTree（T->rchild）;//将无关分支删除
    LMaxPathBiTree（T->lchild）;//递归判断左子树
}
else{
    DelBiTree（T->lchild）;
    if（BiTDepth（T）-BiTDepth（T->rchild）==1）
    LMaxPathBiTree（T->rchild）;//递归判断右子树
    else
    DelBiTree（T->rchild）;
}
}
return OK;
}
```

54．假设以顺序表 sa 表示一棵完全二叉树，sa.elem[sa.last]中存放树中各结点的数据元素。试编写算法由此顺序存储结构建立该二叉树的二叉链表。

答：

```
Status CreateCompleteBiTree（SqList<TElemType>& ST,BiTree& LT）
//根据完全二叉顺序树创建完全二叉链表树
{
BiTree p;
int i=0,len;
if（ST.Length==0） return OK;
p=new BiTNode;
if（!p） return ERROR;
p->data=ST.Get（i）;
p->lchild=NULL;
p->rchild=NULL;
LT=p;
Queue q;
InitQueue（q）;
EnQueue（q,p）;
len=ST.Length（）;
while（!QueueEmpty（q）&&i<len-1）{
DeQueue（q,p）;
if（i<len-1 && i%2==0） //结点编号为偶数，创建左孩子
{
    p->lchild=new BiTNode;
    if（!p->lchild） return ERROR;
    p->lchild->data=ST.Get（++i）;
```

```
    p->lchild->lchild=NULL;
    p->lchild->rchild=NULL;
    EnQueue（q,p->lchild）;
}
if（i<len-1 && i%2==1） //结点编号为奇数，创建右孩子
{
    p->rchild=new BiTNode;
    if（!p->rchild） return ERROR;
    p->rchild->data=ST.Get（++i）;
    p->rchild->lchild=NULL;
    p->rchild->rchild=NULL;
    EnQueue（q,p->rchild）;
}
}
return OK;
}
```

55．为二叉链表的结点增加 DescNum 域。试写一算法，求二叉树的每个结点的子孙数目并存入其 DescNum 域。请给出算法的时间复杂度。

答：

```
Status PreOrderTraverse（BiTree& T）
{
if（T）{
T->DescNum=DescendNum（T）;
PreOrderTraverse（T->lchild）;//递归处理左子树中每个结点的 DescNum 域
PreOrderTraverse（T->rchild）; //递归处理右子树中每个结点的 DescNum 域
}
return OK;
}
int DescendNum（BiTree& T） //递归求结点的子孙数目
{
if（!T） return 0;
if（!T->lchild）{//左子树空
if（!T->rchild） return 0;
else return DescendNum（T->rchild）+1;
}
else//左子树不空
{
if（!T->rchild） return DescendNum（T->lchild）+1;
else//左、右子树都为空
return DescendNum（T->rchild）+DescendNum（T->lchild）+2;
}
}
```

56．试写一个算法，在先序后继线索二叉树中，查找给定结点*p 在先序序列中的后继（假设二叉树的根结点未知）。并讨论实现此算法对存储结构有何要求？

答：先对二叉树 T 进行先序线索，得到先序线索二叉树 Thrt，然后再进行查找。

```
Status PreOrderThreading（BiThrTree& Thrt,BiThrTree& T） //先序线索二叉树算法
{
BiThrTree pre;
Thrt=new BiThrNode; //为先序线索二叉树建立头结点
if（!Thrt） exit（OVERFLOW）;//创建结点失败
Thrt->LTag=Thread;//左标志域
Thrt->RTag=Link;//右标志域
Thrt->lchild=Thrt;//左子树回指
if（!T） Thrt->rchild=Thrt;//若二叉树空，右子树回指
else{
Thrt->rchild=T;
pre=Thrt;
PreThreading（T,pre）; //先序遍历进行先序线索化
pre->rchild=Thrt; //最后一个结点线索化
pre->RTag=Thread;
Thrt->lchild=pre;
}
return OK;
}
Status PreThreading（BiThrTree& T,BiThrTree& pre） //先序线索化
{
if（T）{
if（!T->lchild）{
    T->LTag=Thread;//左标志域为线索
    T->lchild=pre;//左孩子指向前驱结点
}
if（pre && !pre->rchild）{
    pre->RTag=Thread;//前驱结点的右标志域为线索
    pre->rchild=T;//前驱结点的右孩子指向结点
}
pre=T;
if（T->LTag==Link） PreThreading（T->lchild,pre）;//递归线索化左子树
if（T->RTag==Link） PreThreading（T->rchild,pre）;//递归线索化右子树
}
return OK;
}
Status FindNextInBiThrTree（BiThrTree& q,TElemType *p）
//从二叉线索树上任一结点 q 开始查找结点*p
{
BiThrTree pt=q;
if（!pt） return FALSE;
if（pt->data==*p）
//如果找到，将*p 的后继结点指针存于 q 中，返回 TRUE；否则返回 FALSE
```

```
{
if（pt->LTag==Link） q=pt->lchild;
else q=pt->rchild;
return OK;
}
pt=q->rchild;
while（pt!=q && pt->data!=*p）{
if（pt->LTag==Link） pt=pt->lchild;
else pt=pt->rchild;
}
if（pt==q） return FALSE;
if（pt->data==*p）{
if（pt->LTag==Link） q=pt->lchild;
else q=pt->rchild;
}
return OK;
}
```

此算法用到数据域和指针域，二叉树采用链式存储结构较为合适。

57．试写一个算法，在后序后继线索二叉树中，查找给定结点*p 在后序序列中的后继（二叉树的根结点指针并未给出）。并讨论实现算法对存储结构有何要求？

答：先建立线索树再进行查找。

```
Status PostOrderThreading（BiThrTree& T,BiThrTree& pre）;//首先建立后序线索树
Status FindNextInBiThrTree（BiThrTree& q,TElemType *p）;//再进行查找
//后序线索二叉树的算法
Status PostOrderThreading（BiThrTree& Thrt,BiThrTree& T）
{
BiThrTree pre;
Thrt=new BiThrNode; //为线索二叉树建立头结点
if（!Thrt） exit（OVERFLOW）;//创建结点失败
Thrt->LTag=Link;
Thrt->RTag=Thread;
Thrt->rchild=Thrt;//右子树回指
if（!T） Thrt->lchild=Thrt;//若二叉树空，左子树回指
else{
Thrt->lchild=T;
    pre=Thrt;
PostThreading（T,pre）;//后序遍历进行后序线索化
pre->rchild=Thrt;//最后一个结点线索化
pre->RTag=Thread;
Thrt->rchild=pre;
}
return OK;
}
//后序线索化
```

```
Status PostThreading（BiThrTree& T,BiThrTree& pre）
{
if（T）{
if（T->LTag==Link） PostThreading（T->lchild,pre）;//递归线索化左子树
if（T->RTag==Link） PostThreading（T->rchild,pre）;//递归线索化右子树
if（!T->lchild）{
    T->LTag=Thread;
    T->lchild=pre;
}
if（pre && !pre->rchild）{
    pre->RTag=Thread;
    pre->rchild=T;
}
pre=T;
}
return OK;
}
```

此算法用到数据域和指针域，二叉树采用链式存储结构较为合适。

58．试写一个算法，在中序全线索二叉树的结点*p 之下，插入一棵以结点*x 为根、只有左子树的中序全线索二叉树，使*x 为根的二叉树成为*p 的左子树。若*p 原来有左子树，则令它为*x 的右子树。完成插入之后的二叉树应保持全线索化特性。

答：

```
typedef char TElemType; //定义孩子兄弟表示法的二叉树的数据类型
typedef struct CSNode{
TElemType data;
struct CSNode *firstchild,*nextsibling;
}CSNode,*CSTree;
Status CreateTree（CSTree& T） //建立树的二叉链表表示
{
char ch;
cout<<"输入结点的值（一个字符，"@"表示空树）";
cin>>ch;
if（ch=="@"）{
T=NULL;
}
else{
T=new CSNode;
if（!T） return ERROR;
T->data=ch;
CreateTree（T->firstchild）;
CreateTree（T->nextsibling）;
}
return OK;
}
```

```
Status ShowTree（CSTree& T,CSTree& Father） //输出树的各边
{
if（T && Father）
cout<<"（"<<Father->data<<","<<T->data<<"）";
if（T->firstchild） ShowTree（T->firstchild,T）;
if（T->nextsibling） ShowTree（T->nextsibling,Father）;
return OK;
}
```

59．编写算法完成下列操作：无重复地输出以孩子兄弟链表存储的树 T 中所有的边。输出的形式为（k_1,k_2），…,（k_i,k_j）,…,其中 k_i 和 k_j 为树结点中的结点标识。

答：

```
void Print_CSTree（CSTree T）//输出孩子兄弟链表表示的树 T 的各边 {  for（child=T->firstchild;child;child=child->nextsib）//采用循环递归输出  { printf（"（%c,%c）,",T->data,child->data）;  Print_CSTree（child）;  } }//Print_CSTree
```

60．试编写算法，对一棵以孩子-兄弟链表表示的树统计叶子的个数。

答：

```
int LeafNum（CSTree &T）
{
if（T）{
if（!T->firstchild）/*结点 T 的第一个孩子不空，返回的叶子个数为该孩子兄弟子树上叶子个数加 1*/
   return 1+LeafNum（T->nextsibling）;
else/*结点 T 第一个孩子为空，返回的叶子个数为结点 T 第一个孩子子树上的叶子个数加上结点 T 兄弟子树上的叶子个数*/
   return LeafNum（T->firstchild）+LeafNum（T->nextsibling）;
}
else return 0;
}
```

61．试编写算法，求一棵以孩子-兄弟链表表示的树的度。

答：

```
int DegreeNum（CSTree &T）
{
int d,dl,dr;
if（T）{
if（!T->firstchild） d=0;
//结点 T 的第一个孩子不空，则树的度为该孩子兄弟结点的个数加 1
else d=1+RSiblingNum（T->firstchild）;
dl=DegreeNum（T->firstchild）;//递归计算结点 T 第一个孩子子树的度
dr=DegreeNum（T->nextsibling）;//递归计算结点 T 兄弟子树的度
return Max（d,dl,dr）; //三数中求最大者
}
else return 0;
}
int RSiblingNum（CSTree& T） //返回当前结点的兄弟数
```

{
int i=0;
while（T->nextsibling）{
i++;
T=T->nextsibling;
}
return i;
}
62．对以孩子-兄弟链表表示的树编写计算树的深度的算法。
答：递归求结点第一个孩子子树的深度和兄弟子树的度，然后求最大值即可。
int Depth（CSTree& T） //树的深度
{
int d1,d2;
if（T）{
d1=1+Depth（T->firstchild）;
d2=Depth（T->nextsibling）;
return d1>d2?d1:d2;
}
else return 0;
}
63．对以孩子链表表示的树编写计算树的深度的算法。
答：
int GetDepth_CTree（CTree A）//求孩子链表表示的树 A 的深度 { return SubDepth（A.r）; }//GetDepth_CTree
int SubDepth（int T）//求子树 T 的深度 { if（!A.nodes[T].firstchild） return 1; for（sd=1,p=A.nodes[T].firstchild;p;p=p->next） if（（d=SubDepth（p->child））>sd） sd=d; return sd+1; }//SubDepth
64．对以双亲表表示的树编写计算树的深度的算法。
答：
int GetDepth_PTree（PTree T）//求双亲表表示的树 T 的深度 { maxdep=0; for（i=0;i<T.n;i++） { dep=0; for（j=i;j>=0;j=T.nodes[j].parent） dep++;//循环，求每一个结点的深度 if（dep>maxdep） maxdep=dep;//树的深度为所有结点度的最大值 } return maxdep; }//GetDepth_PTree
65．已知一棵二叉树的前序序列和中序序列分别存于两个一维数组中，试编写算法建立该二叉树的二叉链表。
答：
char Pred,Ind; //假设前序序列和中序序列已经分别储存在数组 Pre 和 In 中
Bitree Build_Sub（int Pre_Start,int Pre_End,int In_Start,int In_End）
//由子树的前序和中序序列建立其二叉链表 { sroot=（BTNode*）malloc（sizeof（BTNode））;//动态创建根结点 sroot->data=Pre[Pre_Start];//根结点的数据域即前序序列数组的第一个值 for（i=In_Start;In[i]!=sroot->data;i++）;//在中序序列中查找子树根 leftlen=i-In_Start; rightlen=In_End-i;//计算左右子树的大小 if（leftlen）//建左子树,注意参数表的计算 {

lroot=Build_Sub（Pre_Start+1,Pre_Start+leftlen,In_Start,In_Start+leftlen-1）; sroot->lchild=lroot; }
if（rightlen） //建右子树,注意参数表的计算
{ rroot=Build_Sub（Pre_End-rightlen+1,Pre_End,In_End-rightlen+1,In_End）; sroot->rchild=rroot; } return sroot; //返回子树根 }//Build_Sub
main（） { ... Build_Sub（1,n,1,n）;//初始调用参数,n 为树结点总数 ... }

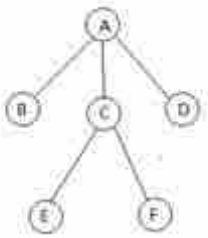
66．假设有 n 个结点的树 T 采用了双亲表示法，写出由此建立树的孩子-兄弟链表的算法。

答：

typedef struct //结点的指针和其最后一个孩子的指针
{ CSNode *ptr; CSNode *lastchild; } NodeMsg; Status Build_CSTree_PTree（PTree T）//由树 T 的双亲表构造其孩子兄弟链表 { NodeMsg Tree[MAXSIZE];
for（i=0;i<T.n;i++）//循环建立链表中的每个结点 { Tree[i].ptr=（CSNode*）malloc（sizeof（CSNode））; Tree[i].ptr->data=T.node[i].data; //动态创建每个结点
if（T.nodes[i].parent>=0） //不是根结点 { j=T.nodes[i].parent; //本算法要求双亲表必须是按层序存储 if（!（Tree[j].lastchild）） //双亲当前还没有孩子 Tree[j].ptr->firstchild=Tree[i].ptr; //成为双亲的第一个孩子 else //双亲已经有了孩子 Tree[j].lastchild->nextsib=Tree[i].ptr; //成为双亲最后一个孩子的下一个兄弟 Tree[j].lastchild=Tree[i].ptr; //成为双亲的最后一个孩子 }//if }//for }//Build_CSTree_PTree

67．假设二元组（F，C）的形式输入一棵树的诸边（其中 F 表示双亲结点的标识，C 表示孩子结点标识），且在输入的二元组序列中，C 是按层次顺序出现的。F="^"时 C 为根结点标识，若 C 也为"^"，则表示输入结束。例如，如下所示树的输入序列为：

^A
AB
AC
AD
CE
CF
^^



答：

typedef struct//定义结点数据类型,结点指针和最后一个孩子的指针
{ char data; CSNode *ptr; CSNode *lastchild; } NodeInfo;
Status CreateCSTree_Duplet（CSTree &T）//输入二元组建立树的孩子兄弟链表 {
NodeInfo Treed; n=1;k=0; if（getchar（）!="^"） return ERROR;//未按格式输入
if（（c=getchar（））=="^"） T=NULL;//空树 Tree[0].ptr=（CSNode*）malloc（sizeof（CSNode））;//动态创建结点指针 Tree[0].data=c; Tree[0].ptr->data=c;
while（（p=getchar（））!="^"&&（c=getchar（））!="^"） { Tree[n].ptr=（CSNode*）

malloc（sizeof（CSNode））; Tree[n].data=c; Tree[n].ptr->data=c; for（k=0;Tree[k].data!=p;k++）;//查找当前边的双亲结点 if（Tree[k].data!=p） return ERROR;//未找到:未按层序输入 r=Tree[k].ptr; if（!r->firstchild） r->firstchild=Tree[n].ptr; else Tree[k].lastchild->nextsib=Tree[n].ptr; Tree[k].lastchild=Tree[n].ptr;//这一段含义同上一题 n++; }//while return OK; }//CreateCSTree_Duplet

68．已知一棵树的由根至叶子结点按层次输入的结点序列及每个结点的度（每层中自左至右输入），试写出构造此树的孩子-兄弟链表的算法。

答：

Status CreateCSTree_Degree（char node[ ],int degree[ ]）/*由结点的层序序列和各结点的度构造树的孩子兄弟链表*/ { CSNode *ptr[MAXSIZE];//树结点指针的辅助存储 ptr[0]=（CSNode*）malloc（sizeof（CSNode））; i=0;k=1;//i 为当前结点序号,k 为当前孩子的序号 while（node[i]） { ptr[i]->data=node[i]; d=degree[i]; if（d） { ptr[k]=（CSNode*）malloc（sizeof（CSNode））;//k 为当前孩子的序号 ptr[i]->firstchild=ptr[k]; //建立 i 与第一个孩子 k 之间的联系 for（j=2;j<=d;j++） { ptr[k]=（CSNode*）malloc（sizeof（CSNode））; ptr[k-1]->nextsib=ptr[k]; //当结点的度大于 1 时,为其孩子建立兄弟链表 k++; }//for ptr[k-1]->nextsib=NULL; }//if i++;//结点序号加 1，继续循环构造表中的结点 }//while }//CreateCSTree_Degree

69．假设以二叉链表存储的二叉树中，每个结点所含数据元素均为单字母，试编写算法，按树状打印二叉树的算法。例如：左下二叉树印为右下形状。

C

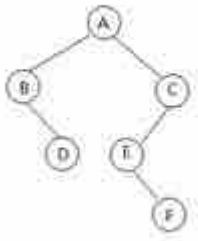
F

E

A

D

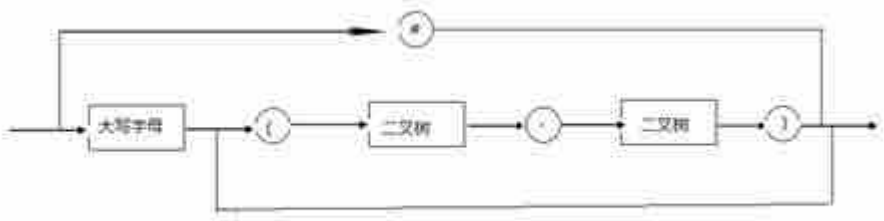
B



答：

void Print_BiTree（BiTree T,int i）/*按树状打印输出二叉树的元素,i 表示结点所在层次,初次调用时 i=0*/ { if（T->rchild） Print_BiTree（T->rchild,i+1）;//递归打印右子树 for（j=1;j<=i;j++） printf（" "）;//打印 i 个空格以表示出层次 printf（"%c\n",T->data）; //打印 T 元素,换行 if（T->lchild） Print_BiTree（T->lchild,i+1）;//递归打印左子树 } //Print_BiTree

70．如果用大写字母标识二叉树结点，则一棵二叉树可以用符合下面语法图的字符序列表示。试写一个递归算法，由这种形式的字符序列，建立相应的二叉树的二叉链表存储结构。



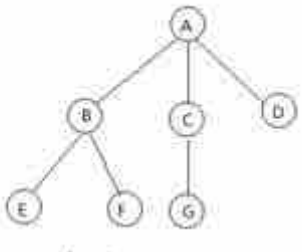
例如 69 题所示二叉树的输入形式为 A（B（#,D）,C（E（#,F）,#））。

答：

```
Status CreateBiTree_GList（BiTree &T）//由广义表形式的输入建立二叉链表 {
c=getchar（）;  if（c=="#"） T=NULL;//空子树  else  { T=（CSNode*）
malloc（sizeof（CSNode））;//动态创建结点 T->data=c; if（getchar（）!="（"） return
ERROR; if（!CreateBiTree_GList（pl））  return ERROR; T->lchild=pl; if（getchar（）!
=","）  return ERROR; if（!CreateBiTree_GList（pr））  return ERROR; T->rchild=pr;
if（getchar（）!="）"） return ERROR; //这些语句是为了保证输入符合 A（B,C）的格式  }
return OK; }//CreateBiTree_GList
```

71．假设树上每个结点所含的数据元素为一个字母，并且以孩子-兄弟链表为树的存储结构，试写一个按凹入表方式打印一棵树的算法。例如：左下所示树印为右下形状。

```
A
    B
        E
        F
    C
        G
    D
```



答：

void Print_CSTree（CSTree T,int i）/*按凹入表形式打印输出树的元素，i 表示结点所在层次，初次调用时 i=0*/ {　for（j=1;j<=i;j++） printf（" "）;//留出 i 个空格以表现出层次 printf（"%c\n",T->data）;//打印元素，换行　for（p=T->firstchild;p;p=p->nextsib） Print_CSTree（p,i+1）;//递归打印子树 }//Print_CSTree

72．以孩子链表为树的存储结构，重做 71 题。

答：

void Print_CTree（int e,int i）//按凹入表形式打印输出树的元素，i 表示结点所在层次 { for（j=1;j<=i;j++） printf（" "）;//留出 i 个空格以表现出层次　printf（"%c\n",T.nodes[e].data）;//打印元素，,换行　for（p=T.nodes[e].firstchild;p;p=p->next） Print_CSTree（p->child,i+1）;//打印子树 }//Print_CSTree

main（） {　...　Print_CTree（T.r,0）;//初次调用时 i=0　... }//main

6.3　考研真题与典型题详解

一、选择题

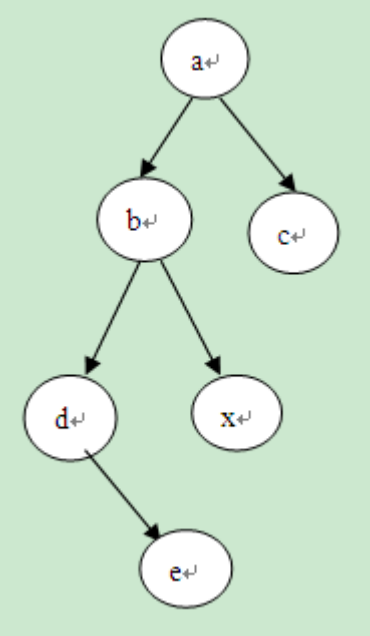
1．若对如下的二叉树进行中序线索化，则结点 x 的左、右线索指向的结点分别是（　　）。[2014 年联考真题]

A．e,c

B．e,a

C．d,c

D．b,a



【答案】D 查看答案

【解析】此二叉树的中序遍历序列为：debxac,由于节点 x 左右孩子都为空，所有进行中序线索化时，它的左右孩子指针分别指向它的中序遍历序列的直接前驱结点 b 和直接后继结点 a，所以选 D

2．将森林 F 转换为对应的二叉树 T，F 中叶结点的个数等于（　　）。[2014 年联考真题]

A．T 中叶结点的个数

B．T 中度为 1 的结点个数

C．T 中左孩子指针为空的结点个数

D．T 中右孩子指针为空的结点个数

【答案】C 查看答案

【解析】森林转化为对应的二叉树是‘孩子-兄弟’存储的，即左孩子指针指向当前节点的孩子节点，右孩子指针指向当前节点的兄弟节点，所以在 T 中左孩子指针为空则代表它在森林中并没有孩子即为叶结点。所以选 C

3．若 X 是后序线索二叉树中的叶结点，且 X 存在左兄弟结点 Y，则 X 的右线索指向的是（　　）。[2013 年联考真题]

A．X 的父结点

B．以 Y 为根的子树的最左下结点

C．X 的左兄弟结点 Y

D．以 Y 为根的子树的最右下结点

【答案】A 查看答案

【解析】根据后续线索二叉树的定义,X 结点为叶子结点且有左兄弟,那么这个结点为右孩子结点,利用后续遍历的方式可知 X 结点的后继是其父结点,即其右线索指向的是父结点。

4．已知三叉树 T 中 6 个叶结点的权分别是 2，3，4，5，6，7，T 的带权（外部）路径长度最小是（　　）。[2013 年联考真题]

A．27

B．46

C．54

D．56

【答案】B 查看答案

【解析】利用三叉树的 6 个叶子结点的权构建最小带权生成树，最小的带权路径长度为（2+3）*3+（4+5）*2+（6+7）*1＝46

5．5 个字符有如下 4 种编码方案，不是前缀编码的是（　　）。[2014 年联考真题]

A．01,0000,0001,001,1

B．011,000,001,010,1

C．000,001,010,011,100

D．0,100,110,1110,1100

【答案】D 查看答案

【解析】在一个字符集中，任何一个字符的编码都不是另一个字符编码的前缀。约定左分支表示字符‘0’，右分支表示字符‘1’，则可以用从根结点到叶子结点的路径上的分支字符串作为该叶子结点字符的编码。如此得到的编码必是前缀编码。D 选项中，编码 110 是编码 1100 的前缀，故不符合前缀编码的定义。

6．先序序列为 a，b，c，d 的不同二叉树的个数是（　　）。[2015 年联考真题]

A．13

B．14

C．15

D．16

【答案】B 查看答案

【解析】二叉树的先序遍历定义为：若二叉树为空，则空操作；否则，访问根节点，然后先序遍历左子树，最后先序遍历右子树。本题中，结点 a 为二叉树的根节点，左右子树的先序遍历可能存在下面四种情况：①左子树为空，bcd 为右子树；②b 为左子树，cd 为右子树；③bc 为左子树，d 为右子树；④bcd 为左子树，右子树为空。然后将左右子树继续分解，如第①种情况的右子树先序遍历（bcd）可能有：a．左子树为空，右子树为 cd；b．左子树为 c，右子树为 d；c．左子树为 cd，右子树为空。按照这种方法继续分解左

右子树，直到不能再分解为止，可得第①和④种情况各包含 5 种不同情况，第②和③种情况各包含 2 种情况，因此总共有 14 种不同的二叉树。

7．下列选项给出的是从根分别到达两个叶节点路径上的权值序列，能属于同一棵哈夫曼树的是（　　）。[2015 年联考真题]

A．24，10，5 和 24，10，7

B．24，10，5 和 24，12，7

C．24，10，10 和 24，14，11

D．24，10，5 和 24，14，6

【答案】D 查看答案

【解析】哈夫曼树是带权路径长度最短的二叉树。由根节点出发到两个叶子节路径中，第二个被访问的两个结点的权值要么相等，要么和为根节点的权值，故 B 项错误。同理，通过第三个被访问的节点排除 A 项。C 项，由两条路径可推出三个叶子节点的权值分别是：3、10 和 11，而根据哈夫曼树的定义可知，权值为 3 的节点应该和权值为 10 的结点结合，故 C 项错误。D 项，反推出有四个叶子节点，权值分别为：5、5、6 和 8，满足哈夫曼树的条件。

8．算术表达式 a+b*（c+d / e）转为后缀表达式后为（　　）。

A．ab+cde / *　B．abcde / +*+　C．abcde / *++　D．abcde* / ++

【答案】B 查看答案

【解析】后缀表达式：在程序语言中，运算符位于两个操作数后面的表达式。

9．设森林 F 对应的二叉树为 B，它有 m 个结点，B 的根为 P，P 的右子树结点个数为 n，森林 F 中第一棵树的结点个数是（　　）。[南京理工大学 2000 研]

A．m-n　B．m-n-1　C．n+1　D．条件不足，无法确定

【答案】A 查看答案

【解析】森林转换成二叉树的原则：将第一棵树的根结点作为根结点，所有结点的第一个左孩子作为左孩子，下一个兄弟结点作为右孩子，其它树作为第一棵树的右孩子。所以森林 F 中第一棵树的结点个数是二叉树总的节点数减去根的右子树的节点数，即 m-n。

10．设森林 F 中有三棵树，第一、第二、第三棵树的结点个数分别为 M1、M2 和 M3。与森林 F 对应的二叉树根结点的右子树上的结点个数是（　　）。[北方交通大学 2001 研]

A．M1　B．M1+M2　C．M3　D．M2+M3

【答案】D 查看答案

【解析】森林转换成二叉树的原则：将第一棵树的根结点作为根结点，所有结点的第一个左孩子作为左孩子，下一个兄弟结点作为右孩子，其它树作为第一棵树的右孩子。所以森林 F 对应的二叉树根结点的右子树上的结点个数是除第一棵树外其他所有树的结点总数。即 M_2+M_3。

11．若二叉树采用二叉链表存储结构，要交换其所有分支结点左、右子树的位置，利用（）遍历方法最合适。

A．前序　B．中序　C．后序　D．按层次

【答案】C 查看答案

【解析】显然后序遍历比较合理。 正常的逻辑是：做好当前结点子树内部的交换，然后交换当前结点的左右子树。刚好符合后续遍历的算法逻辑，但是从实现上来说前序和按层次遍历都是可以的。

12．深度为 h 的满 m 叉树的第 k 层有（　　）个结点（$1\le k\le h$）。[北京航空航天大学 2000 研]

A.m^{k-1}　B．$m^{k}-1$　C．m^{h-1}　D．$m^{h}-1$

【答案】A 查看答案

【解析】满 m 叉树第 k 层必有 m^{k-1} 个结点。

13．一棵树高为 k 的完全二叉树至少有（　　）个结点。

A．2^k-1　B．$2^{k-1}-1$　C．2^{k-1}　D．2^k

【答案】C 查看答案

【解析】完全二叉树结点个数 n 满足 $2^{k-1}<=n<2^k$。

14．高度为 h（h>0）的满二叉树对应的森林由（　　）棵树构成。[北京交通大学 2004 研]

A．1　B．$\log 2^h$　C．h / 2　D．h

【答案】D 查看答案

【解析】满二叉树最右边的右结点有 h-1 个，所以其对应的森里有 h 棵树组成。

15．一棵有 n 个结点的二叉树，按层次从上到下，同一层从左到右顺序存储在一维数组 A[1..n]中，则二叉树中第 i 个结点（i 从 1 开始用上述方法编号）的右孩子在数组 A 中的位置是（　　）。[南京理工大学 2000 研]

A．A[2i]（2i≤n）　B．A[2i+1]（2i+1≤n）

C．A[i-2]　D．条件不充分，无法确定

【答案】D 查看答案

【解析】题目并未明确所给二叉树的形状，因此不能根据第 i 个结点在数组 A 中的存储位置确定其右孩子在数组 A 中的位置。

二、填空题

1．二叉树由__（1）__，__（2）__，__（3）__三个基本单元组成。

【答案】（1）根结点　（2）左子树　（3）右子树查看答案

2．深度为 H 的完全二叉树至少有__（1）__个结点；至多有__（2）__个结点；H 和结点总数 N 之间的关系是__（3）__。

【答案】（1）2^{H-1}　（2）2^H-1　（3）$H=ë\log_2 N⌋+1$ 查看答案

3．设只含根结点的二叉树的高度为 0，则高度为 k 的二叉树的最大结点数为_____，最小结点数为_____。

【答案】（1）$2^{K+1}-1$　（2）k+1 查看答案

【解析】当二叉树是满二叉树时，结点数最多。

4．一个深度为 k 的具有最少结点数的完全二叉树按层次（同层次从左到右），用自然数依次对结点编号，则编号最小的叶子的序号是__（1）__；编号是 i 的结点所在的层次号是__（2）__（根所在的层次号规定为 1 层）。[南京理工大学 2001 研]

【答案】（1）$2^{k-2}+1$（第 k 层 1 个结点，总结点个数是 2^{k-1}，其双亲是 $2^{k-1}/2=2^{k-2}$）查看答案

（2）$ë\log_2 i⌋+1$

5.n（n>1）个结点的各棵树中，其中，深度最小的那棵树的深度是__（1）__，它共有__（2）__个叶结点和__（3）__个非叶结点；深度最大的那棵树的深度是__（4）__，它共有__（5）__个叶结点和__（6）__个非叶结点。[山东大学 2001 研]

【答案】（1）2　（2）n-1　（3）1　（4）n　（5）1　（6）n-16 查看答案

6．二叉树的前序序列和中序序列相同的条件是_____。[合肥工业大学 2000 研]

【答案】空树或任何结点至多只有右子女的二叉树查看答案

【解析】前序遍历的顺序为根左右，中序遍历的顺序为左根右，因此若中序遍历和前序遍历序列相同，则任何结点都没有左子树。

7．将二叉树 bt 中每一个结点的左右子树互换的 C 语言算法如下，其中 ADDQ（Q，bt）、DELQ（Q）、EMPTY（Q）分别为进队、出队和判别队列是否为空的函数，请填写算法中空白之处，完成其功能。[北京科技大学 2000 研]

```
typedef struct node
   {int data ; struct node *lchild, *rchild; }btnode;
void  EXCHANGE(btnode *bt)
        {btnode *p, *q;
         if (bt){ADDQ(Q,bt);
                 while(!EMPTY(Q))
                  {p=DELQ(Q);
        if(p->lchild)__(1)__;
        if(p->rchild)__(2)__;
        q=__(3)__;
        p->rchild=__(4)__;
        __(5)__=q;
        }   }   }
```

【答案】（1）ADDQ（Q，P->lchild）　（2）ADDQ（Q，P->rchild）查看答案

（3）P->rchild　（4）P->lchild　（5）P->lchild

【解析】本题用到队列的知识，首先判断树是否为空，如果不为空，则树根进队，然后取出队列最先进来的元素，判断这个元素的左子树和右子树是否存在，如果存在，则将其存在的子结点进队，同时交换左右子树的位置。同理，继续判断队列是否为空，不为空，继续出队判断，持续这一操作，完成二叉树 bt 中每个结点的左右子树互换。

8．下面的算法在中序线索树中找由指针所指结点的后继并由指针指向该后继结点，试补充完整（线索树的结点有五个域 data，lchild，rchild，左、右标志域 ltag、rtag，并规定标志 0 指向孩子，1 指向线索）。

```
PROC  inorder_next(p);
  __(1)__;
  IF p^.rtag=0 THEN  WHILE__(2)__DO  q:= __(3)__;
  return(q)
ENDP;
```

【答案】若 p^．rtag=1，则 p^．rchild 为后继，否则 p 的后继是 P 的右子树中最左下的结点。查看答案

（1）q=p^．rchild　（2）q^．ltag=0　（3）q^．lchild

【解析】因为是中序遍历，如果 p 结点没有右子树，则 p 结点的后继直接是 p 的右指针所指向的结点。如果 p 结点有右子树，则 p 结点的后继就是 p 的右子树中最左下的结点。

三、综合题

1．证明任一结点个数为 n 的二叉树的高度至少为 0（logn）。[浙江大学 2000 研]

证明：最低高度二叉树的特点是，除最下层结点个数不满外，其余各层的结点数都应达到各层的最大值。设 n 个结点的二叉树的最低高度是 h，则 n 应满足 $2^{h-1} \le n \le 2^h$。解此不等式，并考虑 h 是整数，则有 $h=\ddot{e}\log_2 n\rfloor+1$，即任一结点个数为 n 的二叉树的高度至少为 $O(\log_2 n)$。

2．已知 A[1．．N]是一棵顺序存储的完全二叉树，如何求出 A[i]和 A[j]的最近的共同祖先？[中国人民大学 2001 研]

答：根据顺序存储的完全二叉树的性质，编号为 i 的结点的双亲的编号是 ëi / 2」，故 A[i]和 A[j]的最近公共祖先可如下求出：

while（i / 2 !=j / 2）

if（i>j）i=i / 2；else j=j / 2；

退出 while 后，若 i / 2=0，则最近公共祖先为根结点，否则最近公共祖先是 i / 2。

3．对于具有 n 个叶结点且所有非叶结点都有左、右孩子的二叉树。

（1）试问这种二叉树的结点总数是多少?

（2）试证明 $\sum_{i=1}^{n} 2^{-(l_i-1)}=1$。其中：$l_i$ 表示第 i 个叶结点所在的层号（设根结点所在层号为 1）。

答：（1）根据二叉树中度为 2 的结点个数等于叶结点个数减 1 的性质，故具有 n 个叶结点且非叶子结点均有左子树的二叉树的结点数是 2n-1。

（2）证明：当 i=1 时，$2^{-(1-1)}=2^0=1$，公式成立。设当 i=n-1 时公式成立，证明当 i=n 时公式仍成立。

设某叶结点的层号为 t，当将该结点变为内部结点，从而再增加两个叶结点时，这两个叶结点的层号都是 t+1，对于公式的变化，是减少了一个原来的叶结点，增加了两个新叶结点，反映到公式中，因为 $2^{-(t-1)}=2^{-(t+1-1)}+2^{-(t+1-1)}$，所以结果不变，这就证明当 i=n 时公式仍成立。证毕。

4．已知一棵二叉树的对称序和后序序列如下：

对称序：GLDHBEIACJFK 后序：LGHDIEBJKFCA

（1）（2 分）画出这棵二叉树；

（2）（2 分）转换为对应的森林；

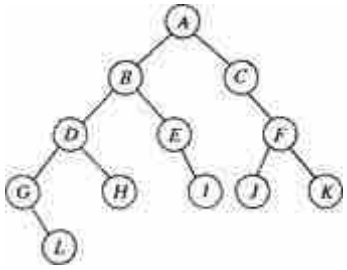
（3）（4 分）画出该森林的带左链的前序表示法：

$$ltag=\begin{cases}1, & \text{无左子女}\\ 0, & \text{有左子女}\end{cases}$$

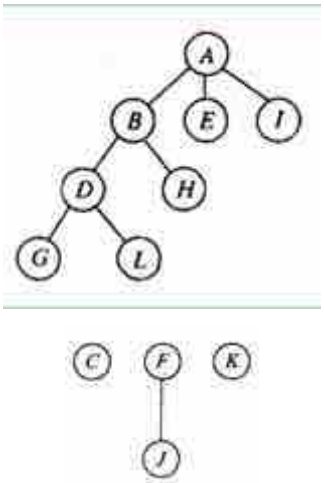
（4）（4 分）画出该森林带度数的后序表示法；

（5）（4 分）在带度数的后序表示法中，不包含指针，但仍能完全反映树的结构。写出以结点 x 为根的子树在后序序列中的前驱的求法（用语言叙述，不用写算法）。

答：（1）



（2）



（3）在前序序列中，任何结点的子树的所有结点都直接跟在该结点之后；每棵子树的所有结点都聚集在一起，中间不会插入其他结点；任何分支结点后面跟的是它的第一个子女。故结点可采用如下结构：（Itag，data，rlink）。其中，ltag 为左标记，ltag=1 表示该结点无子女或二叉树无左子女，ltag=0 表示其后存储其第一个子女或二叉树的左子女；data 是结点的数据；rlink 指向下一兄弟或二叉树的右子女。用左标记代替左指针，占用存储单元少，不会丢失信息。

本题森林的带右链的前序表示法为（注："下标"行不是结点成分）：

下标	1	2	3	4	5	6	7	8	9	10	11	12
rlink	9	7	6	5			8		10	12		

data	A	B	D	G	L	H	E	I	C	F	j	K
ltag	0	0	0	1	1	1	1	1	1	0	1	1

（4）在后序序列中，任何一棵子树的结点聚集在一起，根结点在最后。结点按后序顺序存储在一片连续的存储单元中，结点结构可描述为（data，degree）。

（5）该森林带度数的后序表示法如下：后序序列中任何一棵子树的结点个数减边数为 1。设某结点的度 degree=m，即该结点有 m 个子女，最右边的子女就是该结点的前驱。在后序表示法中最后的第 2 个子女是最右子女为根的子树的前驱，以结点 x 为根的子树的前驱求法如下：令 q=0，从 x 开始从后往前走，每经过一个结点 z，作 q=q+1-degree（z）；到 q=1 时，再往前走一个结点就是以结点 x 为根的子树在后序序列中的前驱。

下标	1	2	3	4	5	6	7	8	9	10	11	12
data	G	L	D	H	B	E	I	A	C	J	F	K
degree	0	0	2	0	2	0	0	3	0	0	1	0

5．假设一个仅包含二元运算符的算术表达式以链表形式存储在二叉树 BT 中，写出计算该算术表达式值的算法。[东北大学 2000 研]

答：

```
typedef struct node
   {ElemType  data;
    char optr;  //只取'+', '-', '*','/'
    struct node *lchild,*rchild }BiNode,*BiTree;
```

```
float PostEval(BiTree bt)        // 以后序遍历算法求以二叉树表示的算术表达式的值
  {float lv,rv;
   if(bt!=null)
     {lv=PostEval(bt->lchild); // 求左子树表示的子表达式的值
      rv=PostEval(bt->rchild); // 求右子树表示的子表达式的值
      switch(bt->optr)
       {case '+': value=lv+rv; break;
        case '-': value=lv-rv;break;
        case '*': value=lv*rv;break;
        case '/': value=lv/rv;
       } }
return(value); }
```

6．编写一算法，利用叶结点中的空指针域将所有叶结点链接为一个带有头结点的双链表，算法返回头结点的地址

答：

```
BiTree head=null,pre; //全局变量链表头指针head,pre
void CreatLeafList(BiTree bt) //将BiTree树中的所有叶结点链成带头结点的双链表
   {if(bt)            //若bt不空
    {CreatLeafList(bt->lchild);                  //中序遍历左子树
     if(bt->lchild==null && bt->rchild==null) //叶结点
       if(head==null)//第一个叶结点
         {head=(BiNode*)malloc(sizeof(BiNode));//生成头结点
          head->lchild=null; head->rchild=bt;  //头结点的左链空,右链指向第一个结点
          bt->lchild=head; pre=bt;//第一个叶结点左链指向头结点,pre指向当前叶结点
         }
       else{pre->rchild=bt; bt->lchild=pre; pre=bt;} //当前叶结点链入双链表
      CreatLeafList(bt->rchild);  //中序遍历右子树
      pre->rchild=null;     //最后一个叶结点的右链置空(链表结束标记)
   }//if(bt)  }//结束CreatLeafList;
```

第 7 章　图

7.1　复习笔记

一、图的定义和术语

1．定义

图是一种数据结构，它的形式化定义为

Graph=（V，R）

数据对象 V：V 是具有相同特性的数据元素的集合，称为顶点集。

数据关系 R：R={VR}

VR={<v,w>|v,w∈V 且 P（v,w），<v,w>表示从 v 到 w 的弧，谓词 P（v,w）定义了弧<v,w>的意义或信息}

2．基本术语

（1）图中的数据元素称做顶点（vertex），V 是顶点的有穷非空集合；VR 是两个顶点之间关系的集合。

（2）若<x，y>∈VR，则<x，y>表示从 x 到 y 的一条弧（arc），且称 x 为弧尾（tail）或初始点（initial node），称 y 为弧头（head）或终端点（terminal node），此时的图称为有向图（digraph）。

（3）若（x，y）∈VR 必有<y，x>∈VR，即 VR 是对称的，则以无序对（x，y）代替这两个有序对，表示 x 和 y 之间的一条边（edge），此时的图称为无向图（undigraph）。这里用 n 表示图中顶点数目，用 e 表示边或弧的数目。

3．完全图

我们用 n 表示图中顶点数目，用 e 表示边或弧的数目。有 n（n-1）/2 条边的无向图称为无向完全图（Completed graph），具有 n（n-1）条弧的有向图称为有向完全图。

4．稀疏图和稠密图

有很少条边或弧（如 e<nlogn）的图称为稀疏图（sparse graph），反之称为稠密图（dense graph）。

5．权和网

有时图的边或弧具有与它相关的数，这种与图的边或弧相关的数称作权（weight）。这些权可以表示从一个顶点到另一个顶点的距离或耗费。这种带权的图通常称为网（net-work）。

6．子图、度、出度和入度

假设有两个图 G=（V，{E}）和 G′=（V′，{E′}），如果 V′⊆V 且 E′⊆E，则称 G′为 G 的子图（subgraph）。

对于无向图 G=（V，{E}），如果边（v，v′）∈E，则称顶点 v 和 v′互为邻接点（adjacent），即 v 和 v′相邻接。边（v，v′）依附（incident）于顶点 v 和 v′，或者说边（v，v′）和顶点 v 和 v′相关联。

顶点 v 的度（degree）是和 v 相关联的边的数目，记为 TD（v）。对于有向图 G=（V，{A}），如果弧（v，v′）∈A，则称顶点 v 邻接到顶点 v′，顶点 v′邻接自顶点 v。弧（v，v′）和顶点 v，v′相关联。以顶点 v 为头的弧的数目称为 v 的入度（Indegree），记为 ID（v）；以 v 为尾的弧的数目称为 v 的出度（outdegree），记为 OD（v）；顶点 v 的度为 TD（v）=ID（v）+OD（v）。

7．路径、路径长度、简单路径、回路和简单回路

无向图 G=（v_1，{E}）中从顶点 v 到顶点 v′的路径（path）是一个顶点序列（v=v_{i0}，v_{i1}，…，v_{im}=v′），其中（v_{ij-1}，v_{ij}）∈E，1≤j≤m。如果 G 是有向图，则路径也是有向的，顶点序列应满足（v_{ij-1}，v_{ij}>E，1≤j≤m。路径的长度是路径上的边或弧的数目。第一个顶点和最后一个顶点相同的路径称为回路或环（cycle）。序列中顶点不重复出现的路径称为简单路径。除了第一个顶点和最后一个顶点之外，其余顶点不重复出现的回路，称为简单回路或简单环。

8．连通图和连通分量、强连通图和强连通分量

在无向图 G 中，如果顶点 v 到顶点 v′有路径，则称 v 和 v′是连通的。如果对于图中任意两个顶点 v_i、v_j∈V，v_i 和 v_j 都是连通的，则称 G 是连通图（connected graph）。连通分量（connected component）指的是无向图中的极大连通子图。在有向图 G 中，如果对于每一对 v_i，v_j∈V，v_i≠v_j，从 v_i 到 v_j 和从 v_j 到 v_i 都存在路径，则称 G 是强连通图。有向图中的极大强连通子图称作有向图的强连通分量。

9．生成树、生成森林、有向树和森林

一个连通图的生成树是一个极小连通子图，它含有图中全部顶点，但只有足以构成一棵树的 n-l 条边。如果在一颗生成树上添加一条边，必定构成一个环。一颗有 n 个顶点的生成树有且仅有 n-1 条边。如果一个图有 n 个顶点和小于 n-1 条边，则是非连通图。但是，有 n-1 条边的图不一定是生成树。

如果一个有向图恰有一个顶点的入度为 0，其余顶点的入度均为 l，则是一棵有向树。一个有向图的生成森林由若干棵有向树组成，含有图中全部顶点，但只有足以构成若干棵不相交的有向树的弧。若连通图中有 n 个顶点，m 个连通分量，则生成森林中有 n-m 条边。

10．图的基本操作

CreateGraph（&G，V，VR）

初始条件：V 是图的顶点集，VR 是图中弧的集合

操作结果：按 V 和 VR 的定义构造图 G。

DestroyGraph（&G）

初始条件：图 G 存在

操纵结果：销毁图 G。

LocateVex（G，u）

初始条件：图 G 存在，u 和 G 中顶点有相同的特征。

操作结果：若 G 总存在顶点 u，则返回该顶点在图中的位置；否则返回其他信息。

GetVex（G，v）

初始条件：图 G 存在，v 是 G 中某个顶点。

操作结果：返回 v 的值。

PutVex（&G，v，value）

初始条件：图 G 存在，v 是 G 种某个顶点。

操作结果：对 v 赋值 value。

FirstAdjVex（G，v）

初始条件：图 G 存在，v 是 G 种某个顶点。

操作结果：返回 v 的第一个邻接顶点。若顶点在 G 中没有邻接顶点，则返回“空”。

NextAdjVex（G，v，w）

初始条件：图 G 存在，v 是 G 中的某个顶点，w 是 v 的邻接顶点。

操作结果：返回 v 的（相对于 w 的）下一个邻接顶点。若 w 是 v 的最后一个邻接点，则返回“空”。

InsertVex（&G，v）

初始条件：图 G 存在，v 和图中的顶点有相同特征。

操作结果：在图 G 中增添新顶点 v。

DeleteVex（&G，v）

初始条件：图 G 存在，v 是 G 的某个顶点。

操作结果：删除 G 中顶点 v 及相关的弧。

InsertArc（&G，v，w）

初始条件：图 G 存在，v 和 w 是 G 中的两个顶点。

操作结果：在 G 中增添<v，w>，若 G 是无向的，则还增添对称弧<w，v>

DeleteArc（&G，v，w）

初始条件：图 G 存在，v 和 w 是 G 中的两个顶点。

操作结果：在 G 中删除弧<v，w>，若 G 是无向的，则还删除对称弧<w，v>

DFSTraverse（G，Visit（））

初始条件：图 G 存在，Visit 是顶点的应用函数。

操作结果：对图进行深度优先遍历。

BFSTraverse（G，Visit（））

初始条件：图 G 存在，Visit 是顶点的应用函数。

操作结果：对图进行广度优先遍历。

二、图的存储结构

1．数组表示法

（1）定义

用两个数组分别存放数据元素（顶点）的信息和数据元素之间的关系（边或弧）的信息。

以二维数组表示 n 个顶点的图时，需存放 n 个顶点和 n^2 个弧信息的存储量，如图 7-1 所示。

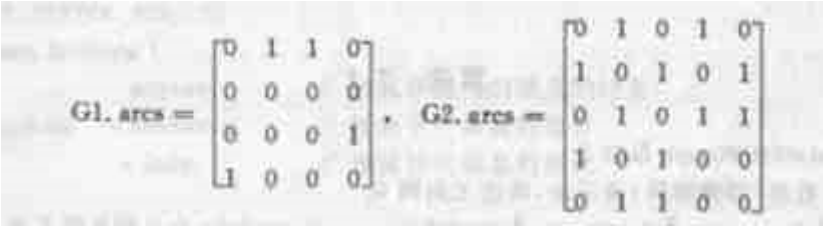


图 7-1 图的邻接矩阵

（2）度的表示

借助于邻接矩阵容易求得各个顶点的度。

①对于无向图，顶点 v_i 的度是邻接矩阵中第 i 行（或第 i 列）的元素之和，即

$$TD(v_i) = \sum_{j=0}^{n-1} A[i][j] \quad (n = MAX_VERTEX_NUM)$$

②对于有向图，第 i 行的元素之和为顶点 v_i 的出度 OD（v_i），第 j 列的元素之和为顶点 v_j 的入度 ID（v_j）。

（3）网的邻接矩阵

网的邻接矩阵可定义为：

$$A[i,j] = \begin{cases} w_{ij} & \text{若} \langle v_i, v_j \rangle \text{或} (v_i, v_j) \in VR \\ \infty & \text{反之} \end{cases},$$

邻接矩阵如图 7-2 所示。

$$\begin{bmatrix} \infty & 5 & \infty & 7 & \infty & \infty \\ \infty & \infty & 4 & \infty & \infty & \infty \\ 8 & \infty & \infty & \infty & \infty & 9 \\ \infty & \infty & 5 & \infty & \infty & 6 \\ \infty & \infty & \infty & 5 & \infty & \infty \\ 3 & \infty & \infty & \infty & 1 & \infty \end{bmatrix}$$

图 7-2 网的邻接矩阵

2．邻接表

（1）定义

邻接表（adjacency list）是图的一种链式存储结构。在邻接表中，对图中每个顶点建立一个单链表，第 i 个单链表中的结点表示依附于点 V_i 的边。表中的每个结点由三个域组成，其中邻接点域（adjvex）指示与顶点 V_i 邻接的点在图中的位置；链域（nextarc）指示下一条边或弧的结点；数据域（info）存储和边或弧相关的信息，如权值等。表结点和头结点如图 7-3 所示。



图 7-3 表结点和头结点

（2）邻接表的特点

若无向图中有 n 个顶点和 e 条边，则它的邻接表需 n 个头结点和 2e 个表结点。显然，在边稀疏（$e \ll \frac{n(n-1)}{2}$）的情况下，用邻接表表示图比邻接矩阵节省存储空间。

3．十字链表

（1）定义

十字链表（orthogonal list）是有向图的另一种链式存储结构。十字链表中，对应于有向图中的每一条弧有一个结点，每个顶点也有一个结点。这些结点的结构如图 7-4 所示。



图 7-4 十字链表的弧结点和顶点结点

（2）有向图的十字链表

若将有向图的邻接矩阵看成是稀疏矩阵的话，则十字链表也可以看成是邻接矩阵的链表存储结构，如图 7-5 所示。

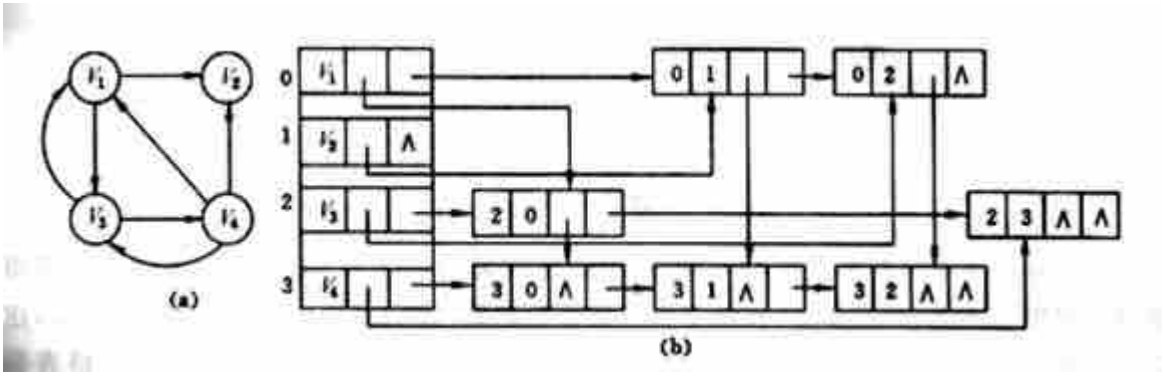


图 7-5 有向图的十字链表

建立十字链表和邻接表的时间复杂度是相同的。

4．邻接多重表

（1）定义

邻接多重表（adjacency multilist）是无向图的另一种链式存储结构。在邻接多重表中，每一条边用一个结点表示，它由如下所示的六个域组成：



每一个顶点也用一个结点表示，它由如下所示的两个域组成：



（2）邻接多重表

如图 7-6 所示为邻接多重表的范例。

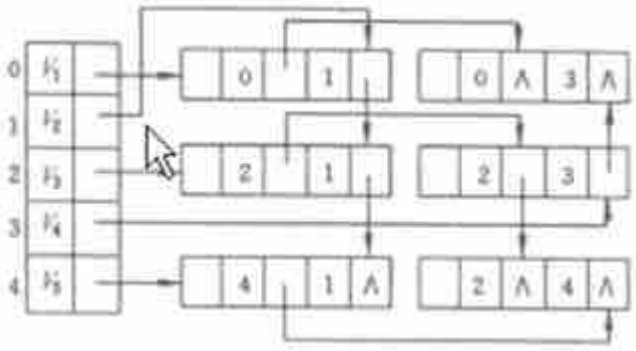


图 7-6 邻接多重表

对无向图而言，其邻接多重表和邻接表的差别，仅仅在于同一条边在邻接表中用两个结点表示，而在邻接多重表中只有一个结点。因此，除了在边结点中增加一个标志域外，邻接多重表所需的存储量和邻接表相同，各种基本操作的实现亦和邻接表相似。

三、图的遍历

从图中某一顶点出发访遍图中其余顶点，且使每一个顶点仅被访问一次。这一过程称作图的遍历（traversing graph）。

1．深度优先搜索

（1）定义

深度优先搜索（depth_first search）遍历类似于树的先根遍历，是树的先根遍历的推广。

（2）遍历过程

假设初始状态是图中所有顶点未曾被访问，则深度优先搜索可从图中某个顶点 v_0 出发，访问此顶点，然后依次从 v_0 的未被访问的邻接点出发深度优先遍历图，直至图中所有和 v_0 有路径相通的顶点都被访问到。若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点。重复上述过程，直至图中所有顶点都被访问到为止。

（3）时间复杂度

当用二维数组表示邻接矩阵作图的存储结构时，查找每个顶点的邻接点所需时间为 $O(n^2)$，其中 n 为图中顶点数。而当以邻接表作图的存储结构时，查找邻接点所需时间为 O（e），其中 e 为无向图中边的数或有向图中弧的数。由此，当以邻接表作存储结构时，深度优先搜索遍历图的时间复杂度为 O（n+e）。

2．广度优先搜索

（1）定义

广度优先搜索（breadth_first search）遍历类似于树的层次遍历的过程，是树的层次遍历的推广。

（2）遍历过程

假设从图中某顶点 v_0 出发，在访问了 v_0 之后依次访问 v_0 的各个未曾访问过的邻接点，然后分别从这些邻接点出发广度优先搜索遍历图，直至图中所有已被访问的顶点的邻接点都被访问到。若此时图中尚有顶点未被访问，则另选图中一个未曾被访问的顶点作起始点。重复上述过程，直至图中所有顶点都被访问到为止。

（3）时间复杂度

遍历图的过程实质上是通过边或弧找邻接点的过程，因此广度优先搜索遍历图的时间复杂度和深度优先搜索遍历相同，即使用二维数组表示邻接矩阵存储图时，时间复杂度为

O（n^2）；使用邻接表存储图时，时间复杂度为 O（n+e），两者不同之处仅仅在于对顶点访问的顺序不同。

四、图的连通性问题

1．无向图的连通分量和生成树

（1）连通图

设 E（G）为连通图 G 中所有边的集合，则从图中任一顶点出发遍历图时，必定将 E（G）分成两个集合 T（G）和 B（G），其中 T（G）是遍历图过程中历经的边的集合，B（G）是剩余的边的集合。显然，T（G）和图 G 中所有顶点一起构成连通图 G 的极小连通子图，即它是连通图的一棵生成树。

（2）非连通图

对于非连通图，每个连通分量中的顶点集和遍历图过程中历经的边一起构成若干棵生成树，这些连通分量的生成树组成非连通图的生成森林。

2．有向图的强连通分量

求强连通分量的步骤如下：

（1）在有向图 G 上，从某个顶点出发沿以该顶点为尾的弧进行深度优先搜索遍历，并按其所有邻接点的搜索都完成的顺序将顶点排列起来。

（2）从最后完成搜索的顶点出发，沿着以该顶点为头的弧作逆向的深度优先搜索遍历，若此次遍历不能访问到有向图中所有顶点，则从余下的顶点中最后完成搜索的那个顶点出发，继续作逆向的深度优先搜索遍历，依次类推，直至有向图中所有顶点都被访问到为止。

3．最小生成树

（1）定义

对于 n 个顶点的连通网可以建立许多不同的生成树，每一棵生成树都可以是一个通信网。其中总的耗费最少树的称作最小代价生成树（minimum cost spanning tree）（简称为最小生成树）。一棵生成树的代价就是树上各边的代价之和。

构造最小生成树有多种算法。其中多数算法利用了最小生成树的 MST 性质：假设 N=（V，{E}）是一个连通网，U 是顶点集 V 的一个非空子集。若（u，V）是一条具有最小权值（代价）的边，其中 u∈U，v∈V-u，则必存在一棵包含边（u，V）的最小生成树。

（2）普里姆算法。

假设 N=（V，{E}）是连通网，TE 是 N 上最小生成树中边的集合。算法从 U={u_0}（u_0∈V），TE=ψ 开始，重复执行下述操作：在所有 u∈U，v∈V-U 的边（u，v）∈E 中找一条代价最小的边（u_0，v_0）并入集合 TE，同时 v_0 并入 U，直至 U=V 为止。此时 TE 中必有 n-1 条边，则 T=（V，{TE}）为 N 的最小生成树。

该算法的时间复杂度为 O（n^2），即只与网中顶点数 n 有关，与网中的边数无关，因此适用于求边稠密的网的最小生成树。

（3）克鲁斯卡尔算法

假设连通网 N=（V，{E}），则令最小生成树的初始状态为只有 n 个顶点而无边的非连通图 T=（V，{ψ}），图中每个顶点自成一个连通分量。在 E 中选择代价最小的边，若该边依附的顶点落在 T 中不同的连通分量上，则将此边加入到 T 中，否则舍去此边而选择下一条代价最小的边。依次类推，直至 T 中所有顶点都在同一连通分量上为止。

该算法的时间复杂度为 O（eloge）（e 为网中边的数目），因此它相对于普里姆算法而言，适合于求边稀疏的网的最小生成树。

4．关节点和重连通分量

假若在删去顶点 v 以及与 v 相关联的各边之后，将图的一个连通分量分割成两个或两个以上的连通分量，则称顶点 v 为该图的一个关节点（articulation point）。一个没有关节点的

连通图称为是重连通图（biconnected graph）。由深度优先生成树可得出两类关节点的特性：

（1）若生成树的根有两棵或两棵以上的子树，则此根顶点必为关节点。
（2）若生成树中某个非叶子顶点 v，其某棵子树的根和子树中的其它结点均没有指向 v 的祖先的回边，则 v 为关节点。

五、有向无环图及其应用

1．有向无环图

一个无环的有向图称做有向无环图（directed acycline graph），简称 DAG 图。

2．拓扑排序

由某个集合上的一个偏序得到该集合上的一个全序，这个操作称之为拓扑排序。

3.AOV-网

（1）定义

用顶点表示活动，用弧表示活动间的优先关系的有向图称为顶点表示活动的网（activity on vertex network），简称 AOV-网。

（2）检测是否存在有向环

在 AOV-网中，不应该出现有向环。检测的办法是对有向图构造其顶点的拓扑有序序列，若网中所有顶点都在它的拓扑有序序列中，则该 AOV-网中必定不存在环。

（3）拓扑排序的步骤

①在有向图中选一个没有前驱的顶点且输出之。
②从图中删除该顶点和所有以它为尾的弧。
③重复上述两步，直至全部顶点均已输出，或者当前图中不存在无前驱的顶点为止。后一种情况则说明有向图中存在环。

4．关键路径

（1）AOE-网

AOE-网（activity on edge）即边表示活动的网。AOE-网是一个带权的有向无环图，其中，顶点表示事件（event），弧表示活动，权表示活动持续的时间。通常，AOE-网可用来估算工程的完成时间。

由于整个工程只有一个开始点和一个完成点，故在正常的情况（无环）下，网中只有一个入度为零的点（称作源点）和一个出度为零的点（称作汇点）。

（2）关键路径

完成工程的最短时间是从开始点到完成点的最长路径的长度。路径长度最长的路径称为关键路径（critical path）。

（3）求关键路径的算法

①输入 e 条弧（j，k），建立 AOE-网的存储结构。
②从源点 v_j 按拓扑有序求其余各顶点的最早发生时间 ve[i]（$2 \le i \le n$）。如果得到的拓扑有序序列中顶点个数小于网中顶点数 n，则说明网中存在环，不能求关键路径，算法终止；否则执行步骤③。
③从汇点 v_n 出发，令 vt[n]=ve[n]，按逆拓扑有序求其余各顶点的最迟发生时间 vz[i]（$n-1 \ge i \ge 1$）。
④根据各顶点的 ve 和 vz 值，求每条弧 S 的最早开始时间 e（s）和最迟开始时间 z（s）。若某条弧满足条件 e（s）=z（s），则为关键活动。

六、最短路径

1．某个顶点到其余各顶点的最短路径

（1）单源点最短路径问题

给定带权有向图 G 和源点 v，求从 v 到 G 中其余各顶点的最短路径。
首先，引进一个辅助向量 D，它的每个分量 D[i]表示当前所找到的从始点 v 到每个终点 v_i 的最短路径的长度。它的初态为：若从 v 到 v_i 有弧，则 D[i]为弧上的权值；否则置 D[i]为∞。显然，长度为

$$D[j]=Min\{D[i]|v_i\in V\}$$

路径就是从 v 出发的长度最短的一条最短路径。此路径为（v，v_j）。
（2）迪杰斯特拉算法
①假设用带权的邻接矩阵 arcs 来表示带权有向图，arcs[i，j]表示弧$<v_i,v_j>$上的权值。若$<v_i，v_j>$不存在，则置 arcs[i，j]为∞（在计算机上可用允许的最大值代替）。S 为已找到从 v 出发的最短路径的终点的集合，它的初始状态为空集，那么，从 v 出发到图上其余各顶点（终点）v_i 可能达到的最短路径长度的初值为：

$$D[i]=arcs[Locate\ Vex（G，v），i]，v_i\in V$$

②选择 v_i，使得

$$D[j]=Min\{D[i]，v_i\in V-S\}$$

v_j 就是当前求得的一条从 v 出发的最短路径的终点。令

$$S=S\cup\{j\}$$

③修改从 v 出发到集合 V-S 上任一顶点 v_k 可达的最短路径长度。如果

$$D[j]+arcs[j，k]<D[k]$$

则修改 D[k]为

$$D[k]=D[j]+arcs[j，k]$$

④重复操作②、③共 n-1 次，由此求得从 v 到图上其余各顶点的最短路径是依路径长度递增的序列。
2．每一对顶点之间的最短路径
解决这个问题的一个办法是：每次以一个顶点为源点，重复执行迪杰斯特拉算法 n 次。这样，便可求得每一对顶点之间的最短路径，时间复杂度为 O（n^3）。
（1）弗洛伊德算法
弗洛伊德算法的时间复杂度是 O（n^3），它仍从图的带权邻接矩阵 cost 出发，其基本思想是：
假设求从顶点 v_i 到 v_j 的最短路径。如果从 v_i 到 v_j 有弧，则从 v_i 到 v_j 存在一条长度为 arcs[i，j]的路径，该路径不一定是最短路径，尚需进行 n 次试探。首先考虑路径（v_i，v_0，v_j）是否存在（即判别弧（v_i，v_0）和（v_1，v_j）是否存在）。如果存在，则比较（v_i，v_j）和（v_i,v_0，v_j）的路径长度取长度较短者为从 v_i 到 v_j 的中间顶点的序号不大于 0 的最短路径。假如在路径上再增加一个顶点 v_1，也就是说，如果（v_i，…，v_1）和（v_1，…，v_j）分别是当前找到的中间顶点的序号不大于 0 的最短路径，那么（v_i，…，v_1，…，v_j）就有可能是从 v_i 到 v_j 的中间顶点的序号不大于 1 的最短路径。将它和已经得到的从 v_i 到 v_j 中间顶点序号不大于 0 的最短路径相比较，从中选出中间顶点的序号不大于 1 的最短路径之后，再增加一个顶点 v_2，继续进行试探。依次类推，在一般情况下，若（v_i，…，v_k）和（v_k，…，v_j）分别是从 v_i 到 v_k 和从 v_k 到 v_j 的中间顶点的序号不大于 k-1 的最短路径，则将（v_j，…，v_k，…，v_j）和已经得到的从 v_i 到 v_j 且中间顶点序号不大于 k-l 的最短路径相比较，其长度较短者便是从 v_i 到 v_j 的中间顶点的序号不大于 k 的最短路径。这样，在经过 n 次比较后，最后求得的必是从 v_i 到 v_j 的最短路径。按此方法，可以同时求得各对顶点间的最短路径。

7.2 强化习题详解

【基础知识题】

1．已知如图 7-1 所示的有向图，请给出该图的

（1）每个顶点的入 / 出度；

（2）邻接矩阵；

（3）邻接表；

（4）逆邻接表；

（5）强连通分量。

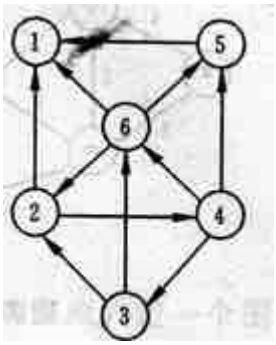


图 7-1

答：

（1）

顶点	1	2	3	4	5	6
入度	3	2	1	1	2	2
出度	0	2	2	3	1	3

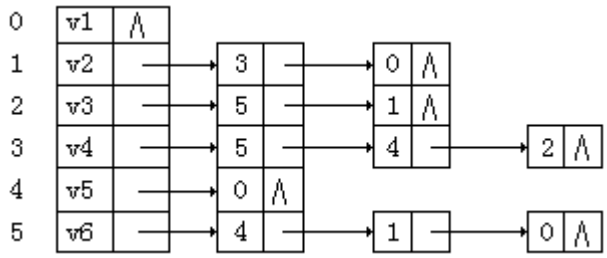
（2）

邻接矩阵

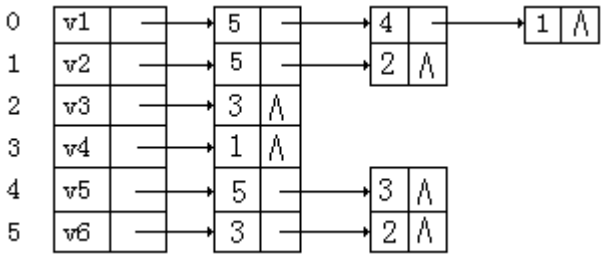
$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \end{bmatrix}$$

（3）

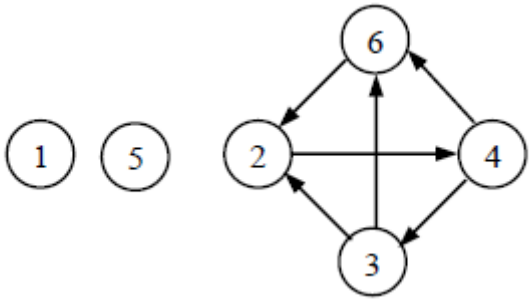
邻接表



（4）逆邻接表



（5）该有向图有 3 个强连通分量。



2．已知有向图的邻接矩阵为 $A_{n\times n}$，试问每一个 $A^{(k)}_{n\times n}$（k=1,2，…,n）各具有何种实际含义？

答：

k=1 时，$A^{(1)}_{n\times n}$ 说明了各结点之间长度为 1 的相互连通关系；

k=2 时，$A^{(2)}_{n\times n}$ 说明了结点之间按路径长度为 2 的相互连通关系；

...

由此递推，可知 $A^{(k)}_{n\times n}$ 表示结点之间长度为 k 的相互连通关系。

3．画出如图 7-2 所示的无向图的邻接多重表，使得其中每个无向边结点中第一个顶点号小于第二个顶点号，且每个顶点的各邻接边的链接顺序，为它所邻接到的顶点序号由小到大的顺序。列出深度优先和广度优先搜索遍历该图所得顶点序列和边的序列。

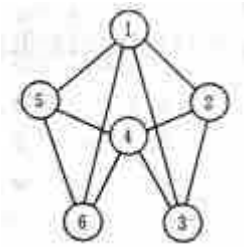
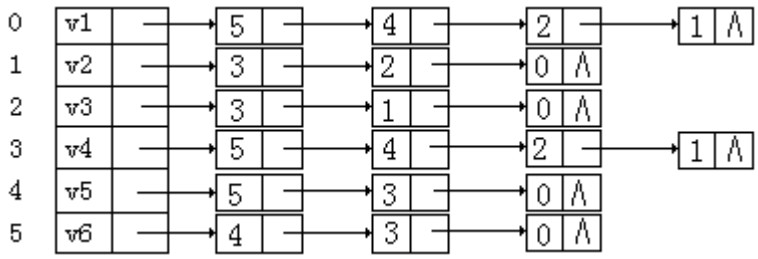


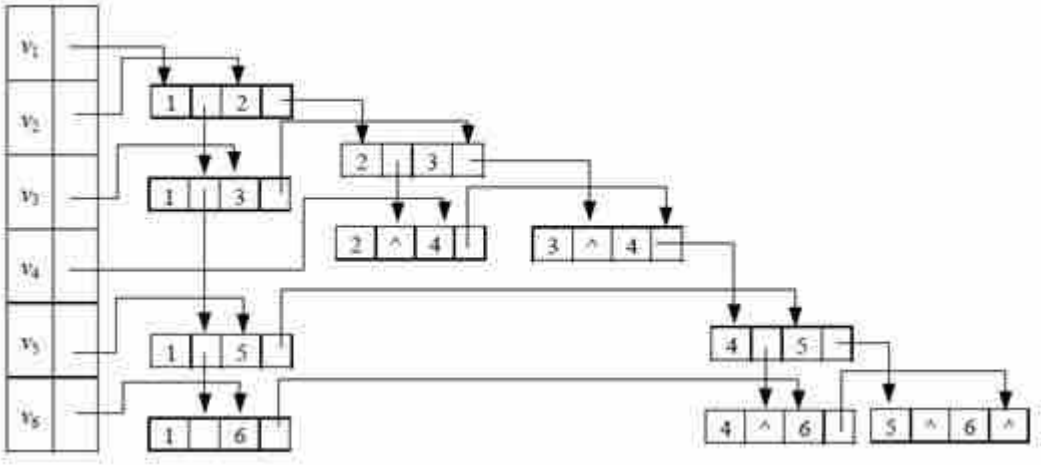
图 7-2

答：

（1）邻接表：



邻接多重表：



（2）深度优先遍历得到的顶点序列和边序列。

顶点序列：1，2，3，4，5，6

边序列：（1，2）（2，3）（3，4）（4，5）（5，6）

（3）广度优先遍历得到的顶点序列和边序列。

顶点序列：1，2，3，5，6，4

边序列：（1，2）（1，3）（1，5）（1，6）（2，4）

4．对如图 7-3 所示的无向图，画出其广度优先生成森林。

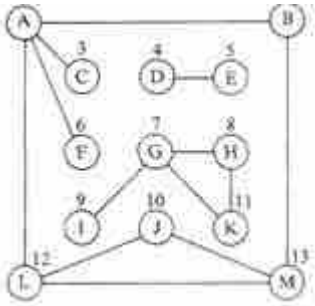
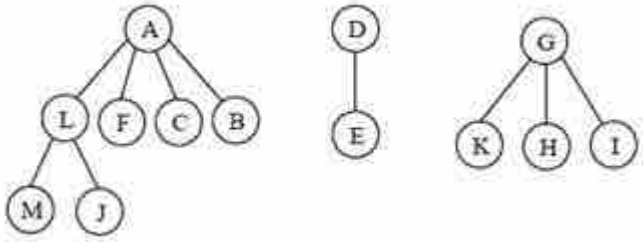


图 7-3

答：从顶点 A 出发进行广度优先遍历得到的广度优先生成森林为：

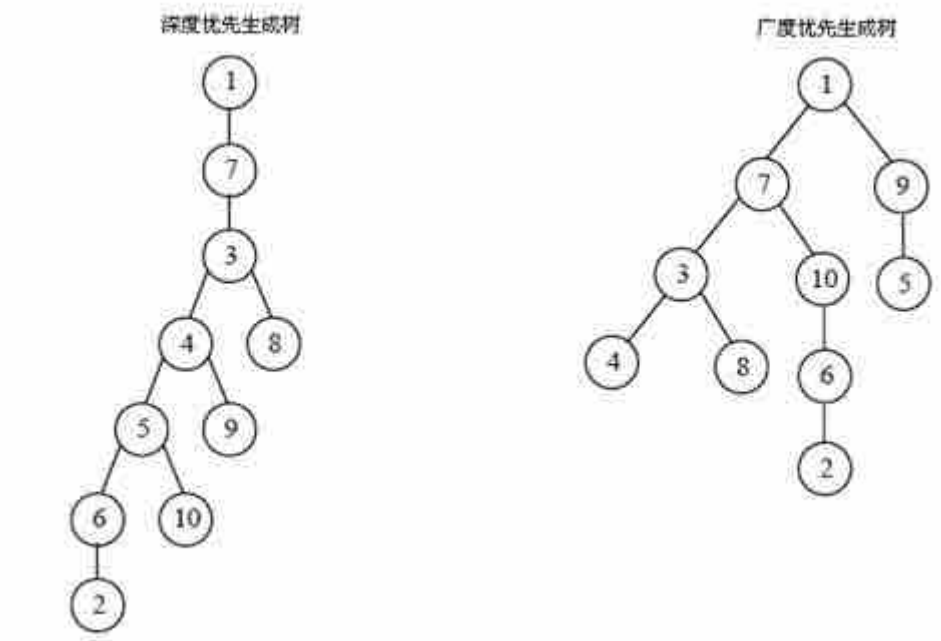


5．已知以二维数组表示的图的邻接矩阵如图 7-4 所示。试分别画出自顶点 1 出发进行遍历所得的深度优先生成树和广度优先生成树。

	1	2	3	4	5	6	7	8	9	10
1	0	0	0	0	0	0	1	0	1	0
2	0	0	1	0	0	0	1	0	0	0
3	0	0	0	1	0	0	0	1	0	0
4	0	0	0	0	1	0	0	0	1	0
5	0	0	0	0	0	1	0	0	0	1
6	1	1	0	0	0	0	0	0	0	0
7	0	0	1	0	0	0	0	0	0	1
8	1	0	0	1	0	0	0	0	1	0
9	0	0	0	0	1	0	1	0	0	1
10	1	0	0	0	0	1	0	0	0	0

图 7-4

答：



6．试证明求强连通分量的算法的正确性。（算法基于深度优先搜索遍历）

答：该算法的操作原理：

（1）算法基于图的深度优先搜索，基于定理：在任何深度优先搜索中，同一强连通分量内的所有顶点均在同一棵深度优先搜索树中。也就是说，强连通分量一定是有向图的某个深搜树子树。

（2）根据强连通分量的定义和性质可知，当一个点既是强连通子图Ⅰ中的点，又是强连通子图Ⅱ中的点，则它是强连通子图Ⅰ∪Ⅱ中的点。

（3）算法用 low 值记录该点所在强连通子图对应的搜索子树的根节点的 dfn 值。注意，该子树中的元素在栈中一定是相邻的，且根节点在栈中一定位于所有子树元素的最下方。
（4）强连通分量是由若干个环组成的，当有环形成时（也就是搜索的下一个点已在栈中），算法将这一条路径的 low 值统一，即这条路径上的点属于同一个强连通分量。
（5）如果遍历完整个搜索树后某个点的 dfn 值等于 low 值，则该点是该搜索子树的根。此时，该点以上（包括该点）一直到栈顶的所有元素组成一个强连通分量。
综上，算法得证。

7．请对如图 7-5 所示的无向带权图，
（1）写出它的邻接矩阵，并按普里姆算法求其最小生成树；
（2）写出它的邻接表，并按克鲁斯卡尔算法求其最小生成树。

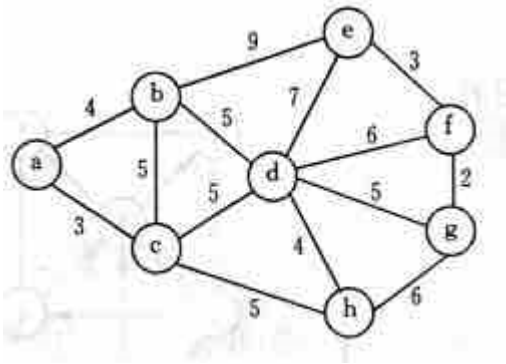
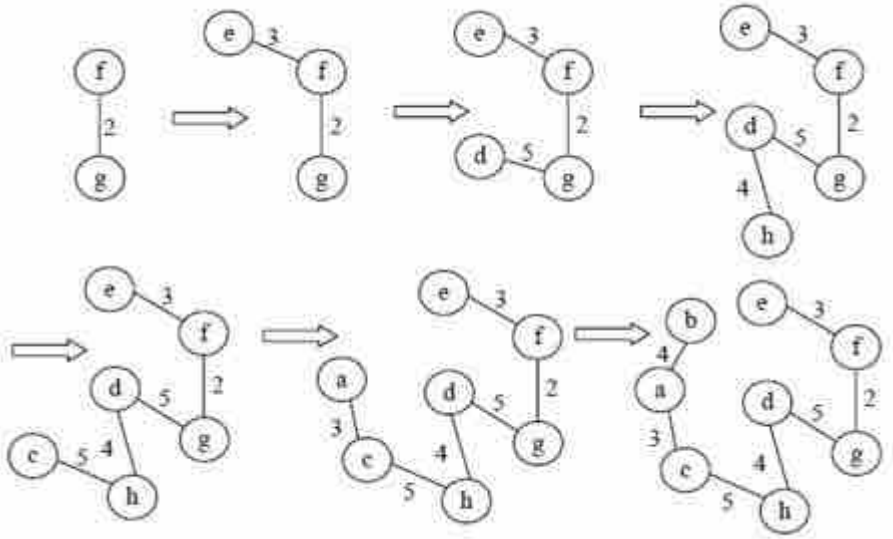


图 7-5

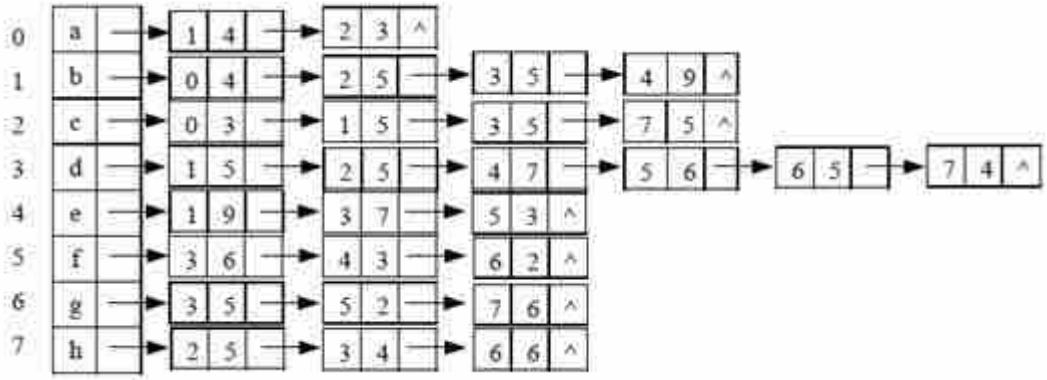
答：
（1）邻接矩阵

	a	b	c	d	e	f	g	h
a	0	1	1	0	0	0	0	0
b	1	0	1	1	1	0	0	0
c	1	1	0	1	0	0	0	1
d	0	1	1	0	1	1	1	1
e	0	1	0	1	0	1	0	0
f	0	0	0	1	1	0	1	0
g	0	0	0	1	0	1	0	1
h	0	0	1	1	0	0	1	0

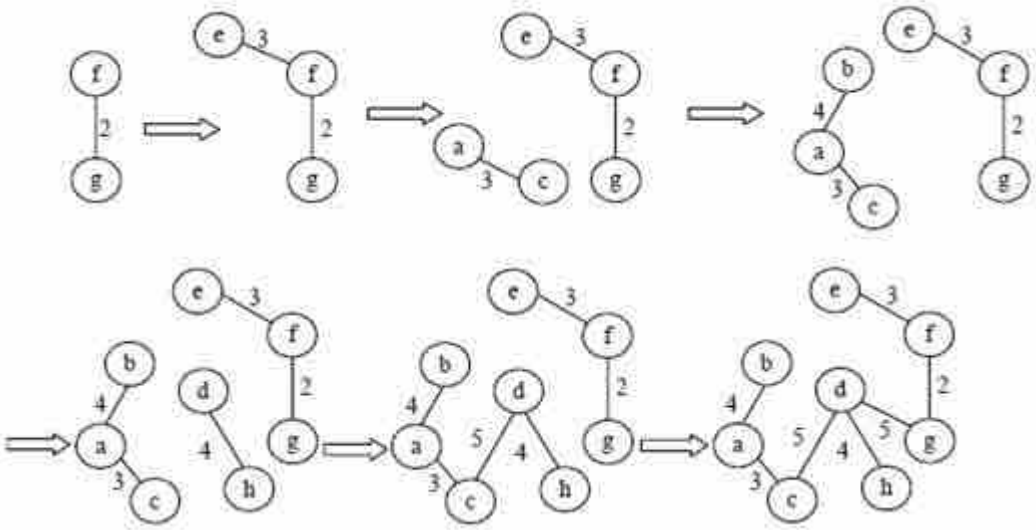
按普里姆算法求得最小生成树为：



（2）邻接表



按克鲁斯卡尔算法求得最小生成树为：



8．试对如图 7-6 所示无向图执行求关节点的算法，分别求出每个顶点的 visited[i]和 low[i]值，i=1,2，…，vexnum。

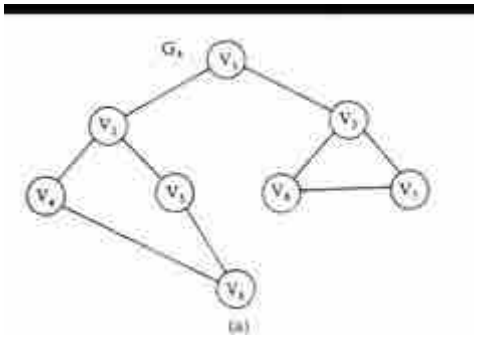


图 7-6

答：

G.vertices[I].data	V_1	V_2	V_3	V_4	V_5	V_6	V_7	V_8
Visited[i]	1	2	6	3	5	7	8	4
Low[i]的产生序号	1	4	7	3	1	6	5	2
Low[i]	1	1	1	2	2	6	6	2

9．试列出如图 7-7 所示图中全部可能的拓扑有序序列，并指出应用 7.5.1 节中算法 TopologicalSort 求得的是哪一个序列（注意：应先确定其存储结构）。

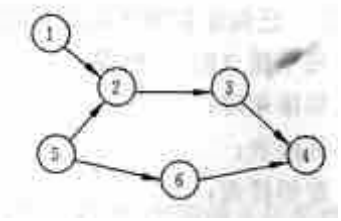


图 7-7

答：

5	6	1			2			3			4
5		1	6		2			3			4
5		1			2	6		3			4
5		1			2			3	6		4
		1	5	6	2			3			4
		1	5		2	6		3			4
		1	5		2			3	6		4

10．对于如图 7-8 所示的 AOE 网络，计算各活动弧的 e（a_i）和 l（a_j）函数值、各事件（顶点）的 ve（v_i）和 vl（v_j）函数值；列出各条关键路径。

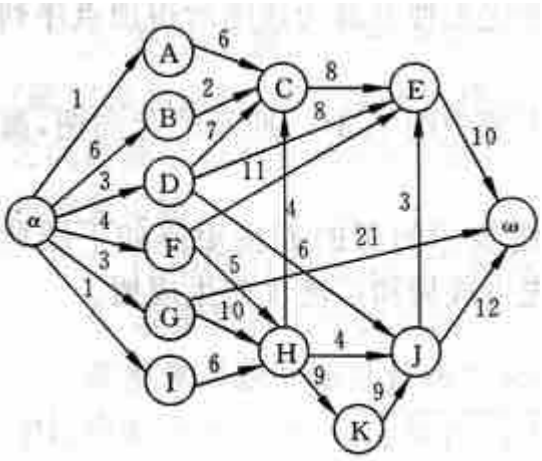


图 7-8

答：

顶点	ve	vi
α	0	0
A	1	20
B	6	24
C	17	26
D	3	19
E	34	34
F	4	8
G	3	3
H	13	13
I	1	7
J	31	31
K	22	22
ω	44	44

边	e	j	j—e
(α,A)	0	19	19
(α,B)	0	18	18
(α,D)	0	16	16
(α,F)	0	4	4
(α,G)	0	0	0
(α,I)	0	6	6
(A,C)	1	20	19
(B,C)	6	24	18
(D,C)	3	19	16
(D, E)	3	26	23
(D,J)	3	25	22
(F,E)	4	23	19
(F, H)	4	8	4
(G,ω)	3	23	20
(G,H)	3	3	0
(I, H)	1	7	6
(C, E)	17	26	9
(H, C)	13	22	9
(H,J)	13	27	14
(H,K)	13	13	0
(K,J)	22	22	0
(J,E)	31	31	0
(J,ω)	31	32	1
(E,ω)	34	34	0

关键路径只有一条：（α，G，H，K，J，E，ω）

11．试利用 Dijkstra 算法求如图 7-9 所示图中从顶点 a 到其他各顶点间的最短路径，写出执行算法过程中各步的状态。

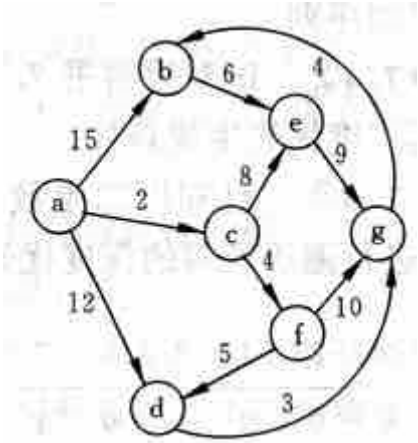
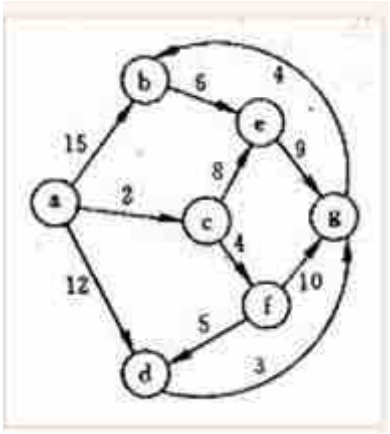


图 7-9

答：



从顶点 a 到其他各点的最短路径的求解过程如下：

终点 / Dist	b	c	d	e	f	g	S（终点集）
K=1	15 (a, b)	**2 (a, c)**	12 (a, d)				{a, c}
K=2	15 (a, b)		12 (a, d)	10 (a, c, e)	**6 (a, c, f)**		{a, c, f}
K=3	15 (a, b)		**11 (a, c, f, d)**	**10 (a, c, e)**		16 (a, c, f, g)	{a, c, f, e}
K=4	15 (a, b)		**11 (a, c, f, d)**			16 (a, c, f, g)	{a, c, f, e, d}
K=5	15 (a, b)					**14 (a, c, f, d, g)**	{a, c, f, e, d, g}
K=6	**15 (a, b)**						{a, c, f, e, d, g, b}

12．试证明求最短路径的 Dijkstra 算法的正确性。

答：Dijkstra 算法给出了从单源点 S 到图 G 的各个顶点的最短通路长度。

假设G中的每个顶点V都被赋予了一个标记L（V）,它要么是一个有限数，要么是∞。假设P是G的顶点的集合，P包含S，满足：

①如果V属于P，则L（V）是从S到V的最短通路的长度，并且存在这样的从S到V的最短通路：通路上的顶点都在P中；

②如果V不属于P，则L（V）是从S到V的满足下面限制的最短通路的长度：V是通路中唯一一个不属于P的顶点。

可以用归纳假设法证明 Dijkstra 算法中的 P 符合上述定义的集合：

①当P中元素个数为1时，P对应算法中的第一步，P={S},显然满足。

②假设P中元素个数为K时，P满足上述定义，下面看算法的的第三步，

③先找出不在P中且带有最小标记的顶点U，标记为L（U）,可以证明从S到U的最短通路中除U外不包含不属于P的元素。

因为若存在除U外其他顶点，则最短通路为SP1P2...PnQ1Q2...QnU（P1,P2..Pn属于P,Q1,Q2,...Qn不属于P）,则由性质2）最短通路长度为L（Q1）+PATH（Q1,U）>L（U），从而大于SP1P2..PnU的通路长度L（U），不是最短通路，所以从S到U的最短通路中除U外不包含不属于P的元素，从而从S到U的最短通路长度由L（U）给出。现把U加入P中构成P" ,显然P"满足性质1）。

取V不属于P",显然V也不属于P,那么从S到V的最短通路且满足除V外所有顶点都在P"中的通路有两种可能，①包含U，②不包含U。

对①SP1P2...PnUV=L（U）+W（U,V）

对②SP1P2…PnV=L（V）

显然二者中的最小值给出了从S到V的最短通路且满足除V外所有顶点都在P"中的长度。

从而算法中（3）给出的 P"含 K+1 个元素且满足（1）和（2）。又归纳，命题得证！

13．试利用 F10yd 算法如图 7-10 所示有向图中各对顶点之间的最短路径。

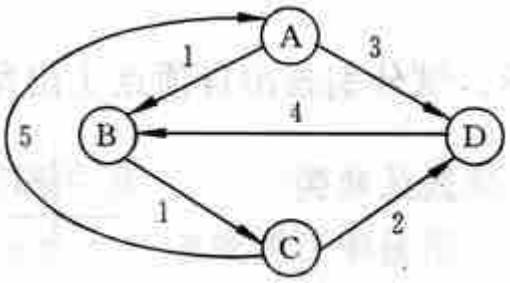


图 7-10

答：

A	A(0)				A(1)			
	1	2	3	4	1	2	3	4
1	0	1	∞	3	0	1	∞	3
2	∞	0	1	∞	∞	0	1	∞
3	5	∞	0	2	5	**6**	0	2
4	∞	4	∞	0	∞	**4**	∞	0

PATH	PATH(0)				PATH(1)			
	1	2	3	4	1	2	3	4
1		AB		AD		AB		AD
2			BC				BC	
3	CA			CD	CA	**CAB**		CD
4		DB				DB		

A	A(2)				A(3)			
	1	2	3	4	1	2	3	4
1	0	1	**2**	3	0	1	2	3
2	∞	0	1	∞	**6**	0	1	**3**
3	5	6	0	2	5	6	0	2
4	∞	4	**5**	0	**10**	4	5	0

PATH	PATH(2)				PATH(3)			
	1	2	3	4	1	2	3	4
1		AB	**ABC**	AD		AB	ABC	AD
2			BC		**BCA**		BC	**BCD**
3	CA	CAB		CD	CA	CAB		CD
4		DB	**DBC**		**DBCA**	DB	DBC	

注：A（4）和 A（3）相同；PATH（4）和 PATH（3）相同。

【算法设计题】

14．编写算法，由依次输入的顶点数目、弧的数目、各顶点的信息和各条弧的信息建立有向图的邻接表。

答：

//由顶点数,边数,顶点信息和边的信息建立有向图的邻接表

Status Build_AdjList（ALGraph &G） { InitALGraph（G）;//初始化表 G
scanf（"%d",&v）; if（v<0） return ERROR;//顶点数不能为负 G.vexnum=v;
scanf（"%d",&a）; if（a<0） return ERROR;//边数不能为负 G.arcnum=a;
for（m=0;m<v;m++） G.vertices[m].data=getchar（）;//输入各顶点的符号
for（m=1;m<=a;m++） { t=getchar（）;h=getchar（）;//t 为弧尾,h 为弧头
if（（i=LocateVex（G,t））<0） return ERROR; if（（j=LocateVex（G,h））<0） return ERROR;//顶点未找到 p=（ArcNode*）malloc（sizeof（ArcNode））;//动态创建顶点
if（!G.vertices.[i].firstarc） G.vertices[i].firstarc=p; else { for（q=G.vertices[i].firstarc;q->nextarc;q=q->nextarc）; q->nextarc=p;//邻接点 } p->adjvex=j;p->nextarc=NULL; }//while return OK; }//Build_AdjList

15．试在邻接矩阵存储结构上实现图的基本操作：
InsertVex（G,v）,InsertArc（G,v,w）,DeleteVex（G,v）和 DeleteArc（G,v,w）。

答：

本题中图 G 均为有向无权图,其余情况容易由此写出 Status Insert_Vex（MGraph &G, char v）//在邻接矩阵表示的图 G 上插入顶点 v { if（G.vexnum+1）>MAX_VERTEX_NUM return INFEASIBLE;//顶点数达到最大，无法插入新顶点 G.vexs[++G.vexnum]=v;//加入邻接矩阵 return OK; }//Insert_Vex
Status Insert_Arc（MGraph &G,char v,char w）//在邻接矩阵表示的图 G 上插入边（v,w） { if（（i=LocateVex（G,v））<0） return ERROR; if（（j=LocateVex（G,w））<0） return ERROR; if（i==j） return ERROR; if（!G.arcs[i][j].adj） { G.arcs[i][j].adj=1;//邻接矩阵插入位置置 1 G.arcnum++;//边数加 1 } return OK; }//Insert_Arc
Status Delete_Vex（MGraph &G,char v）//在邻接矩阵表示的图 G 上删除顶点 v { n=G.vexnum; if（（m=LocateVex（G,v））<0） return ERROR; G.vexs[m]<->G.vexs[n];//将待删除顶点交换到最后一个顶点 for（i=0;i<n;i++） { G.arcs[i][m]=G.arcs[i][n]; G.arcs[m][i]=G.arcs[n][i];//将边的关系随之交换 } G.arcs[m][m].adj=0;//将邻接矩阵删除与顶点邻接的边的位置置 0 G.vexnum--;//顶点个数减 1 return OK; }//Delete_Vex

分析：如果不把待删除顶点交换到最后一个顶点的话，算法将会比较复杂，而伴随着大量元素的移动，时间复杂度也会大大增加。

Status Delete_Arc（MGraph &G,char v,char w）//在邻接矩阵表示的图 G 上删除边（v,w） { if（（i=LocateVex（G,v））<0） return ERROR; if（（j=LocateVex（G,w））<0） return ERROR; if（G.arcs[i][j].adj） { G.arcs[i][j].adj=0;//将邻接矩阵删除的边的位置置 0 G.arcnum--;//边数减 1 } return OK; }//Delete_Arc

16．试对邻接表存储结构重做 15 题。

答：

//为节省篇幅,本题只给出 Insert_Arc 算法，其余算法请自行写出。
Status Insert_Arc（ALGraph &G,char v,char w）//在邻接表表示的图 G 上插入边（v,w） { if（（i=LocateVex（G,v））<0） return ERROR; if（（j=LocateVex（G,w））<0） return ERROR; p=（ArcNode*）malloc（sizeof（ArcNode））;//动态创建表结点 p->adjvex=j;p->nextarc=NULL; if（!G.vertices[i].firstarc） G.vertices[i].firstarc=p; else { for（q=G.vertices[i].firstarc;q->q->nextarc;q=q->nextarc） if（q->adjvex==j） return ERROR; //边已经存在 q->nextarc=p; } G.arcnum++; return OK; }//Insert_Arc

17．试对十字链表存储结构重做 15 题。

答：

//为节省篇幅,本题只给出较为复杂的 Delete_Vex 算法，其余算法请自行写出。

Status Delete_Vex（OLGraph &G,char v）//在十字链表表示的图 G 上删除顶点 v { if（（m=LocateVex（G,v））<0） return ERROR; n=G.vexnum; for（i=0;i<n;i++）//删除所有以 v 为头的边 { if（G.xlist[i].firstin->tailvex==m）//如果待删除的边是十字链表头链上的第一个结点 { q=G.xlist[i].firstin; G.xlist[i].firstin=q->hlink; free（q）;G.arcnum--;//删除结点，边数减 1 } else//否则，循环删除所有以 v 为弧头的边 { for（p=G.xlist[i].firstin;p&&p->hlink->tailvex!=m;p=p->hlink）; if（p） { q=p->hlink; p->hlink=q->hlink; free（q）;G.arcnum--; } }//else }//for for（i=0;i<n;i++）//删除所有以 v 为尾的边 { if（G.xlist[i].firstout->headvex==m）//如果待删除的边是十字链表尾链上的第一个结点 { q=G.xlist[i].firstout; G.xlist[i].firstout=q->tlink; free（q）;G.arcnum--; } else//否则，通过循环删除所有以 v 为弧尾的边 { for（p=G.xlist[i].firstout;p&&p->tlink->headvex!=m;p=p->tlink）; if（p） { q=p->tlink; p->tlink=q->tlink; free（q）;G.arcnum--; } }//else }//for for（i=m;i<n;i++）//顺次用结点 m 之后的顶点取代前一个顶点 { G.xlist[i]=G.xlist[i+1];//修改表头向量 for（p=G.xlist[i].firstin;p;p=p->hlink） p->headvex--; for（p=G.xlist[i].firstout;p;p=p->tlink） p->tailvex--;//修改各链中的顶点序号 } G.vexnum--; return OK; }//Delete_Vex

18．试对邻接多重表存储结构重做 15 题。

答：

//为节省篇幅,本题只给出 Delete_Arc 算法，其余算法请自行写出。

Status Delete_Arc（AMLGraph &G,char v,char w）//在邻接多重表表示的图 G 上删除边（v,w） { if（（i=LocateVex（G,v））<0） return ERROR; if（（j=LocateVex（G,w））<0） return ERROR; if（G.adjmulist[i].firstedge->jvex==j）//如果边（v,w）是 i 链表的第一条边 G.adjmulist[i].firstedge=G.adjmulist[i].firstedge->ilink; else {//在 i 链表上查找边（v,w） for（p=G.adjmulist[i].firstedge;p&&p->ilink->jvex!=j;p=p->ilink）; if （!p） return ERROR; //未找到 p->ilink=p->ilink->ilink; } //在 i 链表中删除该边 if（G.adjmulist[j].firstedge->ivex==i） G.adjmulist[j].firstedge=G.adjmulist[j].firstedge->jlink; else {//在 j 链表上查找边（v,w） for（p=G.adjmulist[j].firstedge;p&&p->jlink->ivex!=i;p=p->jlink）; if （!p） return ERROR; //未找到 q=p->jlink; p->jlink=q->jlink; free（q）; } //在 j 链表中删除该边 G.arcnum--; return OK; }//Delete_Arc

19．编写算法，由依次输入的顶点数目、边的数目、各顶点的信息和各条边的信息建立无向图的邻接多重表。

答：

Status Build_AdjMulist（AMLGraph &G）//输入顶点数,边数,顶点信息和边的信息建立无向图的邻接多重表 { InitAMLGraph（G）;//初始化无向图 scanf（"%d",&v）; if（v<0） return ERROR; //顶点数不能为负 G.vexnum=v; scanf（%d",&a）; if（a<0） return ERROR; //边数不能为负 G.arcnum=a; for（m=0;m<v;m++） G.adjmulist[m].data=getchar（）; //输入表示各顶点的符号 for（m=1;m<=a;m++） { t=getchar（）;h=getchar（）; //t、h 为一条边的两个邻接点 if（（i=LocateVex（G,t））<0） return ERROR; if（（j=LocateVex（G,h））<0） return ERROR; //顶点未找到 p=（EBox*）malloc（sizeof（EBox））; p->ivex=i;p->jvex=j; p->ilink=NULL;p->jlink=NULL; //边结点赋初值 if（!G.adjmulist[i].firstedge） G.adjmulist[i].firstedge=p; else { q=G.adjmulist[i].firstedge; while（q） { r=q; if（q->ivex==i） q=q->ilink; else q=q->jlink; } if（r->ivex==i） r->ilink=p;//注意 i 值既可能出现在边结点的 ivex 域中, else r->jlink=p; //又可能出现在边结点的 jvex 域中 }//else //插入 i 链表尾部 if（!G.adjmulist[j].

firstedge） G.adjmulist[j].firstedge=p; else { q=G.adjmulist[i].firstedge; while（q） { r=q; if（q->jvex==j） q=q->jlink; else q=q->ilnk; } if（r->jvex==j） r->jlink=p; else r->ilink=p; } //else //插入 j 链表尾部 }//for return OK; }//Build_AdjList

20．下面的算法段可以测定图 G=（V,E）是否可传递：

```
trans=TRUE;
for（V 中的每个 x）
    for（N(x)中的每个 y）
        for（N(y)中不等于 x 的每个 z）
            if（z 不在 N(x)中）trans=FALSE;
```

其中 N（x）表示 x 邻接到的所有顶点的集合。试以邻接矩阵存储结构实现判定一个图的可传递性的算法，并通过 n=|V|,m=|E|和 d=结点度数的均值，估计执行时间。

答：

int Pass_MGraph（MGraph G）//判断一个邻接矩阵存储的有向图是不是可传递的,是则返回 1,否则返回 0 { for（x=0;x<G.vexnum;x++） { for（y=0;y<G.vexnum;y++） { if（G.arcs[x][y]） { for（z=0;z<G.vexnum;z++） if（z!=x&&G.arcs[y][z] &&!G.arcs[x][z]） return 0;//图不可传递的条件 }//if }//内层 for }//外层 for return 1; }//Pass_MGraph 分析：本算法的时间复杂度大概是 O（n^2*d）。

21．试对邻接表存储结构重做 20 题。

答：

int Pass_ALGraph（ALGraph G）//判断一个邻接表存储的有向图是不是可传递的,是则返回 1,否则返回 0 { for（x=0;x<G.vexnum;x++） for（p=G.vertices[x].firstarc;p;p=p->nextarc） { y=p->adjvex; for（q=G.vertices[y].firstarc;q;q=q->nextarc） { z=q->adjvex; if（z!=x&&!is_adj（G,x,z）） return 0;//有向图不可传递的条件 }//for }//for } //Pass_ALGraph

int is_adj（ALGraph G,int m,int n）//判断有向图 G 中是否存在边（m,n）,是则返回 1,否则返回 0 { for（p=G.vertices[m].firstarc;p;p=p->nextarc） if（p->adjvex==n） return 1;//G 中存在边（m,n） return 0; }//is_adj

22．试基于图的深度优先搜索策略写一算法，判别以邻接表方式存储的有向图中是否存在由顶点 v_i 到顶点 v_j 的路径（i≠j）。注意：算法中涉及的图的基本操作必须在此存储结构上实现。

答：

int visited[MAXSIZE];//指示顶点是否在当前路径上，在当前路径上值为 1，否则值为 0

int exist_path_DFS（ALGraph G,int i,int j）//深度优先判断有向图 G 中顶点 i 到顶点 j 是否有路径,有则返回 1,否则返回 0 { if（i==j） return 1; //顶点 i 与顶点 j 是一个同一个顶点 else { visited[i]=1; for（p=G.vertices[i].firstarc;p;p=p->nextarc） { k=p->adjvex; if（!visited[k]&&exist_path（k,j）） return 1;//i 下游的顶点到 j 有路径 }//for }//else } //exist_path_DFS

23．同 22 题要求。试基于图的广度优先搜索策略写一算法

答：

int exist_path_BFS（ALGraph G,int i,int j）//广度优先判断有向图 G 中顶点 i 到顶点 j 是否有路径,有则返回 1,否则返回 0 { int visited[MAXSIZE]; //指示顶点是否在当前路径上，在当

前路径上值为 1，否则值为 0　InitQueue（Q）;//使用队列　EnQueue（Q,i）;//先将顶点 i 入列　while（!QueueEmpty（Q））　{ DeQueue（Q,u）; visited[u]=1;//出列的顶点在当前路径上 for（p=G.vertices[i].firstarc;p;p=p->nextarc） { k=p->adjvex; if（k==j） return 1;//达到顶点 j，说明 i 到 j 有路径 if（!visited[k]） EnQueue（Q,k）;//不在路径上的剩余顶点入列 }//for　}//while　return 0; }//exist_path_BFS

24．试利用栈的基本操作编写，按深度优先搜索策略遍历一个强连通图的非递归形式的算法。算法中不规定具体的存储结构，而将图 Graph 看成是一种抽象的数据类型。

答：

void STraverse_Nonrecursive（Graph G）//非递归深度优先搜索遍历强连通图 G {　int visited[MAXSIZE];//指示顶点是否在当前路径上，在则为 1，不在则为 0　InitStack（S）; //借助栈　Push（S,GetVex（S,1））; //将第一个顶点入栈　visit（1）;//遍历第一个顶点 visited[1]=1;//第一个顶点在当前路径上，指示标记为 1　while（!StackEmpty（S））　{ while（Gettop（S,i）&&i）//出栈 { j=FirstAdjVex（G,i）; if（j&&!visited[j]） { visit（j）;　visited[j]=1;//遍历过的顶点，指示标记为 1，在当前路径上　Push（S,j）; //向左走到尽头 } }//while if（!StackEmpty（S）） { Pop（S,j）; Gettop（S,i）; k=NextAdjVex（G,i,j）; //向右走一步 if（k&&!visited[k]） {　visit（k）;　visited[k]=1; Push（S,k）;//遍历过的顶点入栈 } }//if　}//while }//Straverse_Nonrecursive 分析：本算法的基本思想与二叉树的先序遍历非递归算法相同,请参考 6.37。由于是强连通图,所以从第一个结点出发一定能够访问到所有结点.

25．试写一个求有向图 G 中所有简单回路的算法。

答：

int visited[MAXSIZE];//存储顶点的访问状态 int path[MAXSIZE];//暂存当前路径 int cycles[MAXSIZE][MAXSIZE];//储存发现的回路所包含的结点 int thiscycle[MAXSIZE];//储存当前发现的一个回路 int cycount=0;//已发现的回路个数

void GetAllCycle（ALGraph G）//求有向图中所有的简单回路 {

for（v=0;v<G.vexnum;v++） visited[v]=0;　for（v=0;v<G.vexnum;v++） if（!visited[v]） DFS（G,v,0）; //深度优先遍历 }//DFSTraverse

void DFS（ALGraph G,int v,int k）//k 表示当前结点在路径上的序号 {　visited[v]=1; path[k]=v; //记录当前路径　for（p=G.vertices[v].firstarc;p;p=p->nextarc）　{ w=p->adjvex; if（!visited[w]） DFS（G,w,k+1）; else //发现了一条回路 { for（i=0;path[i]!=w;i++）;//找到回路的起点 for（j=0;path[i+j];i++） thiscycle[j]=path[i+j];//把回路复制下来 if（!exist_cycle（）） cycles[cycount++]=thiscycle;//如果该回路尚未被记录过,就添加到记录中 for（i=0;i<G.vexnum;i++） thiscycle[i]=0;//清空目前回路数组 }//else　}//for path[k]=0;　visited[k]=0; //注意只有当前路径上的结点 visited 为 1，因此一旦遍历中发现当前结点 visited 为,1,即表示发现了一条回路 }//DFS

int exist_cycle（）//判断 thiscycle 数组中记录的回路在 cycles 的记录中是否已经存在 {

int temp[MAXSIZE];　for（i=0;i<cycount;i++）//判断已有的回路与 thiscycle 是否相同，即所有结点和它们的顺序都相同　{ j=0;c=thiscycle&#0;//例如,142857 和 857142 是相同的回路 for（k=0;cycles[i][k]!=c&&cycles[i][k]!=0;k++）;//在 cycles 的一个行向量中寻找等于 thiscycle 第一个结点的元素 if（cycles[i][k]） //有与之相同的一个元素 { for（m=0;cycles[i][k+m];m++）　temp[m]=cycles[i][k+m]; for（n=0;n<k;n++,m++）　temp[m]=cycles[i][n]; //调整 cycles 中的当前记录的循环相位并放入 temp 数组中

if（!StrCompare（temp,thiscycle）） //与 thiscycle 比较　return 1; //完全相等

for（m=0;m<G.vexnum;m++） temp[m]=0; //清空这个数组 } }//for return 0; //所有现存回路都不与 thiscycle 完全相等 }//exist_cycle

26．试完成求有向图的强连通分量的算法，并分析算法的时间复杂度。

答：

int visited[MAXSIZE]; int finished[MAXSIZE]; int count;//count 在第一次深度优先遍历中用于指示 finished 数组的填充位置

void Get_SGraph（OLGraph G）//求十字链表结构储存的有向图 G 的强连通分量 { count=0; for（v=0;v<G.vexnum;v++） visited[v]=0;//初始化所有结点均未被访问 for（v=0;v<G.vexnum;v++） //第一次深度优先遍历建立 finished 数组 if（!visited[v]） DFS1（G,v）;//第一次深度优先遍历 for（v=0;v<G.vexnum;v++） visited[v]=0; //清空 visited 数组 for（i=G.vexnum-1;i>=0;i--） //第二次逆向的深度优先遍历 { v=finished（i）; if（!visited[v]） { printf（"\n"）; //不同的强连通分量在不同的行输出 DFS2（G,v）;//第二次深度优先遍历 } }//for }//Get_SGraph

void DFS1（OLGraph G,int v）//第一次深度优先遍历的算法 { visited[v]=1; for（p=G.xlist[v].firstout;p;p=p->tlink） { w=p->headvex; if（!visited[w]） DFS1（G,w）; //如果顶点未被访问过，递归进行第一次深度优先遍历 }//for finished[++count]=v; //在第一次遍历中建立 finished 数组 }//DFS1

void DFS2（OLGraph G,int v）//第二次逆向的深度优先遍历的算法 { visited[v]=1; printf（"%d",v）; //在第二次遍历中输出结点序号 for（p=G.xlist[v].firstin;p;p=p->hlink） { w=p->tailvex; if（!visited[w]） DFS2（G,w）;// 如果顶点未被访问过，递归进行第二次深度优先遍历 }//for }//DFS2 分析：求有向图的强连通分量的算法的时间复杂度和深度优先遍历相同，也为 O（n+e）。

27．试修改普里姆算法，使之能在邻接表存储结构上实现求图的最小生成森林，并分析其时间复杂度（森林的存储结构为孩子—兄弟链表）。

答：

void Forest_Prim（ALGraph G,int k,CSTree &T）//从顶点 k 出发,构造邻接表结构的有向图 G 的最小生成森林 T,用孩子兄弟链表存储 { for（j=0;j<G.vexnum;j++） //以下在 Prim 算法基础上稍作改动 if（j!=k） { closedge[j]={k,Max_int}; for（p=G.vertices[j].firstarc;p;p=p->nextarc） if（p->adjvex==k） closedge[j].lowcost=p->cost;//最小代价 }//if closedge[k].lowcost=0; for（i=1;i<G.vexnum;i++） { k=minimum（closedge）; if（closedge[k].lowcost<Max_int）//找到代价最小的边 { Addto_Forest（T,closedge[k].adjvex,k）; //把这条边加入生成森林中 closedge[k].lowcost=0; for（p=G.vertices[k].firstarc;p;p=p->nextarc） if（p->cost<closedge[p->adjvex].lowcost） closedge[p->adjvex]={k,p->cost}; }//if else Forest_Prim（G,k）; //递归对另外一个连通分量执行算法 }//for } //Forest_Prim

void Addto_Forest（CSTree &T,int i,int j）//把边（i,j）添加到孩子兄弟链表表示的树 T 中 { p=Locate（T,i）; //找到结点 i 对应的指针 p,过程略 q=（CSTNode*）malloc（sizeof（CSTNode））;//动态创建表结点 q->data=j; if（!p） //起始顶点不属于森林中已有的任何一棵树 { p=（CSTNode*）malloc（sizeof（CSTNode））; p->data=i; for（r=T;r->nextsib;r=r->nextsib）;//指向最后一个兄弟结点 r->nextsib=p; p->firstchild=q; } //作为新树插入到最右侧 else if（!p->firstchild） //双亲还没有孩子 p->firstchild=q; //作为双亲的第一个孩子 else //双亲已经有了孩子 { for（r=p->firstchild;r->nextsib;r=r->nextsib）; r->nextsib=q; //作为双亲最后一个孩子的兄弟 } } //Addto_Forest

main（） { ... T=（CSTNode*）malloc（sizeof（CSTNode））; //建立树根 T->data=1; Forest_Prim（G,1,T）; }//main 分析：这个算法是在 Prim 算法的基础上添加了非连通图支持和孩子兄弟链表构建模块而得到的,其时间复杂度为 O（n^2）。

28．已知无向图的边集存放在某个类型为 EdgesetTyPe 的数据结构 Edgeset 中（没有端点相重的环边），并在此结构上已定义两种基本运算：

（1）函数 GetMinEdge（Edgeset,u,v）：若 Edgeset 非空，则必存在最小边，变参 u 和 v 分别含最小边上两顶点，并返回 true；否则返回 false；

（2）过程 DelMinEdge（Edgeset,u,v）：从 Edgeset 中删除依附于顶点 u 和 v 的最小边。

试在上述结构上实现求最小生成树（以孩子一兄弟链表表示）的克鲁斯卡尔算法。

答：

typedef struct { int vex;//顶点序号 int ecno;//顶点所属的连通分量号 } VexInfo;//定义结点的抽象数据类型 VexInfo vexs[MAXSIZE];//记录顶点所属连通分量号的数组

void Init_VexInfo（VexInfo &vexs[ ],int vexnum）//初始化 { for（i=0;i<vexnum;i++） vexs[i]={i,i};//初始状态：每一个顶点都属于不同的连通分量 }//Init_VexInfo

int is_ec（VexInfo vexs[ ],int i,int j）//判断顶点 i 和顶点 j 是否属于同一个连通分量 { if（vexs[i].ecno==vexs[j].ecno） return 1; else return 0; }//is_ec

void merge_ec（VexInfo &vexs[ ],int ec1,int ec2）//合并连通分量 ec1 和 ec2 { for（i=0;vexs[i].vex;i++） if（vexs[i].ecno==ec2） vexs[i].ecno==ec1; }//merge_ec

void MinSpanTree_Kruscal（Graph G,EdgeSetType &EdgeSet,CSTree &T）//求图的最小生成树的克鲁斯卡尔算法 { Init_VexInfo（vexs,G.vexnum）; ecnum=G.vexnum;//连通分量个数 while（ecnum>1） { GetMinEdge（EdgeSet,u,v）;//选出最短边 if（!is_ec（vexs,u,v））//u 和 v 属于不同连通分量 { Addto_CSTree（T,u,v）;//加入到生成树中 merge_ec（vexs,vexs[u].ecno,vexs[v].ecno）;//合并连通分量 ecnum--; } DelMinEdge（EdgeSet,u,v）;//从边集中删除 }//while }//MinSpanTree_Kruscal

void Addto_CSTree（CSTree &T,int i,int j）//把边（i,j）添加到孩子兄弟链表表示的树 T 中 { p=Locate（T,i）;//找到结点 i 对应的指针 p q=（CSTNode*） malloc（sizeof（CSTNode））;//动态创建顶点 q->data=j; if（!p->firstchild） //双亲还没有孩子 p->firstchild=q; //作为双亲的第一个孩子 else //双亲已经有了孩子 { for（r=p->firstchild;r->nextsib;r=r->nextsib）; r->nextsib=q; //作为双亲最后一个孩子的兄弟 } }//Addto_CSTree 分析：本算法使用一维结构体变量数组来表示等价类,每个连通分量所包含的所有结点属于一个等价类。在这个结构上实现了初始化、判断元素是否等价（两个结点是否属于同一个连通分量）、合并等价类（连通分量）的操作。

29．试编写一个算法，给有向无环图 G 中每个顶点赋以一个整数序号，并满足以下条件：若从顶点 i 至顶点 j 有一条弧，则应使 i<j。

答：

//本算法就是拓扑排序

Status TopoSeq（ALGraph G,int new[ ]）//按照题目要求给有向无环图的结点编号,并存入数组 new 中 { int indegree[MAXSIZE]; FindIndegree（G,indegree）;//找到图 G 中每个顶点的入度 Initstack（S）;//初始化栈操作 for（i=0;i<G.vexnum;i++） if（!indegree[i]） Push（S,i）;//入度为 0，进栈 count=0; while（!stackempty（S））{ Pop（S,i）; new[i]=++count; //把拓扑顺序存入数组的对应分量中 for（p=G.vertices[i].firstarc;p;p=p->nextarc） { k=p->adjvex; if（!（--indegree[k]）） Push（S,k）; } }//while if（count<G.vexnum） return ERROR; return OK; }//TopoSeq

30．若在 DAG 图中存在一个顶点 r，在 r 和图中所有其他顶点之间均存在由 r 出发的有向路径，则称该 DAG 图有根。试编写求 DAG 图的根的算法。

答：

```
int visited[MAXSIZE];
void Get_Root（ALGraph G）//求有向无环图的根 {  for（v=0;v<G.vexnum;v++）  { for（w=0;w<G.vexnum;w++） visited[w]=0;//每次都要将访问数组清零 DFS（G,v）; //从顶点 v 出发进行深度优先遍历 for（flag=1,w=0;w<G.vexnum;w++） if（!visited[w]） flag=0; //如果 v 是根,则深度优先遍历可以访问到所有结点 if（flag） printf（"Found a root vertex:%d\n",v）;//如果有的话，找到有向无换图的根  }//for }//Get_Root,这个算法要求图中不能有环,否则会发生误判
void DFS（ALGraph G,int v） {  visited[v]=1;  for（p=G.vertices[v].firstarc;p;p=p->nextarc）  { w=p->adjvex; if（!visited[w]） DFS（G,w）;//递归进行深度优先搜索遍历  } } //DFS
```

31．在图的邻接表存储结构中，为每个顶点增加一个 MPL 域。试写一算法，求有向无环图 G 的每个顶点出发的最长路径的长度，并存入其 MPL 域。请给出算法的时间复杂度。

答：

```
void Fill_MPL（ALGraph &G）//为有向无环图 G 添加 MPL 域 {
FindIndegree（G,indegree）;//图 G 中所有顶点的入度  for（i=0;i<G.vexnum;i++）
if（!indegree[i]） Get_MPL（G,i）;//从每一个零入度顶点出发构建 MPL 域 }//Fill_MPL
int Get_MPL（ALGraph &G,int i）//从一个顶点出发构建 MPL 域并返回其 MPL 值 {
if（!G.vertices[i].firstarc）  { G.vertices[i].MPL=0; return 0; //零出度顶点  }  else  { max=0; for（p=G.vertices[i].firstarc;p;p=p->nextarc） { j=p->adjvex; if（G.vertices[j].MPL==0） k=Get_MPL（G,j）; if（k>max） max=k; //求其直接后继顶点 MPL 的最大者 } G.vertices[i]=max+1;//再加 1,就是当前顶点的 MPL return max+1;  }//else }//Get_MPL
```

32．试设计一个求有向无环图中最长路径的算法，并估计其时间复杂度。

答：

```
int maxlen,path[MAXSIZE];//数组 path 用于存储当前路径 int mlp[MAXSIZE];//数组 mlp 用于存储已发现的最长路径
void Get_Longest_Path（ALGraph G）//求一个有向无环图中最长的路径 {  maxlen=0; FindIndegree（G,indegree）;  for（i=0;i<G.vexnum;i++）  { for（j=0;j<G.vexnum;j++） visited[j]=0;//每次都将访问数组清零 if（!indegree[i]） DFS（G,i,0）;//从每一个零入度结点开始深度优先遍历  }  printf（"Longest Path:"）;  for（i=0;mlp[i];i++） printf（"%d",mlp[i]）;//输出最长路径 }//Get_Longest_Path
void DFS（ALGraph G,int i,int len）//深度优先搜索 {  visited[i]=1;  path[len]=i; if（len>maxlen&&!G.vertices[i].firstarc）//新的最长路径  { for（j=0;j<=len;j++） mlp[j]=path[j];//保存下来 maxlen=len;  }  else  { for（p=G.vertices[i].firstarc;p;p=p->nextarc） { j=p->adjvex; if（!visited[j]） DFS（G,j,len+1）;//如果顶点未被访问过，递归进行深度优先搜索遍历 }  }//else  path[i]=0;  visited[i]=0; }//DFS
```

33．一个四则运算算术表达式以有向无环图的邻接表方式存储，每个操作数原子都由单个字母表示。写一个算法输出其逆波兰表达式。

答：

```
void NiBoLan_DAG（ALGraph G）//输出有向无环图形式表示的表达式的逆波兰式，即后缀表达式 {  FindIndegree（G,indegree）;  for（i=0;i<G.vexnum;i++） if（!indegree[i]）
```

r=i; //找到有向无环图的根　　PrintNiBoLan_DAG（G,i）;//打印逆波兰式 }
//NiBoLan_DAG
void PrintNiBoLan_DAG（ALGraph G,int i）//打印输出以顶点 i 为根的表达式的逆波兰式 {
c=G.vertices[i].data;　　if（!G.vertices[i].firstarc）//c 是原子，直接输出 printf（"%c",c）;
else //子表达式　　{ p=G.vertices[i].firstarc; PrintNiBoLan_DAG（G,p->adjvex）;//递归输出
PrintNiBoLan_DAG（G,p->nexarc->adjvex）; printf（"%c",c）;　　} }//PrintNiBoLan_DAG
34．试编写利用深度优先遍历有向图实现求关键路径的算法。
答：
void Critical_Path（ALGraph G）//利用深度优先遍历求网的关键路径 {
FindIndegree（G,indegree）;　for（i=0;i<G.vexnum;i++） if（!indegree[i]） DFS1（G,i）;
//第一次深度优先遍历：建立 ve　for（i=0;i<G.vexnum;i++） if（!indegree[i]）
DFS2（G,i）; //第二次深度优先遍历：建立 vl　for（i=0;i<=G.vexnum;i++） if（vl[i]
==ve[i]） printf（"%d",i）; //ve=vl 时，输出关键路径 }//Critical_Path
void DFS1（ALGraph G,int i）//第一次深度优先遍历 {　if（!indegree[i]） ve[i]=0;
for（p=G.vertices[i].firstarc;p;p=p->nextarc）　{//建立 ve dut=*p->info; if（ve[i]+dut>ve[p-
>adjvex]） ve[p->adjvex]=ve[i]+dut; DFS1（G,p->adjvex）;　} }//DFS1
void DFS2（ALGraph G,int i）//第二次深度优先遍历 {　if（!G.vertices[i].firstarc） vl[i]
=ve[i];　else　{ for（p=G.vertices[i].firstarc;p;p=p->nextarc） {//建立 vl DFS2（G,p-
>adjvex）; dut=*p->info; if（vl[p->adjvex]-dut<vl[i]）　vl[i]=vl[p->adjvex]-dut; }　}//else }
//DFS2
35．以邻接表作存储结构实现求从源点到其余各顶点的最短路径的 Diikstra 算法。
答：
void ALGraph_DIJ（ALGraph G,int v0,Pathmatrix &P,ShortestPathTable &D）//在邻接表存储
结构上实现迪杰斯特拉算法 {　for（v=0;v<G.vexnum;v++） D[v]=INFINITY;//初始化路
径长度为无穷大　for（p=G.vertices[v0].firstarc;p;p=p->nextarc） D[p->adjvex]=*p->info; //
给 D 数组赋初值　for（v=0;v<G.vexnum;v++）　{ final[v]=0;
for（w=0;w<G.vexnum;w++） P[v][w]=0; //设空路径 if（D[v]<INFINITY）//找到较短的路
径 { P[v][v0]=1; P[v][v]=1; }　}//for　D[v0]=0;final[v0]=1; //初始化
for（i=1;i<G.vexnum;i++）　{ min=INFINITY; for（w=0;w<G.vexnum;w++） {
　if（!final[w]） {
　if（D[w]<min） //尚未求出到该顶点的最短路径　{　v=w;　min=D[w];　}
　}
} final[v]=1; for（p=G.vertices[v].firstarc;p;p=p->nextarc） { w=p->adjvex; if（!final[w]
&&（min+（*p->info）<D[w]）） //符合迪杰斯特拉条件 {　D[w]
=min+edgelen（G,v,w）;//存储最短路径长度　P[w]=P[v];　P[w][w]=1; //构造最短路径 } }
//for　}//for }//ALGraph_DIJ 分析：本算法对迪杰斯特拉算法中直接取任意边长度的语句
作了修改。由于在原算法中，每次循环都是对尾相同的边进行处理,所以可以用遍历邻接表
中的一条链来代替。

7.3　考研真题与典型题详解

一、选择题

1．设图的邻接矩阵 A 如下所示,各顶点的度依次是（　　）。[2013 年联考真题]

$$A=\begin{bmatrix}0&1&0&1\\0&0&1&1\\0&1&0&0\\1&0&0&0\end{bmatrix}$$

A．1，2，1，2

B．2，2，1，1

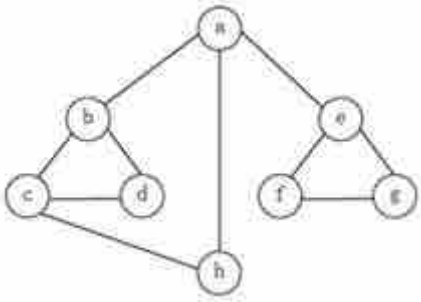
C．3，4，2，3

D．4，4，2，2

【答案】C 查看答案

【解析】当图用邻接矩阵存储时，各顶点的度是矩阵中此结点对应的横行和纵列非零元素之和。

2．若对如下无向图进行遍历，则下列选项中，不是广度优先遍历序列的是（　　）。[2013 年联考真题]



A．h，c，a，b，d，e，g，f

B．e，a，f，g，b，h，c，d

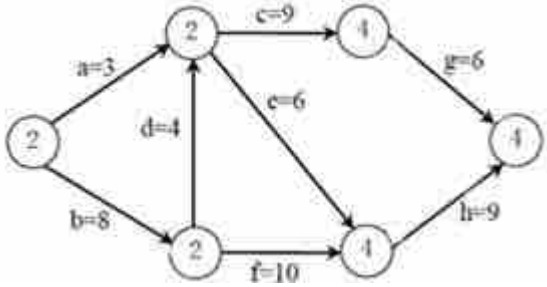
C．d，b，c，a，h，e，f，g

D．a，b，c，d，h，e，f，g

【答案】D 查看答案

【解析】根据广度优先遍历的定义，可知 ABC 三项都为广度优先遍历，而 D 项是深度优先遍历而不是广度优先遍历，故答案为 D。

3．下列 AOE 网表示一项包含 8 个活动的工程。通过同时加快若干进度可以缩短整个工程的工期。下列选项中，加快其进度就可以缩短工程工期的是（　　）。[2013 年联考真题]



A．c 和 e

B．d 和 e

C．f 和 d

D．f 和 h

【答案】C 查看答案

【解析】根据 AOE 网的定义可知，同时缩短几条关键路径上的活动时间，可以缩短整个工期。

4．设有向图 G＝（V，E），顶点集 V＝{V0，V1，V2，V3}，边集 E＝{<V0, V1>，<V0, V2>，<V0, V3>，<V1, V3>}，若从顶点 V0 开始对图进行深度优先遍历，则可能得到的不同遍历序列个数是（　　）。[2015 年联考真题]

A．2

B．3

C．4

D．5

【答案】D 查看答案

【解析】根据题意知有向图的结构如图所示。深度优先遍历的特点是尽可能先对纵深方向进行搜索，所以可能得到的不同遍历序列分别是：

①V0→V2→V1→V3；②V0→V2→V3→V1；③V0→V1→V3→V2；④V0→V3→V2→V1；⑤V0→V3→V1→V2。



5．对如下所示的有向图进行拓扑排序，得到的拓扑序列可能是（　　）[2014 年联考真题]

A．3,1,2,4,5,6

B．3,1,2,4,6,5

C．3,1,4,2,5,6

D．3,1,4,2,6,5



【答案】D 查看答案

【解析】拓扑排序方法如下:

（1）从有向图中选择一个没有前驱（即入度为 0）的顶点并且输出它；

（2）从图中删去该顶点,并且删去从该顶点发出的全部有向边；

（3）重复上述两步,直到剩余的网中不再存在没有前趋的顶点为止。

对于此有向图进行拓扑排序所有序列为：3,1,4,6,2,5 和 3,1,4,2,6,5。所以选 D

6．图中有关路径的定义是（　　）。[北方交通大学 2001 研]

A．由顶点和相邻顶点构成的边所形成的序列　B．由不同顶点所形成的序列

C．由不同边所形成的序列　D．上述定义都不是

【答案】A 查看答案

【解析】顶点 V_p 到顶点 V_q 之间的一条路径是指顶点序列 V_p，V_{i1}，，…，V_{im}，V_q。路径上边的数目称为路径的长度。

7．一个 n 个顶点的连通无向图，其边的个数至少为（　　）。

A．n-1　B．n　C．n+l　D．nlogn

【答案】A 查看答案

【解析】连通无向图边数最少时，任意两个顶点之间都不存在回路。此时用一条线将 n 的顶点连接，边数最少为 n-1。

8．下列说法不正确的是（　　）。[青岛大学 2002 研]

A．图的遍历是从给定的源点出发每个顶点仅被访问一次

B．遍历的基本方法有两种：深度遍历和广度遍历

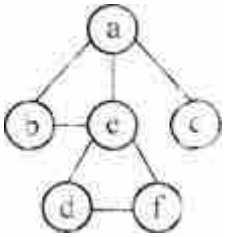
C．图的深度遍历不适用于有向图

D．图的深度遍历是一个递归过程

【答案】C 查看答案

【解析】图的遍历是指从图中的某一个顶点出发，按照某种搜索算法沿着图中的边对图中的所有顶点访问一次且仅访问一次。图的深度遍历类似于树的先序遍历，不仅适合无向图，也适合于有向图。

9．设如图所示，在下面的 5 个序列中，符合深度优先遍历的序列有（　　）个。[南京理工大学 2000 研]



a e b d f c　a c f d e b　a e d f c b　a e f d c b　a e f d b c

A．5 个　B．4 个　C．3 个 D．2 个

【答案】D 查看答案

【解析】深度优先遍历过程：从图中某个初始顶点 v 出发，首先访问初始顶点 v，然后选择一个与顶点 v 相邻且没被访问过的顶点 w 为初始顶点，再从 w 出发进行深度优先搜索，直到图中与当前顶点 v 邻接的所有顶点都被访问过为止。深度优先遍历过程是个递归过程。所给 5 个序列中只有（1）和（5）是正确的序列。

10．在图采用邻接表存储时，求最小生成树的 Prim 算法的时间复杂度为（　　）。[合肥工业大学 2001 研]

A．O（n）　B．O（n+e）　C．O（n^2）　D．O（n^3）

【答案】B 查看答案

【解析】邻接表表示法就是对图 G 中的每个顶点 v_i 建立一个单链表，第 i 个单链表中的结点表示依附于顶点 v_i 的边，这个单链表称为顶点 v_i 的边表。Prim 算法思想：假设 N=（V，{E}）是连通网，TE 是 N 上最小生成树中边的集合。算法从 U={u_0}（u_0∈V），TE=ψ 开始，重复执行下述操作：在所有 u∈U，v∈V-U 的边（u，v）∈E 中找一条代价最小的边（u_0，v_0）并入集合 TE，同时 v_0 并入 U，直至 U=V 为止。此时 TE 中必有 n-1 条边，则 T=（V，{TE}）为 N 的最小生成树。

11．在用邻接表表示图时，拓扑排序算法时间复杂度为（　　）。[合肥工业大学 2000 研；南京理工大学 2001 研；青岛大学 2002 研]

A．0（n）　B．0（n+e）　C．0（n*n）　D．O（n*n*n）

【答案】B 查看答案

【解析】由于输出每个顶点的同时还要删除以它为起点的边，古拓扑排序的时间复杂度为0（n+e）

12．下列关于 AOE 网的叙述中，不正确的是（　　）。[哈尔滨工业大学 2004 研]

A．关键活动不按期完成就会影响整个工程的完成时间

B．任何一个关键活动提前完成，那么整个工程将会提前完成

C．所有的关键活动提前完成，那么整个工程将会提前完成

D．某些关键活动若提前完成，那么整个工程将会提前完成

【答案】B 查看答案

【解析】关键路径是指从有向图的源点到汇点的最长路径。某些关键活动提前完成，那么整个工程将会提前完成，但不是任何一个关键活动提前完成，就能保证整个工程将会提前完成。

二、填空题

1．在一个无向图的的邻接表中，若表结点的个数是 m，则图中边的条数是____条。[西安电子科技大学 2003 研]

【答案】m / 2 查看答案

【解析】对于无向图，在邻接表中，如果存在 n 条边，则会有 2n 个表结点。

2．具有 10 个顶点的无向图，边的总数最多为____。[华中理工大学 2000 研]

【答案】45 查看答案

【解析】当无向图是无向完全图时，边的条数最多，设顶点的数目为 n，则边最多有 n（n-1）/2。

3.G 是一个非连通无向图，共有 28 条边，则该图至少有____个顶点。[西安电子科技大学 2001 研]

【答案】9 查看答案

4．在有 n 个顶点的有向图中，每个顶点的度最大可达____。[中南大学 2002 研]

【答案】2（n-1）查看答案

【解析】当有向图为完全连通图时每个顶点的度达到最大，出度入度均为 n-1。

5．如果具有 n 个顶点的图是一个环，则它有____棵生成树。[中南大学 2005 研]

【答案】n 查看答案

6.N 个顶点的连通图用邻接矩阵表示时，该矩阵至少有____个非零元素。

【答案】2（N-1）查看答案

【解析】连通图一定指的是无向图，有向图会称作强连通图。连接 N 个顶点，至少需要 N-1 条边就可以了。由于无向图的每一条边同时关联了两个顶点。因此用邻接矩阵表示时，该矩阵至少有 2（N-1）个非零元素。

7．为了实现图的广度优先搜索，除了一个标志数组标志已访问的图的结点外，还需____以存放被访问的结点以实现遍历。

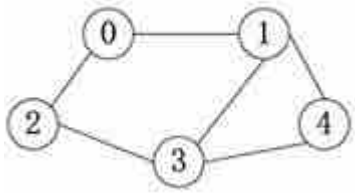
【答案】队列查看答案

8．克鲁斯卡尔算法的时间复杂度为____，它对____图较为适合。

【答案】O（eloge）；边稀疏查看答案

三、综合题

1．已知有 5 个顶点的图 G 如下图所示



请回答下列问题：

（1）写出图 G 的邻接矩阵 A（行、列下标从 0 开始）。

（2）求 A^2，矩阵 A^2 中位于 0 行 3 列元素值的含义是什么？

（3）若已知具有 n（n>=2）个顶点的邻接矩阵为 B，则 B^m（2<=m<=n）非零元素的含义是什么？[2015 年联考真题]

答：（1）邻接矩阵为

$$\begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 0 \end{bmatrix}$$

（2）A^2 为：

$$\begin{bmatrix} 2 & 0 & 0 & 2 & 1 \\ 0 & 3 & 2 & 1 & 1 \\ 0 & 2 & 2 & 0 & 1 \\ 2 & 1 & 0 & 3 & 1 \\ 1 & 1 & 1 & 1 & 2 \end{bmatrix}$$

0 行 3 列的元素的含义是顶点 0 到顶点 3 间是相通的，并且路径长度为 2 的路径有 2 条。

（3）B^m 中非零元素的含义是：假设此顶点位于 i 行 j 列，表示从 i 结点到 j 结点路径长度为 m 的路径的条数。

2．解答问题。设有数据逻辑结构为：

B=（K，R），K={k_1，k_2，…，k_9}

R={<k_1，k_3>，<k_1，k_8>，<k_2，k_3>，<k_2，k_4>，<k_2，k_5>，<k_3，k_9>，<k_5，k_6>，<k_8，k_9>，<k_9，k_7>，<k_4，k_7>，<k_4，k_6>）

（1）画出这个逻辑结构的图示。（3 分）

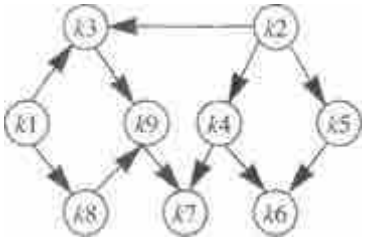
（2）相对于关系 R，指出所有的开始结点和终端结点。（2 分）

（3）分别对关系 R 中的开始结点，举出一个拓扑序列的例子。（4 分）

（4）分别画出该逻辑结构的正向邻接表和逆向邻接表。（6 分）[山东工业大学 1999 研]

答：

（1）



（2）开始结点（入度为 0）：K_1、K_2，终端结点（出度为 0）：K_6、K_7。

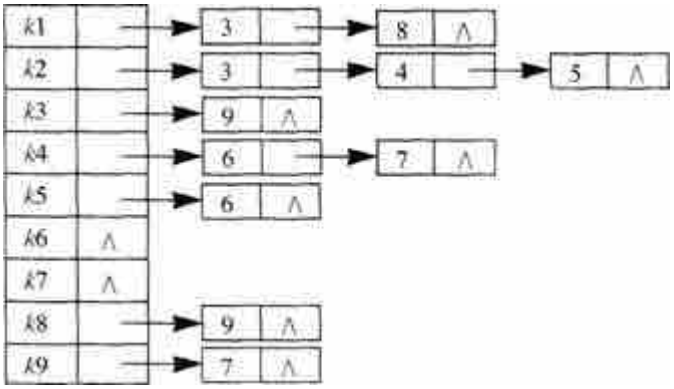
（3）拓扑序列：

K_1，K_2，K_3，K_4，K_5，K_6，K_8，K_9，K_7

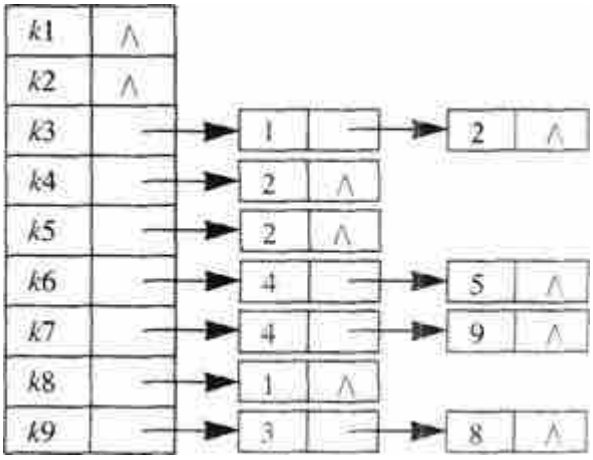
K_2，K_1，K_3，K_4，K_5，K_6，K_8，K_9，K_7

规则：开始结点为 K_1 或 K_2，之后，若遇多个入度为 0 的顶点，按顶点编号顺序选择。

（4）

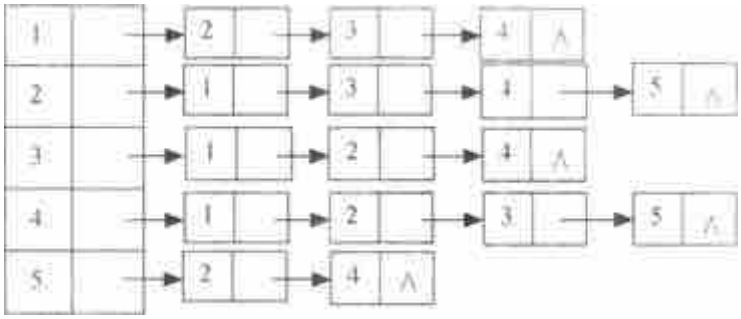


邻接表



逆邻接表

3．设 G=（V，E）以邻接表存储，如图所示，试画出图的深度优先生成树和广度优先生成树。

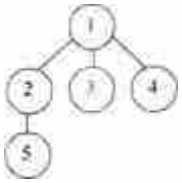


答：设从顶点 1 开始遍历，则深度优先生成树（1）和广度优先生成树（2）为：

（1）



（2）



4．令 G=（V，E）为一个有向无环图，编写一个给图 G 中每一个顶点赋以一个整数序号的算法，并满足以下条件：若从顶点 i 至顶点 j 有一条弧，则应使 i<j。[清华大学 1996 研]

答：

```
void TopNo(AdjList g)
//对以邻接表存储的DAG图g重新编号,使若有<i,j>,则编号i<j
{int new[MAXSIZE],indegree[n]={0},count=0;
 for(i=0;i<n;i++)
    {p=g[i].firstarc;
     while(p) {indegree[p->adjvex]++; p=p->next;} //求各顶点的入度
    }
 StackInit (S);
 for(i=0; i<n; i++) if(!indegree[i]) Push(S,i);
 while(!StackEmpty(S))
   {Pop(S,i);
    new[i]=n--;  //记录结点的逆序序号
    count++;
    for(p=G.vertices[i].firstarc; p; p=p->nextarc)
       {k=p->adjvex; if(!(--indegree[k])) Push(S,k); }
   }//while
 if(count<G.vexnum) return ERROR;
 for(i=1; i<=n; i++) printf("Old No:%d New No %d\n",i,new[i])
 return OK;
}
```

第 8 章　动态存储管理

8.1　复习笔记

一、动态存储管理概述

数据结构中的每个数据元素都占有一定的内存位置，在程序执行的过程中，数据元素的存取是通过对应的存储单元来进行的。

动态存储管理的基本问题是系统如何应用户提出的“请求”分配内存，又如何回收那些用户不再使用而“释放”的内存，以备新的“请求”产生时重新进行分配。

随着用户进入系统，先后提出存储请求，系统则依次进行分配。经过一段时间之后，有的用户运行结束，所占用的内存区变成空闲块，这就使整个内存区呈现占用快与空闲块犬牙交错的状态。假如此时又有新的用户进入系统请求分配内存，此时系统分配内存有两种策略：

（1）一种策略是系统继续从高地址的空闲块中进行分配，而不理会已分配给用户的内存区是否已空闲，直到分配无法进行（即剩余的空闲块不能满足分配的请求）时，才去回收所

有用户不再使用的空闲块，并且重新组织内存，将所有空闲的内存区连接在一起形成一个大的空闲块；

（2）一种策略是用户一旦运行结束，便将它所占内存区释放成为空闲块，同时，每当新的用户请求分配内存时，系统需要巡视整个内存区中所有空闲块，并从中找出一个“合适”的空闲块分配之。由此，系统需建立一张记录所有空闲块的“可利用空间表”，此表的结构可以是“目录表”，也可以是链表。

二、可利用空间表及分配方法

1．可利用空间表

可利用空间表中包含所有可分配的空闲块，每一个空闲块是链表中的一个结点。当用户请求分配时，系统从可利用空间表中删除一个结点分配之；当用户释放其所占内存时，系统回收并将它插入到可利用空间表中。因此，可利用空间表亦称作“存储池”。

根据系统运行的不同情况，可利用空间表可以有下列三种不同的结构形式。

（1）系统运行期间所有用户请求分配的存储量大小相同。

（2）系统运行期间用户请求分配的存储量有若干种大小的规格。

（3）系统在运行期间分配给用户的内存块的大小不固定，可以随请求而变。因此，可利用空间表中的结点即空闲块的大小也是随意的。

2．可利用空间表的分配策略

由于可利用空间表中的结点大小不同，则在分配时就存在一个如何分配的问题。假设某用户需大小为 n 的内存，而可利用空间表仅有大小为 $m \geq n$ 的空闲块，则只需将其中大小为 n 的一部分分配给申请分配的用户，同时将剩余大小为 m-n 的部分作为一个结点留在链表中即可。

（1）三种不同的分配策略

①首次拟合法

从表头指针开始查找可利用空间表，将找到的第一个大小不小于 n 的空闲块的一部分分配给用户。

②最佳拟合法

将可利用空间表中一个不小于 n 且最接近 n 的空闲块的一部分分配给用户。

③最差拟合法

将可利用空间表中不小于 n 且是链表中最大的空闲块的一部分分配给用户。

（2）三种分配策略的比较

①从适用范围上比较，最佳拟合法适用于请求分配的内存大小范围较广的系统，最差拟合法适用于请求分配的内存大小范围较窄的系统，而首次拟合法的分配是随机的，因此它介于两者之间，通常适用于系统事先不掌握运行期间可能出现的请求分配和释放的信息的情况；

②从时间上来比较，首次拟合在分配时需查询可利用空间表，而回收时仅需插入在表头即可；最差拟合恰相反，分配时无需查询链表，而回收时为将新的“空闲块”插入在链表中适当的位置上，需先进行查找；最佳拟合无论分配和回收，均需查找链表，因此最费时间。

三、边界标识法

边界标识法（boundary tag method）是操作系统中用于进行动态分区分配的一种存储管理方法。系统将所有的空闲块链接在一个双重循环链表结构的可利用空间表中，分配可按首次拟合进行，也可按最佳拟合进行。

1．可利用空间表的结构

可利用空间表中的结点结构如图 8-1 所示。

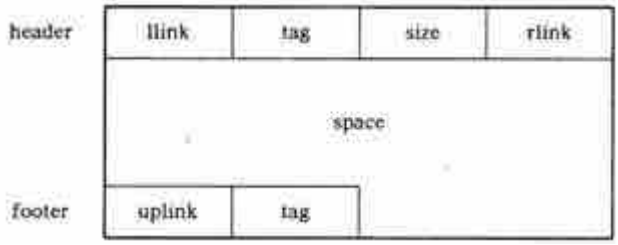


图 8-1 可利用空间表的结点结构

它表示一个空闲块。整个结点由三部分组成：header，space，footr。可利用空间表为双重循环链表。

2．分配算法

分配算法比较简单，假设采用首次拟合法进行分配，则只要从表头指针 pav 所指起点起，在可利用空间表中进行查找，找到第一个容量不小于请求分配的存储量（n）的空闲块时即可进行分配。为了使整个系统更有效地运行，边界标识法还作了如下两条规定：

（1）假设找到的此块待分配的空闲块的容量为 m 个字，链表中会出现一些容量极小总也分配不出去的空闲块，这就大大减慢了分配（查找）的速度。弥补的办法是，选定一个适当的常量 e，当 m-n≤e 时，就将容量为 m 的空闲块整块分配给用户；反之，只分配其中 n 个字的内存块。同时，为了避免修改指针，约定将该结点中的高地址部分分配给用户。

（2）如果每次分配都从同一个结点开始查找，会增加查询较大空闲块的时间。弥补的方法是，在每次分配之后，令指针指向刚进行过分配的结点的后继结点。

3．回收算法

一旦用户释放占用块，系统需立即回收以备新的请求产生时进行再分配。为了使物理地址相邻的空闲块结合成一个尽可能大的结点，则首先需要检查刚释放的占用块的左、右紧邻是否为空闲块（每个内存区（无论是占用块或空闲块）的边界上都设有标志值，标志值为 0 表示空闲块，为 1 表示占用块）。具体分为四种情况：

（1）释放块的左、右邻区均为占用块。只要将此新的空闲块作为一个结点插入到可利用空闲表中任何位置均可，简单的做法就是插入在 pav 指针所指结点之前（或之后）。

（2）释放块的左邻区为空闲块，而右邻区为占用块。只要改变左邻空闲块的结点：增加结点的 size 域的值且重新设置结点的底部即可。

（3）释放块的右邻区为空闲块，而左邻区为占用块。当表中结点由原来的右邻空闲块变成合并后的大空闲块时，结点的底部位置不变，但头部要变，故链表中的指针也要变。

（4）释放块的左、右邻区均为空闲块。为使三个空闲块连在一起成为一个大结点留在可利用表中，只要增加左邻空闲块的 space 容量，同时在链表中删去右邻空闲块结点即可。

在上述后三种的情况下，可利用空间表的变化如图 8-2 所示：

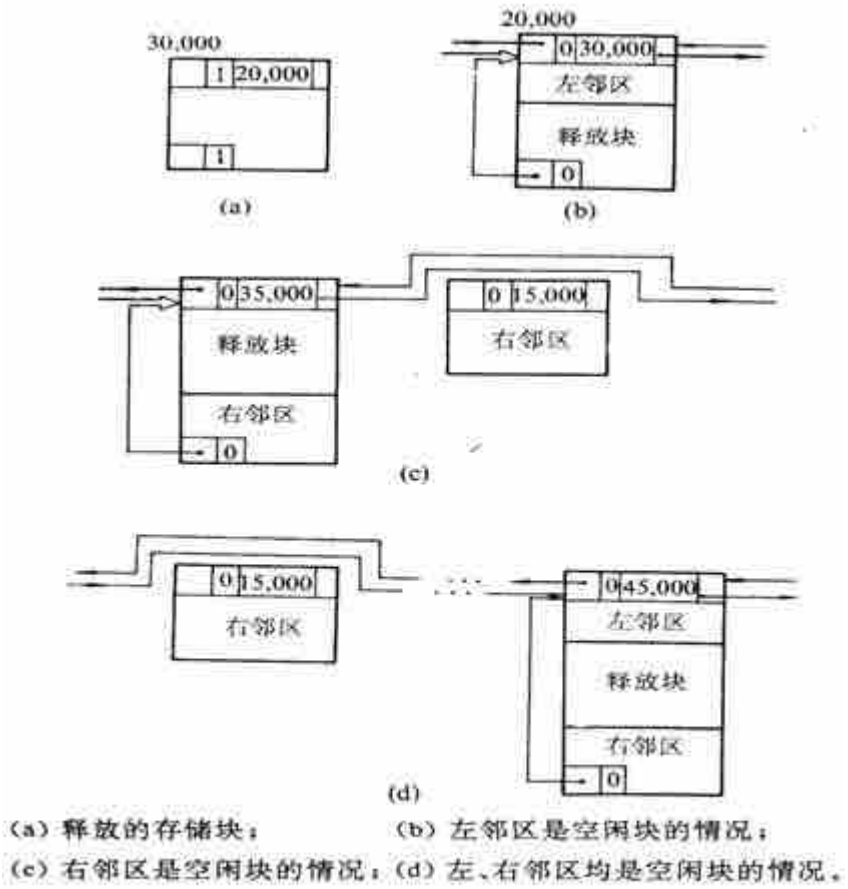


图 8-2 回收存储块后的可利用空间表

四、伙伴系统

伙伴系统（buddy system）是操作系统中用到的另一种动态存储管理方法。

它和边界标识法类似，在用户提出申请时，分配一块大小“恰当”的内存区给用户；反之，在用户释放内存区时回收。

伙伴系统和边界标识法不同的是：在伙伴系统中，无论是占用块或空闲块，其大小均为 2 的 k 次幂（k 为某个正整数）。

1．可利用空间表的结构

双重链表中的结点结构如图 8-3（a）所示，可利用空间表的初始状态如图 8-3（b）所示。

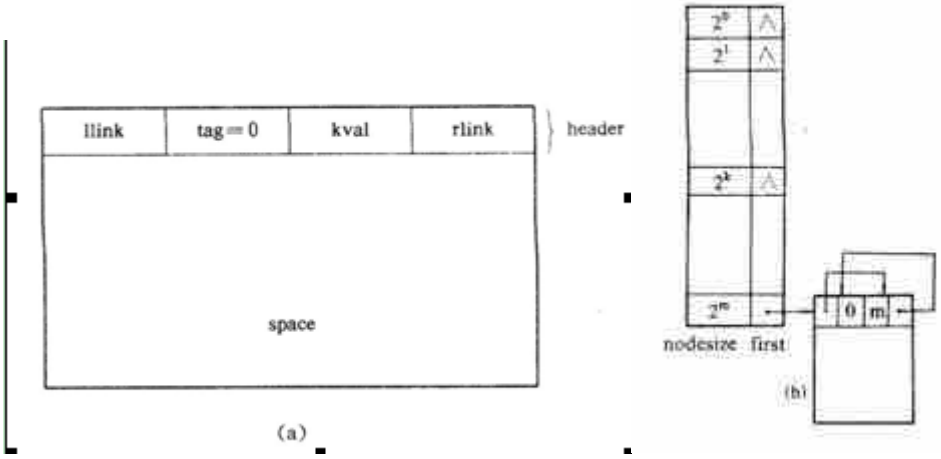


图 8-3 可利用空间表的结构和初始状态

2．分配算法

当用户提出大小为 n 的内存请求时，首先在可利用空间表上寻找结点大小与 n 相匹配的子表，若此子表非空，则将子表中任意一个结点分配之；若此子表为空，则需从结点更大的非空子表中去查找，直至找到一个空闲块，则将其中一部分分配给用户，而将剩余部分插入相应的子表中。

3．回收算法

（1）伙伴系统

在用户释放不再使用的占用块时，系统需将这新的空闲块插入到可利用空间表中去。这里，同样有一个地址相邻的空闲块归并成大块的问题，但在伙伴系统中仅考虑互为“伙伴”的两个空闲块的归并。

在分配时经常需要将一个大的空闲块分裂成两个大小相等的存储区，这两个由同一大块分裂出来的小块就称之“互为伙伴”。在伙伴系统中回收空闲块时，只当其伙伴为空闲块时才归并成大块。

（2）伙伴系统的优缺点

伙伴系统的优点是算法简单、速度快；缺点是只归并伙伴容易产生碎片。

五、无用单元收集

1．无用单元的定义

用户的疏漏或结构本身的原因致使系统在不恰当的时候或没有进行回收而产生“无用单元”或“悬挂访问”的问题。“无用单元”是指那些用户不再使用而系统没有回收的结构和变量。如果所释放的结点被再分配而继续访问指针所指结点，这种访问被称为“悬挂访问”。

“无用单元”指的是那些用户不再使用而系统没有回收的结构和变量。

2．解决结点无法再分配的途径

（1）使用访问计数器

（2）收集无用单元

3．无用单元的收集

收集无用单元应分两步进行：第一步是对所有占用结点加上标志；第二步是对整个可利用存储空间顺序扫描一遍，将所有标志域为“0”的结点链接成一个新的可利用空间表。

4．三种标志算法

（1）递归算法中加标志的操作实质上是遍历广义表，将广义表中所有结点的标志域赋值“l”。

（2）非递归算法程序中附设栈（或队列）实现广义表的遍历。

（3）利用已经标志过的表结点中的 tag、hp 和 tp 域来代替栈记录遍历过程中的路径。

5．三种算法的比较

第三种算法在标志时不需要附加存储，动态分配的可利用空间得到充分利用。但是由于几乎是每个表结点的指针域的值都要作两次改变，因此时间上的开销相当大。而且，一旦发生中断，整个系统瘫痪，无法重新启动运行。

而第二种非递归算法操作简单，时间上要比第三种算法省得多，然而它需要占有一定空间，使动态分配所用的存储量减少。总之，无用单元收集是很费时间的，不能在实时处理的情况下应用。

六、存储紧缩

1．存储紧缩的任务

空间始终是一个地址连续的存储块，回收时必须将所释放的空闲块合并到整个堆上去才能重新使用，这是指“存储紧缩”的任务。通常有两种做法：

（1）一旦有用户释放存储块即进行回收紧缩；

（2）在程序执行过程中不回收用户随时释放的存储块，直到可利用空间不够分配或堆指针指向最高地址时才进行存储紧缩。

2．实现方法

为实现存储紧缩，首先要对占用块进行“标志”，其次需进行下列四步操作：

（1）计算占用块的新地址。从最低地址始巡查整个存储空间，对每一个占用块找到它在紧缩后的新地址。

（2）修改用户的初始变量表，以便在存储紧缩后用户程序能继续正常运行。

（3）检查每个占用块中存储的数据。若有指向其它存储块的指针，则需作相应修改。

（4）将所有占用块迁移到新地址去。这实质上是作传送数据的工作。

8.2 强化习题详解

【基础知识题】

1．假设利用边界标识法首次适配策略分配,已知在某个时刻的可利用空间表的状态如图 8-1 所示:

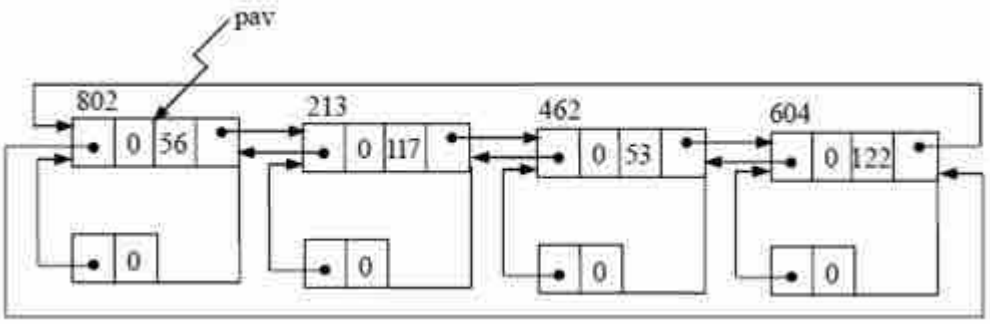


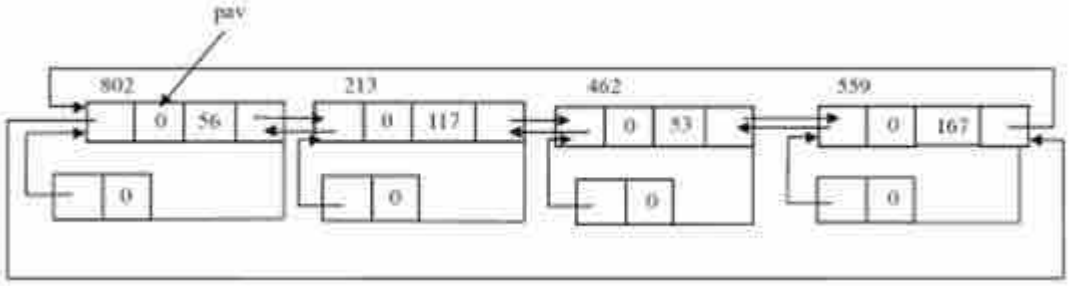
图 8-1

（1）画出当系统回收一个起始地址为 559,大小为 45 的空闲块之后的链表状态；

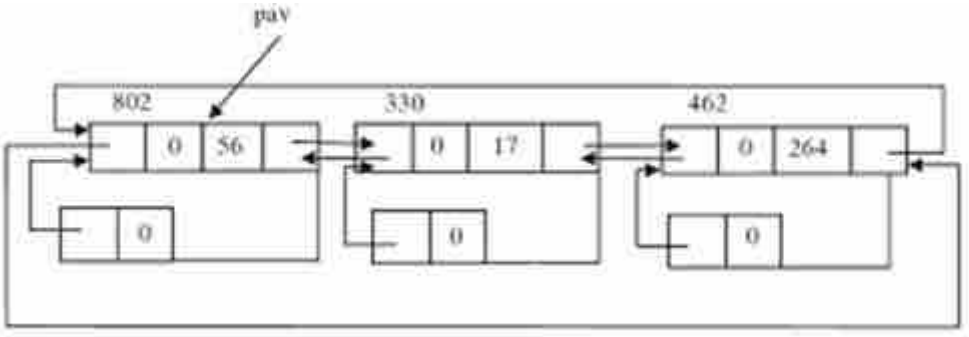
（2）画出系统继而在接受存储块大小为 100 的请求之后,又回收一块起始地址为 515，大小为 44 的空闲块之后的链表状态。

注意：存储块头部中大小域的值个申请分配的存储量均包括头和尾的存储空间。

答：①将原图中第四块的起始地址 604、大小 122 分别改为 559 和 167。



②将原图中第二块的始址和大小分别改为 330 和 17；再将后两块合为一块，始址和大小分别为 462 和 264。



2．组织成循环链表的可利用空间表可附加什么条件时,首次适配策略就转变为最佳适配策略？

答：可利用空间表中的空闲块按由小到大的顺序有序，且头指针恒指最小的空闲块时，首次适配策略就转变为最佳适配策略。

3．设两个大小分别为 100 和 200 的空闲块依次顺序链接成可利用空间表。设分配一块时,该块的剩余部分在可利用空间表中保持原链接状态，试分别给出满足下列条件的申请序列：

（1）最佳适配策略能够满足全部申请而首次适配策略不能；

（2）首次适配策略能够满足全部申请而最佳适配策略不能。

答：可列举出多种实例，如：

（1）110，80，100

（2）110，80，90，20

4．在变长块的动态存储管理方法中,边界标志法的算法效率为什么图 8-2 所示的结点结构组织的可利用空间表的算法效率高？

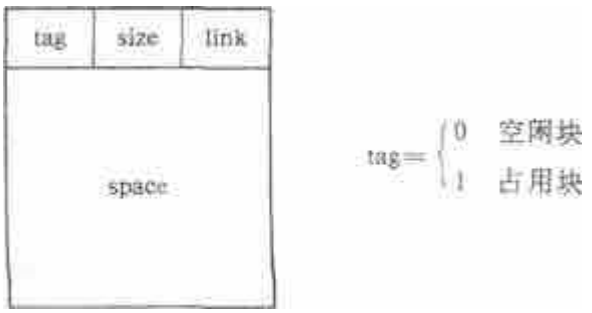


图 8-2

答：（1）边界标识法由于在每个结点头部和底部设立了标识域，使得在回收用户释放的内存块时，很容易判别与它相邻的内存区是否是空闲块，且不需要查询整个可利用空间表便能找到相邻的空闲块并与其合并；

（2）由于可利用空间表上的结点既不需结点大小有序，也不需结点地址有序，所以释放块插入时也不需查找链表。因此，可利用空间表的算法效率高。

5．考虑边界标志法的两种策略（最佳适配和首次适配）：

（1）数据结构的主要区别是什么？

（2）分配算法的主要区别是什么？

（3）回收算法的主要区别是什么？

答：（1）最佳适配策略下空闲块要按由小到大的顺序链接，可以不作成循环表。空闲块表头指针恒指最小空闲块。首次分配则力求使各种大小的块在循环表中均匀分布，所以经常移动头指针；

（2）分配算法无本质区别；

（3）最佳适配策略下（合并后）插入链表时必须保持表的有序性。

6．二进制地址为 011011110000，大小为（4）$_{10}$ 的块的伙伴的二进制地址是什么？若块大小为（16）$_{10}$ 时又如何？

答：（1）011011110100

（2）011011100000

7．已知一个大小为 512 字的内存，假设先后有 6 个用户提出大小分别为 23，45，52，100，11 和 19 的分配请求，此后大小为 45，52 和 11 的占用块顺序被释放。假设以伙伴系统实现动态存储管理，

（1）画出可利用空间表的初始状态；

（2）画出 6 个用户进入之后的链表状态以及每个用户所得存储块的起始地址；

（3）画出在回收三个用户释放的存储块之后的链表状态。

答：（1）只有一个大小为 2^9 的块；可利用空间表的初始状态如图 8-3 所示：

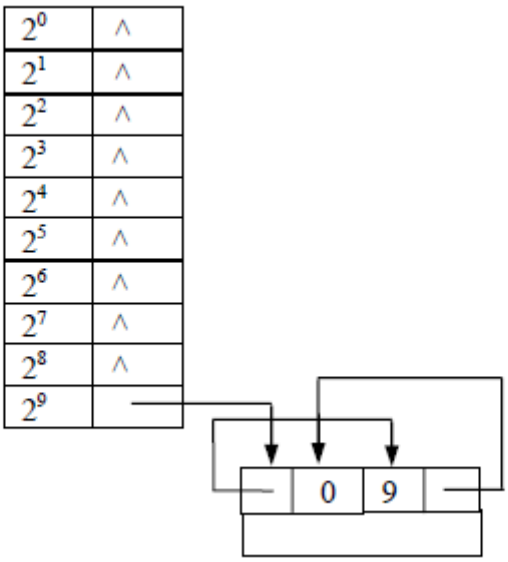


图 8-3

（2）大小为 2 的 4 次方，2 的 5 次方和 2 的 7 次方各一块。为 6 个用户分配所需要的存储空间后可利用空间表的状态如图 8-4 所示：

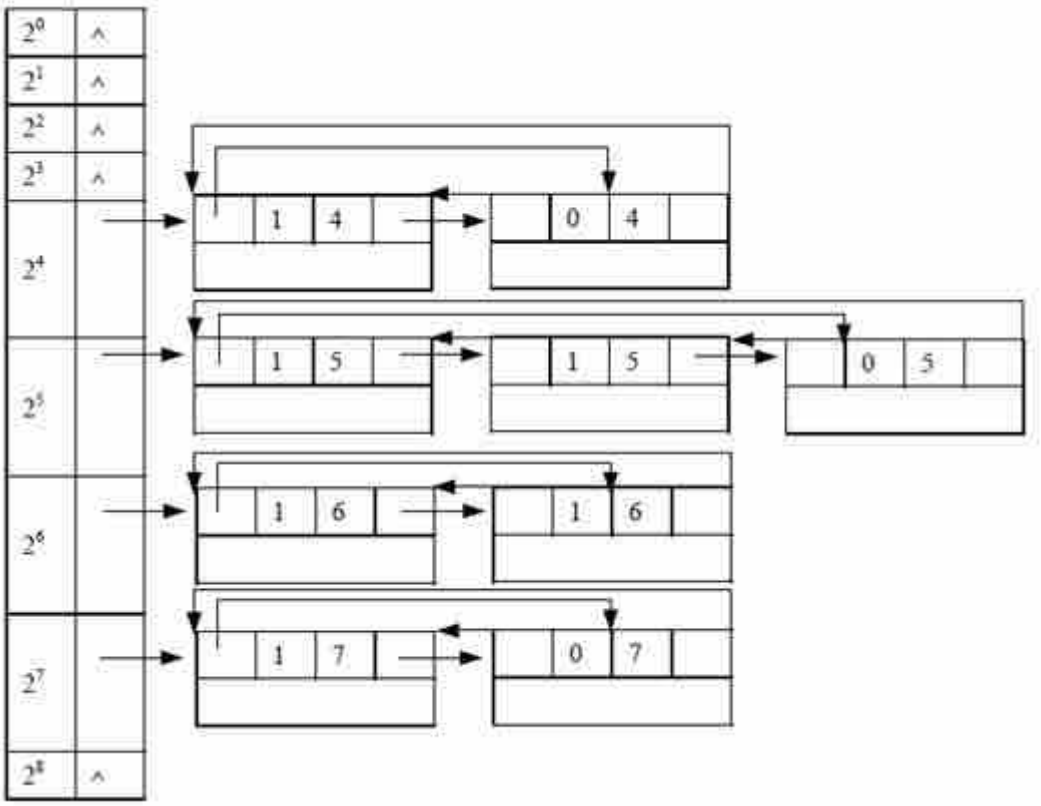


图 8-4

（3）大小为 2 的 5 次方和 2 的 6 次方的块各两块，2 的 7 次方的块 1 块。每个用户得到的存储空间的起始地址如表 8-1 所示：

表 8-1

存储大小	起始地址
23	0
45	64
52	128
100	256
11	32
19	192

（4）在回收 3 个占用块之后可利用空间表的状态如图 8-5 所示。

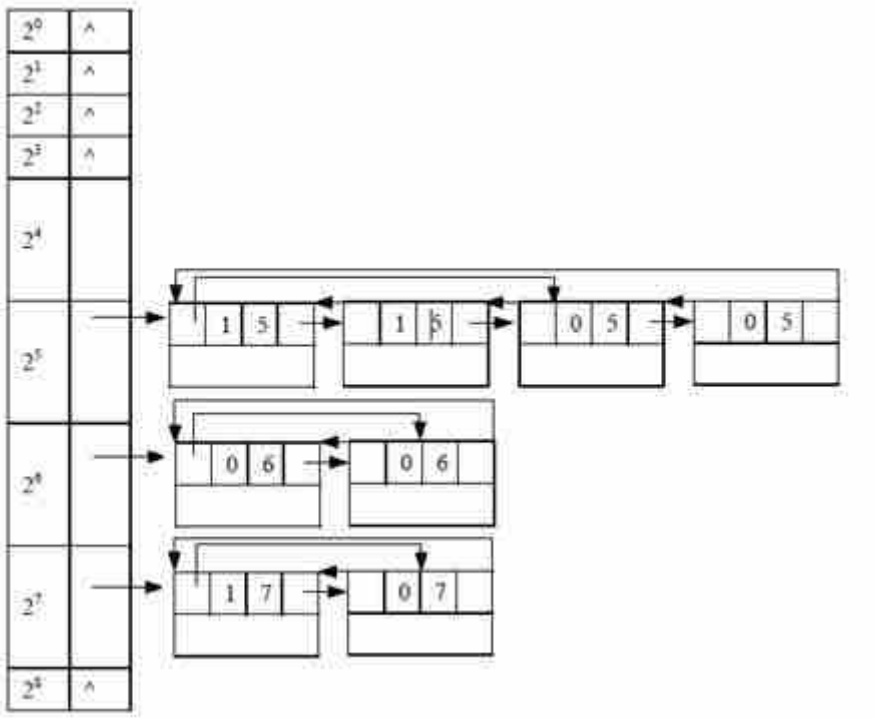


图 8-5

8．试求一个满足以下条件的空间申请序列 a1，…a2，…，an:从可用空间为 25 的伙伴管理系统的初始状态开始，a1, …a2, …,an—1 均能满足，而 an 不能满足，并使 $\sum_{i=1}^{n} a_i$ 最小。

答：（2^{5-1}+1，1）或（1，2^{5-1}+1）

9．设有五个广义表：L=（L_1，L_3），L=（L_2，L_3，L_4），L_2=（L_3），L_3=（ ），L_4=（L_2）。若利用访问计数器实现存储管理，则需对每个表或子表添加一个表头结点，并在其中设一计数域。

（1）试画出表 L 的带计数器的存储结构；

（2）从表 L 中删除子表 L1 时,链表中哪些结点可以释放？各子表的计数器域怎样改变？

（3）若 L_2=（L_3，L_4），将会出现什么现象？

答：

（1）如图 8-6 所示：

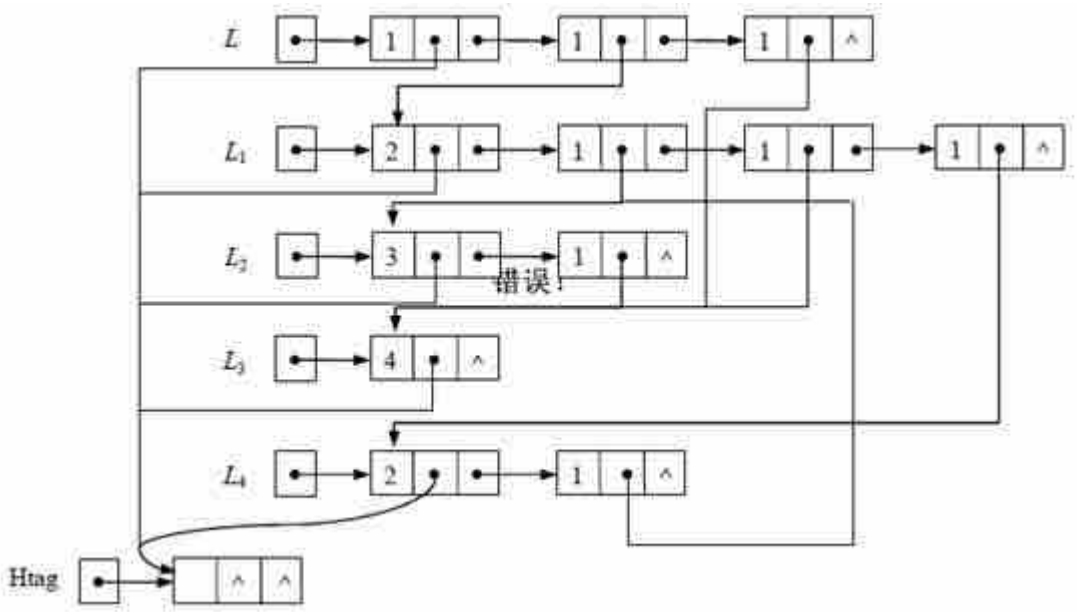


图 8-6

（2）表 L 中首元结点可以释放，即将它从链表中删除，将表 L_1 的头结点计数域减 1。

（3）形成间接递归。若不在实现时进行递归处理，当删除 L_2 时会出现空间不能回收的不一致现象。

【算法设计题】

10．考虑空间释放遵从“最后分配者最先释放”规则的动态存储管理问题，并设每个空间申请中都指定所申请的空闲块大小。

（1）设计一个适当的数据结构实现动态存储管理；

（2）写一个为大小为 n 的空间申请分配存储块的算法。

答：

（1）数据结构的设计

```
typedef    struct    {
    char    *start;
    int    size;
}fmblock;//空闲块类型
```

（2）申请分配存储块

```
char *Malloc_Fdlf（int    n）//遵循最后分配者最先释放规则的内存分配算法
{
while（Gettop（S,b）&&b.size<n）
{
    Pop（S,b）;
    Push（T,b）;//从栈顶逐个取出空闲块进行比较
}
if（StackEmpty（S））    return    NULL; //没有大小足够的空闲块
Pop（S,b）;
b.size-=n;
if（b.size）    Push（S,{b.start+n,b.size}）;//分割空闲块
while（!StackEmpty（T））  //如果栈不空，先出栈再入栈。恢复原来次序
```

```
{
   Pop（T,a）;
   Push（S,a）;
} return    b.start;
}//Malloc_Fdlf
mem_init（）//初始化过程
{
...
InitStack（S）;InitStack（T）;//S 和 T 的元素都是 fmblock 类型
Push（S,{MemStart,MemLen}）;//一开始，栈中只有一个内存整块
...
}//main
```

11．同 11 题条件,写一个回收释放块的算法。

答：

```
void    Free_Fdlf（char    *addr,int    n）//与上一题对应的释放算法
{
while（Gettop（S,b）&&b.start<addr）
{
   Pop（S,b）;
   Push（T,b）;
}//在按地址排序的栈中找到合适的插入位置
if（Gettop（T,b）&&（b.start+b.size==addr））//可以与上邻块合并
{
   Pop（T,b）;
   addr=b.start;n+=b.size;//设置新的空闲块大小
}
if（Gettop（S,b）&&（addr+n==b.start））//可以与下邻块合并
{
   Pop（S,b）;
   n+=b.size; //设置新的空闲块大小
}
Push（S,{addr,n}）;//插入到空闲块栈中
while（!StackEmpty（T））  //恢复原来次序
{
   Pop（T,b）;
   Push（S,b）;
}
}//Free_Fdlf
```

12．试完成边界标志法和依首次适配策略进行分配相应的回收释放块的算法。

答：

```
void    Free_BT（Space    &pav,Space    p）/*在边界标识法的动态存储管理系统中回收空
闲块 p*/
{
n=p->size;
```

```
f=p+n-1;//f 指向空闲块底部
if（（p-1）->tag&&（f+1）->tag）//回收块上下邻块均为占用块
{
    p->tag=0;f->tag=0;
    f->uplink=p;
    if（!pav）
    {//pav 指针为空，只有一个结点
p->llink=p;
p->rlink=p;
    }
    else//否则插入在 pav 指针所指结点之前
    {
q=pav->llink;
p->llink=q;p->rlink=pav;
q->rlink=p;pav->llink=p;
    }
    pav=p;
}//if
else    if（!（p-1）->tag&&（f+1）->tag）//上邻块为空闲块
{
    q=（p-1）->uplink;
    q->size+=n;//增加结点的 size 域的值
    f->uplink=q;//重新设置结点底部
    f->tag=0;
}
else    if（（p-1）->tag&&!（f+1）->tag）//下邻块为空闲块
{
    q=f+1;
    s=q->llink;t=q->rlink;
    p->llink=s;p->rlink=t;
    s->rlink=p;t->llink=p;
    p->size+=q->size;//新的空闲块大小
     （q+q->size-1）->uplink=p;//底部指针指向新结点的头部
    p->tag=0;
}
else//上下邻块均为空闲块
{
    s=（p-1）->uplink;
    t=f+1;
    s->size+=n+t->size;//设置新结点中的 space 空间大小
    t->llink->rlink=t->rlink;
    t->rlink->llink=t->llink;
     （t+t->size-1）->uplink=s; //底部指针指向新结点的头部
}
```

```
}//Free_BT
```

13．试完成伙伴管理系统的存储回收算法。

答：

```
void    Free_BS（freelist &avail,char *addr,int n）//伙伴管理系统的空闲块回收算法
{
buddy=addr%（2*n）?（addr-n）:（addr+n）;//求回收块的伙伴地址
addr->tag=0;
addr->kval=n;
for（i=0;avail[i].nodesize<n;i++）;//找到这一大小的空闲块链
if（!avail[i].first）//尚没有该大小的空闲块
{
addr->llink=addr;
addr->rlink=addr;
avail[i].first=addr;//作为唯一一个该大小的空闲块
}
else
{
    for（p=avail[i].first;p!=buddy&&p!=avail[i].first;p=p->rlink）;//寻找伙伴
    if（p==buddy）//伙伴为空闲块,此时进行合并
    {
if（p->rlink==p）     avail[i].first=NULL;//伙伴是此大小的唯一空闲块
else
{
    p->llink->rlink=p->rlink;
    p->rlink->llink=p->llink;
}//从空闲块链中删去伙伴
new=addr>p?p:addr;//合并后的新块首址
Free_BS（avail,new,2*n）;//递归回收新块
    }//if
    else//伙伴为占用块,此时插入空闲块链的头部
    {
q=p->rlink;
p->rlink=addr;addr->llink=p;
        q->llink=addr;addr->rlink=q;
    }
}//else
}//Free_BS
```

14．设被管理空间的上下界地址分别由变量 lighbound 和 lowbound 给出，形成一个由同样大小的块组成的“堆”。试写出一个算法，将所有的 tag 域的值为 0 的块按始址递增顺序连接成一个可利用空间表（设块大小域为 cellsize）。

答：

```
FBList   *MakeList（char   *highbound,char   *lowbound）/*把堆结构存储的所有空闲块链接成可利用空间表，并返回表头指针*/
{
```

```
p=lowbound;
while（p->tag&&p<highbound）    p++;//查找第一个空闲块
if（p>=highbound）    return    NULL;//没有空闲块
head=p;
for（q=p;p<highbound;p+=cellsize）//建立链表
    if（!p->tag）//如果 p 所指结点为空闲块，插入到可利用空间表的末尾
    {
q->next=p;
q=p;
    }//if
p->next=NULL;
return    head;//返回头指针
}//MakeList
```

15．试完成存储紧缩算法。

答：

```
void    Mem_Contract（Heap    &H）//对堆 H 执行存储紧缩
{
q=MemStart;j=0;
for（i=0;i<Max_ListLen;i++）
    if（H.list[i].stadr->tag）//占用块
    {
s=H.list[i].length;
p=H.list[i].stadr;
for（k=0;k<s;k++）    *（q++）=*（p++）;//紧缩内存空间
H.list[j].stadr=q;
H.list[j].length=s;//紧缩占用空间表
j++;
    }
}//Mem_Contract
```

8.3　考研真题与典型题详解

一、选择题

1．动态存储管理系统中，通常可有（　　）种不同的分配策略。

A．1　B．2　C．3　D．4　E．5

【答案】C 查看答案

【解析】动态存储管理系统中有:①首次拟合法，从表头指针开始查找可利用空间表，将找到的第一个大小不小于 n 的空闲块的一部分分配给用户；②最佳拟合法，将可利用空间表中一个不小于 n 且最接近 n 的空闲块的一部分分配给用户。则系统在分配前首先要对可利用空间表从头到尾扫视一遍，然后从中找出一块不小于 n 且最接近 n 的空闲块进行分配；③最差拟合法，将可利用空间表中不小于 n 且是链表中最大的空闲块的一部分分配给用户。

二、填空题

1．起始地址为 480，大小为 8 的块，其伙伴块的起始地址是_____；若块大小为 32，则其伙伴块的起始地址为_____。

【答案】（1）480+8=488（480%2^{3+1}=O）　（2）480-32=448 查看答案

【解析】起始地址为 P，大小为 2^k 的内存块，其伙伴块的起始地址计算公式如下：

$$buddy(p,k)=\begin{cases}p+2^k & 如p\%2^{k+1}=0\\ p-2^k & 如p\%2^{k+1}=2^k\end{cases}$$

根据上述公式起始地址就为 488。

2．二进制地址为 011011110000，大小为（4）^{10} 和（16）^{10} 块的伙伴地址分别为_____。[上海大学 2002 研]

【答案】011011110100、011011100000 查看答案

【解析】011011110000 是块的起始地址，大小分别为（4）^{10} 和（16）^{10}，其伙伴块的起始地址计算公式如下：

$$buddy(p,k)=\begin{cases}p+2^k & 如p\%2^{k+1}=0\\ p-2^k & 如p\%2^{k+1}=2^k\end{cases}$$

当大小为 4 时，起始地址为 011011110000+0100。当大小为 16 时，起始地址为：011011110000-010000。

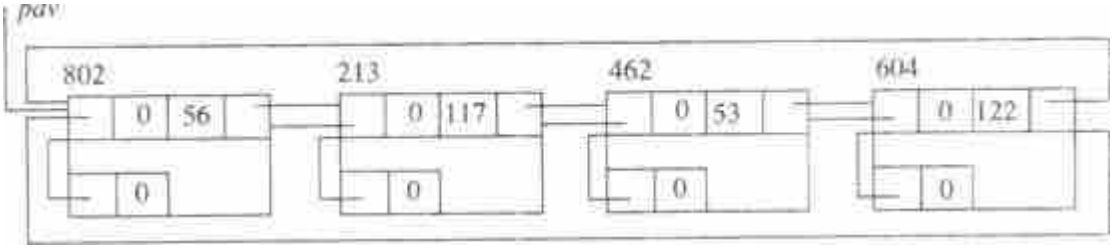
3．无用单元是指_____，例_____。

【答案】（1）用户不再使用而系统没有回收的结构和变量查看答案

（2）p=ma11oc（size）；…，p=nu11

三、综合题

假设利用边界标识法，并以首次拟合策略分配，已知在某个时刻可利用空间表的状态如图所示（注：存储块头部 size 域的值和申请分配的存储量均包括头部和尾部的存储空间））请画出：



（1）当系统回收一个起始地址为 559，大小为 45 的空闲块之后的链表状态；

（2）系统继而在接受存储块大小为 100 的请求后，又回收一个起始地址为 515，大小为 44 的空闲块之后的链表状态。

答：（1）系统回收一个起始地址为 559，大小为 45 的空闲块后，因右侧起始地址 604 为空闲块，应与之合并。合并后，成为起始地址为 559，大小为 167 的空闲块。链表状态如图（1）所示。

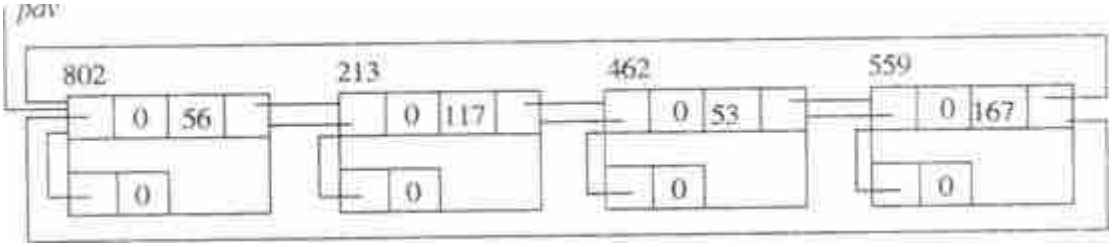
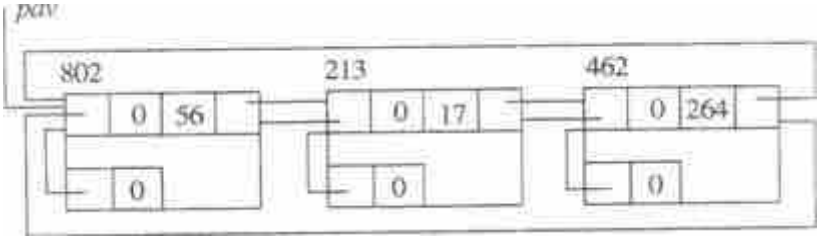


图 1

（2）系统在接受存储块大小为 100 的请求后，将大小为 117 的空闲块分出 100 给予用户。在回收一个起始地址为 515，大小为 44 的空闲块之后，因左侧起始地址为 462、大小为 53

和右侧起始地址为 559、大小为 167 均为空闲块，应与之合并。合并后，起始地址为 462、大小为 264 的空闲块。链表状态如图（2）所示。



图（2）

第 9 章　查　找

9.1　复习笔记

一、查找表

1．定义

查找表（table）是由同一类型的数据元素（或记录）构成的集合。

2．操作

对查找表经常进行的操作有：

（1）查询某个“特定的”数据元素是否在查找表中；

（2）检索某个“特定的”数据元素的各种属性；

（3）在查找表中插入一个数据元素；

（4）从查找表中删去某个数据元素。

3．查找表的分类

若对查找表只进行前两种操作统称为“查找”的操作，则称此类查找表为静态查找表（static search table）；若在查找过程中同时插入查找表中不存在的数据元素，或者从查找表中删除已存在的某个数据元素，则称此类表为动态查找表（dynamic search table）。

4．术语

（1）关键字（key）

关键字是数据元素（或记录）中某个数据项的值，用它可以标识该数据元素（或记录）。

（2）主关键字（primary key）与次关键字（secondary key）

若关键字可以唯一地标识一个数据元素（或记录），则称此关键字为主关键字（对不同的记录，其主关键字均不同）。反之，称用以识别若干记录的关键字为次关键字。当数据元素只有一个数据项时，其关键字即为该数据元素的值。

（3）查找（searching）

根据给定的某个值，在查找表中确定一个其关键字等于给定值的记录或数据元素。若表中存在这样一个记录，则称查找是成功的，此时查找的结果为整个记录的信息，或指示该记录在查找表中的位置；若表中不存在关键字等于给定值的记录，则称查找不成功，此时查找的结果给出一个“空”记录或“空”指针。

二、静态查找表

1．定义

数据元素具有相同的特性，各个数据元素均含有类型相同，可惟一标识数据元素的关键字。其结构为，n 个数据元素有相同属性，属于同一集合。

2．操作

静态查找表的操作有：

（1）Creat（&ST，n）

生成操作。构造一个含 n 个数据元素的静态查找表 ST。

（2）Destory（&ST）
销毁表 ST。
（3）Search（ST，K）
查找操作。若表 ST 中存在其关键字等于给定值 K 的数据元素，则函数值为该元素或它在表中的位置，否则函数值为“空”。
（4）Traverse（ST，Visit（））
输出操作。按某种次序对 ST 的每个元素调用函数 visit（）一次且仅一次。一旦 visit（）失败，则操作失败。
3．顺序表的查找
（1）查找过程
顺序查找（sequential search）的查找过程为：从表中第 n 个记录开始，逐个将记录的关键字和给定值进行比较，若某个记录的关键字和给定值相等，则查找成功，找到所查记录；反之，若直至第一个记录，其关键字和给定值比较都不等，则表明表中没有所查记录，查找不成功。
（2）查找操作的性能分析
通常以“其关键字和给定值进行过比较的记录个数的平均值”作为衡量查找算法好坏的已依据。
为确定记录在查找表中的位置，需和给定值进行比较的关键字个数的期望值称为查找算法在查找成功时的平均查找长度（average search length）。
对于含有 n 个记录的表，查找成功时的平均查找长度为：

$$ASL = \sum_{i=1}^{n} P_i C_i$$

其中，P_i 为查找表中第 i 个记录查找成功的概率，且

$$\sum_{i=1}^{n} P_i = 1$$

C_i 为找到表中其关键字与给定值相等的第 i 个记录时，和给定值已进行过比较的关键字个数。显然，C_i 随查找过程不同而不同。
顺序查找的平均查找长度为

$$ASL=np_1+（n-1）P_2+…+2P_{n-1}+P_n$$

假设每个记录的查找概率相等，即

$$P_i=1/n$$

则在等概率情况下顺序查找的平均查找长度是

$$ASL_{ss}=（n+1）/2$$

假设查找成功与不成功的可能性相同，对每个记录的查找概率也相同，则 $P_i=1/2n$，此时顺序查找不成功的平均查找长度为

$$ASL_{un} = \frac{1}{2n}\sum_{i=1}^{n}(n-i+1) + \frac{1}{2}(n+1)$$

$$= \frac{3}{4}(n+1)$$

4．有序表的查找
（1）折半查找

折半查找（Binary Search）的查找过程是：先确定待查记录所在的范围（区间），以处于区间中间位置记录的关键字和给定的进行比较，若相等则查找成功，若不相等，则逐步缩小范围直到找到或找不到该记录为止。

（2）折半查找的平均查找长度

对任意的 n，当 n 较大（n>50）时，可有下列近似结果：

$$ASL_{bs} = \log_2(n+1) - 1$$

（3）折半查找的性能分析

折半查找法在查找成功时进行比较的关键字个数最多不超过树的深度$\lfloor \log_2 n \rfloor + 1$，其中 n 表示判定树的结点个数即关键字个数；而查找不成功时和给定值进行比较的关键字个数最多也不超过$\lfloor \log_2 n \rfloor + 1$。

可见，折半查找的效率比顺序查找高，但折半查找只能适用于有序表，且限于顺序存储结构（对线性链表无法进行折半查找）。

5．静态树表的查找

（1）静态最优查找树

若只考虑查找成功的情况，则使查找性能达最佳的判定树是其带权内路径长度之和 PH=

$$\sum_{i=1}^{n} w_i h_i$$

取最小值的二叉树。其中，n 为二叉树上结点的个数（即有序表的长度）；h_i 为第 i 个结点在二叉树上的层次数；结点的权 $w_i=cp_i$（i=l，2，…，n），P_i 为结点的查找概率，c 为某个常量。称 PH 值取最小的二叉树为静态最优查找树（static optimal search tree）。

（2）次优查找树

构造静态最优树花费的时间代价高，现构造一棵二叉树，使这棵二叉树的带权内路径长度 PH 值在所有具有同样权值的二叉树中近似为最小，称这类二叉树为次优查找树（nearly optimal search tree）。

（3）两种查找树的比较

次优查找树和最优查找树的查找性能之差很小，但构造次优查找树的算法的时间复杂度为 O（nlogn），远低于最优查找树。

6．索引顺序表的查找

（1）索引顺序查找

若以索引顺序表表示静态查找表，则 Search 函数可用分块查找来实现。分块查找又称索引顺序查找，是顺序查找的一种改进方法。

（2）分块查找

分块查找过程需分两步进行：

①确定待查记录所在的块（子表）；

②在块中顺序查找。

（3）平均查找长度

由索引项组成的索引表按关键字有序，则确定块的查找可以用顺序查找或折半查找，而块中记录是任意排列的，因此在块中只能是顺序查找。分块查找的算法即为这两种查找算法的简单合成。

分块查找的平均查找长度为：

$$ASL_{bs}=L_b+L_w$$

其中，L_b为查找索引表确定所在块的平均查找长度，L_w为在块中查找元素的平均查找长度。

三、动态查找表

1．动态查找表

（1）动态查找表的特点

表结构本身是在查找过程中动态生成的，即对于给定值 key，若表中存在其关键字等于 key 的记录，则返回查找成功，否则插入关键字等于 key 的记录。

（2）动态查找表的操作

InitDSTable（&DT）

操作结果：构造一个空的动态查找表 DT。

DestroDSTable（&DT）

初始条件：动态查找表 DT 存在。

操作结果：销毁动态查找表 DT。

SearcDSTable（DT，key）

初始条件：动态查找表 DT 存在，key 为和关键字类型相同的给定值。

操作结果：若 DT 中存在其关键字等于 key 的数据元素，则函数值为该元素的值或在表中的位置，否则为“空”。

InsertDSTable（&DT，e）

初始条件：动态查找表 DT 存在，e 为待插入的数据元素。

操作结果：若 DT 中不存在其关键字等于 e.key 的数据元素，则插入 e 到 DT。

DeleteDSTable（&DT，key）

初始条件：动态查找表 DT 存在，key 为和关键字类型相同的给定值。

操作结果：若 DT 中存在与其关键字等于 key 的数据元素，则删除之。

TraverseDSTable（DT，Visit（））

初始条件：动态查找表 DT 存在，Visit 是对结点操作的应用函数。

操作结果：按某种次序对 DT 的每一个结点调用 Visit（）一次且至多一次。一旦 Visit（）失败，则操作失败。

2．二叉排序树和平衡二叉树

（1）二叉排序树的定义

二叉排序树（binary sort tree）或者是一棵空树，或者是具有下列性质的二叉树：

①若它的左子树不空，则左子树上所有结点的值均小于它的根结点的值；

②若它的右子树不空，则右子树上所有结点的值均大于它的根结点的值；

③它的左、右子树也分别为二叉排序树。

（2）二叉排序树的特点

二叉排序树又称二叉查找树，它是一种动态树表。其特点是，树的结构通常不是一次生成的，而是在查找过程中，当树中不存在关键字等于给定值的结点时动态插入结点而逐步形成的。

（3）二叉排序树的插入和删除

二叉排序树的插入：若二叉排序树为空，则插入结点应为新的根结点，否则继续在其左子树或右子树上查找，直至某个结点的左子树或右子树空为止，则插入结点应作为一个新的叶结点并成为该结点的左孩子或右孩子。

二叉排序树的删除：假设在二叉排序树上被删结点为*p（指向结点的指针为 p），其双亲结点为*f（结点指针为 f），设*p 是*f 的左指针，分三种情况：

①若*p 结点为叶子结点。删去叶子结点不破坏整棵树的结构，只需修改其双亲结点的指针即可。
②若*p 结点只有左子树 P_L 或者只有右子树 P_R，此时只要令 P_L 或 P_R 直接成为*f 的左子树即可。
③若*p 结点的左子树和右子树均不空。在删去*p 结点之前，中序遍历该二叉树得到的序列为{…C_LC…Q_LQS_LSPP$_R$F…}，在删去*p 之后，可以有两种做法：
a．令*p 的左子树为*f 的左子树，而*p 的右子树为*s 的右子树；
b．令*p 的直接前驱（或直接后继）替代*p，然后再从二叉排序树中删去它的直接前驱（或直接后继）。
（4）二叉排序树的查找性能分析
在随机情况下，二叉排序树的平均查找长度和 logn 是等数量级的。然而，在某些情况下（研究证明，这种情况出现的概率约为 46.5％），尚需在构成二叉排序树的过程中进行“平衡化”处理，成为二叉平衡树。
（5）平衡二叉树
①平衡二叉树的定义
平衡二叉树（balanced binary tree 或 height balanced tree）又称 AVL 树，它或者是一棵空树，或者是具有下列性质的二叉树：它的左子树和右子树都是平衡二叉树，且左子树和右子树的深度之差的绝对值不超过 1。
②平衡因子
二叉树上结点的平衡因子（balance factor）定义为该结点左子树的深度减去它右子树的深度。可见，平衡二叉树上所有结点的平衡因子只可能是-1、0 和 1。只要二叉树上有一个点的平衡因子的绝对值大于 1,该二叉树就是不平衡的。
（6）构造平衡二叉树
假设由于在二叉排序树上插入结点而失去平衡的最小子树的根结点指针为 a（即 a 是离插入结点最近，且平衡因子绝对值超过 l 的祖先结点），则失去平衡后进行调整的规律可归纳为下列四种情况：
①单向右旋平衡处理：由于在*a 的左子树根结点的左子树上插入结点，使*a 的平衡因子由 l 增至 2 而失去平衡，需进行一次顺时针旋转操作，如图 9-1（a）所示。
②双向旋转（先左后右）平衡处理：由于在*a 的左子树跟结点的右子树上插入结点，使*a 的平衡因子由 l 增至 2 而失去平衡，需进行两次旋转操作（先左旋后右旋），如图 9-1（b）所示。
③单向左旋平衡处理：由于在*a 的右子树的右子树上插入结点，使*a 的平衡因子由-l 减至-2 而失去平衡，需进行一次逆时针旋转操作，如图 9-1（c）所示。
④双向旋转（先右后左）平衡处理：由于在*a 的右子树根结点的左子树上插入结点，使*a 的平衡因子由-l 减至-2 而失去平衡，需进行两次旋转操作（先右旋后左旋），如图 9-1（d）所示。

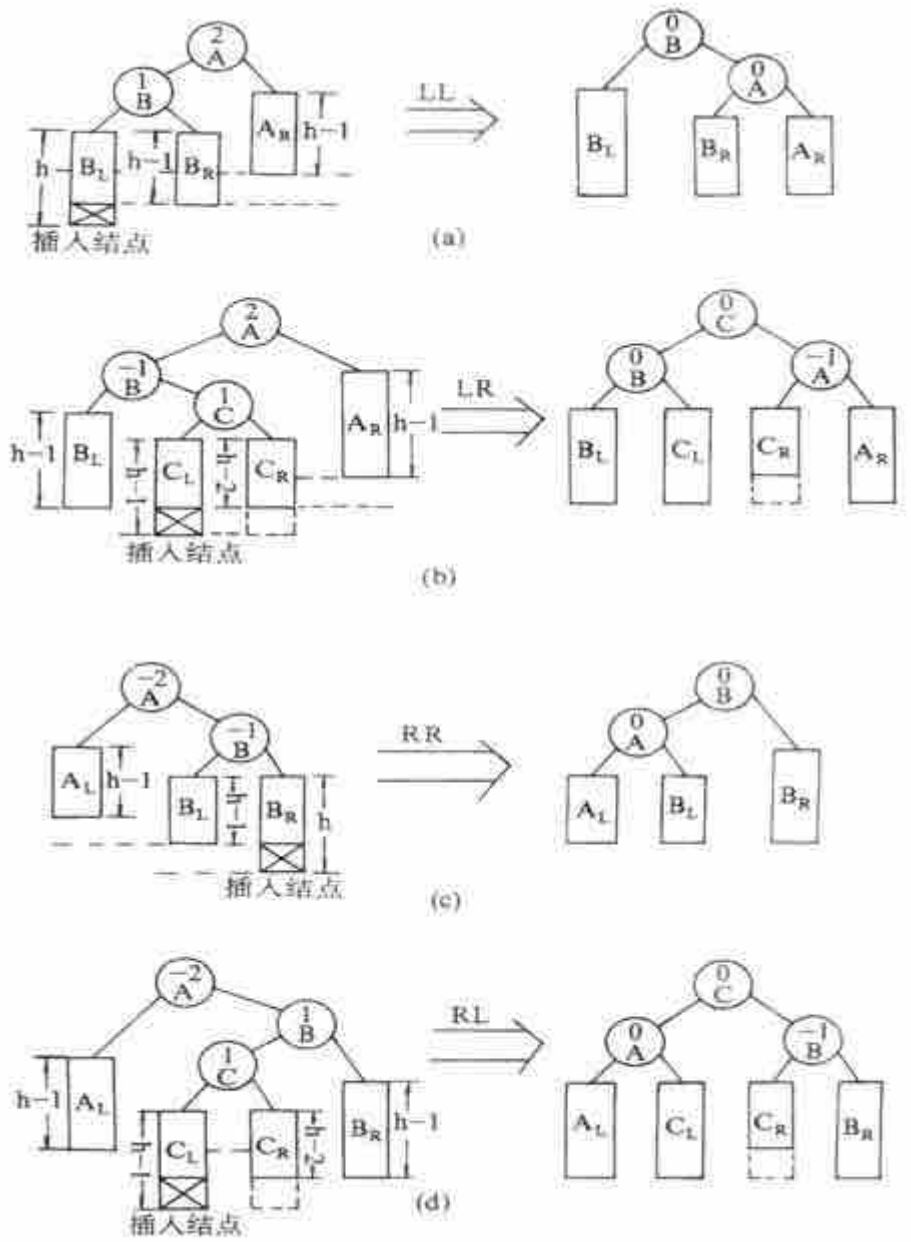


图 9-1 二叉排序树的平衡旋转图例

（a）LL 型；（b）LR 型；（c）RR 型；（d）RL 型。

（7）平衡树查找的性能分析

在查找概率相等的情况下平衡二叉树进行查找的时间复杂度为 O（logn）。

3.B-树和 B+树

（1）B-树的定义

一棵 m 阶的 B-树，或为空树，或为满足下列特性的 m 叉树：

①树中每个结点至多有 m 棵子树；

②若根结点不是叶子结点，则至少有两棵子树；

③除根结点之外的所有非终端结点至少有$\lceil m/2 \rceil$棵子树；

④所有的非终端结点中包含下列信息数据

（n，A_0，K_1，A_1，K_2，A_2，…，K_n，A_n）

其中，K_i（i=1，…，n）为关键字，且 $K_i<K_{i+1}$（i=1，…，n-1）；A_i（i=0，…，n）为指向子树根结点的指针，且指针 A_{i-1} 所指子树中所有结点的关键字均小于 K_i（i=1，…，n），A_n 所指子树中所有结点的关键字均大于 K_n，n（$\lceil m/2 \rceil$-1≤n≤m-1）为关键字的个数（或 n+1 为子树个数）。

⑤所有的叶子结点都出现在同一层次上，并且不带信息。

（2）B-树查找的定义

在 B-树上进行查找是一个顺指针查找结点和在结点的关键字中进行查找交叉进行的过程。

（3）B-树查找的性能分析

在含有 N 个关键字的 B-树上进行查找时，从根结点到关键字所在结点的路径上涉及的结点数不超过 $\log_{\lceil m/2 \rceil}(\frac{N+1}{2})+1$

（4）B-树的插入

B-树中结点的关键字个数必大于或者等于 $\lceil m/2 \rceil$-1，因此，若在 B-树上插入一个关键字，则，①首先在最低层的某个非终端结点中添加一个关键字，若该结点的关键字个数不超过 m-1，则插入完成，否则要产生结点的“分裂”。

②假设*p 结点中已有 m-1 个关键字，当插入一个关键字之后，结点中含有信息为：

$$m，A_0，（K_1，A_1），…，（K_m，A_m）$$

且其中

$$K_i<K_{i+1}$$

$$1\le i<m$$

此时可将*p 结点分裂为*p 和*p’两个结点，其中*p 结点中含有信息为

$$\lceil m/2 \rceil - 1, A_0, (K_1, A_1), \cdots, (K_{\lceil m/2 \rceil - 1}, A_{\lceil m/2 \rceil - 1})$$

P′结点中含有信息

$$m - \lceil m/2 \rceil, A_{\lceil m/2 \rceil}, (K_{\lceil m/2 \rceil + 1}, A_{\lceil m/2 \rceil + 1}), \cdots, (K_m, A_m)$$

而关键字 $K_{\lceil m/2 \rceil}$ 和指针*p’一起插入到*p 的双亲结点中。

（5）B-树的删除

若在 B-树上删除一个关键字，应：

①找到该关键字所在结点，并从中删除之，若该结点为最下层的非终端结点，且其中的关键字数目不少于 $\lceil m/2 \rceil$，则删除完成，否则要进行“合并”结点的操作。

②若所删关键字为非终端结点中的 K_i，则可用指针 A_i 所指子树中的最小关键字 Y 替代 K_i，然后在相应的结点中删去 Y。因此只需删除最下层非终端结点中的关键字，有下列三种情形：

a．被删关键字所在结点中的关键字数目不小于 $\lceil m/2 \rceil$，则只需从该结点中删去该关键字 K_i 和相应指针 A_i，树的其它部分不变；

b．被删关键字所在结点中的关键字数目等于 $\lceil m/2 \rceil$-1，而与该结点相邻的右兄弟（或左兄弟）结点中的关键字数目大于 $\lceil m/2 \rceil$-1，则需将其兄弟结点中的最小（或最大）的关键

字上移至双亲结点中，而将双亲结点中小于（或大于）该上移关键字的关键字下移至被删关键字所在的结点中；

c．被删关键字所在结点和其相邻的兄弟结点中的关键字数目均等于$\lceil m/2 \rceil$-1。假设该结点有右兄弟，且其右兄弟结点地址由双亲结点中的指针 A_i 所指，则在删去关键字之后，它所在结点中剩余的关键字和指针，加上双亲结点中的关键字 K_i 一起，合并到 A_i 所指兄弟结点中（若没有右兄弟，则合并至左兄弟结点）。

（6）B+树的定义

B+树是应文件系统所需而出的一种 B-树的变型树。一棵 m 阶的 B+树和 m 阶的 B-树的差异在于：

①有 n 棵子树的结点中含有 n 个关键字；

②所有的叶子结点中包含了全部关键字的信息，及指向含这些关键字记录的指针，且叶子结点本身依关键字的大小按照自小而大的顺序链接；

③所有的非终端结点可以看成是索引部分，结点中仅含有其子树（根结点）中的最大（或最小）关键字。

4．键树

（1）键树的定义

键树又称数字查找树（digital search trees），它是一棵度≥2 的树，树中的每个结点中不是包含一个或几个关键字，而只含有组成关键字的符号。

（2）键树的两种存储结构。

①以树的孩子兄弟链表来表示键树，则每个结点包括三个域：

a．symbol 域：存储关键字的一个字符；

b．first 域：存储指向第一棵子树根的指针；

c．next 域：存储指向右兄弟的指针。

②若以树的多重链表表示键树，则树的每个结点中含有 d 个指针域，此时键树又称 Trie 树。若从键树中某个结点到叶子结点的路径上每个结点都只有一个孩子，则可将该路径上所有结点压缩成一个“结子节点”，且在该叶子结点中存储关键字及指向记录的指针等信息。

四、哈希表

1．哈希函数

（1）定义

若结构中存在关键字和 K 值相等的记录，则必定在 f（K）的存储位置上，由此，不需要进行比较便可直接取得所查记录。在此，称这个对应关系 f 为哈希（Hash）函数，按这个思想建立的表为哈希表。

（2）特点

①哈希函数是一个映象，因此哈希函数的设定很灵活，只要使得任何关键字由此所得的哈希函数值都落在表长允许范围之内即可；

②对不同的关键字可能得到同一哈希地址，即 keyl≠key2，而 f（keyl）=f（key2），这种现象称冲突（collision）。在一般情况下，冲突只能尽可能地少，而不能完全避免。

2．哈希函数的构造方法

（1）构造方法

①直接定址法

取关键字或关键字的某个线性函数值为哈希地址。即：

H（key）=key 或 H（key）=a·key+b

其中 a 和 b 为常数（这种哈希函数称作自身函数）。

由于直接定址所得地址集合和关键字集合的大小相同。因此，对于不同的关键字不会发生冲突，但实际中能使用这种哈希函数的情况很少。

②数字分析法

假设关键字是以 r 为基的数（如以 10 为基的十进制数），并且哈希表中可能出现的关键字都是事先知道的，则可取关键字的若干数位组成哈希地址。

③平方取中法

取关键字平方后的中间几位为哈希地址，取的位数由表的长度决定。

④折叠法

将关键字分割成位数相同的几部分（最后一部分的位数可以不同），然后取这几部分的叠加和（舍去进位）作为哈希地址。

⑤除留余数法

取关键字被某个不大于哈希表表长 m 的数 P 除后所得余数为哈希地址。即

$$H(key)=key \ MOD \ p \quad (p \le m)$$

这是一种最简单，也最常用的构造哈希函数的方法，它不仅可以对关键字直接取模（MOD），也可在折叠、平方取中等运算之后取模。

在使用除留余数法时，对除数 p 的选择很重要。若 p 选的不好，容易产生同义词。一般情况下，可以选 p 为质数或不包含小于 20 的质因素的合数。

⑥随机数法

选择一个随机函数，取关键字的随机函数值为它的哈希地址，即

$$H(key)=random(key)$$

其中 random 为随机函数。通常，当关键字长度不等时采用此法构造哈希函数较恰当。

（2）选取哈希函数的因素

①计算哈希函数所需时间（包括硬件指令的因素）；

②关键字的长度；

③哈希表的大小；

④关键字的分布情况；

⑤记录的查找频率。

3．处理冲突的方法

（1）开放定址法

$$H_i=(H(key)+d_i) \ MoD \ m \quad i=1，2，\ldots，k \quad (k \le m-1)$$

其中：H（key）为哈希函数，m 为哈希表表长，d_i 为增量序列，可有下列三种取法：

a．$d_i=1$，2，3，…，m-1，称线性探测再散列；

b．$d_i=1^2$，-1^2，2^2，-2^2，3^2，…，$\pm k^2$，（$k \le m/2$）称二次探测再散列；

c．d_i=伪随机数序列，称伪随机探测再散列。

（2）再哈希法

$$H_i=RH_i(key) \quad i=1，2，\ldots，k$$

RH_i 均是不同的哈希函数，即在同义词产生地址冲突时计算另一个哈希函数地址，直到冲突不再发生，这种方法不易产生“聚集”，但增加了计算的时间。

（3）链地址法

将所有关键字为同义词的记录存储在同一线性链表中。

（4）建立一个公共溢出区

假设哈希函数的值域为[0，m-1]，则设向量 hashtable（0：m-1）为基本表，每个分量存放一个记录，另设立向量 overtable[0：v]为溢出表，所有关键字和基本表中关键字为同义词的记录，不管它们由哈希函数得到的哈希地址是什么，一旦发生冲突，都填入溢出表。

4．哈希表的查找及其分析

（1）哈希表的查找和性能分析

在哈希表上进行查找的过程和构造哈希表的过程基本一致。给定 K 值，根据造表时设定的哈希函数求得哈希地址，若表中此位置上没有记录，则查找不成功；否则比较关键字，若和给定值相等，则查找成功；否则根据造表时设定的处理冲突的方法找“下一地址”，直至哈希表中某个位置为“空”或者表中所填记录的关键字等于给定值时为止。

从哈希表的查找过程可见：

a．哈希表虽然在关键字与记录的存储位置之间建立了直接映象，但由于“冲突”的产生，使得哈希表的查找过程仍然是一个给定值和关键字进行比较的过程。因此，仍需以平均查找长度作为衡量哈希表的查找效率的量度。

b．查找过程中需和给定值进行比较的关键字的个数取决于三个因素：哈希函数，处理冲突的方法和哈希表的装填因子。

哈希函数的“好坏”首先影响出现冲突的频繁程度，但是，对于“均匀的”哈希函数可以假定：不同的哈希函数对同一组随机的关键字，产生冲突的可能性相同，因为一般情况下设定的哈希函数是均匀的，则可不考虑它对平均查找长度的影响。

（2）哈希表的装填因子

哈希表的装填因子定义为

$$\alpha = \frac{\text{表中填入的记录数}}{\text{哈希表的长度}}$$

α 标志哈希表的装满程度。直观地看，α 越小，发生冲突的可能性就越小；α 越大，表中已填入的记录越多，再填记录时，发生冲突的可能性就越大，则查找时，给定值需与之进行比较的关键字的个数也就越多。

9.2　强化习题详解

【基础知识题】

1．若对大小均为 n 的有序的顺序表和无序的顺序表进行顺序查找，试在下列三种情况下分别讨论两者在等概率时的平均查找长度是否相同？

（1）查找不成功，即表中没有关键字等于给定值 K 的记录；

（2）查找成功，且表中只有一个关键字等于给定值 K 的记录；

（3）查找成功，且表中有若干个关键字等于给定值 K 的记录，一次查找要求找出所有记录，此时的平均查找长度应考虑找到所有记录时所用的比较次数；

答：（1）相同，平均查找长度为 $\frac{3}{4}(n+1)$

（2）相同，平均查找长度为 $\frac{1}{2}(n+1)$

（3）不同，对于有序顺序表，平均查找长度为 $\frac{n-k+1}{2}$，对于无序顺序表，则为 $\frac{1}{2}(n+1)$。

2．试分别画出在线形表（a,b,c,d,e,f,g）中进行折半查找，以查关键字等于 e，f 和 g 的过程。

答：

查找 e 的过程如下：



3．画出对长度为 10 的有序表进行折半查找的判定树，并求其概率时查找成功的平均查找长度。

答：

$$ASL_{succ}=\frac{1}{10}[1\times1+2\times2+4\times3+3\times4]=2.9$$

长度为 10 的有序表进行折半查找的判定树如图 9-1 所示。

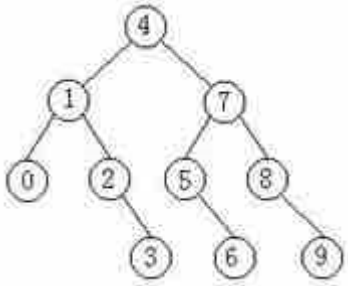


图 9-1 判定树

4．假设下述递归方法进行顺序表的查找，若表长＜＝10，则进行顺序查找，否则进行折半查找。试画出对表长 n=50 的顺序表进行上述查找时，描述该查找的判定树，并求出在等概率情况下查找成功的平均查找长度。

答：

$$ASL_{succ}=\frac{1}{50}[1+2\times2+4\times3+8\times(4+5+6+7+8)+9\times3]=5.68$$

对表长 n=50 的顺序表进行上述查找时，描述该查找的判定树如图 9-2 所示。

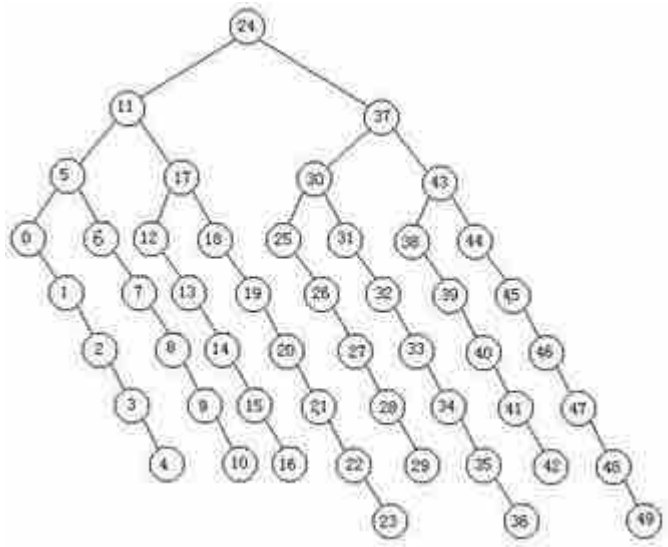


图 9-2 判定树

5．下列算法为斐波那契查找的算法：

```
int FibSearch（SqList r, KeyType K ） {
j=1;
while （fib（j）<n+1） j=j+1;
mid=n-fib（fib（j-2） +1;// 若 fib（j）=n+1 则 mid=fib（j-1）
f1=fib（j-2）;  f2=fib（j-3）;  found=FALSE;
while （（mid<>0）&&!found）
switch{
case K=r[mid].key:  found=TRUE; break;
case K<r[mid].key:
if （!f2） mid=0;
else{mid=mid-f2; t=f1-f2;f1=f2; f2=t; }
break;
case K>r[mid].key:
if （f1=1） mid=0;
else{mid=mid+f2;f1=f1-f2;f1=f2; f2=f2-f1; }
break;
}
    if （found） return mid;
else return 0;
  }// FibSearch
```

其中 fib（i）为斐波那契序列（参见 9.1.2 节注）。试画出对长度为 20 的有序表进行斐波那契查找的判定树，并求在等概率时查找成功的平均查找长度

答：

$$ASL_{succ}=\frac{1}{20}\left[1+2\times 2+4\times 3+7\times 4+5\times 5+6\right]=3.8$$

对长度为 20 的有序表进行斐波那契查找的判定树如图 9-3 所示。

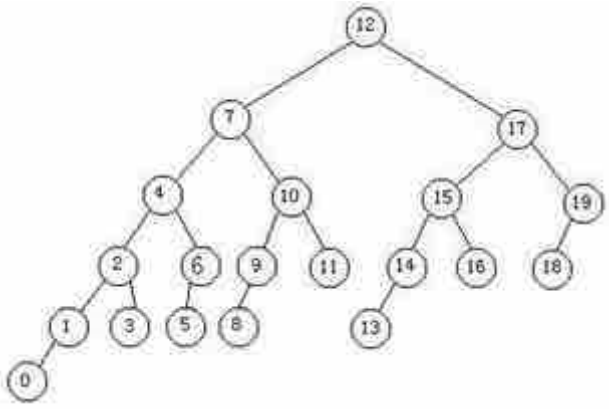


图 9-3 判定树

6．假设在某程序中有如下一个 if—then 嵌套的语句

if （C_1）

if（C_2）

if（C_3）

…

if（C_n）

S；

其中 C_i 为布尔表达式。显然只有当所有的 C_i 都为 TRUE 时，语句 S 才能执行。假设 t（i）为判别 C_i 是否为 TRUE 所须的时间，p（i）为 C_i 是 TRUE 的概率，试讨论这 n 个布尔表达式 C_i（i=1，2，…，n）应如何排列才能使该程序最有效地执行？

答：

先列出该语句执行时间的期望值，假设语句 S 的执行时间为 T，则该语句执行时间的期望值为：

T=（1-p（1））t（1）+p（1）（1-p（2））（t（1）+t（2））+…

+p（1）p（2）…p（n-1）（1-p（n））（t（1）+t（2）+…

+t（n））+p（1）p（2）+…p（n）（t（1）+t（2）…+t（n）+t）

令 p（0）=1,则上式可写为

$$T=(\prod_{j=1}^{n}p(j))t+\sum_{j=1}^{n}t(j)(\sum_{i=1}^{n}(\prod_{j=0}^{i-1}p(j))(1-p(i))+\prod_{j=1}^{n}p(j))$$

$$=(\prod_{j=1}^{n}p(j))t+\sum_{j=0}^{n}(\prod_{j=0}^{i-1}p(j))t(i)$$

可以证明，当这 n 个布尔表达式 Ci 的排列满足下式时，将使 T 达极小值：

$$\frac{t(1)}{1-p(1)}<\frac{t(2)}{1-p(2)}<\cdots<\frac{t(n)}{1-p(n)}$$

因此，应按 $\frac{t(i)}{1-p(i)}$ 从小到大的次序来排列，将使该语句的执行最有效。

7．已知一个有序表的表长为 8N，并且表中没有关键字相同的记录。假设按如下方法查找一个关键字等于给定值 K 的记录：先在第 8，16，24，…，8K，…，8N 个记录中进行顺序查找，或者查找成功，或者由此确定出一个继续进行折半查找的范围，画出描述上述查找过程的判定树，并求等概率查找成功的平均查找长度。

答：

$$a_1 = 1\times1+1\times2+2\times3+4\times4$$

$$a_2 = 1\times2+1\times3+2\times4+4\times5$$

$$a_n = 1\times n+1\times(n+1)+2\times(n+2)+4\times(n+3)$$

$$s_N = \sum_{n=1}^{N} a_n = \sum_{n=1}^{N}[8N+17] = \frac{8N(N+1)}{2}+17N$$

$$ASL_{succ} = \frac{N+1}{2}+\frac{17}{8}$$

描述上述查找过程的判定树如图 9-4 所示。

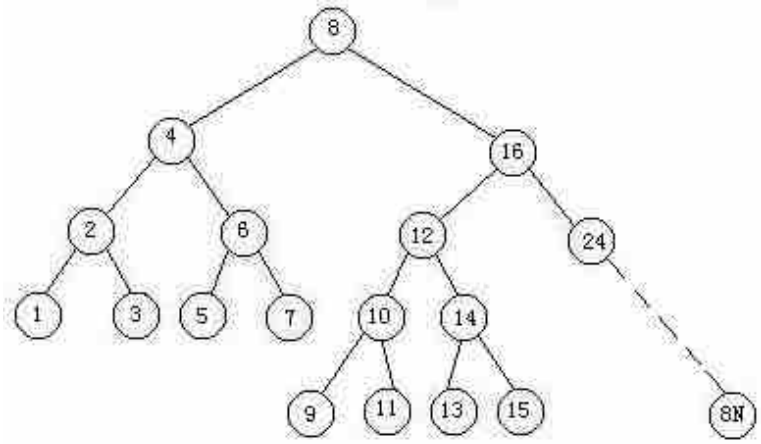


图 9-4 判定树

8．已知含 12 个关键字的有序表及其相应权值为：

关键字	A	B

（1）试按次优查找树的构造算法并加适当调整画出由这 12 个关键字构造所得的次优查找树，并计算它的 PH 值；

（2）画出对以上有序表进行折半查找的判定树，并计算它的 PH 值。

答：

（1）次优查找树如图 9-5 所示，PH 值为 133；

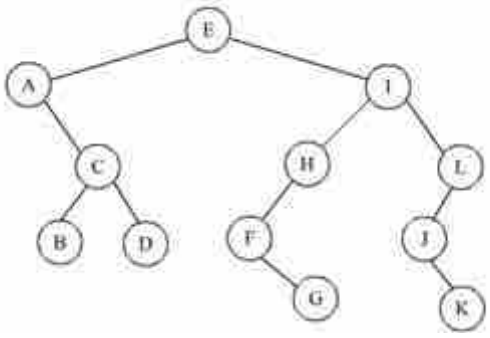


图 9-5

（2）折半查找的判定树的 PH 值为 156

9．已知如下所示长度为 12 的表

（Jan，Feb，Mar，Apr，May，June，July，Aug， Sep，Oct， Nov， Dec）

（1）试按表中元素的顺序依次插入一棵初始为空的二叉排列树，画出插入完成之后的二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。

（2）若对表中元素先进行排序构成有序表，求在等概率的情况下对此有序表进行拆半查找时查找成功的平均查找长度。

（3）按表中元素顺序构造一棵平衡二叉排序树，并求其在等概率的情况下查找成功的平均查找长度。

答：

（1） $$ASL_{succ}=\frac{1}{12}[1\times1+2\times2+3\times3+3\times4+2\times5+1\times6]=3.5$$

如图 9-6 所示为插入完成之后的二叉排序树。

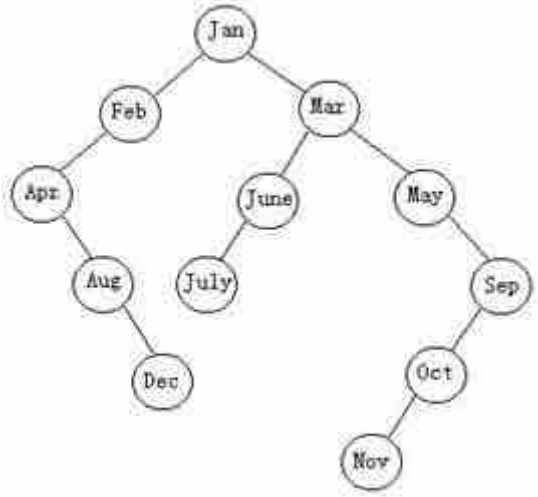


图 9-6

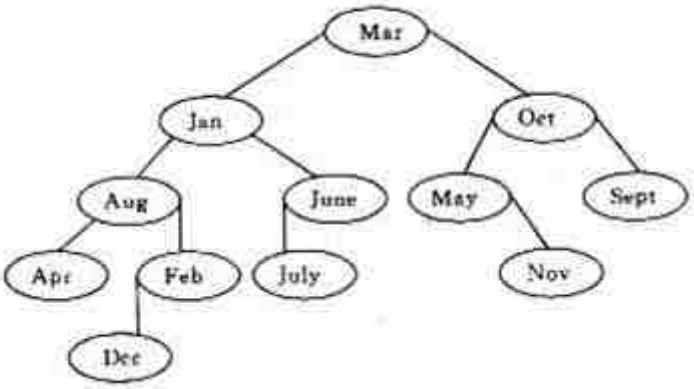
（2）经排序后的表及在折半查找时找到表中元素所经比较的次数对照如下：

Apr	Aug	Dec	Feb	Jan	July	June	Mar	May	Nov	Oct	Sept
3	4	2	3	4	1	3	4	2	4	3	4

等概率情况下查找成功时的平均查找长度为

$$ASL_{SUCC}=\frac{1}{12}(1\times1+2\times2+3\times4+4\times5)=\frac{37}{12}$$

（3）求得的平衡二叉树为



它在等概率条件下的平均查找长度为

$$ASL = \frac{1}{12}(1\times1 + 2\times2 + 3\times4 + 4\times4 + 4\times5) = \frac{38}{12}$$

10．可以生成如图 9-7 所示的二叉排序树的关键字的初始排列有几种？请写出其中的任意 5 个。

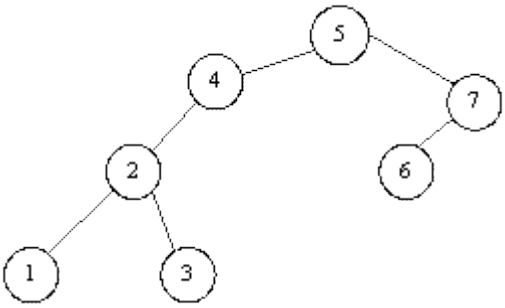


图 9-7

答：

①5 4 7 6 2 3 1

②5 4 7 6 2 1 3

③5 7 4 6 2 3 1

④5 7 4 6 2 1 3

⑤5 7 6 4 2 3 1

11．试推导含 12 结点的平衡二叉树的最大深度，并画出一棵这样的树。

答：令 Fk 表示含有最少结点的深度为 k 的平衡二叉树的结点数目，则：

F1=1，F2=2，…，F（n）=F（n-2）+F（n-1）+1。因此，含有 12 个结点的平衡二叉树的最大深度为 5，例如图 9-8 所示：

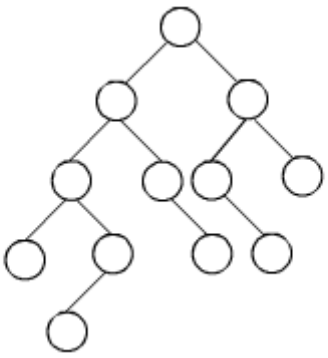


图 9-8

12．在 B-树定义中，特性（3）的意图是什么？试思考：若把“$\lceil m/2 \rceil$”改为“$\lceil 2m/3 \rceil$”或“$\lceil m/3 \rceil$”是否可行？所得到的树结构和 B-树有何区别？

答：特性（3）的意图在于保证 B-树中结点空间的利用率不低于某个下限。改为$\lceil 2m/3 \rceil$是不行的，因为当某个节点因插入关键字而是其中关键字数目为 m 时，无法分裂成两个子树个数均大于$\lceil 2m/3 \rceil$的结点；改为$\lceil m/3 \rceil$是可行的，但它的结点空间利用率较低，不过分裂不如 B-树那样频繁。

13．含 9 个叶子结点的 3 阶 B-树中至少有多少个非叶子结点？含 10 个叶子结点的 3 阶 B-树中至多有多少个非叶子结点？

答：①含 9 个叶子结点的 3 阶 B-树中至少有 4 个非叶子结点。

②含 10 个叶子结点的 3 阶 B-树中至多有 8 个非叶子结点。

14．试从空树开始，画出按以下次序向 2-3 树即 3 阶 B-树中插入关键码的建树过程：20，30，50，52，60，68，70。如果此后删除 50 和 68，画出每一步执行后 2-3 树的状态。

答：建树过程如图 9-9，9-10,9-11 所示：

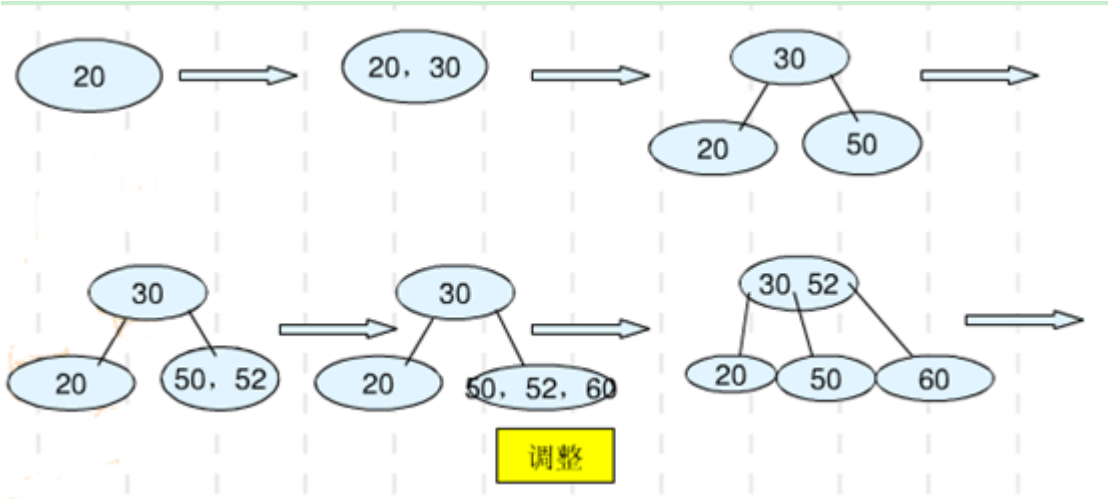


图 9-9

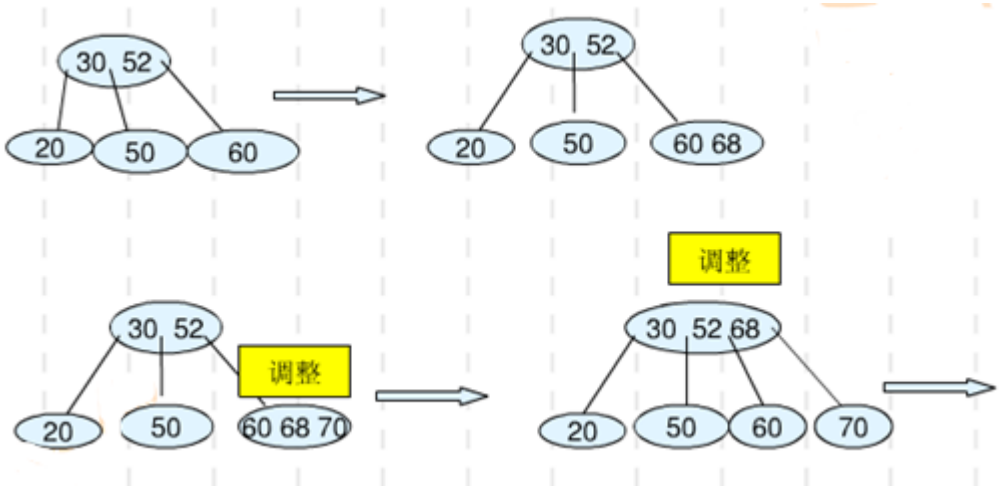


图 9-10

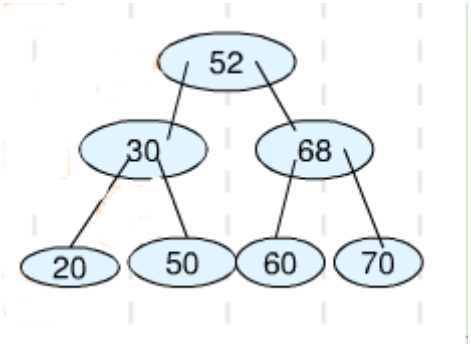


图 9-11

删除过程如图 9-12 所示：

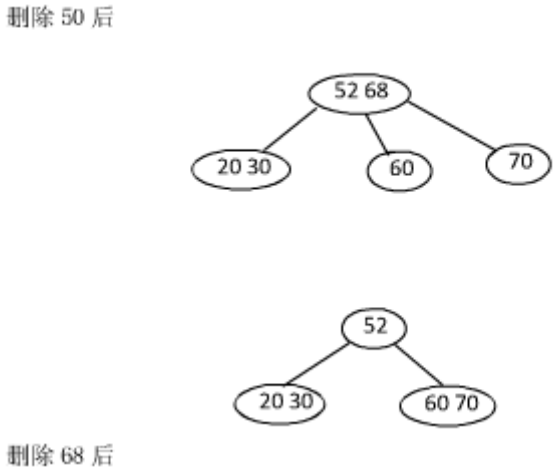


图 9-12

15．试证明：高度 h 的 2-3 树中叶子结点的数目在 2^{h-1} 与 3^{h-1} 之间。

答：因为 2-3 树即为 3 阶 B-树，其中每个结点至少有 2 棵子树，而二叉树第 h 层的结点总数至多为 2^{h-1}，所以在每层的结点数都是最小的情况下，第 h 层的叶子数为 2^{h-1}。又因为 2-3 树每个结点至多有 3 棵子树，所以每层的结点数都是最大情况下，第 h 层的叶子数为 3^{h-1}。所以高度为 h 的 2-3 树中叶子结点的数目在 2^{h-1} 与 3^{h-1} 之间。

16．在含有 n 个关键码的 m 阶 B-树中进行查找时，最多访问多少个结点？

答：根据 B-树的定义，第一层至少有一个结点；第二层至少有 2 个结点；由于除根结点之外的每个非终端结点至少有$\lceil m/2 \rceil$棵子树，则第三层至少有 2（$\lceil m/2 \rceil$）个结点；……；依次类推，第$l+1$层至少有$2\left(\lceil m/2 \rceil\right)^{l-1}$个结点。而$l+1$层得结点为叶子结点。若 m 阶 B-树中具有 n 个关键字，则叶子结点即查找不成功的结点为 n+1，由此有：

$$n+1 \geq 2*\left(\lceil m/2 \rceil\right)^{l-1}$$

反之

$$l \leq \log_{\lceil m/2 \rceil}\left(\frac{n+1}{2}\right)+1$$

即，在含有 n 个关键字的 B-树上进行查找时，从根结点到关键字所在结点的路径上涉及的结点数不超过

$$\log_{\lceil m/2 \rceil}\left(\frac{n+1}{2}\right)+1$$

。

17.B 树和 B-树的主要差异是什么？

答：这两种处理索引的数据结构的不同之处：

①B 树中同一键值不会出现多次，并且它有可能出现在叶结点，也有可能出现在非叶结点中。而 B+树的键一定会出现在叶结点中，并且有可能在非叶结点中也有可能重复出现，以维持 B+树的平衡。

②因为 B 树键位置不定，且在整个树结构中只出现一次，虽然可以节省存储空间，但使得在插入、删除操作复杂度明显增加。B+树相比来说是一种较好的折中。

③B 树的查询效率与键在树中的位置有关,最大时间复杂度与 B+树相同（在叶结点的时候），最小时间复杂度为 1（在根结点的时候）。而 B+树的时候复杂度对某建成的树是固定的。

18．试画出一个对应于关键字集{program，programmer，programming，processor，or}的 Trie 树，对每个关键字从右向左取样，每次一个字母。

答：构造 Trie 树，对关键字集作如下分割，首先按最后一个字符不同分成若干子集，然后按倒数第二的字符不同分成若干子集，结果如图 9-13 所示。

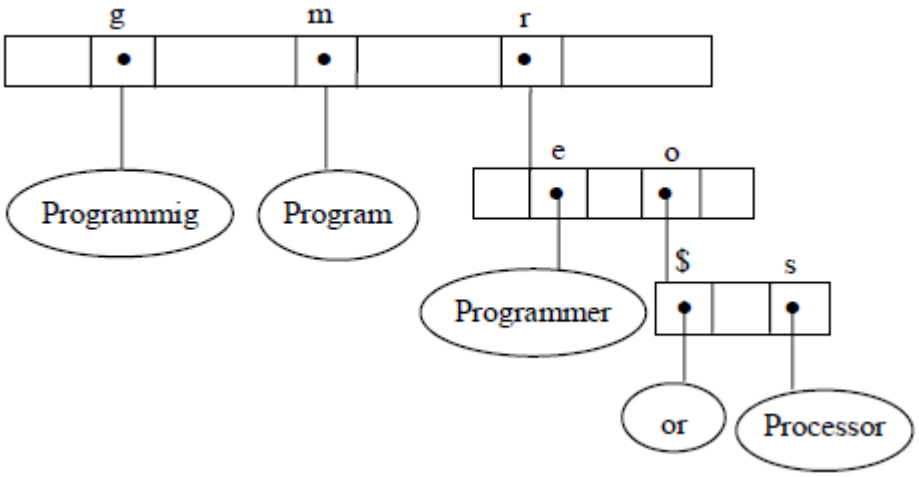


图 9-13

19．选取哈希函数 H（k）=（3k）MOD 11。用开放定址法处理冲突，d=I（（7k）MOD 10+1）（I=1，2，3，…）。试在 0~10 的的散列地址空间中对关键字序列（22，41，53，46，30，13，　01，67）造哈希表，并求等概率情况下查找成功时的平均查找长度。

答：构建的哈希表如表 9-1 所示，

表 9-1

	0	1	2	3	4	5	6	7	8	9	10
关键序列	22	30	41	1	13	53	46				
成功次数	1	2	1	1	3	1	1				

查找成功是的平均查找长度：$ASL_{succ}=\frac{1}{8}(1+1+1+1+2+2+6+3)=\frac{17}{8}$。

20．试为下列关键字建立一个装载因子不小于 0.75 的哈希表，并计算你所构造的哈希表的平均查找长度。

（ZHAO，QIAN，SUN，LI，ZHOU，WU，ZHENG，WANG，CHANG，CHAO，YANG，JIN）

答：步骤如下，

①由装填因子求出表长；

②可利用字母序号设计哈希函数；

③确定解决冲突的方法。

由装填因子=（表中填入的记录数）/（哈希表的长度），可求得哈希表的长度应小于等于 16。

设计如下哈希表：

①地址空间为 0~15；

②设哈希函数为 H（key）=$\lfloor i/3 \rfloor$，其中 i 为关键字中第一个字母在字母表中的序号；

③用线形探测开放定址法处理冲突

$$H_i = (H(x) + d_i)\ MOD\ m$$

m=16; d_i=1,2,3,…,15

哈希函数值为：

	ZHAO	QIAN	SUN	LI	ZHOU	WU	ZHENG	WANG	CHANG	CHAO	YANG	JIN
i	26	17	19	12	26	23	26	23	3	3	25	7
H（key）	8	5	6	4	8	7	8	7	0	0	8	2

哈希表如表 9-2 所示：

表 9-2

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
关键序列	CHANG	CHAO	JIN	LI	QIAN	SUN	WU	ZHAO	WANG	ZHOU	ZHENG	YANG			
成功次数	1	2	1	1	1	1	1	1	3	3	4	5			
失败次数	4	3	2	1	2	3	4	5	6	7	8	9	10	1	1	1

查找成功时的平均查找长度：

$$ASL_{SUCC} = \frac{1}{12}(7+2+3\times 2+4+5) = 2$$

查找不成功时的平均查找长度：

$$ASL_{UNSUCC} = \frac{1}{16}(10+9+8+7+6+5+4+3+2+1+1+1+1+2+3+4) = \frac{69}{16}$$

21．在地址空间为 0~16 的散列地区中，对以下关键字序列构造两个哈希表：

（（Jan，Feb，Mar，Apr，May，June，July，Aug，Sep，Oct，Nov，Dec）

（1）用线形探测开放定址法处理

（2）用链地址法处理。

并分别求这两个哈希表在等概率情况下查找成功和不成功的平均查找长度。

设哈希函数为 H（x）=$\lfloor i/2 \rfloor$，其中 i 为关键字中第一个字母在字母表中的序号。

答：由题意哈希函数值为

	Jan	Feb	Mar	Apr	May	June	July	Aug	Sep	Oct	Nov	Dec
i	7	6	13	1	13	7	7	1	19	15	14	4
H（x）	3	3	6	0	6	3	3	0	9	7	7	2

（1）查找成功时的平均查找长度：

$$ASL_{SUCC} = \frac{31}{12}$$

查找不成功时的平均查找长度：

$$ASL_{UNSUCC} = \frac{60}{14}$$

（2）

查找成功时的平均查找长度：

$$ASL_{SUCC}=\frac{18}{12}$$

查找不成功时的平均查找长度：

$$ASL_{UNSUCC}=\frac{12}{14}$$

22．已知一个含有 1000 个记录的表，关键字为中国人姓氏的拼音，请给出此表的一个哈希表设计方案，要求它在等概率情况下查找成功的平均长度不超过 3。

答：解题思路：

设计哈希表的步骤为：

①根据所选择的处理冲突的方法求出装载因子 a 的上界；

②由 a 值设计哈希表的长度 m；

③根据关键字的特性和表长 m 选定合适的哈希函数。

注意：题目要求 ASL≤3，则 m 必然要尽量长，以减少冲突。

23．设有一个关键字取值范围为正整数的哈希表，空表项的值为-1，用开放定址法解决冲突。现有两种删除策略：一是将待删表项的关键字置为-1；二是将探测序列上的关键字顺序递补，即用探测序列上下一个关键字覆盖待删关键字，并将原序列上之后一个关键字置为-1。这两种方法是否可行？为什么？给出一种可行的方法，并叙述它对查找和插入算法所产生的影响。

答：两种策略都不可行。前者切断了探测链；后者可能移动了非同义词。

一种可行的做法是将待删表项的关键字置为 0，以区别于空表项。该做法对查找和插入算法产生的影响是：查找和插入算法都应相应地进行调整。

24．某校学生学号由 8 为十进制数字组成：$c_1c_2c_3c_4c_5c_6c_7c_8$。c_1c_2 为入学时年份的后两位；c_3c_4 为系别：00~24 分别代表该校的 25 个系；c_5 为 0 或 1，0 表示本科生，1 表示研究生；$c_6c_7c_8$ 为对某级某系某类学生的顺序编号：对于本科生，它不超过 199，对于研究生，它不超过 049，共有 4 个年级，四年级学生 1996 年入学。

（1）当在校生人数达极限情况时，将他们的学号散列到 0~24999 的地址空间，问装载因子是多少？

（2）求一个无冲突的哈希函数 H_1，它将在校学生学号散列到 0~24999 的地址空间。其簇族性如何？

（3）设在校生总数为 15000 人，散列地址空间为 0~19999，你是否能找到一个（2）中要求的 H_1？若不能，试设计一个哈希函数 H_2 极其解决冲突的方法，使得多数学号可只经一次散列得到（可设各系各年级本科生平均人数为 130，研究生平均人数为 20）。

（4）用算法描述语言表达 H_2，并写出相应的查找函数。

答：

（1）在校生人数达极限情况，哈希表装满记录，故装填因子 α=1;

（2）这样的哈希函数不惟一，一种方案是

H_1（C）=C_{678}+C_5×200+C_{34}×（200+50）+（C_{12}-96）×（（200+50）×25）

其中，C=$C_1C_2C_3C_4C_5C_6C_7C_8$

C_{678}=C_6×100+C_7×10+C_8

C_{34}=C_3×10+C_4

C_{12}=C_1×10+C_2

此哈希函数在无冲突的前提下非常集聚。

（3）不能（虽然 α <1）。哈希函数可设计为

H_2（C）=（1-2C_5）C_{678}+C_5（l-1）+C_{34}l+（C_{12}-96）×25l

其中，$C_{678}=C_6\times100+C_7\times10+C_8$，其余类推。
方案一：l=150，最后 5000 个表项作为公共溢出区；
方案二：l=200，用开放定址处理冲突。也可有其他折衷方案。
（4）H_2（C）=（1-2C_5）C_{678}+C_5（l-1）+C_{34}l+（C_{12}-96）×25l
【算法设计题】
25．假设顺序表按关键字自大至小有序，下面的顺序查找算法，将监视哨设在高下标端。然后画出描述此查找过程的判定树，分别求出等概率情况下查找成功和不成功时的平均查找长度。

```
FUNC  seqsrch(r : sqlisttp; K : keytype) : integer;
  {K 为给定值,返回 i 为关键字等于 K 的记录在表 r 中的序号,i 值为零表明查找不成功}
   r[0].key :=K;   i :=n;
   WHILE r[i].key≠K   DO   i :=i-1
   RETURN(i)
ENDF.  {seqsrch}
```

答：
int Search_Sq（SSTable ST,int key）//在有序表上顺序查找的算法,监视哨设在高下标端 {
ST.elem[ST.length+1].key=key;//监视哨 for（i=1;ST.elem[i].key>key;i++）;
if（i>ST.length||ST.elem[i].key<key） return ERROR; return i; }//Search_Sq 分析：本算法查找成功情况下的平均查找长度为 ST.length/2，不成功情况下为 ST.length。
26．试将折半查找的算法改写成递归算法。
答：
int Search_Bin_Recursive（SSTable ST,int key,int low,int high）//折半查找的递归算法 {
if（low>high） return 0; //未查找到时返回 0 mid=（low+high）/2; if（ST.elem[mid].key==key） return mid;//查找到，返回位置 else if（ST.elem[mid].key>key）//待查元素的关键字大于中间结点的关键字，则去左半部分递归查找 return Search_Bin_Recursive（ST,key,low,mid-1）;
//否则，去右半部分递归查找 else return Search_Bin_Recursive（ST,key,mid+1,high）; } }
//Search_Bin_Recursive
27．试改写如下的折半查找的算法，当 r[i].kcy<=K<r[i+1]．kcy（i=1,2，…，n-1）时，返回 i；当 K＜r[1].key 时，返回 0；当 K>=r[n].key 时，回 n。

```
UNIT Ordered_List;
  INTERFACE
    TYPE  rectype=RECORD
                key:keytype;
                ...
          END;
          ordlisttp=ARRAY[1..n] OF rectype  ;{有序表}
  FUNC  binsrch (r:ordlisttp;K:keytype):integer;
  { r[1..n]中存储 n 个记录,且 r[i+1].key>r[i].key,i=1,2,…,n-1,折半查找记录,若存在其
    关键字等于给定值 K 的记录,则函数值 i 是记录在有序表 r 中的序号,否则函数值为0}
  ...
```

答：

int Locate_Bin（SSTable ST,int key）//折半查找，返回小于或等于待查元素的最后一个结点号 { int *r; r=ST.elem; if（key<r.key） return 0; else if（key>=r[ST.length].key） return ST.length; low=1;high=ST.length; while（low<=high） { mid=（low+high）/2; if（key>=r[mid].key&&key<r[mid+1].key） //查找结束的条件 return mid; else if（key<r[mid].key） high=mid; else low=mid; } //本算法不存在查找失败的情况,不需要 return 0; }//Locate_Bin

28．试编写利用折半查找确定记录所在块的分块查找算法。并讨论在块中进行顺序查找时使用“监视哨”的优缺点，以及必要时如何在分块查找的算法中实现设置“监视哨”的巧。

答：

typedef struct { int maxkey; int firstloc; } Index;

typedef struct { int *elem; int length; Index idx[MAXBLOCK];//每块起始位置和最大元素,其中 idx[0]不利用,其内容初始化为{0,0}便于折半查找 int blknum; //块的数目 } IdxSqList;//索引顺序表类型

int Search_IdxSeq（IdxSqList L,int key）//分块查找,先用折半查找法确定记录所在块,块内采用顺序查找法 { if（key>L.idx[L.blknum].maxkey） return ERROR; //超过最大元素 low=1;high=L.blknum; found=0; while（low<=high&&!found）//折半查找记录所在块号 mid { mid=（low+high）/2; if（key<=L.idx[mid].maxkey&&key>L.idx[mid-1].maxkey） found=1; else if（key>L.idx[mid].maxkey） low=mid+1; else high=mid-1; } i=L.idx[mid].firstloc; //块的下界 j=i+blksize-1; //块的上界 temp=L.elem[i-1]; //保存相邻元素 L.elem[i-1]=key; //设置监视哨 for（k=j;L.elem[k]!=key;k--）; //顺序查找 L.elem[i-1]=temp; //恢复元素 if（k<i） return ERROR; //未找到 return k; }//Search_IdxSeq 分析：在块内进行顺序查找时,如果需要设置监视哨，则必须先保存相邻块的相邻元素，以免数据丢失。

29．已知一非空有序表，表中记录按关键字递增排列，以不带头结点的单循环链表作存储结构，外设两个指针 h 和 t，其中 h 始终指向关键字最小的结点，t 则在表中浮动，其初始位置和 h 相同，在每次查找之后指向刚查到的结点。查找算法的策略是：首先将给定值 K 和 t->key 进行比较，若相等，则查找成功；否则因 K 小于或大于 t->key 而从 h 所指结点或 t 所指结点的后继结点起进行查找。

（1）按上述查找过程编写查找算法；

（2）画出描述此查找过程的判定树，并分析在等概率查找时查找成功的平均查找长度（假设表长为 n，待查关键码 K 等于每个结点关键码的概率为 1/n，每次查找都是成功的，因此在查找时，t 指向每个结点的概率也为 1/n）。

答：

typedef struct { LNode *h;//h 指向最小元素 LNode *t;//t 指向上次查找的结点 } CSList;

LNode *Search_CSList（CSList &L,int key）//在有序单循环链表存储结构上的查找算法，假定每次查找都成功 { if（L.t->data==key） return L.t;//查找到 else if（L.t->data>key）//未查找到，更新指针 for（p=L.h,i=1;p->data!=key;p=p->next,i++）; else for（p=L.t,i=L.tpos;p->data!=key;p=p->next,i++）; L.t=p; //更新 t 指针 return p; } //Search_CSList 分析：由于题目中假定每次查找都是成功的，所以本算法中没有关于查找失败的处理，由微积分知识可得，等概率情况下，平均查找长度约为 n/3。

30．将 29 题的存储结构改为双向循环链表，且只外设一个指针 sp，其初始位置指向关键字最小的结点，在每次查找之后指向刚查到的结点。查找算法的策略是：首先将给定值 K 和 sp->key 进行比较，若相等，则查找成功；否则依 K 小于或大于 sp->key 继续从，sp 的前驱或后继结点起进行查找。编写查找算法并分析等概率查找时查找成功的平均查找长度。

答：

typedef struct { DLNode *pre; int data; DLNode *next; } DLNode; //定义结点的数据类型

typedef struct { DLNode *sp; int length; } DSList; //供查找的双向循环链表类型

DLNode *Search_DSList（DSList &L,int key）//在有序双向循环链表存储结构上的查找算法,假定每次查找都成功 { p=L.sp; if（p->data>key） { while（p->data>key） p=p->pre; L.sp=p; } else if（p->data<key） { while（p->data<key） p=p->next; L.sp=p; } return p; } //Search_DSList 分析：本题的平均查找长度与上一题相同，也是 n/3。

31．试写一个判别给定二叉树是否为二叉排序树的算法，设此二叉树以二叉链表作存储结构。且树中结点的关键字均不同。

答：

int last=0,flag=1;

int Is_BSTree（Bitree T）//判断二叉树 T 是否二叉排序树，是则返回 1，否则返回 0 { if（T->lchild&&flag） Is_BSTree（T->lchild）;//递归判断左子树 if（T->data<last） flag=0; //与其中序的前驱结点相比较 last=T->data; if（T->rchild&&flag） Is_BSTree（T->rchild）; //递归判断右子树 return flag; }//Is_BSTree

32．已知一棵二叉排序树上所有关键字中的最小值为-max，最大值为 max，-max<x<max。编写递归算法，求该二叉排序树上的小于 x 且最靠近 x 的值 a 和大于 x 且最靠近 x 的值 b。

答：

//本算法仍是借助中序遍历来实现

int last=0;

void MaxLT_MinGT（BiTree T,int x）//找到二叉排序树 T 中小于 x 的最大元素和大于 x 的最小元素 { if（T->lchild） MaxLT_MinGT（T->lchild,x）;//递归判断左子树

if（last<x&&T->data>=x）//找到了小于 x 的最大元素 printf（"a=%d\n",last）;

if（last<=x&&T->data>x） //找到了大于 x 的最小元素 printf（"b=%d\n",T->data）;

last=T->data; if（T->rchild） MaxLT_MinGT（T->rchild,x）;//递归判断右子树 } //MaxLT_MinGT

33．编写递归算法，从大到小输出给定二叉排序树中所有关键字不小于 x 的数据元素。

答：

void Print_NLT（BiTree T,int x）//从大到小输出二叉排序树 T 中所有不小于 x 的元素 { if（T->rchild） Print_NLT（T->rchild,x）;//递归判断右子树 if（T->data<x） exit（）; //当遇到小于 x 的元素时立即结束运行 printf（"%d\n",T->data）; if（T->lchild） Print_NLT（T->lchild,x）; //先右后左的中序遍历 }//Print_NLT

34．试写一时间复杂度为 O（$\log_2 n+m$）的算法，删除二叉排序树中所有关键字不小于 x 的结点，并释放结点空间。其中 n 为树中所含结点数，m 为被删除的结点个数。

答：

void Delete_NLT（BiTree &T,int x）//删除二叉排序树 T 中所有不小于 x 元素结点,并释放空间 { if（T->rchild） Delete_NLT（T->rchild,x）;//递归遍历右子树中所有不小于 x 元素的结点 if（T->data<x） exit（）; //当遇到小于 x 的元素时立即结束运行 q=T; T=T->lchild; free（q）; //如果树根不小于 x,则删除树根,并以左子树的根作为新的树根 if（T） Delete_NLT（T,x）; //继续在左子树中执行算法 }//Delete_NLT

35．假设二叉排序树以后继线索链表作存储结构，编写输出该二叉排序树中所有大于 a 且小于 b 的关键字的算法。

答：

void Print_Between（BiThrTree T,int a,int b）//打印输出后继线索二叉排序树 T 中所有大于 a 且小于 b 的元素 { p=T; while（!p->ltag） p=p->lchild; //找到最小元素 while（p&&p->data<b）//j 关键字小于 b 的元素 { if（p->data>a） printf（"%d\n",p->data）; //输出符合条件的元素 if（p->rtag） p=p->rtag; else { p=p->rchild; while（!p->ltag） p=p->lchild; } //转到中序后继 }//while }//Print_Between

36．同 35 题的结构，编写在二叉排序树中插人一个关键字的算法。

答：

void BSTree_Insert_Key（BiThrTree &T,int x）//在后继线索二叉排序树 T 中插入元素 x { if（T->data<x） //插入到右侧 { if（T->rtag） //T 没有右子树时,作为右孩子插入 { p=T->rchild; q=（BiThrNode*）malloc（sizeof（BiThrNode））;//动态创建结点 q->data=x; T->rchild=q;T->rtag=0; q->rtag=1;q->rchild=p; //修改原线索 } else BSTree_Insert_Key（T->rchild,x）;//T 有右子树时,插入右子树中 }//if else if（T->data>x） //插入到左子树中 { if（!T->lchild） //T 没有左子树时,作为左孩子插入 { q=（BiThrNode*）malloc（sizeof（BiThrNode））;//动态创建结点 q->data=x; T->lchild=q; q->rtag=1;q->rchild=T; //修改自身的线索 } else BSTree_Insert_Key（T->lchild,x）;//T 有左子树时,插入左子树中 }//if }//BSTree_Insert_Key

37．同 37 题的结构，编写从二叉排序树中删除一个关键字的算法。

答：

Status BSTree_Delete_key（BiThrTree &T,int x）//在后继线索二叉排序树 T 中删除元素 x { BTNode *pre,*ptr,*suc;//ptr 为元素 x 所在的结点，指针 pre 和 suc 分别指向 ptr 的前驱和后继 p=T;last=NULL;//last 始终指向当前结点 p 的前驱结点 while（!p->ltag） p=p->lchild;//找到中序起始元素 while（p） { if（p->data==x）//找到元素为 x 的结点 { pre=last; ptr=p; } else if（last&&last->data==x） suc=p;//找到了 x 的后继 if（p->rtag） p=p->rtag; else { p=p->rchild; while（!p->ltag） p=p->lchild; } //转到中序后继 last=p; } //while //借助中序遍历找到元素 x 及其前驱和后继结点 if（!ptr） return ERROR;//未找到待

删结点 Delete_BSTree（ptr）;//删除 x 结点 if（pre&&pre->rtag） pre->rchild=suc;//修改线索 return OK; }//BSTree_Delete_key
void Delete_BSTree（BiThrTree &T）//根据教材中给出的删除二叉排序树的子树 T 的算法，按照线索二叉树的结构作了一些改动 { q=T; if（!T->ltag&&T->rtag）//结点无右子树,此时只需重接其左子树 T=T->lchild; else if（T->ltag&&!T->rtag）//结点无左子树,此时只需重接其右子树 T=T->rchild; else if（!T->ltag&&!T->rtag）//结点既有左子树又有右子树 { p=T;r=T->lchild; while（!r->rtag） { s=r; r=r->rchild;//找到结点的前驱 r 和 r 的双亲 s } T->data=r->data;//用 r 代替 T 结点 if（s!=T） s->rchild=r->lchild; else s->lchild=r->lchild;//重接 r 的左子树到其双亲结点上 q=r; }//else free（q）; //删除结点 }
//Delete_BSTree 分析：本算法采用了先求出 x 结点的前驱和后继，再删除 x 结点的办法，这样修改线索时会比较简单，直接让前驱的线索指向后继就行了。如果试图在删除 x 结点的同时修改线索，则问题反而复杂化了。
38．试写一算法，将两棵二叉排序树合并为一棵二叉排序树。
答：
void BSTree_Merge（BiTree &T,BiTree &S）//把二叉排序树 S 合并到 T 中 { if（S->lchild）BSTree_Merge（T,S->lchild）;//递归合并左子树 if（S->rchild） BSTree_Merge（T,S->rchild）;//递归合并右子树 Insert_Key（T,S）;//插入元素 }//BSTree_Merge
void Insert_Node（Bitree &T,BTNode *S）//把树结点 S 插入到 T 的合适位置上 { if（S->data>T->data）//大于当前结点的元素值 { if（!T->rchild） T->rchild=S; else Insert_Node（T->rchild,S）;//插入到右子树中 } else if（S->data<T->data）//小于当前结点的元素值 { if（!T->lchild） T->lchild=S; else Insert_Node（T->lchild,S）;//插入到左子树中 } S->lchild=NULL; //插入的新结点必须和原来的左右子树断绝关系 S->rchild=NULL; //否则会导致树结构的混乱 }//Insert_Node 分析：这是一个与课本上不同的插入算法，在合并过程中，并不释放或新建任何结点，而是采取修改指针的方式来完成合并。这样就必须按照后序序列把一棵树中的元素逐个连接到另一棵树上，否则将会导致树的结构的混乱。
39．试写一算法，将一棵二叉排序树分裂为两棵二叉排序树，使卿其中一棵树的所有结点的关键字都小于或等于 x，另一棵树的任一结点的关键字均大于 x。
答：
void BSTree_Split（BiTree &T,BiTree &A,BiTree &B,int x）//把二叉排序树 T 分裂为两棵二叉排序树 A 和 B，其中 A 的元素全部小于等于 x，B 的元素全部大于 x { if（T->lchild）BSTree_Split（T->lchild,A,B,x）;//递归分裂左子树 if（T->rchild） BSTree_Split（T->rchild,A,B,x）;//递归分裂右子树 if（T->data<=x） Insert_Node（A,T）; else Insert_Node（B,T）;//将元素结点插入合适的树中 }//BSTree_Split
void Insert_Node（Bitree &T,BTNode *S）//把树结点 S 插入到 T 的合适位置上 { if（!T）T=S;//考虑到刚开始分裂时树 A 和树 B 为空的情况 else if（S->data>T->data）//其余部分与上一题同 { if（!T->rchild） T->rchild=S; else Insert_Node（T->rchild,S）; } else if（S->data<T->data） { if（!T->lchild） T->lchild=S; else Insert_Node（T->lchild,S）; } S->lchild=NULL; S->rchild=NULL; }//Insert_Key
40．在平衡二叉排序树的每个结点中增设一个 lsize 域，其值为它的左子树中的结点数加 1。试写一时间复杂度为 O（logn）的算法，确定树中第 k 小的结点的位置。
答：
typedef struct { int data; int bf; int lsize;//lsize 域表示该结点的左子树的结点总数加 1 BlcNode *lchild,*rchild; } BlcNode,*BlcTree;//含 lsize 域的平衡二叉排序树类型

BTNode *Locate_BlcTree（BlcTree T,int k）//在含 lsize 域的平衡二叉排序树 T 中确定第 k 小的结点指针 { if（!T） return NULL;//k 小于 1 或大于树结点总数 if（T->lsize==k） return T;//找到结点 else if（T->lsize>k） return Locate_BlcTree（T->lchild,k）;//在左子树中寻找 else return Locate_BlcTree（T->rchild,k-T->lsize）;//在右子树中寻找,注意要修改 k 的值 }//Locate_BlcTree

41．为 B＋树设计结点类型并写出算法，随机（而不是顺序）地查找给定的关键字 K，求得它所在的叶子结点指针和它在该结点中的位置（提示：B＋树中有两种类型的指针，结点结构也不尽相同，考虑利用记录的变体。

答：

typedef struct { enum {LEAF,BRANCH} tag;//结点类型标识 int keynum; BPLink parent;//双亲指针 int key[MAXCHILD];//关键字 union { BPLink child[MAXCHILD]; //非叶结点的孩子指针 struct { rectype *info[MAXCHILD];//叶子结点的信息指针 BPNode *next;//指向下一个叶子结点的链接 } leaf; } } BPNode,*BPLink,*BPTree;//B+树及其结点的数据类型

Status BPTree_Search（BPTree T,int key,BPNode *ptr,int pos）//B+树中按关键字随机查找的算法，返回包含关键字的叶子结点的指针 ptr 以及关键字在叶子结点中的位置 pos { p=T; while（p.tag==BRANCH）//沿分支向下查找 { for（i=0;i<p->keynum&&key>p->key[i]; i++）;//确定关键字所在子树 if（i==p->keynum） return ERROR;//关键字太大 p=p->child[i]; } for（i=0;i<p->keynum&&key!=p->key[i];i++）;//在叶子结点中查找 if（i==p->keynum） return ERROR;//未找到关键字 ptr=p;pos=i; return OK; }//BPTree_Search

42．假设 Trie 树上叶子结点的最大层次为 h，同义词放在同一叶子结点中，试写在 Trie 树中插人一个关键字的算法。

答：

void TrieTree_Insert_Key（TrieTree &T,StringType key）//在 Trie 树 T 中插入字符串 key（StringType 的结构见第四章） { q=（TrieNode*）malloc（sizeof（TrieNode））;//动态创建结点 q->kind=LEAF; q->lf.k=key; //建叶子结点 klen=key[0]; p=T;i=1; while（p&&i<=klen&&p->bh.ptr[ord（key[i]）]） {//自上而下查找 last=p; p=p->bh.ptr[ord（key[i]）]; i++; } if（p->kind==BRANCH）//如果最后落到分支结点（无同义词） { p->bh.ptr[ord（key[i]）]=q;//直接连上叶子 p->bh.num++; } else//如果最后落到叶子结点（有同义词） { r=（TrieNode*）malloc（sizeof（TrieNode））;//建立新的分支结点 last->bh.ptr[ord（key[i-1]）]=r;//用新分支结点取代老叶子结点和上一层的联系 r->kind=BRANCH;r->bh.num=2; r->bh.ptr[ord（key[i]）]=q; r->bh.ptr[ord（p->lf.k[i]）]=p;//新分支结点与新老两个叶子结点相连 } }//TrieTree_Insert_Key 分析：当自上而下的查找结束时,存在两种情况。①树中没有待插入关键字的同义词,此时只要新建一个叶子结点并连到分支结点上即可。②有同义词，此时要把同义词的叶子结点与树断开，在断开的部位新建一个下一层的分支结点，再把同义词和新关键字的叶子结点连到新分支结点的下一层。

43．同 42 的假设，试写在 Trie 树中删除一个关键字的算法。

答：

Status TrieTree_Delete_Key（TrieTree &T,StringType key）//在 Trie 树 T 中删除字符串 key { p=T;i=1; while（p&&p->kind==BRANCH&&i<=key[0]）//查找待删除元素 { last=p; p=p->bh.ptr[ord（key[i]）]; i++; } if（p&&p->kind==LEAF&&p->lf.k=key）//查找到待删除元素 { last->bh.ptr[ord（key[i-1]）]=NULL; free（p）;//删除元素 return OK; } else return ERROR;//未找到待删除元素 }//TrieTree_Delete_Key

44．已知某哈希表的装载因子小于1，哈希函数H（key）为关键字（标识符）的第一个字母在字母表中的序号，处理冲突的方法为线性探测开放定址法。试编写一个按第一个字母的顺序输出哈希表中所有关键字的算法。

答：

void Print_Hash（HashTable H）//按第一个字母顺序输出Hash表中的所有关键字，其中处理冲突采用线性探测开放定址法 { for（i=1;i<=26;i++） for（j=i;H.elem[j].key;j=（j+1）%hashsize[sizeindex]）//线性探测 if（H（H.elem[j].key）==i） printf（"%s\n",H.elem[j]）; } //Print_Hash

int H（char *s）//构造Hash函数 { if（s） return s[0]-96;//求关键字第一个字母的小写字母序号 else return 0; }//H

45．假设哈希表长为m，哈希函数为H（x），用链地址法处理冲突。试编写输人一组关键字并建造哈希表的算法。

答：

typedef *LNode[MAXSIZE] CHashTable;//链地址Hash表类型

//本算法不考虑排序问题.

Status Build_Hash（CHashTable &T,int m）//输入一组关键字,建立Hash表，表长为m，用链地址法处理冲突 { if（m<1） return ERROR; T=malloc（m*sizeof（WORD））;//建立表头指针向量 for（i=0;i<m;i++） T[i]=NULL; while（（key=Inputkey（））!=NULL）//假定Inputkey函数用于从键盘输入关键字 { q=（LNode*）malloc（sizeof（LNode））;//动态创建包含关键字的结点 q->data=key;q->next=NULL; n=H（key）; if（!T[n]） T[n]=q;//作为链表的第一个结点 else { for（p=T[n];p->next;p=p->next）; p->next=q;//插入链表尾部 } }//while return OK; }//Build_Hash

46．假设有一个1000×1000的稀疏矩阵，其中1%的元素为非零元素，现要求用哈希表作存储结构。试设计一个哈希表并编写相应算法，对给定的行值和列值确定矩阵元素在哈希表上的位置。请将你的算法与在稀疏矩阵的三元组表存储结构上存取元素的算法（不必写出）进行时间复杂度的比较。

答：

Status Locate_Hash（HashTable H,int row,int col,KeyType key,int &k）//根据行列值在Hash表表示的稀疏矩阵中确定元素key的位置k { h=2*（100*（row/10）+col/10）;//构造Hash函数 while（H.elem[h].key&&!EQ（H.elem[h].key,key）） h=（h+1）%20000;//处理冲突 if（EQ（H.elem[h].key,key）） k=h; else k=NULL; }//Locate_Hash 分析：本算法所使用的Hash表长20000，装填因子为50%，Hash函数为行数前两位和列数前两位所组成的四位数再乘以二，用线性探测法处理冲突，当矩阵的元素是随机分布时，查找的时间复杂度为O（1）。

9.3 考研真题与典型题详解

一、选择题

1．在一株高度为2的5阶B树中，所含关键字的个数最少是（ ）[2013年联考真题]

A．5

B．7

C．8

D．14

【答案】A 查看答案

【解析】根据 B 树的定义可知，跟结点最少含有 max（2,（m-1））个关键字，高度为 2 的阶 B 树最少有（5-1）+1＝5 个关键字，其中根节点含有（5-1）个关键字，第 2 层结点含有 1 关键字。

2．若将关键字 1，2，3，4，5，6，7 依次插入到初始为空的平衡二叉树 T 中，则 T 中平衡因子为 0 的分支结点的个数是（　　）[2013 年联考真题]

A．0

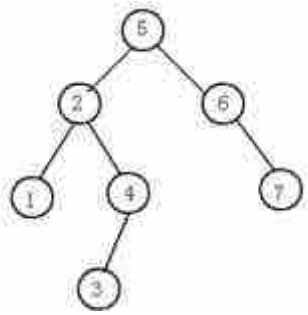
B．1

C．2

D．3

【答案】D 查看答案

【解析】将图中给定的关键字序列依次插入到平衡树中，构成的平衡树如下图所示,由图可知平衡因子为 0 的分支结点为 3 个叶子结点，故答案为 D。



3．在任意一棵非空二叉排序树 T1 中，删除某结点 v 之后形成二叉排序树 T2，再将 v 插入 T2 形成二叉排序树 T3。下列关于 T1 与 T3 的叙述中，正确的是（　　）。[2013 年联考真题]

I．若 v 是 T1 的叶结点，则 T1 与 T3 不同

II．若 v 是 T1 的叶结点，则 T1 与 T3 相同

III．若 v 不是 T1 的叶结点，则 T1 与 T3 不同

IV．若 v 不是 T1 的叶结点，则 T1 与 T3 相同

A．仅 I、III

B．仅 I、IV

C．仅 II、III

D．仅 II、IV

【答案】C 查看答案

【解析】在一棵二叉排序树中删除一个结点后再将此结点插入到二叉排序树中，如果删除的结点是叶子结点那么在插入结点后，后来的二叉排序树与删除结点之前相同。如果删除的结点不是叶子结点，那么再插入这个结点后，后来的二叉树可能发生变化，不完全相同。

4．现在有一颗无重复关键字的平衡二叉树（AVL 树），对其进行中序遍历可得到一个降序序列。下列关于该平衡二叉树的叙述中，正确的是（　　）。[2015 年联考真题]

A．根节点的度一定为 2

B．树中最小元素一定是叶节点

C．最后插入的元素一定是叶节点

D．树中最大元素一定是无左子树

【答案】D 查看答案

【解析】二叉树的中序遍历定义是“若二叉树为空，则空操作；否则：①中序遍历左子树；②访问根节点；③中序遍历右子树”。A 项错误，当树中仅有一个或者两个结点时，根节点的度就可能不为 2；B 项错误，树中最小元素是中序遍历时最后访问的节点，当没有右子树时，最后访问的节点是根节点；C 项错误，当最后插入的元素破坏树的平衡后，树会进行调整，使其成为中间节点；D 项正确，由中序遍历的特点可知，左子树的值大于根节点，所以最大元素一定没有左子树。

5．用哈希（散列）方法处理冲突（碰撞）时可能出现堆积（聚集）现象，下列选项中，会受堆积现象直接影响的是（　　）[2014 年联考真题]

A．存储效率

B．数列函数

C．装填（装载）因子

D．平均查找长度

【答案】D 查看答案

【解析】哈希方法冲突会使在查找冲突的关键字时，还要根据冲突处理办法多次比较关键字，则直接影响了平均查找长度。

6．在一棵具有 15 个关键字的 4 阶 B 树中，含关键字的结点数最多是（　　）[2014 年联考真题]

A．5

B．6

C．10

D．15

【答案】D 查看答案

【解析】m 阶 B 树非根结点含关键字个数　⌈m/2⌉ - 1 <= j <= m – 1。

4 阶 B 树非根结点含关键字 1～3 个，所以要使关键字结点数量最多，那么每个结点只有一个关键字，一共有 15 个关键字那么最多有 15 个含有关键字的结点

7．下列选项中，不能构成折半查找中关键字比较序列的是（　　）。[2015 年联考真题]

A．500，200，450，180

B．500，450，200，180

C．180，500，200，450

D．180，200，500，450

【答案】A 查看答案

【解析】折半查找的过程是：先确定待查找记录所在的范围，然后逐步缩小范围直到找到或找不到该记录为止。折半查找的关键字序列满足：对每一个关键字，其后面的所有关键字序列或者都小于等于该关键字或者都大于等于该关键字。A 项错误，第三次比较的关键字为 450，说明待查关键字位于 200～450 间，所以第四次比较时不会遇到关键字 180。

8．若查找每个记录的概率均等，则在具有 n 个记录的连续顺序文件中采用顺序查找法查找一个记录，其平均查找长度 ASL 为（　　）。[北京航空航天大学 2000 研]

A．（n-1）/ 2　B．n / 2　C．（n+1）/ 2　D．n

【答案】C 查看答案

【解析】最快查找一次成功，最慢查找 n 次成功。平均查找次数为（1+2+3+…+n）/n，那么 ASL=（n+1）/2。

9．对线性表进行折半查找时，要求线性表必须（　　）。[燕山大学 2001 研]

A．以顺序方式存储

B．以顺序方式存储，且数据元素有序
C．以链接方式存储
D．以链接方式存储，且数据元素有序
【答案】B 查看答案
【解析】二分查找又称折半查找，优点是比较次数少，查找速度快，平均性能好；其缺点是要求待查表为有序表，且插入删除困难。因此，折半查找方法适用于不经常变动而查找频繁的有序列表。折半查找方法适用于对以顺序方式存储的有序表的查找，查找效率较高。

10．下列二叉排序树中查找效率最高的是（　　）。[中南大学 2003 研]
A．平衡二叉树
B．二叉查找树
C．没有左子树的二叉排序树
D．没有右子树的二叉排序树
【答案】A 查看答案
【解析】平衡二叉树的左子树和右子树的深度之差的绝对值不超过 1。这就保证了二叉树的深度是 $\log_2 n$ 级别的。二叉查找树或者是一颗空数；或者是具有下列性质的二叉树：①若左子树不空，则左子树上所有结点的值均小于它的根结点的值；②若右子树不空，则右子树上所有结点的值均大于它的根结点的值；③左、右子树也分别为二叉排序树。
B、C、D 三项均不能保证左子树和右子树的深度之差的绝对值不超过 1，甚至很大，因此查找效率低。
11．如果要求一个线性表既能较快地查找，又能适应动态变化的要求，可以采用下列哪一种查找方法。[北京交通大学 2005 研]
A．分块　B．顺序　C．折半　D．哈希
【答案】A 查看答案
【解析】分块查找，把线形表分成若干块，块间是顺序存储的，所以查找速度较快。在每一块中的数据元素的存储顺序是任意的，所以便于线性表的动态变化。
12．分别以下列序列构造二叉排序树，与用其他三个序列所构造的结果不同的是（　　）。[合肥工业大学 2000 研]
A．（100，80，90，60，120，110，130）
B．（100，120，110，130，80，60，90）
C．（100，60，80，90，20，110，130）
D．（100，80，60，90，120，130，110）
【答案】C 查看答案
【解析】二叉排序树：左右子树都是二叉排序树，且保证右子树都比根结点大，左子树都比根结点小。据以上两点建立二叉排序树。
13．下列关于 m 阶 B-树的说法错误的是（　　）。[南京理工大学 1997 研]
A．根结点至多有 m 棵子树
B．所有叶子都在同一层次上
C．非叶结点至少有 m / 2（m 为偶数）或 m / 2+1（m 为奇数）棵子树
D．根结点中的数据是有序的
【答案】C 查看答案
【解析】B-树是一种多路搜索树（并不是二叉的），一棵 m 阶的 B-树除根结点之外的所有非终端结点至少有 ém/2ù 棵子树。
二、填空题

1．在各种查找方法中，平均查找长度与结点个数 n 无关的查找方法是_____。[中南大学 2005 研]

【答案】哈希查找查看答案

【解析】哈希查找中，平均查找长度分为成功查找长度和不成功平均查找长度，在计算过程中平均查找长度均与结点个数 n 无关。

2．在有序表 A[1．.20]中，按折半查找方法进行查找，查找长度为 4 的元素的下标从小到大依次是_____。[合肥工业大学 2000 研]

【答案】1，3，6，8，11，13，16，19 查看答案

【解析】下面是 1-20 元素需要查找的次数表。

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
4	3	4	5	2	4	3	4	5	1	4	3	4	5	2	4	5	3	4	5

3．假定有 k 个关键字互为同义词，若用线性探测再哈希法把这 k 个关键字存入哈希表中，至少要进行_____次探测。[西安电子科技大学 2001 研]

【答案】k（k+1）／2 查看答案

4．在含有 n 个结点的二叉排序树中查找一个关键字，进行关键字比较次数最大值是_____。[北京交通大学 2004 研]

【答案】n 查看答案

【解析】当二叉排序树是单枝树时，高度最高，为 n，这时关键字比较次数可能达到最大，最大值是 n。

三、综合题

1．对下面的关键字集{30，15，21，40，25，26，36，37}若查找表的装填因子为 0.8，采用线性探测再哈希方法解决冲突，做：

（1）设计哈希函数；

（2）画出哈希表；

（3）计算查找成功和查找失败的平均查找长度；

（4）写出将哈希表中某个数据元素删除的算法。

答：由于装填因子为 0.8，关键字有 8 个，所以表长为 8／0.8=10。

（1）用除留余数法，哈希函数为 14（key）=key％7。

（2）

哈希地址	0	1	2	3	4	5	6	7	8	9
关键字	21	15	30	36	25	40	26	37		
比较次数	1	1	1	3	1	1	2	6		

（3）计算查找失败时的平均查找长度，必须计算不在表中的关键字，当其哈希地址为 i（0≤i≤m-1）时的查找次数。本例中 m=10。故查找失败和查找成功时的平均查找长度分别为：

$ASL_{unsucc}=(9+8+7+6+5+4+3+2+1+1)/10=4.6$　$ASL_{succ}=16/8=2$

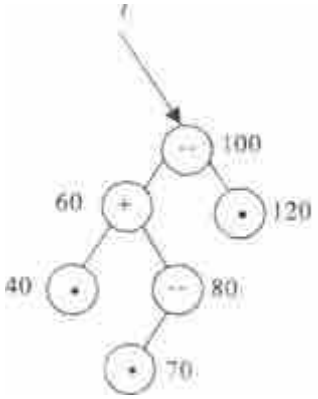
（4）

```
int Delete(int h[n], int k)
// 从哈希表h[n]中删除元素k,若删除成功返回1,否则返回0
{i=k%7;// 哈希函数用H(key)=key % 7
if(h[i]== maxint)//maxint解释成空地址
   printf("无关键字%d\n",k);return (0);}
 if(h[i]==k){h[i]=-maxint ;return (1);} //被删元素换成最大机器数的负数
 else // 采用线性探测再散列解决冲突
   {j=i;
    for(d=1;d<=n-1;d++)
       {i=(j+d)%n;                    // n为表长,此处为10
        if(h[i]== maxint)return (0); //maxint解释成空地址
        if(h[i]==k){ h[i]=-maxint ;return (1);}
        }//for }
 printf("无关键字%d\n",k);return (0)
}
```

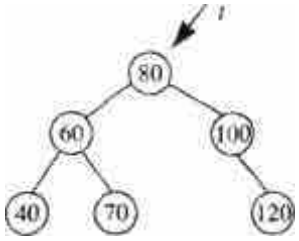
2．如图所示是一棵正在进行插入运算的 AVL 树，关键码 70 的插入使它失去平衡，按照 AVL 树的插入方法，需要对它的结构进行调整以恢复平衡。[北京大学 1997 研]

（1）请画出调整后的 AVL 树。

（2）假设 AVL 树用 llink—rlink 法存储，t 是指向根结点的指针，请用 PASCAL 语言（或 C 语言）语句表示出这个调整过程。（说明：不必写出完整的程序，只需用几个语句表示出在本题中所给出的具体情况下调整过程中指针的变化。在调整过程中还有两个指针变量 p 和 q 可以使用。



答：（1）



（2）

```
p=t->lchild; q=p->rchild; //中间变量
p->rchild=q->lchild; //q的左子女是调整后p的右子女
t->lchild=q->rchild; //q的右子女是调整后t的左子女
                     //本例中为空指针
q->lchild=p; q->rchild=t; //调整后以q为根
t=q; //调整根结点指针
```

第 10 章　内部排序

10.1　复习笔记

一、概述

排序（sorting）是计算机程序设计中的一种重要操作，它的功能是将一个数据元素（或记录）的任意序列，重新排列成一个按关键字有序的序列。

由于待排序的记录数量不同，使得排序过程中涉及的存储器不同，可将排序方法分为两大类：一类是内部排序；另一类是外部排序。本章主要介绍内部排序。

1．稳定排序方法和不稳定排序方法定义

K_m（$1\leq m\leq n$）是关键字，假设 $K_i=K_j$（$1\leq i\leq n$，$1\leq j\leq n$，$i\neq j$），且在排序前的序列中 R_i 领先于 R_j，则称所用的排序方法是稳定的；反之，若可能使排序后的序列中 R_j 领先于 R_i，则称所用的排序方法是不稳定的。

2．内部排序和外部排序的定义

（1）内部排序

内部排序指的是待排序记录存放在计算机随机存储器进行的排序过程。

（2）外部排序

外部排序指的是排序记录的数量很大，以致内存一次不能容纳全部记录，在排序过程中尚需对外存进行访问的排序过程。

3．内部排序的分类

（1）按依据的原则分类

按排序过程中依据的不同原则对内部排序方法进行分类，则大致可分为插入排序、交换排序、选择排序、归并排序和基数排序等五类；

（2）按所需的工作量分类

如果按内排过程中所需的工作量来区分，则可分为三类：

①简单的排序方法，时间复杂度为 O（n^2）；

②先进的排序方法，时间复杂度为 O（nlogn）；

③基数排序，时间复杂度为 O（d·n）。

4．内部排序的主要操作

通常，在排序的过程中需进行两种基本操作：

（1）比较两个关键字的大小；

（2）将记录从一个位置移动至另一个位置。前一个操作对大多数排序方法来说都是必要的，而后一个操作可以通过改变记录的存储方式来予以避免。待排序的记录序列可有下列三种存储方式：

①待排序的一组记录存放在地址连续的一组存储单元上，在这种存储方式中，记录之间的次序关系由其存储位置决定，则实现排序必须移动记录；

②待排序的一组记录存放在静态链表中，记录之间的次序关系由指针指示，则实现排序不需要移动记录，仅需修改指针即可；

③待排序的记录本身存储在一组地址连续的存储单元内，同时另设一个指示各个记录存储位置的地址向量，在排序过程中不移动记录本身，而移动地址向量中这些记录的“地址”，排序结束之后再按照地址向量中的值调整记录的存储位置。

二、插入排序

1．直接插入排序

（1）直接插入排序的定义

直接插入排序（straight insertion sort）是一种最简单的排序方法。它的基本操作是将一个记录插入到已排好序的有序表中，从而得到一个新的记录数增 l 的有序表。

（2）排序过程

整个排序过程进行 n-l 趟插入，即：先将序列中的第 1 个记录看成是一个有序的子序列，然后从第 2 个记录起逐个进行插入，直至整个序列变成按关键字非递减的有序序列为止。

（3）效率

直接插入排序的算法简洁，容易实现，时间复杂度为 O（n^2）。

2．其它插入排序

（1）折半插入排序

由于插入排序的基本操作是在一个有序表中进行查找和插入，因此这个“查找”可利用“折半查找”来实现，由此进行的插入排序称之为折半插入排序（Binary Insertion Sort）。折半插入排序所需的附加存储空间和直接插入排序相同，从时间上比较，折半插入排序仅减少了关键字间的比较次数，而记录的移动次数不变。因此，折半插入排序的时间复杂度仍为

O（n^2）。

（2）2-路插入排序

2-路插入排序是在折半插入排序的基础做了进一步改进，其目的是减少排序过程中移动记录的次数，但为此需要 n 个记录的辅助空间。在 2-路插入排序中，移动记录的次数约为

n^2/8。因此，2-路插入排序只能减少移动记录的次数，而不能绝对避免移动记录。并且，当 r[1]是待排序记录中关键字最大或最小的记录时，2-路插入排序就完全失去它的优越性。

（3）表插入排序

表插入排序的基本操作仍是将一个记录插入到已排好序的有序表中，与直接插入排序的不同之处仅在以修改 2n 次指针值代替移动记录，排序过程中所需进行的关键字间的比较次数

相同。因此，表插入排序的时间复杂度仍是 O（n^2）。

3．希尔排序

希尔排序（Shell Sort）又称“缩小增量排序”（diminishing increment sort），它也是一种插入排序，但在时间效率上有较大的改进。

希尔排序的基本思想是：先将整个待排记录序列分割成为若干子序列分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。

希尔排序的特点是：子序列的构成不是简单地“逐段分割”，而是将相隔某个“增量”的记录组成一个子序列。希尔排序示例如图 10-1 所示。

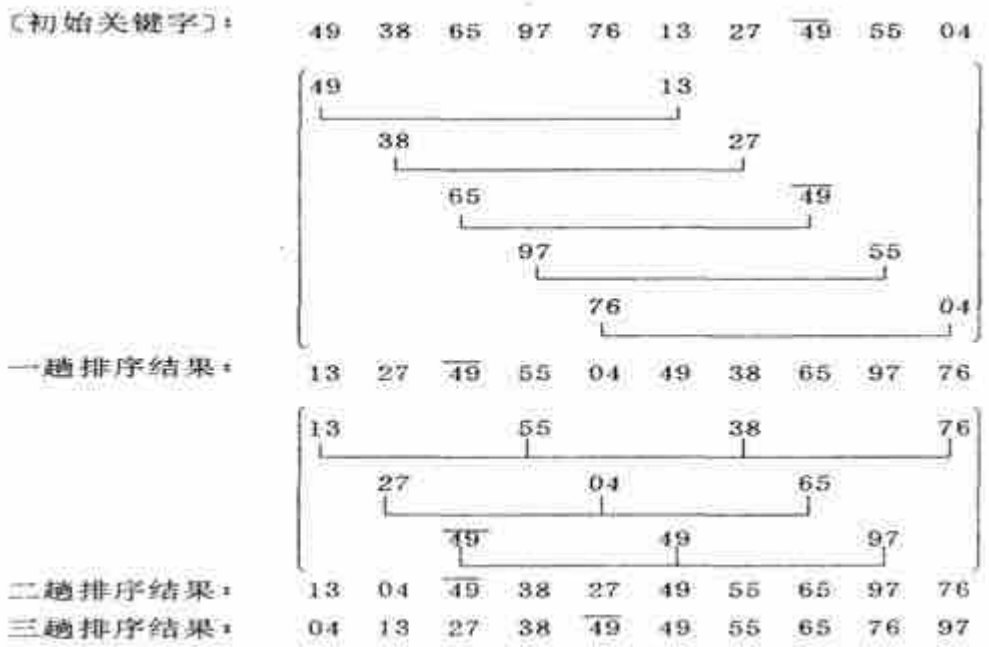


图 10-1 希尔排序示例

三、快速排序

快速排序是一类借助“交换”进行排序的方法。

1．起泡排序

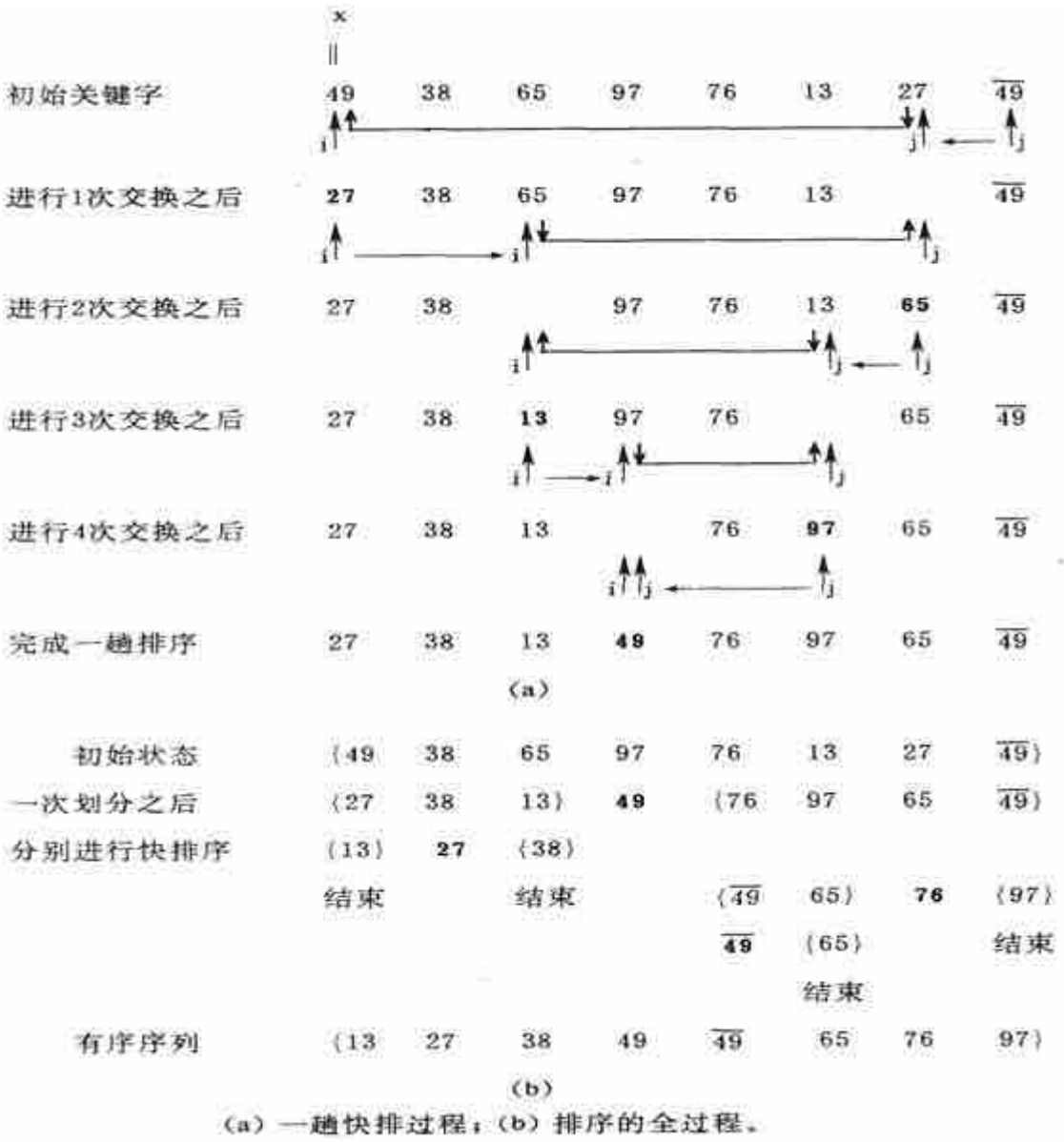
在起泡排序（bubble sort）的过程中，关键字较小的记录逐趟向上移动，而关键字较大的记录向下移动，每一趟排序关键字最大的记录被筛选出。起泡排序总的时间复杂度为

O（n^2）。

2．快速排序

快速排序是对起泡排序的一种改进。快速排序（quick sort）的基本思想是，通过一趟排序将待排记录分割成独立的两部分，其中一部分记录的关键字均比另一部分记录的关键字小，则可分别对这两部分记录继续进行排序，以达到整个序列有序。

一趟快速排序的过程如图 l0-2（a）所示，整个快速排序的过程可递归进行。若待排序列中只有一个记录，显然已有序，否则进行一趟快速排序后再分别对分割所得的两个子序列进行快速排序，如图 10-2（b）所示。



(a) 一趟快排过程；(b) 排序的全过程。

图 10-2 快速排序示例

四、选择排序

选择排序（selection sort）的基本思想是：每一趟在 n-i+1（i=1，2，…，n-1）个记录中选取关键字最小的记录作为有序序列中第 i 个记录。

1．简单选择排序

一趟简单选择排序的操作为：通过 n-i 次关键字间的比较，从 n-i+1 个记录中选出关键字最小的记录，并和第 i（1≤i≤n）个记录交换之。

简单选择排序总的时间复杂度是 O（n^2）。

2．树形选择排序

（1）定义

树形选择排序（tree selection sort），又称锦标赛排序，是一种按照锦标赛的思想进行选择排序的方法。

（2）排序过程

首先对 n 个记录的关键字进行两两比较，然后在其中较小者之间再进行两两比较，如此重复，直至选出最小关键字的记录为止。这个过程可用一棵有 n 个叶子结点的完全二叉树表示。

（3）优缺点

优点：树形选择排序总的时间复杂度为 O（$\log_2 n$）。

缺点：这种排序方法有辅助存储空间较多、“最大值”进行多余的比较等缺点。

3．堆排序

（1）排序过程

若在输出堆顶的最小值之后，使得剩余 n-1 个元素的序列又重建成一个堆，则得到 n 个元素中的次小值。如此反复执行，便能得到一个有序序列，这个过程称之为堆排序。

这种自堆顶至叶子的调整过程称为“筛选”。堆排序示例如图 10-3 所示。

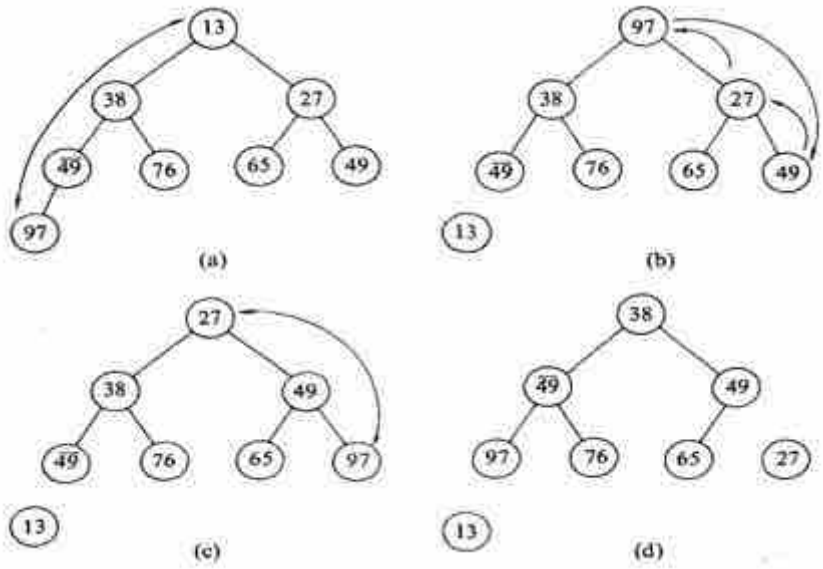


图 10-3 堆排序示例

（2）优缺点

堆排序方法对记录数较少的文件并不值得提倡，但对 n 较大的文件还是很有效的。因为其运行时间主要耗费在建初始堆和调整建新堆时进行的反复“筛选”上。堆排序只需要一个记录大小的辅助空间，每个待排序的记录仅占有一个存储空间。

堆排序在最坏的情况下，其时间复杂度也为 O（nlogn）。相对于快速排序来说，这是堆排序的最大优点。此外，堆排序仅需一个记录大小供交换用的辅助存储空间。

五、归并排序

1．定义

“归并”的含义是将两个或两个以上的有序表组合成一个新的有序表。

2．排序过程

假设初始序列含有 n 个记录，则可看成是 n 个有序的子序列，每个子序列的长度为 1，然后两两归并，得到 $\lceil \frac{n}{2} \rceil$ 个长度为 2 或 1 的有序子序列；再两两归并，…，如此重复，直至得到一个长度为 n 的有序序列为止，这种排序方法称为 2-路归并排序。2-路归并排序中的核心操作是将一维数组中前后相邻的两个有序序列归并为一个有序序列。

3．效率及优点

实现归并排序需和待排记录等数量的辅助空间，其时间复杂度为 O（nlogn）。与快速排序和堆排序相比，归并排序的最大特点是，它是一种稳定的排序方法。

六、基数排序

1．多关键字排序

为了实现多关键字排序，通常有两种方法：最高位优先法（Most Significant Digit first，简称 MSD 法）和最低位优先法（Least Significant Digit first 简称 LSD 法）。

2．链式基数排序

链式基数排序示例如图 10-4 所示

```
listtp= ARRAY [1..n] OF rednode;      (静态链表)
arrtp=ARRAY [chtype] OF integer;      (指针数组)
```

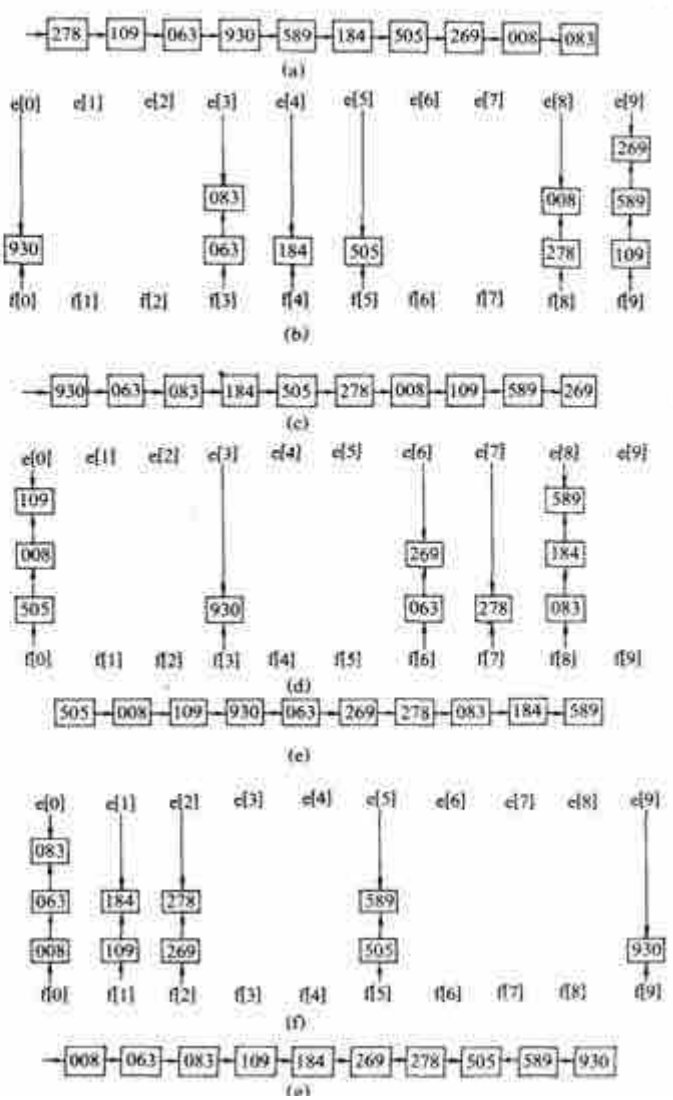


图 10-4 链式基数排序示例

七、排序方法比较

综合比较各种内部排序方法的结果如表 10-1 所示：

表 10-1 内部排序算法比较

排序方法	平均时间	最坏情况	辅助存储
简单排序	$O(n^2)$	$O(n^2)$	O(1)
快速排序	O(nlogn)	$O(n^2)$	O(logn)
堆排序	O(nlogn)	O(nlogn)	O(1)
归并排序	O(nlogn)	O(nlogn)	O(n)
基数排序	O(d(n+rd))	O(d(n+rd))	O(rd)

根据表 10-1 可以得出以下几个结论：

（1）从平均时间性能而言，时间复杂度为 O（nlogn）的算法所需时间最少，但快速排序在最坏情况下的时间性能不如堆排序和归并排序，而后两者相比较的结果是，在 n 较大时，归并排序所需时间较堆排序少，但它所需的辅助存储最多。

（2）表中的“简单排序”包括除希尔排序之外的所有插入排序、起泡排序和简单选择排序，其中直接插入排序最简单，当序列中的记录“基本有序”或 n 值较小时，它是最佳的排序方法，因此常将它和快速排序、归并排序等其它的排序方法结合在一起使用。

（3）基数排序的时间复杂度也可表示成 O（d·n），它最适用于 n 值很大而关键字较小的序列。若关键字也很大，而序列中大多数记录的“最高位关键字”均不同，则亦可先按“最高位关键字”不同将序列分成若干“小”的子序列，然后进行直接插入排序。

（4）从方法的稳定性来比较，基数排序是稳定的内排方法，所有时间复杂度为 O（n^2）的简单排序法也是稳定的，而快速排序、堆排序和希尔排序等时间性能较好的排序方法都是不稳定的。一般来说，排序过程中的“比较”是在“相邻的两个记录关键字”间进行的排序方法是稳定的。注意，稳定性是由方法本身决定的，对不稳定的排序方法而言，不管其描述形式如何，总能举出一个说明不稳定的实例来。反之，对稳定的排序方法，总能找到一种不引起不稳定的描述形式。由于大多数情况下排序是按记录的主关键字进行的，则所用的排序方法是否稳定无关紧要，若排序按记录的次关键字进行，则应根据问题所需慎重选择排序方法及其描述算法。

综上所述，在所有排序方法中，没有哪一种是绝对最优的。因此，在使用时根据不同状况适当选用，甚至可以将多种方法结合起来使用。

10.2 强化习题详解

【基础知识题】

1．以关键码序列（503

（1）直接插入排序;

（2）希尔排序（增量 d[1]=5）;

（3）快速排序;

（4）堆排序;

（5）归并排序;

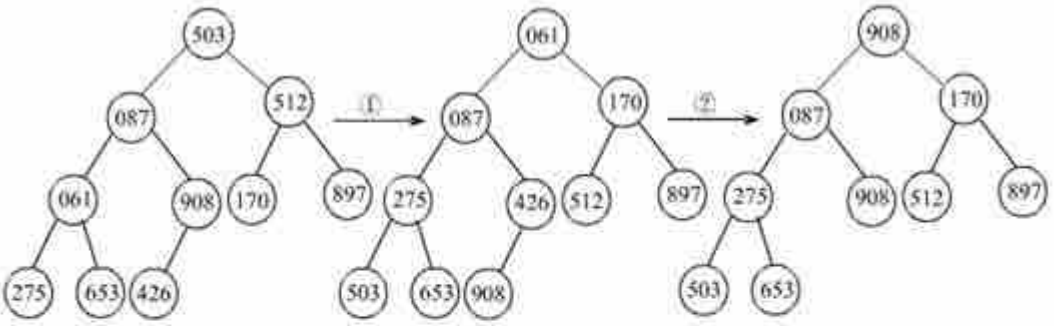
（6）基数排序。

答：

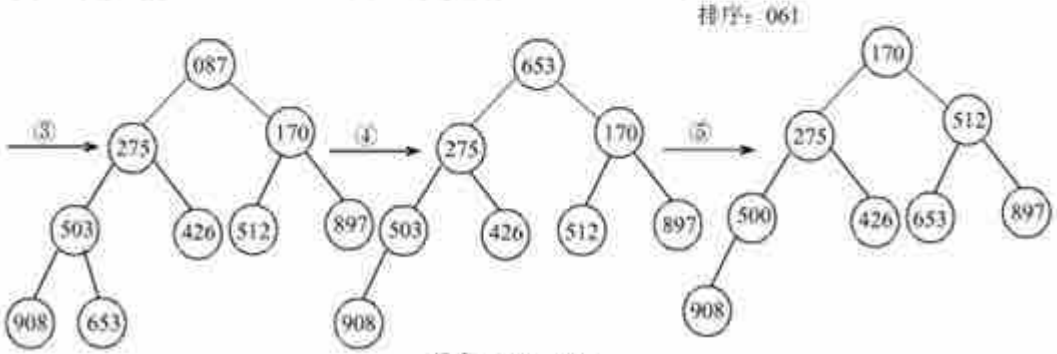
（1）直接插入排序：

初始关键字：　　（503）　087 512 061 908 170 897 275 653 426

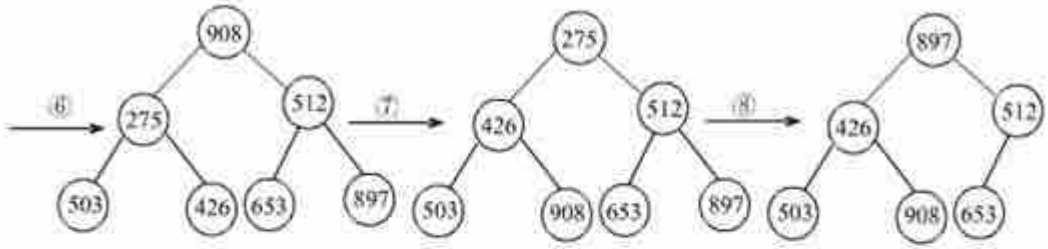
第一趟结果：（087）　（087 503） 512 061 908 170 897 275 653 426
第二趟结果：（512）　（087 503 512） 061 908 170 897 275 653 426
第三趟结果：（061）　（061 087 503 512） 908 170 897 275 653 426
第四趟结果：（908）　（061 087 503 512 908） 170 897 275 653 426
第五趟结果：（170）　（061 087 170 503 512 908） 897 275 653 426
第六趟结果：（897）　（061 087 170 503 512 897 908） 275 653 426
第七趟结果：（275）　（061 087 170 275 503 512 897 908） 653 426
第八趟结果：（653）　（061 087 170 275 503 512 653 897 908） 426
第九趟结果：（426）　（061 087 170 275 426 503 512 653 897 908）
（2）希尔排序：
初始关键字：　503 087 512 061 908 170 897 275 653 426
第一趟结果：d[1]=5 170 087 275 061 426 503 897 512 653 908
第二趟结果：d[2]=3 061 087 275 170 426 503 897 512 653 908
第三趟结果：d[3]=1 061 087 170 275 426 503 512 653 897 908
（3）快速排序：
初始关键字：503 087 512 061 908 170 897 275 653 426
第一次交换：426 087 512 061 908 170 897 275 653 503
第二次交换：426 087 503 061 908 170 897 275 653 512
第三次交换：426 087 275 061 908 170 897 503 653 512
第四次交换：426 087 275 061 503 170 897 908 653 512
第五次交换：426 087 275 061 170 503 897 908 653 512
第一趟结果：426 087 275 061 170 503 897 908 653 512
将关键码序列从 503 处分开，前半部分和后半部分分别进行快速排序：
前半部分：　426 087 275 061 170
第一次交换：170 087 275 061 426
后半部分：　897 908 653 512
第一次交换：512 908 653 897
第二次交换：512 897 653 908
第三次交换：512 653 897 908
第二趟结果：170 087 275 061 426 503 512 653 897 908
将关键码序列从 426 处分开，前半部分和后半部分分别进行快速排序：
前半部分：　170 087 275 061
第一次交换：061 087 275 170
第二次交换：061 087 170 275
后半部分已经有序，不必进行交换操作。
第三趟结果：061 087 170 275 426 503 512 653 897 908
（4）堆排序
初始关键字：503 087 512 061 908 170 897 275 653 426



排序：061

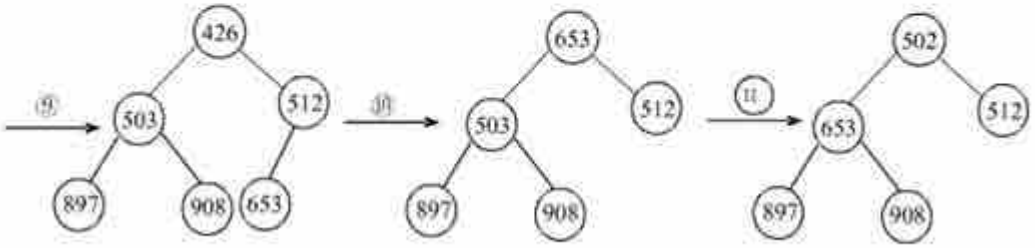


排序：061　087

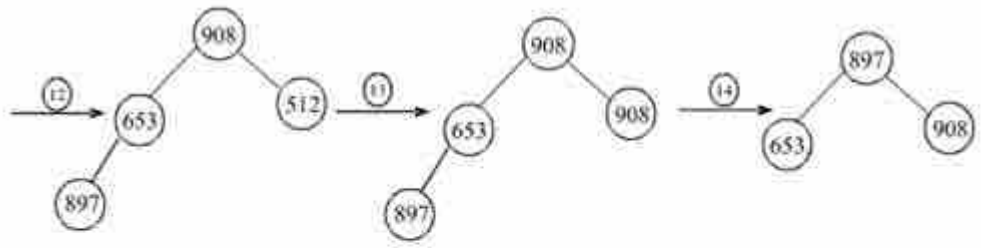


排序：061　087　170

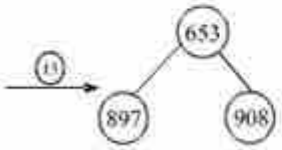
排序：061　087　170　275



排序：061　087　170　275　426



排序：061 087 170 275 426 502　　排序：061 087 170 275 426 502 512



排序：061 087 275 426 502 512 653 897 908

（5）归并排序

初始关键字：503　087 512　061 908　170 897 275 653 426

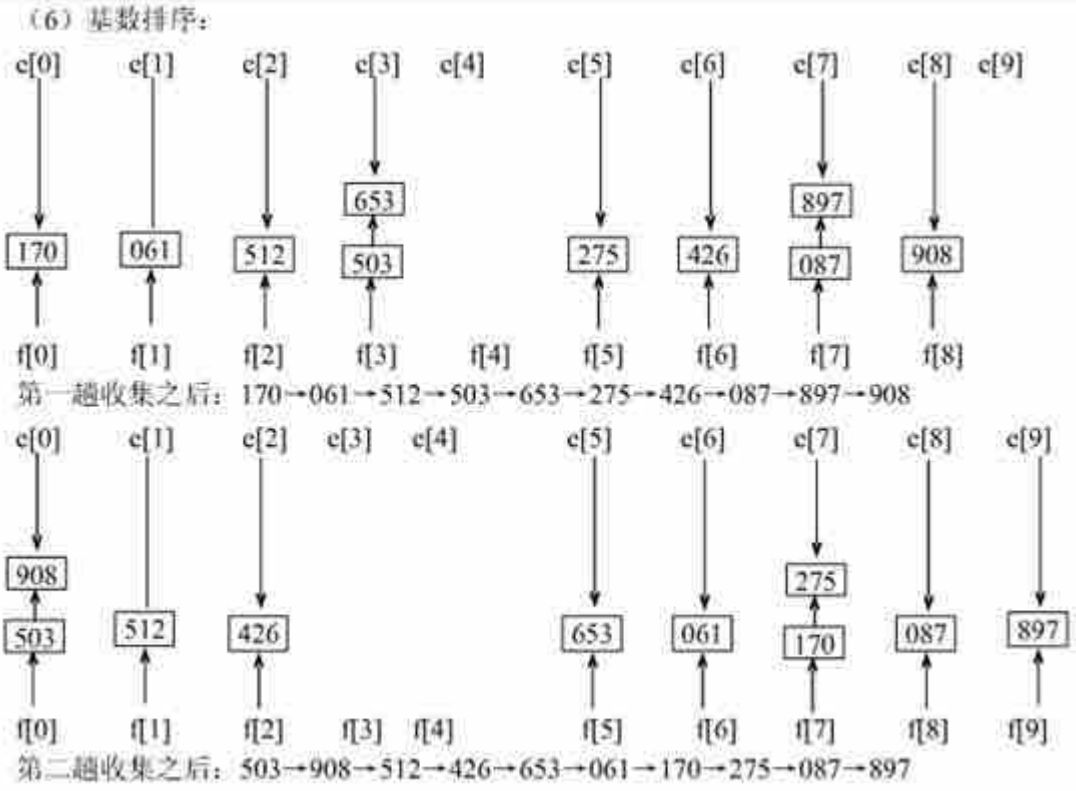
第一趟结果：（087 503）　（061 512）　（170 908）　（275 897）　（426 653）

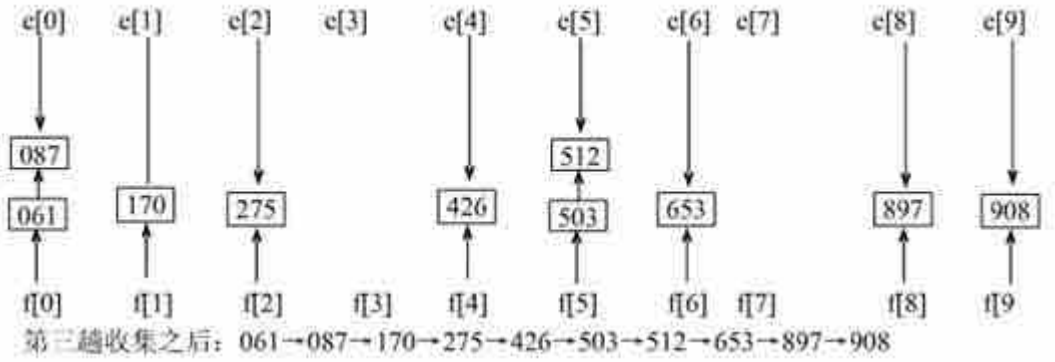
第二趟结果：（061 087 503 512）　（170 275 897 908）　（426 653）

第三趟结果：（061 087　503 512）　（170 275　426　653　897 908）

第四趟结果：（061 087　170 275　426 503　512　653　897 908）

（6）基数排序





2．若对下列关键字序列按教科书 10.3 节和 10.5 节中所列算法进行快速排序和归并排序,分别写出三次调用过程 Partition 和过程 Merge 后的结果。

（Tim, Kay, Eva,Roy, Dot, Jon, Kim, Ann, Tom, Jim, Guy, Amy）

答：

三次调用过程 Partition 即三趟快速排序，三次调用过程 Merge 即三趟归并排序。

三次调用过程Partition的结果分别为

（Amy, Kay, Eva, Roy, Dot, Jon, Kim, Ann, Guy, Jim）Tim（Tom）

Amy（Kay, Eva, Roy, Dot, Jon, Kim, Ann, Guy, Jim）Tim（Tom）

Amy（Jim, Eva, Guy, Dot, Jon, Ann）Kay（Kim, Roy）Tim（Tom）

三次调用过程Merge的结果分别为

（Kay, Tim, Eva, Roy, Dot, Jon, Kim, Ann, Tom, Jim, Guy, Amy）

（Eva, Kay, Tim, Roy, Dot, Jon, Kim, Ann, Tom, Jim, Guy, Amy）

（Eva, Kay, Tim, Dot, Roy, Jon, Kim, Ann, Tom, Jim, Guy, Amy）

3．试问在 1 题所列各种排序方法中,哪些是稳定的？哪些是不稳定的？并为每一种不稳定的排序方法举出一个不稳定的实例。

答：稳定的排序：直接插入排序、归并排序、基数排序；

不稳定的排序：快速排序、堆排序、希尔排序。

举例：下例子中的 275*和 275 是关键码相等的不同数据元素，为区分两者，在后一个 275 上加*，以示区别。如果排序后 275*跑到 275 前面，表示此排序方法不稳定。

（1）快速排序： {512 275 275*} 基准 512，与 275*对调

{275* 275 512} 结果

（2）堆排序： {275 275* 061 170} 已经是最大堆，交换 275 与 170

{170 275* 061 275} 对前三个调整

{275* 170 061 275} 前三个大堆，交换 275*与 061

{061 170 275* 275} 对前 2 个调整

{170 061 275* 275} 前 2 个最大堆，交换 170 与 061

{061 170 275* 275}

（3）希尔排序： {512 275 275* 061} 增量 2：1 与 3，2 与 4 逆序交换

{275* 061 512 275} 增量 1：直接插入排序

{061 275* 275 512} 结果

4．试问：对初始状态如下（长度为 n）的各序列进行直接插入排序时，至多需进行多少次关键间的比较（要求排列按关键字自小至大顺序有序）？

（1）关键字（自小至大）顺序有序：（$key_1<key_2<\ldots<key_n$）

（2）关键字（自大至小）顺序有序：（$key_1>key_2>\ldots>key_n$）

（3）序号为奇数的关键字顺序有序，序号为偶数的关键字顺序有序：
（$key_1<key_3<key_5<\ldots,key_2<key_4<\ldots$）
（4）前半个序列中的关键字顺序有序,后半个序列中的关键字逆序有序：
（$key_1<key_2<\ldots key_{n/2},key_{n/2+1}>\ldots key_n$）
答：（1）0

（2）$\frac{n(n-1)}{2}$

（3）$n-1+\sum_{i=1}^{\lceil \frac{n}{2} \rceil} i$

（4）$\lceil \frac{n}{2} \rceil -1+\sum_{i=\lceil \frac{n}{2} \rceil +1}^{n} i$

5．假设把 n 个元素的序列{a_1，a_2，…，a_n}中满足条件$a_k < \max_{1\le t\le k}\{a_t\}$的元素 a_k 称为“逆序元素”。若在一个无序序列中有一对元素 $a_i>a_j$（i<j），试问当将 a_i 和 a_j 相互交换之后（即序列由{…a_i…a_j…}变为{…a_j…a_i…}），该序列中逆序元素的个数会有什么变化？为什么？

答：考虑序列中位于a_i和a_j之间的元素$a_k(i<k<j)$的几种情况，当$a_k>a_i>a_j$时，有可能使a_i有非逆序元素变为逆序元素，但整个序列的“逆序对”不变，即该序列中逆序元素的个数不会发生改变。

6．奇偶交换排序如下所述：第一趟对所有奇数 i，将 a[i]和 a[i+1]进行比较；第二趟对所有的偶数 i，将 a[i]和 a[i+1]进行比较，若 a[i]>a[i+1]，则将两者交换；第三趟对奇数 i；第四趟对偶数 i，…，依次类推直至整个序列有序为止。
（1）试问这种排序方法的结束条件是什么？
（2）分析当初始序列为正序或逆序两种情况下，奇偶交换排序过程中所需进行的关键字比较的次数。
答：（1）设有一个布尔变量 exchange，判断在每一次做过一趟奇数项扫描和一趟偶数项扫描后是否有过交换。若 exchange=1，表示刚才有交换，还需继续做下一趟奇数项扫描和偶数项扫描；若 exchange=0，表示刚才没有交换，可以结束排序。
（2）设待排序元素序列中有 n 个元素，序列中的元素序号从 0 开始。当所有待排元素序列中的元素按关键码从大到小初始排列时，执行$m=\lfloor (n+1)/2 \rfloor$奇偶排序。当所有待排元素序列中的元素按关键码从小到大初始排列时，执行 1 趟奇偶排序。
在一趟奇偶排序过程中，若 n 为奇数，对所有的奇数项和所有偶数项扫描一遍，关键码比较次数都是$m=\lfloor n/2 \rfloor$次，总比较次数为$2\times\lfloor n/2 \rfloor=n-1$；若 n 为偶数，对所有奇数项扫描一遍，关键码比较 n/2 次，再对所有偶数项扫描一遍，排序码比较 n/2-1 次。关键码总比较次数为 n/2+n/2-1=n-1。

总之，当序列各元素按关键码值从小到大有序时，元素比较次数为 n-1，移动次数为 0；当序列各元素按关键码值从大到小逆序时，元素比较次数为$\lfloor (n+1)/2 \rfloor \times (n-1)$，移动次数为$3\times\lfloor (n+1)/2 \rfloor \times (n-1)$。

7．对长度为 n 的记录序列进行快速排序时，所需进行的比较次数依赖于这 n 个元素的初始排列。

（1）n=7 时在最好的情况下需进行多少次比较？请说明理由。

（2）对 n=7 给出一个最好情况的初始排列实例。

答：（1）快速排序的最好情况是指排序所需的“关键字间的比较次数”和“记录的移动次数”最少的情况，在 n=7 时，至少需进行 2 趟排序。

（2）例如：初始排列为 4 1 3 2 6 5 7 时，只需进行 10 次比较，2 趟排序即可完成，为最好情况。

8．试证明：当输入序列已经呈现为有序状态时，快速排序的时间复杂度为$O(n^2)$。

答：当初始记录序列按关键字有序或基本有序时，快速排序将蜕化为起泡排序。

若初始序列为“正序”序列，则只需进行一趟排序，在排序过程中进行 n-1 次关键字间的比较，且不移动记录；反之，若初始序列为“逆序”时，则需进行 n-1 趟排序，需进行$\sum_{i=2}^{n}(i-1)=n(n-1)/2$次比较，并作等数量级的记录移动。因此，总的时间复杂度为$O(n^2)$。

9．若将快速排序的一次划分改写为如下形式，重写快速排序的算法，并讨论对长度为 N 的记录序列进行快速排序时在最好的情况下所需进行的关键字间比较的次数（包括三者求中）。

```
int partition （ SqList&L, int low, int high, bool ci, bool cj）{
int i,j,m; KeyType x;
i=s; j=t; m=（s+t） DIV 2 ; x=Midkey（s,m,t）;// 三者取中值
ci=cj=FALSE;
while（i<J）{
while（（i<J）&&（R[I].KEY<=X））{
i=i+1;
if （r[i].keyi）&&（r[j].key>x））{
j=j-1;
if （r[j].key>r[j+1].key）{
r[j]←→r[j+1];cj=TRUE;
}
if （i<J）{
r[i] r[j];
if （（i>s）&&（r[i].key
if （（jr[j+1].key））cj=TRUE;
}
}
```

return i;
}// partition
答：长度为 N 的记录序列在进行快速排序时最好的情况下所需进行的关键字间的比较次数为 O（nlogn）。
10．阅读下列排序算法，并与已学算法相比较，讨论算法中基本操作的执行次数。
Void sort （SqList&r, int n）{
i=1;
while （i<N-I+1）{
min=max=1;
for （j=i+1;j<=n-i+1;++j）{
if （r[j].keyr[max].key） max=j;）
}
if （min !=i） r[min] ←→r[i];
if （max !=n-i+1）{
if （max=i） r[min] ←→r[n-i+1];
else if r[max] ←→r[n-i+1];
}
i++;
}
}//sort
答：该算法基于直接插入排序算法，是直接插入排序的改进版，基本操作为元素的比较和交换，执行次数为 O（n^2）。
11．试问:按锦标赛排序的思想，决出八名运动员之间的名次排列，至少需排多少场次的比赛（应考虑最坏的情况） ？
答：不能简单地用（n-1）+（n-2）log2n 来计算比赛场次，因为，随着 n/2 的叶子结点被调整完毕之后，树的深度会逐层减少。分别对 n=8 和 n=7 的情况推导并归纳，得到如下计算公式：比赛场次=（n-1）+n/2（k-1）+ （$n/2^2$）（k-2）+…+ n/2（k-1），其中 k=$\lceil \log_2 n \rceil$，当 n＝8 时，k=3,比赛场次=7+8/2（2）+8/4=17 场（这是最坏情况，即每次都先从叶子调整起）。
12．判别以下序列是否为堆（小顶堆或大顶堆）。如果不是,则把它调整为堆（要求记录交换次数最少）.
（1）（100,86,48,73,35,39,42,57,66,21）;
（2）（12,70,33,65,24,56,48,92,86,33）;
（3）（103,97,56,38,6623,42,12,30,52,06,20）;
（4）（05,56,20,23,40,38,29,61,35,76,28,100）.
答：（1）是大顶堆;
（2）不是堆，重新排列成小顶堆（12，24，33，65，33，56，48，92，86，70）;
（3）是大顶堆;
（4）不是堆，重新排列成小顶堆
（05，23，20，35，28，38，29，61，56，76，40，100）。
13．一个长度为 n 的序列,若去掉其中少数 k（k<<n）个记录后,序列是按关键字有序的,则称为近似有序序列。试对这种序列讨论各种简单排序方法的时间复杂度。

答：对近似有序的序列可用直接插入排序法，其时间复杂度为 O（k·n），当 k<<n，且 n 很大时，将比快速排序更有效，而起泡排序和简单选择排序时间复杂度均为 O（n^2），和 k 无关。

14．对一个由 n 个关键字不同的记录构成的序列，你能否用比 2n-3 少的次数选出这 n 和记录中关键字取最大值和关键字取最小值的记录？若能，请说明如何实现？在最坏情况下至少进行多少次比较？

答：（1）能用比 2n-3 少的次数。

（2）可以设立两个变量 max 和 min，用于存放最大元和最小元（的位置），第一次取两个元素进行比较，大的放入 max，小的放入 min；从第 2 次开始，每次取一个元素先和 max 比较，如果大于 max 则以它替换，并结束本次比较；若小于 max 则再与 min 相比较，在最好的情况下，一路比较下去都不用和 min 相比较。所以这种情况下，至少要进行 n-1 次比较就能找到最大元和最小元。

（3）最坏情况下，要进行 2n-3 次比较才能得到结果。

15．已知一个含有 n 个记录的序列，其关键字均为介于 0 和 n^2 之间的整数。若利用堆排序等方法进行排序，则时间复杂度为 O（nlogn）。如果将每个关键字 K_i，认作 $K_i=K_i^1+K_i^2$，其中 K_i^1 和 K_i^2 都是范围[0，n]中的整数，则利用基数排序只需用 O（n）的时间。推广之，若整数关键字的范围为[0，n^k]，则可得到只需时间 O（kn）的排序方法，试讨论如何实现之。

答：将每个关键字 K_i ,认作 $K_i=K_i^1+K_i^2$ 的过程使用基数排序，若整数关键字范围为[0，n^k]，使用基数排序便可得到时间复杂度为 O（kn）的排序方法。

16．已知一个单链表由 3000 个元素组成,每一个元素是一整数，其值在 1~1000000 之间。试考察在第 10 章给出的几种排序方法中，哪些方法可用于解决这个链表的排序问题？哪些不能？简述理由。

答：①快速排序、堆排序和归并排序可以解决这个链表的排序问题，而插入排序、简单选择排序和起泡排序不能。

②原因：该单链表元素个数为 3000 个，数量较大，使用简单排序时间复杂度为 O（n^2），代价太高。而快速排序、堆排序和归并排序尽管可能会占用较多的存储空间，但是时间复杂度为 O（nlogn），对于元素个数较多的情况下，所用的时间和空间都是完全可以接受的。

17．在进行多关键字排序的两种方法中,试思考在什么条件下 MSD 法比 LSD 法效率更高？

答：①LSD 即低关键字优先算法，是从最低位的关键字开始，每次都按相应的关键字位上的大小不同将全部记录进行分配和收集，直至最高位的关键字为止；

②MSD 即高关键字优先算法，需将待排序的列按高关键字大小不同进行分配形成若干较小的子序列，直至按最低位的关键字分配，最后一次性地将所有记录收集起来。

扑克牌按花色、面值从大到小排序中介绍的方法一即是 MSD 法；扑克牌按花色、面值从小到大排序中介绍的方法二即是 LSD 法；若要求最终的序列按照关键字从大到小排列，则 MSD 法比 LSD 法效率更高。

18．假设某大旅店共有 5000 个床位，每天需根据住宿旅客的文件制造一分花名册，该名册要求按省（市）的次序排列，每一省（市）按县（区）排列,又同一县（区）的旅客按姓氏排列。请你为旅店的管理人员设计一个制作这份花名册的方法。

答：采用哪种排序方法，关键要看记录结构是如何定义的，如果旅客填表如表 10-1 所示：

表 10-1

省（市）	县（区）	姓名	床位号

则按题意要求，应当采用 MSD 法，否则无法满足。但若“姓名”项在前，则必须用 LSD 才符合题意要求。

19．已知待排序的三个整数 a,b 和 c（a≠b≠c≠a）可能出现的六种排列情况的概率不等,且如表 10-2 所示:

表 10-2

a<b<c	b<a<c	a<c<b	c<a<b	b<c<a	c<b<a
0.13	0.24	0.08	0.19	0.20	0.16

试为该序列设计一个最佳排序方案,使排序过程中所需进行的关键字间的比较次数的期望值达到最小.

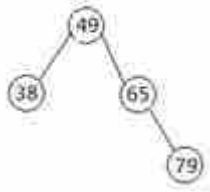
答：待排序的元素只有 3 个，元素个数较少，且基本有序，采用直接插入排序，关键字间的比较次数的期望值达到最小。

20．分别利用折半插入排序法和 2-路归并排序法对含四个记录序列进行排序,画出描述该排列过程的判定树,并比较他们所需进行的关键字间的比较次数的最大值。

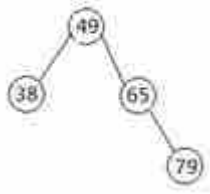
答：

例如，含有 4 个记录的序列：49,38,65,79

折半插入排序判定树：



2-路归并排序法判定树：



折半查找比较次数为 5 次，2-路归并排序比较次数为 3 次，故比较次数最大值为 5 次。

【算法设计题】

21．试编写 2-路插入排序的算法。

答：

```
void BiInsert_Sort（SqList &L）//二路插入排序的算法
{
int d[MAXSIZE]; //辅助存储空间
x=L.r[0].key; d[0]=x;
first=1;final=1;
for（i=2;i<=L.length;i++）
{
if（L.r[i].key>=x） //待插入关键字较小，插入前部
{
    for（j=final;d[j]>L.r[i].key;j--）
```

```
d[j+1]=d[j];
    d[j+1]=L.r[i].key;
    final++;
}
else //插入后部
{
    for（j=first;d[j]<L.r[i].key;j++）
    d[j-1]=d[j];
    d[（j-2）%MAXSIZE+1]=L.r[i].key;
    first=（first-2）%MAXSIZE+1; //这种形式的表达式是为了兼顾 first=1 的情况
}
}//for
for（i=first,j=1;d[i];i=i%MAXSIZE+1,j++）//将序列复制回去
L.r[j].key=d[i];
}//BiInsert_Sort
```

22．试编写链表插入的算法。

答：

```
void SLInsert_Sort（SLList &L）//静态链表的插入排序算法
{
L.r[0].key=0;L.r[0].next=1;
L.r[1].next=0; //建初始循环链表
for（i=2;i<=L.length;i++） //逐个插入
{
p=0;x=L.r[i].key;
while（L.r[L.r[p].next].key<x&&L.r[p].next）//查找插入位置
    p=L.r[p].next;
q=L.r[p].next;
L.r[p].next=i;
L.r[i].next=q;
}//for
p=L.r[0].next;
for（i=1;i<L.length;i++） //重排记录的位置
{
while（p<i） p=L.r[p].next;
q=L.r[p].next;
if（p!=i）
{
    L.r[p]<->L.r[i];
    L.r[i].next=p;
}
p=q;
}//for
}//SLInsert_Sort
```

23．改写起泡算法；将用以起控制作用的布尔变量 change 改为一个整型变量，指示每一趟排序中进行交换的最后一个记录的位置，并以它作为下一趟起泡排序循环终止的控制值。

答：

```
void Bubble_Sort1（int a[ ],int n）//对包含 n 个元素的数组 a 进行改进的冒泡排序
{
    change=n-1; //change 指示上一趟冒泡中最后发生交换的元素
    while（change）
    {
for（c=0,i=0;i<change;i++）
{
if（a[i]>a[i+1]）
{
    a[i]<->a[i+1];//交换
    c=i+1; //c 指示这一趟冒泡中发生交换的元素
}
change=c;
}//for
    }//while
}//Bubble_Sort1
```

24．编写一个双向起泡的排序算法,即相邻两遍向相反方向起泡。

答：

```
void Bubble_Sort2（int a[ ],int n）//相邻两趟是反方向起泡的冒泡排序算法
{
low=0;high=n-1; //冒泡的上下界
change=1;
while（low<high&&change）
{
change=0;
for（i=low;i<high;i++） //从上向下起泡
{
    if（a[i]>a[i+1]）
    {
    a[i]<->a[i+1];//交换
    change=1;
    }
}//for
high--; //修改上界
for（i=high;i>low;i--） //从下向上起泡
{
    if（a[i]<a[i-1]）
    {
    a[i]<->a[i-1]; //交换
    change=1;
    }
```

}//for
low++; //修改下界
}//while
}//Bubble_Sort2
25．按下述原则编写快排的非递归算法:
（1）一趟排序之后，若子序列已有序（无交换），则不参加排序，否则先对长度较短的子序列进行排序，且将另一子序列的上，下界入栈保存；
（2）若待排记录数≤3,则不再进行分割,而是直接进行比较排序。
答：

```
typedef struct {
    int low;
    int high;
} boundary; //子序列的上下边界类型
void QSort_NotRecurve（int SQList &L）//快速排序的非递归算法
{
    low=1;high=L.length;
    InitStack（S）;//S 的元素为 boundary 类型
    while（low<high&&!StackEmpty（S））//排序算法结束的条件
    {
if（high-low>2） //如果当前子序列长度大于 3 且尚未排好序
{
pivot=Partition（L,low,high）;//进行一趟划分
if（high-pivot>pivot-low）
{
    Push（S,{pivot+1,high}）;//把长的子序列边界入栈
    high=pivot-1; //短的子序列留待下次排序
}
else
{
    Push（S,{low,pivot-1}）;//把短的子序列边界入栈
    low=pivot+1; //长的子序列留待下次排序
}
}//if
else if（low<high&&high-low<3）//如果当前子序列长度小于 3 且尚未排好序
{
Easy_Sort（L,low,high）; //直接进行比较排序
low=high; //当前子序列标志为已排好序
}
else //如果当前子序列已排好序但栈中还有未排序的子序列
{
Pop（S,a）; //从栈中取出一个子序列
low=a.low;
high=a.high;
}
```

```
    }//while
}//QSort_NotRecurve
int Partition（SQList &L,int low,int high）//一趟划分的算法,与书上相同
{
    L.r[0]=L.r[low];
    pivotkey=L.r[low].key;
    while（low<high）
    {
while（low<high&&L.r[high].key>=pivotkey）
high--;//依次往前移
L.r[low]=L.r[high];
while（low<high&&L.r[low].key<=pivotkey）
low++;//依次往后移
L.r[high]=L.r[low];
    }//while
    L.r[low]=L.r[0];
    return low;
}//Partition
void Easy_Sort（SQList &L,int low,int high）//对长度小于 3 的子序列进行比较排序
{
    if（high-low==1） //子序列只含两个元素
if（L.r[low].key>L.r[high].key） L.r[low]<->L.r[high];
    else //子序列含有三个元素，进行交换
    {
if（L.r[low].key>L.r[low+1].key） L.r[low]<->L.r[low+1];
if（L.r[low+1].key>L.r[high].key） L.r[low+1]<->L.r[high];
if（L.r[low].key>L.r[low+1].key） L.r[low]<->L.r[low+1];
    }
}//Easy_Sort
```

26．编写算法，对 n 个关键字取整数值的记录序列进行整理，以使所有关键字为负值的记录排在关键字为非负值的记录之前，要求:

（1）采用顺序存储结构，至多使用一个记录的辅助存储空间；

（2）算法的时间复杂度为 O（n）；

（3）讨论算法中记录的最大移动次数。

答：

```
void Divide（int a[ ],int n）//把数组 a 中所有值为负的记录调到非负的记录之前
{
low=0;high=n-1;
while（low<high）
{
while（low<high&&a[high]>=0） high--; //以 0 作为虚拟的枢轴记录
a[low]<->a[high];//交换
while（low<high&&a[low]<0） low++;
a[low]<->a[high];
```

}//while
}//Divide
27．荷兰国旗问题：设有一个仅由红，白，蓝三种颜色的条块组成的条块序列。请编写一个时间复杂度为 O（n）的算法，使得这些条块按红，白，蓝的顺序排好，即排成荷兰国旗图案。
答：
typedef enum {RED,WHITE,BLUE} color; //三种颜色
void Flag_Arrange（color a[ ],int n）/*把由三种颜色组成的序列重排为按照红，白，蓝的顺序排列*/
{
    i=0;j=0;k=n-1;
    while（j<=k）
    {
switch（a[j]）
{
case RED://如果是红色
    a[i]<->a[j];//交换
    i++;
    j++;
break;
case WHITE://如果是白色
    j++;
break;
case BLUE://如果是蓝色
    a[j]<->a[k];
    k--; //这里没有 j++;语句是为了防止交换后 a[j]仍为蓝色的情况
break;
        }//switch
    }//while
}//Flag_Arrange
分析：这个算法中设立了三个指针，其中指针 j 表示当前元素；指针 i 以前的元素全部为红色；指针 k 以后的元素全部为蓝色。这样就可以根据 j 的颜色，把其交换到序列的前部或者后部。
28．试以单链表为存储结构实现简单选择排序的算法.
答：
void LinkedList_Select_Sort（LinkedList &L）//单链表上的简单选择排序算法
{
for（p=L;p->next->next;p=p->next）
{
q=p->next;x=q->data;
for（r=q,s=q;r->next;r=r->next） //在 q 后面寻找元素值最小的结点，逐个比较
{
    if（r->next->data<x）
    {

```
    x=r->next->data;
    s=r;
    }
}//for
if（s!=q）//找到了值比 q->data 更小的最小结点 s->next，并交换 q 和 s->next 两个结点
{
    p->next=s->next;s->next=q;
    t=q->next;q->next=p->next->next;
    p->next->next=t;
}
}//for
}//LinkedList_Select_Sort
```

29．已知（k1,k2…kp）是堆，则可以写一个时间复杂度为 O（logn）的算法将（k1，k2…kp，kp+1）调整为堆。试编写“从 p=1 起,逐个插入建堆”的算法，并讨论由此方法建堆的时间复杂度。

答：

```
void Build_Heap（Heap &H,int n）//从低下标到高下标逐个插入建堆的算法
{
for（i=2;i<n;i++）
{//此时从 H.r[1]到 H.r[i-1]已经是大顶堆
j=i;
while（j!=1） //把 H.r[i]插入
{
    k=j/2;
    if（H.r[j].key>H.r[k].key）
    H.r[j]<->H.r[k];//交换
    j=k;
}//while
}//for
}//Build_Heap
```

30．假设定义堆为满足如下性质的完全三叉树：（1）空树为堆；（2）根结点的值不小于所有子数根的值，且所有子树均为堆。编写利用上述定义的堆进行排序的算法，并分析推导算法的时间复杂度。

答：

```
void TriHeap_Sort（Heap &H）//利用三叉树形式的堆进行排序的算法
{
for（i=H.length/3;i>0;i--）
Heap_Adjust（H,i,H.length）;
for（i=H.length;i>1;i--）
{
H.r[1]<->H.r[i];
Heap_Adjust（H,1,i-1）;
}
}//TriHeap_Sort
```

```
void Heap_Adjust（Heap &H,int s,int m）/*顺序表 H 中,H.r[s+1]到 H.r[m]已经是堆,把 H.r[s]
插入并调整成堆*/
{
    rc=H.r[s];
    for（j=3*s-1;j<=m;j=3*j-1）
    {
if（j<m&&H.r[j].key<H.r[j+1].key） j++;
if（j<m&&H.r[j].key<H.r[j+1].key） j++;
H.r[s]=H.r[j];
s=j;
    }//for
    H.r[s]=rc;
}//Heap_Adjust
```

分析：本算法与课本上的堆排序算法相比，只有两处改动；

①建初始堆时,i 的上限从 H.length/3 开始；

②调整堆的时候,要从结点的三个孩子结点中选择最大的那一个，最左边的孩子的序号的计算公式为 j=3*s-1。

31．可按如下所述实现归并排序:假设序列中有 k 个长度为≤2l 的有序子序列.利用过程归并插入（Merge Insertion）排序过程对他们进行两两归并,得到 ┌k/2┐ 个长度≤2l 的有序子序列,称为一趟归并排序.反复调用一趟归并排序过程,使有序子序列的长度自 l=1 开始成倍地增加,直至使整个序列成为一个有序序列.试对序列实现上述归并排序的递推算法,并分析你的算法的时间复杂度。

答：

```
void Merge_Sort（int a[ ],int n）//归并排序的非递归算法
{
for（l=1;l<n;l*=2） //l 为一趟归并段的段长
{
for（i=0;（2*i-1）*l<n;i++） //i 为本趟的归并段序号
{
    start1=2*l*i; //求出待归并的两段的上下界
    end1=start1+l-1;
    start2=end1+1;
    end2=（start2+l-1）>（n-1）?（n-1）:（start2+l-1）;//注意 end2 可能超出边界
    Merge（a,start1,end1,start2,end2）; //归并
}//for
}//for
}//Merge_Sort
void Merge（int a[ ],int s1,int e1,int s2,int e2）/*将有序子序列 a[s1]到 a[e1]和 a[s2]到 a[e2]归
并为有序序列 a[s1]到 a[e2]*/
{
    int b[MAXSIZE];//设立辅助存储数组 b
    for（i=s1,j=s2,k=s1;i<=e1&&j<=e2;k++）
    {
if（a[i]<a[j]） b[k]=a[i++];
```

```
else b[k]=a[j++];
    }
    while（i<=e1） b[k++]=a[i++];
    while（j<=e2） b[k++]=a[j++]; //归并到 b 中
    for（i=s1;i<=e2;i++） //复制回去
a[i]=b[i];
}//Merge
```

32．2-路归并排序的另一策略是，先对待排序序列扫描一遍，找出并划分为若干个最大有序子列，将这些子序列作为初始归并段。试写一个算法在链表结构上实现这一策略。

答：

```
void LinkedList_Merge_Sort2（LinkedList &L）/*初始归并段为最大有序子序列的归并排序,采用链表存储结构*/
{
LNode *end[MAXSIZE]; //设立一个数组来存储各有序子序列的尾指针
for（p=L->next->next,i=0;p;p=p->next） //求各有序子序列的尾指针
{
if（!p->next||p->data>p->next->data） end[i++]=p;
while（end[0]->next） //当不止一个子序列时进行两两归并
{
    j=0;k=0; //j 为当前子序列尾指针存储位置；k 为归并后的子序列尾指针存储位置
    for（p=L->next,e2=p;p->next;p=e2） //两两归并所有子序列
    {
    e1=end[j];e2=end[j+1]; //确定两个子序列
    if（e1->next） LinkedList_Merge（L,p,e1,e2）; //归并
    end[k++]=e2; //用新序列的尾指针取代原来的尾指针
    j+=2; //转到后面两个子序列
    }
}//while
}//for
}//LinkedList_Merge_Sort2
void LinkedList_Merge（LinkedList &L,LNode *p,LNode *e1,LNode *e2）//对链表上的子序列进行归并,第一个子序列是从 p->next 到 e1,第二个是从 e1->next 到 e2
{
q=p->next;r=e1->next;
while（q!=e1->next&&r!=e2->next） //q 和 r 为两个子序列的起始位置
{
    if（q->data<r->data） //选择关键字较小的那个结点接在 p 的后面
    {
p->next=q;p=q;
q=q->next;
    }
    else//否则接到 r 的后面
    {
p->next=r;p=r;
```

```
r=r->next;
    }
}//while
while（q!=e1->next） //接上剩余部分
{
    p->next=q;p=q;
    q=q->next;
}
while（r!=e2->next） //接上剩余部分
{
    p->next=r;p=r;
    r=r->next;
}
}//LinkedList_Merge，与上一题完全相同
```

33．已知两个有序序列（a_1，a_2，…，a_m）和（a_{m+1}，a_{m+2}，…，a_n）,并且其中一个序列的记录个数少于 s，且 s=$\lfloor\sqrt{n}\rfloor$。试写一个算法，用 O（n）时间和 O（1）附加空间完成这两个有序序列的归并。

答：

```
void SL_Merge（int a[ ],int l1,int l2）/*把长度分别为 l1，l2 且 l1^2<（l1+l2）的两个有序子序列归并为有序序列*/
{
start1=0;start2=l1; //分别表示序列 1 和序列 2 的剩余未归并部分的起始位置
for（i=0;i<l1;i++） //插入第 i 个元素
{
for（j=start2;j<l1+l2&&a[j]<a[start1+i];j++）; //寻找插入位置
k=j-start2; //k 为要向右循环移动的位数
RSh（a,start1,j-1,k）;//将 a[start1]到 a[j-1]之间的子序列循环右移 k 位
start1+=k+1;
start2=j; //修改两序列尚未归并部分的起始位置
}
}//SL_Merge
void RSh（int a[ ],int start,int end,int k）//将 a[start]到 a[end]之间的子序列循环右移 k 位
{
len=end-start+1;
for（i=1;i<=k;i++）
if（len%i==0&&k%i==0） p=i; //求 len 和 k 的最大公约数 p
    for（i=0;i<p;i++） //对 p 个循环链分别进行右移
    {
j=start+i;
l=start+（i+k）%len;
temp=a[j];
while（l!=start+i）
{
```

```
a[j]=temp;
temp=a[l];
a[l]=a[j];
j=l;
l=start+（j-start+k）%len; //依次向右移
}//while
a[start+i]=temp;
    }//for
}//RSh
```

34．假设有 1000 个关键字为小于 10000 的整数的记录序列，请设计一种排序方法，要求以尽可能少的比较次数和移动次数实现排序，并按你的设计编出算法。

答：利用哈希排序。

```
void Hash_Sort（int a[ ]）//对 1000 个关键字为四位整数的记录进行排序
{
int b[10000];
for（i=0;i<1000;i++） //直接按关键字散列
{
for（j=a[i];b[j];j=（j+1）%10000）;
b[j]=a[i];
}
for（i=0,j=0;i<1000;j++） //将散列收回 a 中
{
if（b[j]）//不为空
{
    for（x=b[j],k=j;b[k];k=（k+1）%10000）
    {
    if（b[k]==x）
    {
a[i++]=x;
b[k]=0;
    }
    }//for
}//if
}//for
}//Hash_Sort
```

35．序列的“中值记录”指的是：如果将此序列排序后,它是第$\left\lceil \frac{n}{2} \right\rceil$个记录，试写一个求中值记录的算法。

答：

```
typedef struct {
    int gt; //大于该记录的个数
    int lt; //小于该记录的个数
} place; //整个序列中比某个关键字大或小的记录个数
```

```
int Get_Mid（int a[ ],int n）//求一个序列的中值记录的位置
{
    place b[MAXSIZE];
    for（i=0;i<n;i++） //对每一个元素统计比它大和比它小的元素个数 gt 和 lt
    {
for（j=0;j<n;j++）
{
if（a[j]>a[i]） b[i].gt++;//比它大的元素个数加 1
else if（a[j]<a[i]） b[i].lt++;//比它小的元素个数加 1
}
    }//for
    mid=0;
    min_dif=abs（b[0].gt-b[0].lt）;
    for（i=0;i<n;i++） //找出 gt 值与 lt 值最接近的元素，即为中值记录
if（abs（b[i].gt-b[i].lt）<min_dif） mid=i;
    return mid;
}//Get_Mid
```

36．已知记录序列 a[1…n]中的关键字各不相同，可按如下所述实现计数排列，另设数组 c[1…n]，对每个记录 a[i]，统计序列中关键字比它小的记录个数存于 c[i]，则 c[i]=0 的记录必为关键字最小的记录，然后依 c[i]值的大小对 a 中记录进行重新排列，试编写算法实现上述排列方法。

答：

```
void Count_Sort（int a[ ],int n）//计数排序算法
{
int c[MAXSIZE];//辅助存储空间
for（i=0;i<n;i++）//对每一个元素
{
for（j=0,count=0;j<n;j++）//统计关键字比它小的元素个数
    if（a[j]<a[i]） count++;
c[i]=count;
}//for
for（i=0;i<n;i++） //依次求出关键字最小，第二小，…，最大的记录
{
min=0;
for（j=0;j<n;j++）
    if（c[j]<c[min]） min=j;//求出最小记录的下标 min
a[i]<->a[min];//与第 i 个记录交换
c[min]=INFINITY;//修改该记录的 c 值为无穷大以便下一次选取
}//for
}//Count_Sort
```

37．假设含 n 个记录的序列中，其所有关键字为值介于 v 和 w 之间的整数，且其中很多关键字的值是相同的。则可按如下方法进行排序，另设数组 number[v..w]且令 number[i]统计关键字取整数 i 的记录数，之后按 number 重排序列以达到有序。试编写算法实现上述排列方法，并讨论此种方法的优缺点。

答：

```
void Enum_Sort（int a[ ],int n）//对关键字只能取 v 到 w 之间任意整数的序列进行排序
{
int number[w+1],pos[w+1];
for（i=0;i<n;i++）
number[a[i]]++; //计数
for（pos[0]=0,i=1;i<n;i++）
pos[i]=pos[i-1]+num[i];/*pos 数组可以把关键字的值映射为元素在排好的序列中的位置*/
for（i=0;i<n;i++） //构造有序数组 c
c[pos[a[i]]++]=a[i];
for（i=0;i<n;i++）
a[i]=c[i];
}//Enum_Sort
```

分析:本算法参考了第五章三元组稀疏矩阵转置的算法思想，其中的 pos 数组和那里的 cpot 数组起的是相类似的作用。

38．试编写算法，借助“计数”实现基数排序。

答：

```
typedef enum {0,1,2,3,4,5,6,7,8,9} digit; //定义个位数类型
typedef digit[3] num; //定义 3 位自然数类型，假设低位存储在低下标，高位存储在高下标
void Enum_Radix_Sort（num a[ ],int n）/*利用计数实现基数排序，其中关键字为 3 位自然数，共有 n 个自然数*/
{
    int number ,pos ;
    num c[MAXSIZE];
    for（j=0;j<3;j++） //依次对个位，十位和百位排序
    {
for（i=0;i<n;i++）
number[a[i][j]]++; //计数，数组 number 记录元素 a[i][j]出现的次数
for（pos[0]=0,i=1;i<n;i++）
pos[i]=pos[i-1]+num[i];//把关键字的值映射为元素在排好的序列中的位置
for（i=0;i<n;i++） //构造有序数组 c
c[pos[a[i][j]]++]=a[i];
for（i=0;i<n;i++）
a[i]=c[i];
    }//for
}//Enum_Radix_Sort
```

分析；计数排序是一种稳定的排序方法。正因为如此，它才能够被用来实现基数排序。

39．序列 b 的每个元素是一个记录，每个记录占的存储量比其关键字占的存储量大的多，因而记录的移动操作是极为费时的。试写出一个算法，将序列 b 的排序结果放入序列 a 中，且每个记录只拷贝一次而无其他移动。你的算法可以调用第 10 章中给出的任何排序算法。

思考：当记录存于链表中时，若希望利用快速排序算法对关键字排序，从而最后实现链表的排序，如何模仿上述方法实现？

答：

```
typedef struct {
```

```
    int key;
int pos;
} Shadow; //影子序列的记录类型
void Shadow_Sort（Rectype b[ ],Rectype &a[ ],int n）/*对元素很大的记录序列 b 进行排序，结果放入 a 中，不移动元素*/
{
    Shadow d[MAXSIZE];
    for（i=0;i<n;i++） //生成影子序列
    {
d[i].key=b[i].key;
d[i].pos=i;
    }//for 复制数组，生成影子序列
    for（i=n-1,change=1;i>1&&change;i--） //对影子序列执行冒泡排序
    {
change=0;
for（j=0;j<i;j++）
{
if（d[j].key>d[j+1].key）
{
    d[j]<->d[j+1];
    change=1;//改变标识
}
}//for
    }//for
    for（i=0;i<n;i++） //按照影子序列里记录的原来位置复制原序列
a[i]=b[d[i].pos];
}//Shadow_Sort
```

10.3 考研真题与典型题详解

一、选择题

1．下列排序算法中元素的移动次数和关键字的初始排列次序无关的是（　　）。[2015 年联考真题]

A．直接插入排序

B．起泡排序

C．基数排序

D．快速排序

【答案】C 查看答案

【解析】C 项，基数排序是采用分配和收集实现的，不需要进行关键字的比较。ABD 三项都依赖关键字的比较，不同的初始排列次序下元素移动的次数有很大变化，最好情况元素正序，则不用移动，最坏情况元素反序，则需要移动 n（n-1）/2 次（n 为元素个数）。

2．已知小根堆为 8，15，10，21，34，16，12，删除关键字 8 之后需重建堆，在此过程中，关键字之间的比较数是（　　）。[2015 年联考真题]

A．1

B．2

C．3

D．4

【答案】C 查看答案

【解析】堆排序中，依次输出堆顶的最小值，然后重新调整堆，如此反复执行，便得到一个有序序列。本题中，删除堆顶元素 8 后将最后一个元素 12 置于堆顶，然后调整堆：首先与 15 比较，12 小于 15，所以不用交换；然后与 10 比较，因为 10 小于 12，所以交换 10 和 12 的位置；调整后 12 再与 16 比较，12 小于 16，调整堆过程结束。因此 12 共与 15、10、16 进行了三次比较。

3．希尔排序的组内排序采用的是（　　）。[2015 年联考真题]

A．直接插入排序

B．折半插入排序

C．快速排序

D．归并排序

【答案】A 查看答案

【解析】希尔排序基本思想是：先将整个待排元素序列按某个增量分割成若干个子序列，在子序列内进行直接插入排序，然后依次缩减增量再进行排序，待整个序列中的元素基本有序（增量足够小）时，再对全体元素进行一次直接插入排序。

4．用希尔排序方法对一个数据序列进行排序时，若第 1 趟排序结果为 9,1,4,13,7,8,20,23,15，则该趟排序采用的增量（间隔）可能是（　　）[2014 年联考真题]

A．2

B．3

C．4

D．5

【答案】B 查看答案

【解析】对于 A，增量为 2，那么 9,4,7,20,15 是一组，而它们是无序的，所以 A 错误

对于 C，增量为 4，那么 9,7,15 是一组，而它们是无序的，所以 C 错误

对于 D，增量为 5，那么 9,8 是一组，降序，1,20 是一组，而它们是升序，所以 D 也错误。

对于 B，分为 3 组：9,13,20；1,7,23；4,8,15 都是升序有序，所以 B 正确

5．下列选项中，不可能是快速排序第 2 趟排序结果的是（　　）[2014 年联考真题]

A．2,3,5,4,6,7,9

B．2,7,5,6,4,3,9

C．3,2,5,4,7,6,9

D．4,2,3,5,7,6,9

【答案】C 查看答案

【解析】对于快速排序，每一趟都会使一个元素位于有序时的位置，而有序序列为 2,3,4,5,6,7,9，与 C 进行对比，只有 9 位于它有序的时候的位置，显然不是第二趟快速排序的结果。

6．对给定的关键字序列 110，119，007，911，114，120，122 进行基数排序，则第 2 趟分配收集后得到的关键字序列是（　　）[2013 年联考真题]

A．007，110，119，114，911，120，122

B．007，110，119，114，911，122，120

C．007，110，911，114，119，120，122

D．110，120，911，122，114，007，119

【答案】C 查看答案

【解析】基数排序的第 1 趟排序是按照个位数字来排序的,第 2 趟排序是按然十位数字的大小进行排序的,故答案是 C 项。

7．下面给出的四种排序法中（　　）排序法是不稳定性排序法。

A．插入　B．起泡　C．二路归并　D．堆

【答案】D 查看答案

8．稳定的排序方法是（　　）。[北方交通大学 2000 研]

A．直接插入排序和快速排序

B．折半插入排序和起泡排序

C．简单选择排序和四路归并排序

D．树形选择排序和希尔排序

【答案】B 查看答案

【解析】直接插入排序、四路归并排序、起泡排序、树形选择排序和折半插入排序是稳定的排序。

9．若需在 O（$nlog_2n$）的时间内完成对数组的排序，且要求排序是稳定的，则可选择的排序方法是（　　）。

A．快速排序　B．堆排序　C．归并排序　D．直接插入排序

【答案】C 查看答案

【解析】稳定排序有：插入排序、起泡排序、归并排序、基数排序。不稳定排序有：快速排序、堆排序、shell 排序。时间复杂度平均为 O（$nlog_2n$）的有：归并排序、堆排序、shell 排序、快速排序。

10．排序趟数与序列的原始状态有关的排序方法是（　　）排序法。

A．插入　B．选择　C．起泡　D．快速

【答案】CD 查看答案

【解析】若数据初始情况就基本有序，则快速排序的时间性能会退化为 O（n^2）；若数据初始情况就基本顺序与起泡排序顺序相反，则起泡排序趟数会增多。

11．对序列{15，9，7，8，20，-1，4}进行排序，进行一趟后数据的排列变为{4，9，-1，8，20，7，15}；则采用的是（　　）排序。

A．选择　B．快速　C．希尔　D．起泡

【答案】C 查看答案

【解析】希尔排序的基本思想：希尔排序也是一种插入排序方法．实际上是一种分组插入方法。先取定一个小于 n 的整数 d_1 作为第一个增量，把表的全部记录分成 d_1 个组．所有距离为 d_1 的倍数的记录放在同一个组中，在各组内进行直接插入排序；然后，取第二个增量 d_2（$<d_1$），重复上述的分组和排序，直至所取的增量 $d_1=1$（$d_i<d_i-1<\ldots<d_2<d_1$），即所有记录放在同一组中进行直接插入排序为止。

12．在下列排序方法中，（　　）方法可能出现这种情况：在最后一趟开始之前，所有的元素都不在其最终应在的正确位置上。[武汉理工大学 2003 研]

A．快速排序　B．起泡排序　C．堆排序　D．插入排序

【答案】D 查看答案

【解析】选择排序、起泡排序和堆排序一趟排序后，在序列的首部或尾部应该有最大或最小值，快速排序一趟排序后，能够确定序列中第一个元素的位置。因此在最后一趟开始之前，快速排序、起泡排序、堆排序基本将所有元素位置确定了。

13．采用简单选择排序，比较次数与移动次数分别为（　　）。[南京理工大学 2000 研]

A．O（n），O（logn）　B．O（logn），O（n*n）

C．O（n*n），O（n）　D．O（nlogn），O（n）

【答案】C 查看答案

【解析】简单选择排序只在要交换的时候交换位置，及移动位置，共需移动 n 次。而需要比较的次数为 n-1+n-2+...+1=n（n-1）/2。

14．以下序列不是堆的是（　　）。[西安电子科技大学 2001 研]

A．（100，85，98，77，80，60，82，40，20，10，66）

B．（100，98，85，82，80，77，66，60，40，20，10）

C．（10，20，40，60，66，77，80，82，85，98，100）

D．（100，85，40，77，80，60，66，98，82，10，20）

【答案】D 查看答案

【解析】堆的定义：

n 个关键字序列 K_1，K_2，…，K_n 称为堆，当且仅当该序列满足如下性质（简称为堆性质）：

①$K_i \le K_{2i}$ 且 $K_i \le K_{2i+1}$

②$K_i \ge K_{2i}$ 且 $K_i \ge K_{2i+1}$（$1 \le i \le [n/2]$）

小根堆：满足第①种情况的堆；

大根堆：满足第②种情况的堆。

根据堆定义即可得出答案。

二、填空题

1．分别采用堆排序，快速排序，起泡排序和归并排序，对初态为有序的表，则最省时间的是_____算法，最费时间的是_____算法。

【答案】起泡，快速查看答案

【解析】当初态为有序表时，冒泡排序只需要进行一趟比较即可，此时时间复杂度为 O（n），而快速排序算法需要比较的次数达到最大，时间复杂度为 O（n^2）。

2．从平均时间性能而言，_____排序最佳。[青岛大学 2001 研]

【答案】快速查看答案

3．对于 7 个元素的集合{1，2，3，4，5，6，7}进行快速排序，具有最小比较和交换次数的初始排列次序为_____。

【答案】（4，1，3，2，6，5，7）查看答案

三、综合题

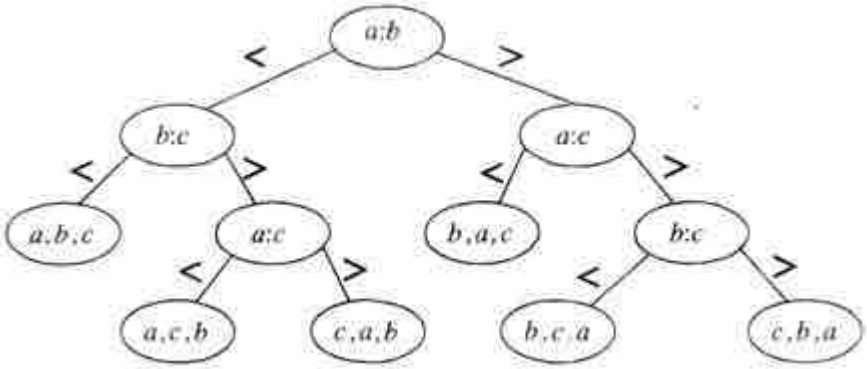
1．设有 n 个元素采用起泡排序法进行排序，通常需要进行多少趟排序?对于第 J 趟起泡通常需要进行多少次关键字比较?在程序设计中如何设置判断条件，有可能使起泡趟数可以减少并且能完成排序。[北京交通大学 2005 研]

答：n 个元素采用起泡排序法进行排序，通常需要进行 n-1 趟排序。第 j 趟起泡排序要进行 n-j 次比较。在一趟排序中，若没有记录交换，则表示排序完成。因而，可通过设标记来控制排序结束，以下语句段说明了标记 flag 的使用。

```
int j=1,flag=1;        //flag用作控制标记
 while(j<n && flag) //j是起泡排序趟数,最多n-1趟
  {flag=0;           //设标记,若本趟不发生交换,本趟起泡排序后,算法结束
   for(i=1;i<=n-j;i++)
     if(r[i]>r[i+1]) {flag=1; r[i]<-->r[i+1];}//发生了交换,还要进行下趟
   }
```

2．考虑由三个不同关键词构成的序列：{a，b，c}，试画出直接插入排序算法的二叉判定树。[吉林大学 2001 研]

答：



4．写出下列排序算法的基本思想，并写出对序列（23，12，35，47，16，25，36，19，21，16）进行排序时每一趟的结果。[山东科技大学 2002 研]

```
PROC   bbsort(VAR  r: sequence;  n: integer);
{r是一个数组}
  d:=1;  pos[-1]:=1; pos[1]:=n;  i:=1;  exchanged:= true;
  WHILE  exchanged  DO
    [ exchanged:= false;
     WHILE  i<>pos[d]  DO
        [IF (r[i]-r[i+d])*d>0 THEN [r[i]与r[i+d]交换; exchanged:=true;];
         i:=i+d;
        ]
     pos[d]:=pos[d]-d; i:=pos[d]; d:=-d;
  ]
ENDP;
```

答：此排序为双向起泡排序：从前向后一趟排序下来得到一个最大值，若其中发生交换，则再从后向前一趟排序，得到一个最小值。

第一趟：12，23，35，16，25，36，19，21，16，47

第二趟：12，16，23，35，16，25，36，19，21，47

第三趟：12，16，23，16，25，35，19，21，36，47

第四趟：12，16，16，23，19，25，35，21，36，47

第五趟：12，16，16，19，23，25，21，35，36，47

第六趟：12，16，16，19，21，23，25，35，36，47

第七趟：12，16，16，19，21，23，25，35，36，47

5．按由大到小的顺序对一含有Ⅳ个元素的数组 A[N]进行排序，利用如下改进的简单选择排序方法：第一次选出最大者存入 A[1]，第二次选出最小者存入 A[N]，第三次选出次大者存入 A[2]，第四次选出次小者存入 A[N-1]，如此大小交替地选择，直到排序完毕。

答：

```
void Simple_Selection(int A[],int n)
//改进的选择排序,选大和选小交替进行,按题意,元素下标从1开始
{for(i=1;i<=n/2;i++)
```

```
    {k=i;    //k记最大元素下标
     for(j=i+1;j<=n-i+1;j++)
         if(A[j]>A[k]) k=j;    //选第i个最大元素
     if(k!=i) A[i]<-->A[k];    //选出第i个最大元素
     k=n-i;   //k记最小元素下标
     for(j=n-i;j<=i;j--)
         if(A[j]<A[k]) k=j;    //选第i个最小元素
     if(k!=i) A[i]<-->A[k];    //选出第i个最小元素
}//高层for }//结束算法
```

第 11 章　外部排序

11.1　复习笔记

一、外存信息的存取

1．磁带信息的存取

（1）磁带

磁带是薄薄涂上一层磁性材料的一条窄带。

（2）磁带的启停

磁带是一种启停设备，启停时间约为 5 毫秒，它可以根据读/写的需要随时启动和停止。由于读/写信息应在旋转稳定时进行，故磁带从静止状态启动后，要经过一个加速的过程才能达到稳定状态；反之，在读/写结束后，从运动状态到完全停止，要经过一个减速的过程。

（3）间隙 IRG

在磁带的启停过程中，磁带上相邻两组字符组（记录）之间要留一空白区，称作间隙 IRG（Inter Record Gap）。由于这个间隙 IRG 很大，所以磁带的利用率很低。

（4）间隙 IBG

为有效地利用磁带，用组成块的办法来减少 IRG 的个数。在每次写信息时，不是按用户给出的字符组记入磁带，而是将若干个字符组合并成一块后一次写入磁带。于是，每个字符组间就没有 IRG，而变成块间的间隙 IBG（Inter Block Gap）。

（5）成块的优点

①减少 IRG 的数目，从而提高磁带的利用率。

②减少 I/0 操作，提高读写效率。

（6）读写时间

磁带上读写一块信息所需的时间由两部分组成：

$$T_{I/O}=t_a+n\times t_w$$

其中 t_a 为延迟时间，读/写头到达传输信息所在物理块起始位置所需时间；t_w 为传输一个字符的时间。

2．磁盘信息的存取

（1）磁盘

磁盘是一种直接存取的存储设备（DASD）。它是一个扁平的圆盘，盘面上有许多称为磁道的圆圈，信息就记载在磁道上。由于磁道的圆圈为许多同心圆，所以可以直接存取。

（2）磁盘的特点

①存取时间变化不大；

②容量大、速度快，存取速度比磁带快得多。

③由于磁道的圆圈为许多同心圆，所以可以直接存取。

（3）磁盘的读/写

磁盘驱动器执行读/写信息的功能。盘片装在一个主轴上，并绕主轴高速旋转，当磁道在读写头下通过时，便可以进行信息的读/写。

（4）读写时间

在磁盘上读写一块信息所需的时间由三部分组成：

$T_{I/O}=t_{seek}+t_{la}+n\times t_{wm}$

其中：t_{seek} 为寻查时间（seek time），即读/写头定位的时间；t_{la} 为等待时间（latency time），即等待信息块的初始位置旋转到读写头下的时间；t_{wm} 为传输时间（transmission time）。

二、外部排序

1．外部排序的方法

外部排序基本上由两个相对独立的阶段组成：

第一阶段，按可用内存大小，将外存上含 n 个记录的文件分成若干长度为 z 的子文件或段，依次读入内存并利用有效的内部排序方法对它们进行排序，并将排序后得到的有序子文件重新写入外存，通常称这些有序子文件为归并段或顺串（run）。

第二阶段，对这些归并段或顺串进行逐趟归并，使其逐渐由小至大，直至得到整个有序文件为止。

2．外部排序所需时间

一般情况下，外部排序所需总的时间。

=内部排序（产生初始归并段）所需的时间（$m*t_{IS}$ ）

+外存信息读写的时间（$d*t_{IO}$）

+内部归并所需的时间（$S*ut_{mg}$）

其中：t_{IS} 是为得到一个初始归并段进行内部排序所需时间的均值；

t_{IO} 是进行一次外存读/写时间的均值；

ut_{mg} 是对 u 个记录进行内部归并所需时间；

m 为经过内部排序之后得到的初始归并段的个数；

s 为归并的趟数；

d 为总的读/写次数。

3．归并

对同一文件而言，进行外排时所需读/写外存的次数和归并的趟数 s 成正比，对 m 个初始归并段进行 k-路平衡归并时，归并的趟数 $s=\lceil \log_k m \rceil$。

三、多路平衡归并的实现

1．败者树

在双亲结点中记下刚进行完的比赛中的败者，而让胜者去参加更高级别的比赛，便可得到一颗“败者树”。败者树是树形选择排序的一种变型。如图 11-1 所示为一棵实现 5-路归并的败者树。

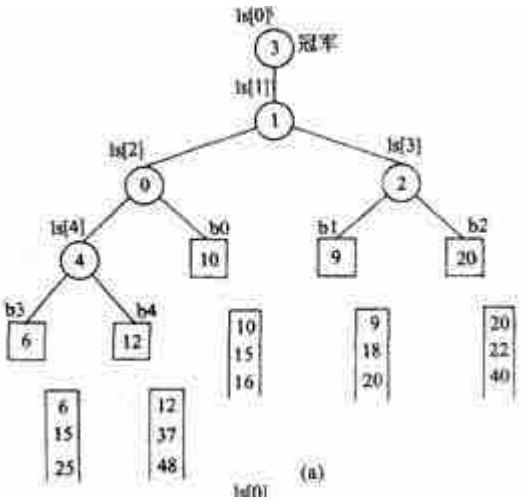


图 11-1 5-路归并的败者树

2.k-路归并

如图 11-2 所示的算法简单描述利用败者树进行 k-路归并的过程。

```
TYPE    loser_tree=ARRAY [0..k-1] OF integer;
            (败者树为完全二叉树,可用向量作存储结构,且此败者树不含叶子结点(即外结点))
        exnode=RECORD
                 key : keytype
               END;
               (败者树的外结点中只存放待归并记录的关键字)
        external=ARRAY [0..k] OF exnode;
PROC k_merge (VAR ls : loser_tree;    VAR b : external);
  (利用败者树 ls 将编号从 0 至 k-1 的 k 个输入归并段中的记录归并到输出归并段中,b[0]至 b[k-1]为败者树上 k
   个叶子结点,分别存放 k 个输入归并段中当前参加归并的记录的关键字)
  FOR i:=0 TO k-1 DO input(b[i].key);        (分别从 k 个输入归并段中读入一个记录的关键字)
  crt_loser_tree(ls);        (建败者树 ls,选得最小关键字为 b[ls[0]].key)
  WHILE b[ls[0]].key≠maxkey DO
    [ q:=ls[0];
      output(q);  (将编号为 q 的归并段中当前参加归并(即关键字为 b[q].key)的记录写至输出归并段)
      input(b[q].key);  (从编号为 q 的归并段中读入下一个记录的关键字)
      adjust(ls,q);  (调整败者树选择新的最小关键字记录)
    ]: output(ls[0])    (将最大关键字记录写入输出归并段)
  ENDP; {k_merge}
```

图 11-2 利用败者树进行 k-路归并的过程

四、置换-选择排序

1．定义

置换-选择排序（replacement-selection sorting）是在树形选择排序的基础上得来的，特点是：在整个排序（得到所有初始归并段）的过程中，选择最小（或最大）关键字和输入、输出交叉或平行进行。

2．过程

假设初始待排文件为输入文件 FI，初始归并段文件为输出文件 FO，内存工作区为 WA，FO 和 WA 的初始状态为空，并设内存工作区 WA 可容纳 w 个记录，则置换-选择排序的操作过程为：

（1）从 FI 输入 w 个记录到工作区 WA；
（2）从 WA 中选出最小关键字记录，记为 MINIMAX 记录；
（3）将 MINIMAX 输出到 FO 中去；
（4）若 FI 不空，则从 FI 输入下一个记录到 WA 中；
（5）从 WA 中所有关键字比 MINIMAX 大的记录中选出最小关键字记录，作为新的 MINIMAX 记录；
（6）重复（3）-（5），直至在 WA 中选不出新的 MINIMAX 为止，由此得到一个初始归并段，输出一个归并段的结束标志到 FO 中去；
（7）重复（2）-（6），直至 WA 为空。由此得到全部初始归并段。

3．性能分析

若不计输入、输出的时间，则对 n 个记录的文件而言，生成所有初始归并段所需时间为 O（nlogw）。

五、最佳归并树

1．定义

对长度不等的 m 个初始归并段，构造一棵赫夫曼树作为归并树，便可使在进行外部归并时对外存进行的读/写次数达最少，这棵归并树便称作最佳归并树。

2．实现方案

若按最佳归并树的归并方案进行磁盘归并排序，需在内存建立一张索引表，记录归并段的长度和它在磁盘上的物理位置。

11.2　强化习题详解

（略）

11.3　考研真题与典型题详解

一、选择题

1．排序算法的稳定性是指（　　）。[北京理工大学 2005 研]

A．经过排序之后，能使值相同的数据保持原顺序中的相对位置不变

B．经过排序之后，能使值相同的数据保持原顺序中的绝对位置不变

C．算法的排序性能与被排序元素的数量关系不大

D．算法的排序性能与被排序元素的数量关系密切

【答案】A 查看答案

【解析】假定在待排序的记录序列中，存在多个具有相同的关键字的记录，若经过排序，这些记录的相对次序保持不变，即在原序列中，ri=rj，且 ri 在 rj 之前，而在排序后的序列中，ri 仍在 rj 之前，则称这种排序算法是稳定的；否则称为不稳定的。

2．下列排序算法中，占用辅助空间最多的是（　　）。[厦门大学 2002 研]

A．归并排序　B．快速排序　C．希尔排序　D．堆排序

【答案】A 查看答案

【解析】归并排序的辅助空间为 O（n），快速排序所占用的辅助空间为 O（logn），堆排序所占用的辅助空间为 O（1）。

3．从未排序序列中依次取出一个元素与已排序序列中的元素依次进行比较，然后将其放在已排序序列的合适位置，该排序方法称为（　　）排序法。[北京航空航天大学 1999 研]

A．插入　B．选择　C．希尔　D．二路归并

【答案】A 查看答案

【解析】解此题需要熟知各种排序方法的基本思想。

插入排序的基本思想是：假设待排序的记录存放在数组 R[0．．n-1]中，排序过程的某一中间时刻，R 被划分成两个子区间 R[0．．i-1]和 R[i．．n-1]，其中：前一个子区间是已排好序的有序区，后一个子区间则是当前未排序的部分，不妨称其为无序区。
将当前无序区的第 1 个记录 R[i]插入到有序区 R[0．．i-1]中适当的位置上。使 R[0．．i]变为新的有序区。这种方法通常称为增量法，因为它每次使有序区增加 1 个记录。
4．对 n 个记录的线性表进行快速排序为减少算法的递归深度，以下叙述正确的是（　　）。[北方交通大学 2000 研]
A．每次分区后，先处理较短的部分　　B．每次分区后，先处理较长的部分
C．与算法每次分区后的处理顺序无关　D．以上三者都不对
【答案】A 查看答案
【解析】令递归函数为 f，第一次进行递归函数认为递归深度为 1，以后从深度为 n 的递归函数 f 中再调用递归函数 f，此时深度为 n+1。整个 f 的最大深度为递归深度。
5．当 n 个整型数据是有序时，对这 n 个数据用快速排序算法排序，则时间复杂度是（6），当用递归算法求 n!时，算法的时间复杂度是（7），则：（6）-（7）=（　　）。[南京理工大学 1999 研]
A．O（n）　B．O（nlogn）　C．O（n*n）　D．O（logn）
【答案】C 查看答案
【解析】当 n 个整型数据是有序时，对这 n 个数据用快速排序算法排序，快速排序会退化为冒泡排序，其时间复杂度为 O（n*n）：当用递归算法求 n!时，算法的时间复杂度是 O（n）。
6．将两个各有 N 个元素的有序表归并成一个有序表，其最少的比较次数是（　　）。[中科院计算所 1998 研；中国科技大学 1998 研]
A．N　B．2N-1　C．2N　D．N-1
【答案】A 查看答案
【解析】归并排序基本思想：
归并排序是多次将两个或两个以上的有序表合并成一个新的有序表。最简单的归并是直接将两个有序的子表合并成一个有序的表。归并排序最好情况下的复杂度为 O（n）。

二、填空题

1．若不考虑基数排序，则在排序过程中，主要进行的两种基本操作是关键字的_____和记录的_____。[北京邮电大学 2001 研]
【答案】比较；移动查看答案
2．外排序的基本操作过程是_____和_____。
【答案】生成有序归并段（顺串）；归并查看答案
3．外部排序的基本方法是归并排序，但在之前必须先生成_____。
【答案】初始归并段（顺串）查看答案
4．在基于关键字比较且时间为 O（$nlog_2n$）的排序中，若要求排序是稳定的，则可选用_____排序；若要求就地排序（及辅助空间为 O（1））， 则可选用_____排序。
【答案】归并；堆查看答案
5．按 LSD 进行关键字排序，除最次位关键字之外，对每个关键字进行排序时，只能用_____的排序方法。[北京交通大学 2004 研]
【答案】稳定查看答案

三、综合题

1．利用比较的方法进行排序，在最坏的情况下，能达到的最好时间复杂性是什么?请给挂；详细证明。

答：假定待排序的记录有 n 个。由于含 n 个记录的序列可能出现的状态有 n!个，则描述 n 个记录排序过程的判定树必须有 n!个叶结点。因为若少一个叶子，则说明尚有两种状态没有分辨出来。我们知道，若二叉树高度是 h，则叶结点个数最多为 2^{h-1}；反之，若有 u 个叶结点，则二叉树的高度至少为$|\log_2 u|+1$。这就是说，描述 n 个记录排序的判定树必定存在一条长度为$|\log_2(n!)|$的路径。即任何一个借助“比较”进行排序的算法，在最坏情况下所需进行的比较次数至少是$|\log_2(n!)|$。根据斯特林公式，有$|\log_2(n!)|=O(n\log_2 n)$。即借助于“比较”进行排序的算法在最坏情况下能达到的最好时间复杂度为 $O(n\log_2 n)$。证毕。

2．简述直接插入排序、简单选择排序、2 路归并排序的基本思想以及在时间复杂度和排序稳定性上的差别。

答：（1）直接插入排序

直接插入排序的基本思想是基于插入，开始假定第一个记录有序，然后从第二个记录开始，依次插入到前面有序的子文件中。即将记录 R[i]（$2\le i\le n$）插入到有序子序列 R[1．.i-1]中，使记录的有序序列从 R[1．．i-1]变为 R[1．．i]，最终使整个文件有序。共进行 n-1 趟插入。最坏时间复杂度是 $O(n^2)$，平均时间复杂度是 $O(n^2)$，空间复杂度是 O（1），是稳定排序。

（2）简单选择排序

简单选择排序的基本思想是基于选择，开始有序序列长度为零，第 i（$1\le i<n$）趟简单选择排序是，从无序序列 R[i．．n]的 n-i+1 记录中选出关键字最小的记录，和第 i 个记录交换，使有序序列逐步扩大，最后整个文件有序。共进行 n-1 趟选择。最坏时间复杂度是 $O(n^2)$，平均时间复杂度是 $O(n^2)$，空间复杂度是 O（1），是不稳定排序。

（3）二路并归排序

二路并归排序的基本思想是基于归并，开始将具有 n 个待排序记录的序列看成是 n 个长度为 1 的有序序列，然后进行两两归并，得到「n / 2ù 个长度为 2 的有序序列，再进行两两归并，得到「n / 4ù 个长度为 4 的有序序列。如此重复，经过$|\log_2 n|$趟归并，最终得到一个长度为 n 的有序序列。最坏时间复杂度和平均时间复杂度都是 $O(n\log_2 n)$，空间复杂度是 O（n），是稳定排序。

第 12 章　文　件

12.1　复习笔记

一、文件的基本概念

1．文件及其类别

（1）文件的定义

文件（file）是由大量性质相同的记录组成的集合。

（2）按文件记录类型分类

①操作系统文件。操作系统文件仅是一维的连续的字符序列，无结构、无解释；

②数据库文件。数据库文件是带有结构的记录的集合，由一个或多个数据项组成，它也是文件中可存取的数据的基本单位。数据库文件还可按记录中关键字的多少再分成单关键字文件和多关键字文件。

（3）按记录长度是否确定分类

①定长记录文件

②不定长记录文件。

2．记录的逻辑结构和物理结构

（1）逻辑结构

记录的逻辑结构是指记录在用户或应用程序员面前呈现的方式，是用户对数据的表示和存取方式。

（2）物理结构

记录的物理结构是数据在物理存储器上存储的方式，是数据的物理表示和组织。

（3）逻辑记录和物理记录的三种关系

①一个物理记录存放一个逻辑记录；

②一个物理记录包含多个逻辑记录；

③多个物理记录表示一个逻辑记录；

3．文件的操作（运算）

文件的操作有两类：检索和修改。

（1）检索

文件的检索有下列三种方式：

①顺序存取：存取下一个逻辑记录；

②直接存取：存取第 i 个逻辑记录。

以上两种存取方式都是根据记录序号（即记录存入文件时的顺序编号）或记录的相对位置进行存取的；

③按关键字存取：给定一个值，查询一个或一批关键字与给定值相关的记录。对数据库文件可以有如下四种查询方式：

a.简单询问；

b.区域询问；

c.函数询问；

d.布尔询问。

（2）修改

文件的修改包括插入一个记录、删除一个记录和更新一个记录三种操作。文件的操作可以有实时和批量两种不同方式。

4．文件的物理结构

文件在存储介质（磁盘或磁带）上的组织方式称为文件的物理结构。文件可以有各种各样的组织方式，其基本方式有三种：顺序组织、随机组织和链组织。

二、顺序文件

1．定义

顺序文件（sequential file）是记录按其在文件中的逻辑顺序依次进入存储介质而建立的，即顺序文件中物理记录的顺序和逻辑记录的顺序是一致的。若次序相继的两个物理记录在存储介质上的存储位置是相邻的，则又称连续文件；若物理记录之间的次序由指针相连表示，则称串联文件。

2．优缺点

顺序文件的优点是连续存取的速度快，因此主要用于只进行顺序存取、批量修改的情况。若对应答时间要求不严时亦可进行直接存取。

三、索引文件

1．定义

索引表是指除了文件本身（称作数据区）之外，另外建立的一张指示逻辑记录和物理记录之间一一对应关系的表。这类包括文件数据区和索引表两大部分的文件称作索引文件。

索引表中的每一项称作索引项。不论主文件是否按关键字有序，索引表中的索引项总是按关键字（或逻辑记录号）顺序排列。

索引表是由系统程序自动生成的。在记录输入建立数据区的同时建立一个索引表，表中的索引项按记录输入的先后次序排列，待全部记录输入完毕后再对索引表进行排序。

2．检索方式

索引文件的检索方式为直接存取或按关键字存取，检索过程和分块查找相类似。在索引文件中进行检索只访问外存两次，即一次读索引，一次读记录。并且由于索引表是有序的，查找索引表时可用折半查找法。

3．操作

索引文件的修改也容易进行。删除一个记录时，仅需删去相应的索引项；插入一个记录时，将记录置于数据区的末尾，同时在索引表中插入索引项；更新记录时，将更新后的记录置于数据区的末尾，同时修改索引表中相应的索引项。当记录数目很大时，索引表也很大，可以对索引表建立一个索引，称为查找表。检索记录时，先查找查找表，再查找索引表，然后读取记录。访问三次外存即可。

当数据文件在使用过程中记录变动较多时，应采用动态索引。如二叉排序树（或二叉平衡树）、B-树以及键树，这些都是树表结构，插入、删除都很方便。索引文件只能是磁盘文件。

四、ISAM 文件和 VSAM 文件

1.ISAM 文件

索引顺序存取方法 ISAM（Indexed Sequential Access Method）是一种专为磁盘存取设计的文件组织方式。

在 ISAM 文件上检索记录时，先从主索引出发找到相应的柱面索引，再从柱面索引出发找到记录所在柱面的磁道索引，最后从磁道索引上找到记录所在磁道的第一个记录的位置，由此出发在该磁道上进行顺序查找直至找到记录为止。

通常需要周期地整理 ISAM 文件，把记录读入内存，重新排列，复制成一个新的 ISAM 文件，填满基本区而空出溢出区。

2.VSAM 文件

虚拟存储存取方法 VSAM（Virtual Storage Access Method）利用了操作系统的虚拟存储器的功能，给用户提供方便。

VSAM 文件由三部分组成：索引集、顺序集和数据集。

VSAM 文件没有溢出区，解决插入的办法是在初建文件时留有空间。

在 VSAM 文件中删除记录时，需将同一控制区间中较删除记录关键字大的记录向前移动，把空间留给以后插入的新记录。若整个控制区间变空，则需修改顺序集中相应的索引项。

VSAM 文件的缺点：VSAM 文件占有较多的存储空间，一般只能保持约 75％的空间利用率。但它的优点是：动态地分配和释放存储，不需要对文件进行重组，并能较快地对插入的记录进行查找，查找一个后插入记录的时间与查找一个原有记录的时间是相同的。

五、直接存取文件（散列文件）

1．定义

直接存取文件指的是利用杂凑（hash）法进行组织的文件。它类似于哈希表，即根据文件中关键字的特点设计一种哈希函数和处理冲突的方法将记录散列到存储设备上，故又称散列文件。

2．操作

在直接文件中进行查找时，首先根据给定值求得哈希地址（即基桶号），将基桶的记录读入内存进行顺序查找，当找到关键字等于给定值的记录时，则检索成功；否则，若基桶没有填满记录或其指针域为空，则文件内不含有给定记录；否则根据指针域的值的指示将溢出桶的记录读入内存继续进行顺序查找，直至检索成功或不成功。

在直接存取文件中删除记录时，和哈希表一样，仅需对被删记录作一标记即可。

3．优缺点

直接存取文件的优点是：文件随机存放，记录不需进行排序；插入、删除方便，存取速度快，不需要索引区，节省存储空间。

其缺点是：不能进行顺序存取，只能按关键字随机存取，且询问方式仅限于简单询问，并且在经过多次的插入、删除之后，可能造成文件结构不合理，即溢出桶满而基桶内多数为被删除的记录，此时亦需重组文件。

六、多关键字文件

多关键字文件的特点是，在对文件进行检索操作时，不仅要对主关键字进行简单询问，还经常要对次关键字进行其它类型的询问检索。

1．多重链表文件

多重链表文件（multilist file）的特点是：

（1）记录按主关键字的顺序构成一个串联文件，并建立主关键字的索引（称为主索引）；

（2）对每一个次关键字项建立次关键字索引（称为次索引），所有具有同一次关键字的记录构成一个链表。主索引为非稠密索引，次索引为稠密索引。每个索引项包括次关键字、头指针和链表长度。

多重链表文件易于编程，也易于修改。如果不要求保持链表的某种次序，则插入一个新记录是容易的，此时可将记录插在链表的头指针之后。但是，要删去一个记录却很繁琐，需在每个次关键字的链表中删去该记录。

2．倒排文件

倒排文件和多重链表文件的区别在于次关键字索引的结构不同。通常，称倒排文件中的次关键字索引为倒排表，具有相同次关键字的记录之间不设指针相链，而在倒排表中该次关键字的一项中存放这些记录的物理记录号。若数据文件非串联文件，而是索引顺序文件（如 ISAM 文件），则倒排表中应存放记录的主关键字而不是物理记录号。

倒排表作索引的好处在于检索记录较快，其缺点是维护困难。

12.2　强化习题详解

1．试比较顺序文件,索引文件和索引顺序文件各有什么特点。

答：（1）顺序文件，是按照从头到尾的顺序进行存取操作的，文件中的信息就像在一条长长的队列中排列一样。特点是：

①存取第 i 个记录，必须先搜索在它之前的 i-1 个记录；

②插入新的记录时只能加在文件的末尾；

③若要更新文件中的某个记录，则必须将整个文件进行复制。

（2）索引文件是指在主文件之外再建立一个表示关键字与其物理记录之间对应关系的表，称为索引表。索引表与主文件共同构成索引文件。特点是：

①检索方式为直接存取或按关键字（进行简单询问）存取；

②只能是磁盘文件。

（3）索引顺序文件是一种专为磁盘存取设计的文件组织方式，索引表是由系统程序自动生成的。与索引文件相比，特点是：

可对一组记录建立一个索引项，而不用对每个记录建立一个索引项。

2．直接存取文件为什么不用链地址法存储结构而要按桶散列？桶的大小 m 是如何确定的？

答：直接存取文件指的是利用杂凑（hash）法进行组织的文件。它类似于哈希表，即根据文件中关键字的特点设计一种哈希函数和处理冲突的方法将记录散列到存储设备上，故又称散列文件。

与哈希表不同的是，对于文件来说，磁盘上的文件记录通常是成组存放的。若干个记录组成一个存储单位，在散列文件中，这个存储单位称作桶（bucket）。假若一个桶能存放 m 个记录，即 m 个同义词的记录可以存放在同一地址的桶中，而当第 m+1 个同义词出现时才发生“溢出”。处理溢出也可采用哈希表中处理冲突的各种方法，但对散列文件，主要采用链地址法。

3．试综述文件有哪几种常用的组织方式？他们各有什么特点？

答：文件的基本组织方式有顺序组织、索引组织、散列组织和链组织。文件的存储结构可以采用将基本组织结合的方法，常用的结构有顺序结构、索引结构、散列结构。

（1）顺序结构，相应文件为顺序文件，其记录按存入文件的先后次序顺序存放。顺序文件本质上就是顺序表。若逻辑上相邻的两个记录在存储位置上相邻，则为连续文件；若记录之间以指针相链接，则称为串联文件。顺序文件只能顺序存取，要更新某个记录，必须复制整个文件。顺序文件连续存取的速度快，主要适用于顺序存取，批量修改的情况。

（2）索引结构，相应文件为索引文件。索引文件包括索引表和数据表，索引表中的索引项包括数据表中数据的关键字和相应地址，索引表有序，其物理顺序体现了文件的逻辑次序，实现了文件的线性结构。索引文件只能是磁盘文件，既能顺序存取，又能隋机存取。

（3）散列结构，也称计算寻址结构，相应文件称为散列文件，其记录是根据关键字值经散列函数计算确定其地址，存取速度快，不需索引，节省存储空间。不能顺序存取，只能随机存取。

12.3　考研真题与典型题详解

一、选择题

1．哈希文件使用哈希函数将记录的关键字值计算转化为记录的存放地址，因为哈希函数是一对一的关系，则选择好的（　　）方法是哈希文件的关键。[哈尔滨工业大学 2001 研]

A．哈希函数　B．除余法中的质数　C．冲突处理　D．哈希函数和冲突处理

【答案】D 查看答案

【解析】哈希表是根据文件中关键字的特点设计一种哈希函数和处理冲突的方法将记录散列到存储设备上。

2．顺序文件采用顺序结构实现文件的存储，对大型的顺序文件的少量修改，要求重新复制整个文件，代价很高，采用（　　）的方法可降低所需的代价。

A．附加文件　B．按关键字大小排序　C．按记录输入先后排序　D．连续排序

【答案】A 查看答案

4．下述文件中适合于磁带存储的是（　　）。[中科院计算所 2000 研]

A．顺序文件　B．索引文件　C．哈希文件　D．多关键字文件

【答案】A 查看答案

【解析】磁带存储是一种顺序存储，顺序文件（sequential file）是记录按其在文件中的逻辑顺序依次进入存储介质而建立的，即顺序文件中物理记录的顺序和逻辑记录的顺序是一致的。因此顺序文件适合磁带存储。

5.ISAM 文件和 VASM 文件属于（　　）。[山东大学 2001 研]

A．索引非顺序文件　B．索引顺序文件　C．顺序文件　D．哈希文件

【答案】B 查看答案

6.B+树应用在（　　）文件系统中。[北京邮电大学 2001 研]

A．ISAM　B．VSAM

【答案】B 查看答案

【解析】VSAM 文件的组成 VSAM 文件由 3 个部分组成：索引集、顺序集与数据集，其中顺序集与索引集一起构成一棵 B+树。

二、填空题

1．文件可按其记录的类型不同而分成两类，即_____和_____文件。

【答案】操作系统文件、数据库查看答案

2．物理记录之间的次序由指针相链表示的顺序文件称为_____。

【答案】串联文件查看答案

3．顺序文件中，要存取第 I 个记录，必须先存取_____个记录。[哈尔滨工业大学 2001 研]

【答案】第 I-1 查看答案

4．索引顺序文件是最常用的文件组织之一，通常用_____结构来组织索引。

【答案】树查看答案

5．倒排序文件的主要优点在于_____。

【答案】检索记录快查看答案

6．检索是为了在文件中寻找满足一定条件的记录而设置的操作。检索可以按_____检索。也可以按_____检索；按_____检索又可以有_____检索和_____检索。

【答案】关键字；记录号；记录号；顺序；直接查看答案

7．哈希检索技术的关键是_____和_____。

【答案】构造哈希函数；解决冲突的方法查看答案

8.VSAM 系统是由_____、_____、_____构成的。

【答案】索引集；顺序集；数据集查看答案

三、综合题

1．文件存储结构的基本形式有哪些?一个文件采用何种存储结构应考虑哪些因素?

答：文件的基本组织方式有顺序组织、索引组织、散列组织和链组织。文件的存储结构可以采用将基本组织结合的方法，常用的结构有顺序结构、索引结构、散列结构。

（1）顺序结构

相应文件为顺序文件，其记录按存入文件的先后次序顺序存放。顺序文传本质上就是顺序表。若逻辑上相邻的两个记录在存储位置上相邻，则为连续文件；若记录之间以指针相链接，则称为串联文件。顺序文件只能顺序存取，要更新某个记录，必须复制整个文件。顺序文件连续存取的速度快，主要适用于顺序存取，批量修改的情况。

（2）带索引的结构

相应文件为索引文件。索引文件包括索引表和数据表，索引表中的索引项包括数据表中数据的关键字和相应地址，索引表有序，其物理顺序体现了文件的逻辑次序，实现了文件的线性结构。索引文件只能是磁盘文件，既能顺序存取，又能随机存取。

（3）散列结构

也称计算寻址结构，相应文件称为散列文件，其记录是根据关键字值经哈希函数计算确定其地址，存取速度快，不需索引，节省存储空间。不能顺序存取，只能随机存取。

其他文件均由以上文件派生而得。文件采用何种存储结构应综合考虑各种因素，如：存储介质类型、记录的类型、大小和关键字的数目以及对文件作何种操作。

2．索引顺序存取方法（ISAM）中，主文件已按关键字排序，为何还需要主关键字索引?

答：ISAM 是专为磁盘存取设计的文件组织方式。即使主文件关键字有序，但因磁盘是以盘组、柱面和磁道（盘面）三级地址存取的设备，因此通常对磁盘上的数据文件建立盘组、柱面和磁道（盘面）三级索引。

在 ISAM 文件上检索记录时，先从主索引（柱面索引的索引）找到相应柱面索引。再从柱面索引找到记录所在柱面的磁道索引，最后从磁道索引找到记录所在磁道的第一个记录的

位置，由此出发在该磁道上进行顺序查找直到查到为止；反之，若找遍该磁道而未找到所查记录，则文件中无此记录。

3．已知职工文件中包括职工号、职工姓名、职务和职称 4 个数据项（见下表）。

职工文件

职工号	职工姓名	职务	职称
001	张军	教员	讲师
002	沈灵	系主任	教授
003	叶明	校长	教授
004	张蓬	室主任	副教授
005	叶宏	系主任	教授
006	周芳	教员	教授
007	刘光	系主任	教授
008	黄兵	教员	讲师
009	李民	室主任	教授
010	赵松	教员	副教授
…	…	…	…

职务有校长、系主任、室主任和教员；校长领导所有系主任，系主任领导他所在系的所有室主任，室主任领导他所在室的全体教员；职称有教授、副教授和讲师 3 种。请在职工文件的数据结构中设置若干指针和索引，以满足下列两种查找的需要：

（1）能够检索出全体职工间领导与被领导的情况；

（2）能够分别检索出全体教授、全体副教授、全体讲师。

答：在职务项中增加一个指针项，指向其领导者。因题目中未提出具体的隶属关系，如哪个系的系主任，哪个系哪个室的室主任，哪个室的教员等。这里假设每个室主任隶属于他前边离他最近的那个系主任，每个教员隶属于他前边离他最近的那个室主任，如下多重表文件。在职称项中增加一个指针项，指向同一职称的下一个职工，增加一个次关键字索引表。

“职称”索引表

关键字	头指针	长度
讲师	001	2
副教授	004	2
教授	002	6

多重表文件

记录号	职工号	职工姓名	职务		职称	
01	001	张军	教员	04	讲师	08
02	002	沈灵	系主任	03	教授	03
03	003	叶明	校长	∧	教授	05
04	004	张蓬	室主任	02	副教授	10
05	005	叶宏	系主任	03	教授	06
06	006	周芳	教员	04	教授	07
07	007	刘光	系主任	03	教授	09
08	008	黄兵	教员	04	讲师	∧
09	009	李民	室主任	07	教授	∧
10	010	赵松	教员	09	副教授	∧
	…	…	…	…		